

2. Követelmény, projekt, funkcionalitás

23 – githappens

Konzulens:

Haragos Gergő Viktor

Csapattagok

Czap László	NS7V2T	czap.laszlo@edu.bme.hu
Bocskai Zsófia	UGSTW9	bocskaizsofi@gmail.com
Tóth Márton	K01YXA	toth.marton21@gmail.com
Dénes Péter István	PO6MVA	denespeter03tab@gmail.com
Jandó Barnabás Tamás	AWQ77G	jandobarnabas@gmail.com

2026.02.25.

2. Követelmény, projekt, funkcionalitás

2.1 Bevezetés

2.1.1 Cél

Ezen dokumentum célja a követelmények és a specifikáció meghatározása illetve a használati esetek bemutatása, mindezen által a fejlesztők és a megrendelő számára egyértelműbb a projekt.

2.1.2 Szakterület

A projekt keretein belül megvalósítandó szoftver játék, célterület tehát a szórakoztatás.

2.1.3 Definíciók, rövidítések

MVC - model-view-controller

2.1.4 Hivatkozások

Kari felhő elérési linkje: <https://niif.cloud.bme.hu/>

Feladatleírás (Hókotrók): <https://www.iit.bme.hu/file/11582/feladat>

Követelmény, projekt, funkcionalitás segédlet:

<https://www.iit.bme.hu/file/13619/2-k%C3%B6vetelm%C3%A9ny-projekt-funcionalit%C3%A1s/>

2.1.5 Összefoglalás

Jelen dokumentumban, áttekintjük a feladatban megvalósítandó szoftvert és annak funkcióit, használati eseteit illetve követelményeit. Szó esik még a projekt tervről azon belül pedig a részletes ütemtervről.

Az áttekintés fejezetben általános leírást teszünk a kiadandó szoftverről, a funkcióiról, a felhasználók típusáról és milyenségéről, a korlátozásokról és megemlítyük a projekt feltételezéseit.

A követelmények fejezetben tárgyaljuk a funkcionális, erőforrásokkal és átadással kapcsolatos követelményeket illetve az egyéb nem funkcionális követelményeket.

A lényeges use casek fejezetben, találhatóak a use case leírások táblázatos formában illetve a use-case diagram.

A szótár fejezet a követelményekben írt szavakat tartalmazza és azok jelentéseit.

Az utolsó fejezet pedig tartalmazza a részletes projekt tervet ahol a projekt lebonyolításához szükséges eszközöket tárgyaljuk továbbá a csoporttagok közötti kommunikáció mikéntjét a munkamegosztást és a részletes ütemtervet, heti felelősöket.

2.2 Áttekintés

2.2.1 Általános áttekintés

Miután elindítottuk a programot, megjelenik a kezdőképernyő, ahol be tudjuk állítani a kívánt paramétereket a játékhoz (például a járművek számát és a játékhosszt). Ezt követően el tudjuk indítani a játékot, aminek következtében megjelenik a pálya a havas utakkal, a civil autókkal és a játékos által irányított hókotrókkal, illetve buszokkal. A játék végén megjelenik az eredményjelző, ami tudatja a felhasználóval az elért végeredményt.

A legmagasabb szintű szervezés MVC architektúrával fog történni, mely szétválasztja a játék logikáját az aktuális adatoktól és a megjelenéstől. Nem szükséges hálózati kapcsolat, mivel a játék egyetlen számítógépen fut. Az adattárolás a játék során a memóriában történik, ehhez elég minimális szintű memória (néhány megabájt).

2.2.2 Funkciók

A feladat egy stratégiai és erőforrás-menedzsment játék megalkotása és annak dokumentálása. A helyszín Zúzmaraváros, amelyet váratlanul ér egy folyamatos, intenzív havazás. A játékos feladata, hogy egy személyben menedzselje a város közlekedését: irányítsa a takarító cégek hókotróit, valamint a helyi közlekedési vállalat buszait. A játék indítása előtt a felhasználó beállíthatja a szimuláció alapvető paramétereit (pl. a játék hosszát körökben mérve, a kezdő járművek számát).

A játék központi eleme a város úthálózata. Mivel a város tektonikusan aktív területen fekszik, az utak nem hagyományos koordinátarendszerben helyezkednek el, hanem csomópontok és az azokat összekötő útszakaszok, hidak és alagutak hálózatát alkotják. Egy kereszteződésben akár négyenél több út is összefuthat. Az utakon a civil autósok a lakásuk és a munkahelyük között közlekednek, mindig a legrövidebb járható utat keresve.

A város időjárása és az úthálózat állapota folyamatosan változik, ami dinamikus kihívást nyújt. A havazás folyamatos, így a már letakarított utakat is újra beledi a hó. Ha túl sok autó halad át egy havas szakaszon, a hó jégpáncéllá válik. A vékony hórétegben az autók még tudnak haladni, de a mély hóban elakadnak, a jégpáncélon pedig megcsúsznak. Ha két autó összeütközik, baleset történik, ami az adott sávot járhatatlanná teszi, egészen addig, amíg egy hókotró a szomszédos sávot le nem takarítja.

Takarítók és Hókotrók

A játékos takarítóként a hókotrók mozgását irányítja, és gazdasági döntéseket hoz. A hókotrók minden letisztított útszakaszért pénzt kapnak. Ezt a bevételt a játékos visszaforgathatja a vállalatba: vehet új hókotrókat, vagy lecserélheti a meglévő gépek kotrófejeit. Kezdetben csak egyszerű hányó vagy jégtörő fejes hókotrók állnak rendelkezésre. Egy hókotró egyszerre csak egy sávot tud letakarítani, és a felszerelt fejtől függ, hogy milyen munkát végez:

- **Söprő és Hányó fej:** A havat a nyomvonal mellé (a szomszédos sávba vagy az út mellé) tolják, de a jeget nem tudják eltakarítani.
- **Jégtörő fej:** Csak a jeget töri fel, de a havat nem takarítja el.
- **Sószóró fej:** Sót szór az útra, ami megakadályozza a hó megmaradását, és idővel felolvasztja a jeget. Az olvadási idő a jég és hó vastagságától függ.
- **Sárkány fej:** Egy gázturbinával azonnal és egyszerűen elolvasztja a havat és a jeget a jármű előtt.

A játékosnak figyelnie kell a nyersanyagokra is: a sószóróhoz sót, a sárkányhoz biokerozint kell vásárolnia. Ha ezek elfogynak, a fej hatástalanná válik, és a játékosnak vagy újat kell vennie, vagy le kell cserélnie a fejet egy másik típusra.

Buszvezetők

A játékos buszvezetőként a helyi tömegközlekedés fenntartásáért felel. A buszoknak meghatározott két végállomás között kell minél többször megfordulniuk a játék ideje alatt. A játékos feladata a megfelelő útvonal kijelölése a buszok számára. Hatalmas kockázatot jelent egy balesetekkel teli, jeges útra hajtani: ha a busz megcsúszik, vagy egy másik jármű

belecsúszik, a busz azonnal mozgásképtelenné válik. Egy balesetet szenvedett busz a büntetés miatt több körből is kimarad, és csak a rongcsok eltakarítása (a szomszédos sáv felszabadítása) után tudja folytatni az útját, ami jelentős idővesztést és pontkiesést okoz.

Pontszerzés és a játék vége

A játék célja a buszok számára, hogy az adott (körökben mért) idő alatt a lehető legtöbb sikeres fordulót teljesítsék a végállomások között. A takarítók sikere a gazdasági mutatókban mérhető: a cél, hogy a letakarított utakból a játék végére minél nagyobb vagyont és járműparkot halmozzanak fel. A szimuláció végén a rendszer kiértékeli a játékos döntéseit, és egy eredményjelző felületen összesíti a teljesítményt.

2.2.3 Felhasználók

A játék tematikájából kiindulva a felhasználók több korosztályból is kikerülhetnek. Mivel a program egy egygépes, körökre osztott logikai-stratégiai szimuláció, olyan játékosokat céloz meg, akik élvezik az erőforrás-menedzsmentet és az előretervezést. A felhasználóknak nem szükséges rendelkezniük semmilyen mélyebb informatikai háttértudással. Elegendő a játékszabályok megismerése, valamint a rendszer által vizuálisan nyújtott információk (utak állapota, balesetek) értelmezése.

A játékot legalább két játékos játssza egymás ellen, de mindkét fajtából több játékos is részt vehet egyszerre a szimulációban. A felhasználók két eltérő szerepkört tölthetnek be, amelyek más-más gondolkodásmódot és célokat követelnek meg:

- **Takarítók:** Ezek a játékosok a hókotrók flottáját irányítják. Jellemzőjük a stratégiai gondolkodás, a gazdasági erőforrás-menedzsment (bevételek visszaforgatása, új gépek és eszközök vásárlása), valamint a város közlekedési gócpontjainak átlátása.
- **Buszvezetők:** Az ő feladatuk a buszok biztonságos navigálása a végállomások között. Jellemzőjük a kockázatfelmérő képesség és a logikus útvonaltervezés, mivel folyamatosan reagálniuk kell a jégpáncélokra és balesetekre, hogy minimalizálják a buszok elakadásának esélyét.

2.2.4 Korlátozások

A fejlesztendő szoftverre vonatkozó főbb technikai és architektúrális korlátozások a következők:

- **Futtatási környezet:** A játéknak a megrendelő által rendelkezésre bocsátott, távolról elérhető virtuális gépen kell hibátlanul futnia. A környezet specifikációja: "Windows 10 20H2 - JDK-Eclipse-WSU" sablon. A játék PC-re készül.
- **Fejlesztési eszközök:** A szoftver megvalósítása Java nyelven történik, a futtatáshoz szükséges a JVM (Java Virtual Machine) telepítése. A fordításhoz és a futtatáshoz pusztán a standard JDK eszközkészlet használható (javac és java parancsok), külső keretrendszerek használata nélkül.
- **Architektúra és Kialakítás:** A program a Modell-Nézet-Vezérlő (MVC) tervezési mintára épül. Hálózati kapcsolatot nem igényel, egygépes (lokális) futtatásra terveztük. Az adatokat memóriában tárolja, külső adatbázist nem használ. A megjelenítés nem valós idejű, hanem körökre osztott, ugrásszerű frissítéseken alapul a magasabb szintű stabilitás érdekében.

- Hardveres korlátozások: A szoftver kezelése egyszerű, futtatásához és a játékosok interakcióihoz kizárólag egy standard billentyűzet és egy egér (vagy touchpad) csatlakoztatása szükséges.

2.2.5 Feltételezések, kapcsolatok

A fejlesztési folyamat és a dokumentáció elkészítése során az alábbi forrásokra és felületekre támaszkodunk:

- Feladatleírás (Hókotrók): A tárgy hivatalos oldalán közzétett dokumentum, amely rögzíti a játék alapvető szabályait, a gazdasági rendszert és a járművek viselkedését.
- Követelmény, projekt, funkcionalitás segédlet: A dokumentáció formai és tartalmi elvárásait rögzítő leírás, amely a futtatási környezetet is specifikálja.
- Verziókezelés és Feladatkövetés: A kód megosztását és a csapatmunka összehangolását a Github, a munkafolyamatok követését a Jira, dokumentumok szerkesztését Google Docs segíti.

2.3 Követelmények

2.3.1 Funkcionális követelmények

Azonosító	Leírás	Ellenőrzés	Prioritás	Forrás	Use-case	Komment
FK001	A rendszernek a szimuláció során folyamatosan növelnie kell az utakon lévő hóréteg vastagságát.	A játék indítása után látszódnia kell az idő elteltével a hóréteg növekedésének.	MUST	Feladatleírás	Hóréteg növekedése	
FK002	A járműveknek a haladásuk során meg kell találniuk a céljukhoz vezető legrövidebb, akadálymentes utat.	A jármű a láthatóan legrövidebb úton éri el a célt. Ha az eredeti útvonalát elzárjuk egy akadállyal, kikerüli azt, és a megmaradt lehetőségek közül a legrövidebb úton halad tovább.	MUST	Feladatleírás	Jármű irányítása/legrövidebb elérhető út keresése	
FK003	Ha a járművek	Hogyha 3-4 jármű	MUST	Feladatleírás	Vékony hórétegben	

	vékony hórétegen haladnak át, a hó letaposódik és a sáv jégpáncéllá válik.	áthalad a letakarítatlan úton. Az autók elkezdene csúszkálni. Grafikusan megjelenik.			irányítás, Hó taposása	
FK004	Jeges útszakaszon a járművek megcsúszhatnak és összeütközhetnek, ami az adott sávon elakadást eredményez.	Hogyha jeges úton két egymás után haladó jármű sebessége különböző és a hátsó jármű gyorsabb akkor összeütköznek.	MUST	Feladatleírás	Jégpáncélon csúszkálás kezelése, Ütközés kezelése, Elakadás kezelése	A baleset blokkolja az utat a letakarításig
FK005	A hókotróknak a felszerelt mechanikus fej típusától függően (söprő, hányó, jégtörő) el kell távolítaniuk vagy fel kell törniük a havat/jeget.	Ha megfelelő kotrófejjel egy hókotró elhalad az úton akkor a jégpáncél grafikusan eltűnik és az autók nem csúsznak össze.	MUST	Feladatleírás	Sáv takarítása kotrófejjel, Hó eldalra tolása, Hó messzire szórása, Jégtörés	
FK006	A speciális fejeknek (sósóró, gázturbina) nyersanyag (só/keozin) felhasználásával el kell olvasztaniuk a havat és a jeget.	A sónak és a keozinnak fogynia kell, a hónak vagy jégnek pedig el kell tűnnie.	MUST	Feladatleírás	Hó és jég olvasztása/ Sóval olvasztás, Gázturbinával olvasztás	Nyersanyag hiányában hatástalan
FK007	A takarító játékosoknak minden sikeresen	Hogyha egy útról eltűnik a jég vagy hó a játékos	MUST	Feladatleírás	Pénz beszédése	A gazdasági

	megtisztított sáv után pénzjutalmat kell kapniuk.	által egy adott útszakasz egyik sávján a játékos számlájának értéke nő.				ciklus alapja
FK008	A takarítóknak lehetőséget kell biztosítani a felhalmozott pénzből új gépek, nyersanyagok vásárlására és a fejek cserélésére.	Grafikus felületen megjelenő menüből kiválaszthatóak eszközök amelynek hatására a játékos számlájának értéke csökken.	MUST	Feladatleírás	Segédeszközök vásárlása, Kotrófej cserélés	
FK009	A buszvezetőknek a buszaikkal végállomások közötti utakat kell megtenniük, amelyet a rendszernek számolnia kell és új útvonaltervezést kell indítania a másik végállomás felé.	Ha busz eléri a célként megadott végállomást a rendszer frissíti a játékos pontszámát. A busz nem áll meg véglegesen, új útvonalterv jön létre a másik végállomásra.	MUST	Feladatleírás	Buszok irányítása, Forduló teljesítése	Ez a buszvezetők fő pontszerzési módja.
FK010	Ha egy jármű előtt a sáv járhatatlanná válik (baleset, túl mély hó), de a szomszédos sáv szabad, a rendszernek engedélyezni	A jármű az új sávra folytatja az útját a célja felé.	MUST	Feladatleírás	Sáv váltás	

	e kell a sávváltást legrövidebb út megtalálása érdekében.					
FK011	A rendszernek meg kell különböztetni a felszíni utakat az alagutaktól. Az alagutakban futó sávokon a havazás nem növelheti a hóréteg vastagságát.	Ha egy út alagút akkor ott nem lehet hó, sem grafikusán és nem okozhat elakadást sem, továbbá nem alakulhat ki jégpáncél sem.	MUST	Feladatleírás	Hóréteg növekedése	
FK012	Ha a jégpáncél miatt két autó összeecsúszik, a rendszernek az adott sávot járhatatlanná kell tennie a forgalom számára (buszok esetében az adott időre mozgásképtelenné kell tenni a járművet).	Szimuláljunk egy ütközést egy jeges sávban. Győződjünk meg róla, hogy a baleset bekövetkezése után az adott sáv státusza járhatatlanná vált, és a mögöttes forgalom nem tud áthaladni rajta. Busz bevonása esetén ellenőrizzük, hogy a megadott idő letelte után a busz újra	MUST	Feladatleírás	Elakadás kezelése	

		mozgásképe ssé válik-e.				
FK013	Ha a havazás miatt a hóréteg eléri a mély hó szintjét, a személyautóknak el kell akadniuk, amíg egy hókotró meg nem tisztítja az utat.	Megnő a hóréteg az útszakaszon “mély hó” szintre, majd figyeljük az arra haladó autókat amiknek el kell akadniuk. Majd oda küldünk egy kotrót és takarítás után az autóknak újra el kell tudni indulniuk.	MUST	Feladateleírás	Elakadás kezelése	
FK014	A söprő és hányó kotrófejek esetében a rendszernek tiltania kell a jeges állapotú sávok takarítását. Ezek a fejek csak a havat és a már feltört jeget távolíthatják el.	Szereljük fel egy söprő vagy hányó kotrófejet, majd irányítsuk a járművet egy jeges még fel nem tört sávra és ellenőrizzük, hogy a jég megmarad-e. Egy sima havas, illetve már feltört jeges sávon pedig sikeres-e a takarítás?	MUST	Feladateleírás	Hó és feltört jég takarítása	
FK015	A járműveknek előnyben kellene részesítenie a letakarított	Hozzunk létre egy szituációt két lehetséges útvonallal	SHOULD	Csapat	Legrövidebb elérhető út keresése.	

	útszakaszokat , akkor is ha távolságban kis mértékben hosszabbak a vékony hóval borított rövidebb utaknál.	ugyanazon célállomás felé: egy rövidebb, de vékony hóval borított és egy kissé hosszabb, de letakarított úttal. Adjunk parancsot a járműnek a célba jutásra, és ellenőrizzük , hogy automatikusan a hosszabb, de tiszta útvonalat választja-e.				
FK016	Baleset Bekövetkezés ekor vizuálisan jól látható jelzéssel ki kellene emelnie a térképen az érintett területet.	Balesetet idézünk elő és ellenőrizzük , hogy megjelent-e a baleset pillanatában egy vizuális figyelmeztet és	SHOULD	Csapat	Ütközések kezelése	
FK017	A takarítóknak figyelmeztetést kellene kapnia ha olyan fejet próbálnak felrakni amihez nincsen használható nyersanyagjuk.	Próbáljunk meg olyan fejet feltenni amihez nincsen szükséges nyersanyag	SHOULD	Csapat	Kotrófej cserélés	

FK018	A takarító játékosoknak folyamatosan és egyértelműen mutatnia kellene az aktuális pénzügyi egyenlegét.	Játék közben folyamatosan látható a játékos aktuális egyenlege. Tranzakció esetén a kijelzőn szereplő összeg frissül.	SHOULD	Csapat	Segédeszközök vásárlása	
FK019	A takarítók számára biztosítani kell a lehetőséget, hogy új hókotró járművet vásároljon, amelynek az ára a legmagasabb a segédeszközök között.	A bolt menüjében ellenőrizni kell, hogy a hókotró ára a legmagasabb. Megfelelő egyenleg esetén sikeres a vásárlás.	MUST	Feladatleírás	Segédeszközök vásárlása	
FK020	A hókotróknak szigorúan egyetlen sávot szabad megtisztítaniuk egyszerre, függetlenül a felszerelt kotrófej típusától.	Bármilyen kotrófej van használatban, ha egy havas sávon végigmegyünk, kizárólag abból a sávból tűnik el a hó, amiben a jármű halad.	MUST	Feladatleírás	Sáv takarítása kotrófejjel	
FK021	A hányó fej használatakor a rendszernek a havat nem a közvetlenül mellette lévő sávba, hanem annál távolabbra	A hányó fejjel elkotort hó nem kerülhetett a szomszédos sávba	MUST	Feladatleírás	Hó messzire szórása	

	kell áthelyeznie.					
FK022	A játék kezdetekor minden takarító játékos kizárólag egy hányó fejes vagy egy jégtörő fejes hókotrórt kaphat.	A játék kezdetekor csak egy hókotró van, ami vagy hányó fejes vagy jégtörős	MUST	Feladatleírás	Hókotrók irányítása	

2.3.2 Erőforrásokkal kapcsolatos követelmények

Azonosító	Leírás	Ellenőrzés	Prioritás	Forrás	Komment
EKK1	A forráskódnak futtatható kell lennie fizikai gépen illetve a kari felhőben is.	Bemutató	MUST	Feladatleírás	Kari felhő elérési linkje: (https://niif.cloud.bme.hu/ vagy https://fured.cloud.bme.hu/)
EKK2	A program JAVA nyelven fog íródni.	Csapaton belüli megegyezés	MUST	Csapat	
EKK3	Tárhely a szoftver telepítéséhez	Futtatáskor	MUST	Tóth Márton (K01YXA)	
EKK4	Egér és billentyűzet a szoftver használatához	Futtatáskor	MUST	Tóth Márton (K01YXA)	

2.3.3 Átadással kapcsolatos követelmények

Azonosító	Leírás	Ellenőrzés	Prioritás	Forrás	Komment
ÁKK1	A dokumentumokat	Beadás előtt csapat ellenőrzi,	MUST	Feladatleírás	

	mind papírra kinyomtatva be kell nyújtani, mind a hercules rendszerben PDF fájlként fel kell tölteni.	leadás utáni aláírás.			
ÁKK2	Valamennyi leadott dokumentum címlapjára a letölthető sablon szerintinek kell lennie	Beadás előtt csapat ellenőrzi	MUST	Feladatleírás	
ÁKK3	A nyomtatott lapokat nem szabad összetűzni, hanem lefűzhető, átlátszó műanyag irattartóban (bugyi) kell beadni.	Beadó személy gondoskodik róla	MUST	Feladatleírás	
ÁKK4	Minden leadott anyag végén a csapat naplója áll. Naplót a leadott anyaghoz MINDIG kell készíteni.	Beadás előtt csapat egyezteti a napló tartalmát	MUST	Feladatleírás	
ÁKK5	A feltöltendő szoftver anyagot	Beadás előtt a feltöltő ellenőrzi	MUST	Feladatleírás	(tar, rar, 7z, arj, stb)

	egyetlen zip fájlba kell becsomagolni.				
ÁKK6	Minden szakaszhoz maximum 3 alkalommal lehet anyagot feltölteni.	Beadás előtt csapat ellenőrzi	MUST	Feladatleírás	
ÁKK7	A forráskódok mellett meg kell adni az installálási útmutatást és kezelési leírást.	Beadás előtt a feltöltő ellenőrzi	MUST	Feladatleírás	

2.3.4 Egyéb nem funkcionális követelmények

Azonosító	Leírás	Ellenőrzés	Prioritás	Forrás	Komment
ENFK1	Hiba esetén a szoftver adjon érthető hibaüzenetet.	Futtatáskor	MUST	Csapat	
ENFK2	A szoftver adjon sugót.	Futtatáskor	SHOULD	Csapat	
ENFK3	A kód tartalmazzon megfelelő kommentelést.	Fejlesztéskor	SHOULD	Csapat	Jobb karbantarthatóságot biztosít
ENFK4	A grafikus felület kezelőfelülete intuitív.	Futtatáskor	MUST	Csapat	

2.4 Lényeges use-case-ek

2.4.1 Use-case leírások

Use-case neve	Jármű irányítása
Rövid leírás	Absztrakt, általános járműirányítási folyamat, amelyet az autók, buszok és hókotrók irányítása use-case-ek terjesztenek ki.
Aktorok	(nincs közvetlen aktor – absztrakt use-case)
Forgatókönyv	A jármű haladni próbál az úton.

Use-case neve	Autók irányítása
Rövid leírás	A sofőr az autót irányítja a „Jármű irányítása” általános viselkedésének kiterjesztésével.
Aktorok	Sofőr
Forgatókönyv	A sofőr közlekedik az autóval a munkahelye és a lakóhelye között a legrövidebb elérhető útvonalon, reagál a hó vastagságára, jégre, akadályokra és szükség esetén sávot vált vagy megáll. Az autó halad, amíg a hó nem túl mély. Ha a hó túl mély, elakad. Ha sok jármű tapossa a havat, jégpáncél képződik és csúszkálás történhet.

Use-case neve	Buszok irányítása
Rövid leírás	A buszvezető irányítja a buszt a „Jármű irányítása” általános viselkedésének kiterjesztésével.
Aktorok	Buszvezető
Forgatókönyv	A buszvezető vezeti a buszt a két végállomás között a legrövidebb elérhető útvonalon, reagál a hó vastagságára, jégre, akadályokra és szükség esetén sávot vált vagy megáll. A buszvezetők célja minél több forduló teljesítése. A busz halad, amíg a hó nem túl mély. Ha a hó túl mély, elakad. Ha sok jármű tapossa a havat, jégpáncél képződik és csúszkálás történhet. Ha ütközés történik, akkor a busz ideiglenesen mozgásképtelenné válik.

Use-case neve	Forduló teljesítése
Rövid leírás	A buszvezetők célja, akkor következnek be, amikor megfordulnak a végállomások között.
Aktorok	Buszvezető
Forgatókönyv	A buszvezetők oda.vissza közlekednek a két végállomás között és teljesítik a fordulókat.

Use-case neve	Legrövidebb elérhető út keresése
Rövid leírás	Absztrakt, általános folyamat, amelyet az autók és buszok irányítása use-case-ek terjesztenek ki. A járművek (autó,

	busz) a legrövidebb elérhető úton közlekednek két pont között.
Aktorok	(nincs közvetlen aktor – absztrakt use-case)
Forgatókönyv	A legrövidebb elérhető út meghatározása után a jármű eljut az útvonalon egyik végpontból a másikba.

Use-case neve	Sávváltás
Rövid leírás	Absztrakt, általános folyamat, amelyet az autók, buszok és hókotrók irányítása use-case-ek terjesztenek ki. A jármű sávot vált.
Aktorok	(nincs közvetlen aktor – absztrakt use-case)
Forgatókönyv	A jármű az aktuális sávban akadályt érzékel. Ha a mellette levő sáv járható, sávot vált. Ha nem, elakad.

Use-case neve	Vékony hórétegben irányítás
Rövid leírás	Absztrakt, általános folyamat, amelyet az autók és buszok irányítása use-case-ek terjesztenek ki. A járművek a vékony hórétegben közlekednek.
Aktorok	(nincs közvetlen aktor – absztrakt use-case)
Forgatókönyv	A járművek a vékony hóréteggel borított útszakaszra lépnek. A havazás folyamatos és a hóréteg növekszik. Ha túl mély lesz a hó, a járművek elakadnak. Ha sok jármű tapossa a havat, jégpáncél képződhet rajta.

Use-case neve	Hó taposása
Rövid leírás	Absztrakt, általános folyamat, amelyet az autók és buszok irányítása use-case-ek terjesztenek ki. A járművek a vékony hóban haladva tapossák a havat.
Aktorok	(nincs közvetlen aktor – absztrakt use-case)
Forgatókönyv	Sok jármű a vékony hóréteget letapossa és jégpáncélt alakít ki. A jégpáncélon a járművek csúszkálhatnak, illetve ha összeecsúsznak, a sáv járhatatlanná válik.

Use-case neve	Jégpáncélon csúszkálás kezelése
Rövid leírás	Absztrakt, általános folyamat, amelyet az autók és buszok irányítása use-case-ek terjesztenek ki. A járművek a jégpáncélon csúszkálhatnak.
Aktorok	(nincs közvetlen aktor – absztrakt use-case)
Forgatókönyv	A hóréteg letaposása után kialakult jégpáncélon a járművek csúszkálnak és egymásba csúszhatnak, aminek eredményeként a sáv járhatatlanná válik.

Use-case neve	Ütközés kezelése
Rövid leírás	Absztrakt, általános folyamat, amelyet az autók és buszok irányítása use-case-ek terjesztenek ki. A járművek összeecsúsznak a jégen.

Aktorok	(nincs közvetlen aktor – absztrakt use-case)
Forgatókönyv	A jégpáncélon csúszkálás következtében a járművek összeecsúszhatnak. A sáv járhatatlanná válik.

Use-case neve	Elakadás kezelése
Rövid leírás	Absztrakt, általános folyamat, amelyet az autók és buszok irányítása use-case-ek terjesztenek ki. A járművek elakadhatnak.
Aktorok	(nincs közvetlen aktor – absztrakt use-case)
Forgatókönyv	A jármű elakad, ha a hó túl mély vagy ütközés miatt. Ha a jármű tud, sávot vált, ha nem, várakozik a hókotróra.

Use-case neve	Hókotrók irányítása
Rövid leírás	A takarítók irányítják a hókotrókat.
Aktorok	Takarító
Forgatókönyv	A takarítók a hókotrókkal a az utcákat járkák és céljuk tisztítani az útszakaszokat..

Use-case neve	Sáv takarítása kotrófejjel
Rövid leírás	A takarító a kotrófejjel takarít egyszerre egy sávot.
Aktorok	Takarító
Forgatókönyv	A takarító a felszerelt kotrófejjel takarítja az aktuális sávban lévő útszakaszokat.. Takarítás közben lehet fejet cserélni a már meglévő fejek között.

Use-case neve	Hó és feltört jég takarítása
Rövid leírás	A söprő és a hányó fej a lefagyott jeget nem, csak a havat és a feltört jeget takarítja.
Aktorok	Takarító
Forgatókönyv	A feltört jég takarítása előtt fel kell törni a jeget. A söprő és a hányó fej a lefagyott jeget nem, csak a havat és a feltört jeget tudja eltakarítani.

Use-case neve	Hó oldalra tolása
Rövid leírás	A söprő fej oldalra tolja a havat, közvetlenül a hókotró nyomvonala mellé, azaz a szomszédos sávba vagy az út mellé.
Aktorok	Takarító
Forgatókönyv	A hó a hókotró nyomvonala mellé, a szomszédos sávba tolása.

Use-case neve	Hó messzire szórása
Rövid leírás	A hányó fej oldalra, de messzire szórja a havat.
Aktorok	Takarító
Forgatókönyv	A hó messzire szórása.

Use-case neve	Jégtörés
Rövid leírás	A jégtörő fej a jeget töri fel, de nem takarítja el azt.
Aktorok	Takarító
Forgatókönyv	A jégtörő fej feltöri a jeget, ezt később a söprő és hányó fej eltakaríthatja.

Use-case neve	Hó és jég olvasztása
Rövid leírás	A sószóró fej és a sárkány fejjel elolvasztható a hó és a jég.
Aktorok	Takarító
Forgatókönyv	A takarító a sószóró fej vagy a sárkány fejjel elolvasztja a havat vagy jeget.

Use-case neve	Gázturbinával olvasztás
Rövid leírás	A sárkány fejjel elolvasztható a hó és a jég a hókotró előtt.
Aktorok	Takarító
Forgatókönyv	A takarító sárkány fejjel elolvasztja a havat vagy jeget. A sárkányfej biokerozinnal működik, ha elfogy a fej hatástalanná válik, ekkor a fejet le kell cserélni vagy biokerozint venni.

Use-case neve	Sóval olvasztás
Rövid leírás	A sószóró fejjel elolvasztható a hó és a jég a hókotró előtt.
Aktorok	Takarító
Forgatókönyv	A sószóró fej által kiszórt só egy idő után elolvasztja a havat és a jeget, és megakadályozza, hogy a lehulló hó megmaradjon az utakon. A só által kifejtett olvadási idő a hó és a jég vastagságától függ. A sószóró sóval működik, ha elfogy a fej hatástalanná válik, ekkor a fejet le kell cserélni vagy sót venni.

Use-case neve	Kotrófej cserélés
Rövid leírás	A hókotróra egyszerre egyfajta fej szerelhető, ezeket lehet cserélgetni.
Aktorok	Takarító
Forgatókönyv	A takarító a meglévő fejek közül kiválaszthatja, mire szeretné lecserélni a jelenlegi fejet. Létezik söprő, hányó, jégtörő, sószóró és sárkány fej is.

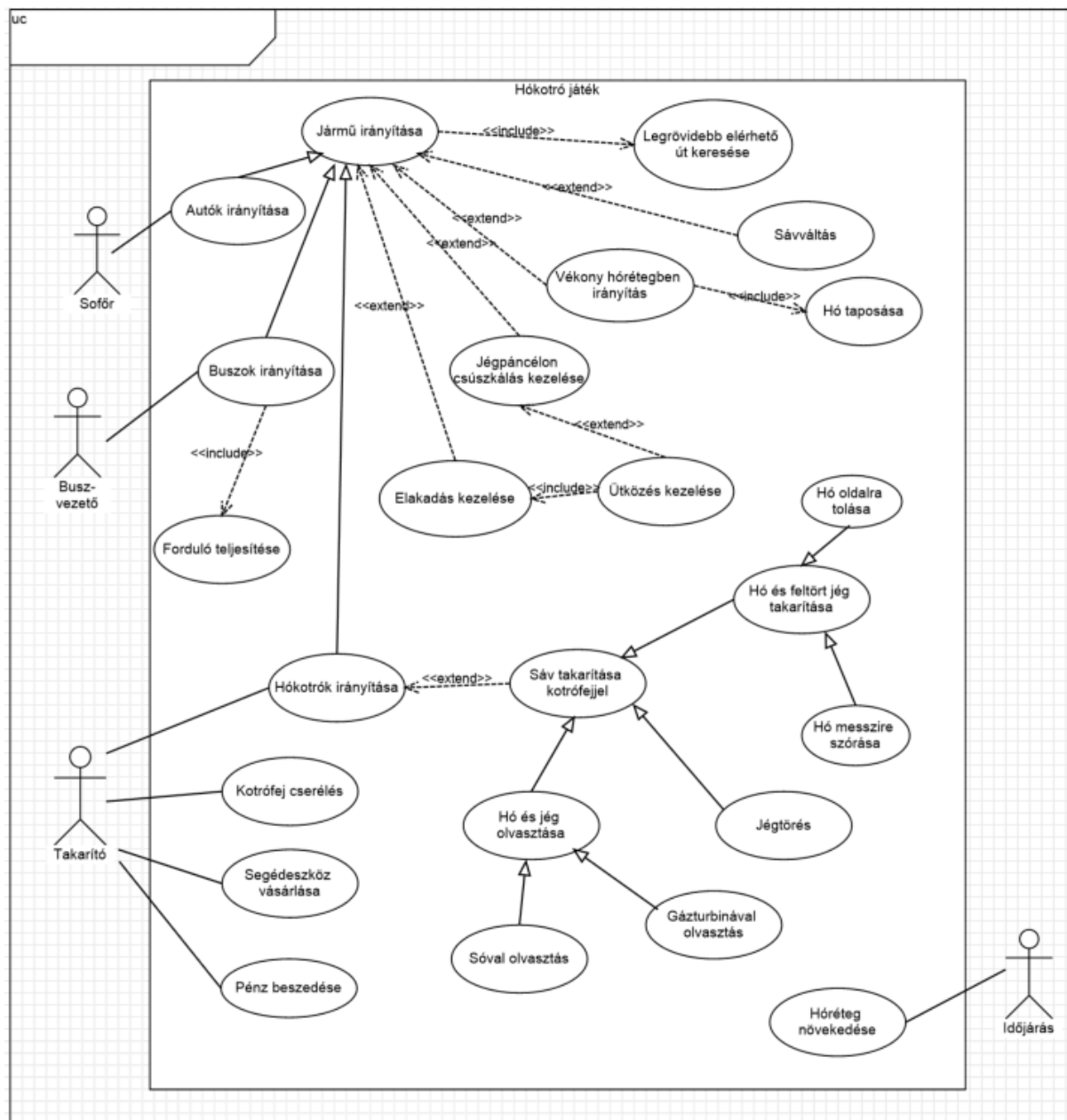
Use-case neve	Segédeszköz vásárlása
Rövid leírás	A takarító pénzből új fejeket, sót, biokerozint vagy új hókotrókat vásárol.
Aktorok	Takarító
Forgatókönyv	A takarítóknak kezdetben csak egy hányó fejes vagy egy jégtörő fejes hókotró áll rendelkezésükre, de később újabb fejek és újabb hókotrók is vásárolhatók.

	A fejek árai növekednek az alábbi sorrendben: söprő, hányó, jégtörő, sószóró, sárkány. A sószóró működéséhez só, a sárkány működéséhez biokerozint kell vásárolni. A legdrágább segédeszköz az új hókotró.
--	---

Use-case neve	Pénz beszédése
Rövid leírás	A hókotró minden letisztított útszakasz után pénzt kap.
Aktorok	Takarító
Forgatókönyv	A hókotró megtisztít egy útszakaszt, amiért jutalmat kap. A pénz jóváírásra kerül a játékos számláján.

Use-case neve	Hórégteg növekedése
Rövid leírás	A havazás folyamatos, emiatt a hóréteg folyamatosan növekszik, illetve a letakarított utakat is újra behavazza.
Aktorok	Időjárás (másodlagos aktor)
Forgatókönyv	A havazás elindul, behavazza az utakat. Az alacsony hórétegű utakban a járművek még tudnak közlekedni, ám a vastag hórétegben elakadnak. A takarítók az utakat kotrófejjel takarítják. A letakarított utakat a hó fokozatosan újra ellepi.

2.4.2 Use-case diagram



2.5 Szótár

Alagút: Olyan útszakasz, amely a felszín alatt halad. Alagutak által az utak nem csak szintben kereszteződhetnek.

Autó (személyautó): Civil jármű, amely a lakóhely és a munkahely között közlekedik a legrövidebb elérhető útvonalon.

Biokerozin: A sárkány fej működéséhez szükséges nyersanyag, amely a hó és a jég azonnali elolvasztásához kell.

Busz: Játékos által irányított tömegközlekedési jármű, amely két végállomás között közlekedik, és fordulókat teljesít céljából.

Buszvezető: Olyan játékos, aki a buszokat irányítja, útvonalat tervez, és a biztonságos közlekedésért felel.

Csomópont (útkereszteződés): Az úthálózat olyan pontja, ahol több útszakasz találkozik, és ahol a járművek irányt válthatnak. Az utak nem csak szintben kereszteződhetnek, vannak hidak és alagutak is. A keresztezésekben pedig akár négynél több út is összefuthat.

Elakadás: Olyan állapot, amikor egy jármű nem tud továbbhaladni mély hó, baleset miatt.

Forduló: A busz egyik végállomástól a másikig történő eljutása, majd visszaindulása.

Gázturbina (Sárkány fej): Speciális kotrófej, amely biokerozinnal működik, és azonnal elolvasztja a hókotró előtti havat és jeget.

Hányó fej: Olyan kotrófej, amely oldalra, de messzire szórja a havat. A lefagyott jeget nem, csak a havat és a feltört jeget tudja eltakarítani.

Havazás: Folyamatos időjárási jelenség a játékban, amely minden körben növeli a hóréteg vastagságát a felszíni utakon.

Híd: Olyan útszakasz, amely a felszín felett halad. Hidak által az utak nem csak szintben kereszteződhetnek.

Hó: Az útfelületen felhalmozódó csapadék, amelynek vastagsága befolyásolja a járművek közlekedését. (Vékony hóban a járművek még képesek haladni, de mélyben elakadnak.)

Hókotró: Speciális jármű, amely maga előtt eltávolítja a sávokon felhalmozódott havat és jeget különböző kotrófejek segítségével.

Hóréteg: Az útfelületen található hó, amely lehet vékony, mély.

Jármű: Általános fogalom, amely magában foglalja az autókat, buszokat és hókotrókat.

Járhatatlan sáv: Olyan sáv, amelyen mély hó vagy baleset miatt nem lehet közlekedni.

Jégtörő fej: Olyan kotrófej, amely a jégpáncélt töri fel, de nem távolítja el a havat.

Jégpáncél: A vékony hó letaposása után kialakuló jeges réteg, amelyen a járművek megcsúszhatnak.

Kotrófej: A hókotróra szerelhető eszköz, amely meghatározza, hogy a jármű milyen módon képes a havat vagy jeget kezelni. Egy hókotróra egyszerre egy fej szerelhető fel, de a fejeket lehet cserélni.

Lakóhely: Az a csomópont, ahonnan egy autó indul és ahová visszatér.

Legrövidebb elérhető út: Két pont közötti olyan útvonal, amely a legkisebb hosszúságú és járható sávokból áll.

Mély hó: Olyan vastagságú hóréteg, amelyen az autók és buszok elakadnak.

Munkahely: Az a csomópont, amely az autók egyik célállomása.

Nyersanyag: A speciális kotrófejek működéséhez szükséges erőforrás (só vagy biokerozin).

Pontszám: A játék során elért teljesítményt mérő érték, amely buszok esetén a fordulók számán, takarítók esetén a vagyonon alapul.

Összecszúsás: Jégpáncélon csúszkáláskor történő ütközéses baleset, aminek következtében a járművek elakadnak.

Sáv: Az útszakasz egy irányított része, amelyen a járművek közlekednek és amelyet a hókotró tisztíthat.

Sáv váltás: Az a művelet, amikor egy jármű az aktuális sávból egy szomszédos, járható sávba tér át.

Só: A sószóró fej működéséhez szükséges nyersanyag, amely idővel elolvasztja a havat és jeget.

Sofőr: Olyan szerep, ami az autókat irányítja, útvonalat tervez, és az autókat a munkahely és a lakóhely között vezeti. (A sofőrök nem játékosok.)

Sószóró fej: Olyan kotrófej, amely sót szór az útra, megakadályozva a hó megmaradását és idővel felolvasztva a jeget.

Söprő fej: Olyan kotrófej, amely a havat a szomszédos sávba vagy az út mellé tolja, de jeget nem képes eltávolítani.

Szomszédos sáv: Az aktuális sáv mellett közvetlenül elhelyezkedő másik sáv.

Takarító: Olyan játékos, aki a hókotrókat irányítja és gazdasági döntéseket hoz.

Úthálózat: A csomópontokból és az azokat összekötő sávokból, hidakból és alagutakból álló rendszer.

Útszakasz: Egy út egy sávjának egysége, a takarítók a útszakaszok takarításáért kapják a pénzt.

Végállomás: A buszok kiinduló és célállomása, amelyek között a fordulókat teljesítik.

Vékony hó: Olyan kis vastagságú hóréteg, amelyen a járművek még képesek közlekedni, de letaposás esetén jégpáncéllá alakulhat.

2.6 Projekt terv

A forráskódot Github segítségével osztjuk meg, a dokumentációt Google Driveon szerkesztjük, Jira segítségével pedig a feladok kiosztását követjük nyomon. Heti meetengekhez Discordot használunk, alapvető és informális kommunikációra pedig messenger csoportot.

hét	feladat	határidő	felelős
1	csapatalakítás	febr. 20. 13:00	Tóth Márton
2	Követelmény, projekt, funkcionalitás	márc. 2. 14:15	Dénes Péter István
3	Analízis modell (I. változat)	márc. 9. 14:15	Bocskai Zsófia
4	Analízis modell (II. változat)	márc. 16. 14:15	Czap László
5	Szkeleton tervezése	márc. 23. 14:15	Jandó Barnabás
6	Szkeleton elkészítése	márc. 30. 14:15	Tóth Márton
7-8	Prototípus koncepciója	ápr. 13. 14:15	Dénes Péter István
9	Részletes tervek	ápr. 20. 14:15	Bocskai Zsófia
10-11	Prototípus elkészítése	máj. 4. 14:15	Czap László
12	Grafikus változat tervei	máj. 11. 14:15	Jandó Barnabás
13-14	Grafikus változat elkészítése	máj. 27. labor	Tóth Márton
15	Egyesített dokumentáció	máj. 29. 13.00	Dénes Péter István

2.7 Napló

Kezdet	Időtartam	Résztevők	Leírás
2026.02.25. 14:00	1 óra	Czap Bocskai Tóth Jandó Dénes	Értekezlet. Döntés: A projekthez használt platformok a Github, Drive és Jira. A Projekt terv részeként definiált heti felelősök elosztásra kerülnek. Bocskai elkészíti a Use case diagramokat és a Szótárat. Czap az áttekintést, amiből Jandó készíti el a funkciókat és a funkcionális követelményeket. Tóth a további követelményeket. Dénes létrehozza a platformokat a Projekt tervet és a dokumentum maradék részeit írja meg.
2026.02.25.	1,5 óra	Dénes	Tevékenység: Dénes létrehozza a megfelelő platformokat és hozzáad minden csapattagot, továbbá elkezd a Követelmény dokumentum bevezető szövegeit megírni
2026.02.25	3 óra	Laci	Tevékenység: Általános áttekintés, 2.2.3-2.2.5-ös pontok kidolgozása, funkció elkezdése
2026.02.27	2,5 óra	Jandó	Tevékenység: Funkciók kiegészítése, 2.3.1 Funkcionális

			követelmények kidolgozása
2026. 02. 27	3 óra	Bocskai	Use-case leírások, use-case diagram, szótár
2026.02.26.	1,5 óra	Tóth	Tevékenység: 2.3.2, 2.3.3 és 2.3.4-es pontok kidolgozása
2026.03.01.	1,5 óra	Dénes	Tevékenység: Teljes dokumentum átolvasása, esetleges ütközések felderítése. Projekt terv megírása és a dokumentum végső formázása.
2026.03.01.	1 óra	Czap Bocskai Tóth Jandó Dénes	Értekezlet: Döntés: Az egyéni munka elfogadása.

3. Analízis modell I.

23 – githappens

Konzulens:

Haragos Gergő Viktor

Csapattagok

Czap László	NS7V2T	czap.laszlo@edu.bme.hu
Bocskai Zsófia	UGSTW9	bocskaizsofi@gmail.com
Tóth Márton	K01YXA	toth.marton21@gmail.com
Dénes Péter István	PO6MVA	denespeter03tab@gmail.com
Jandó Barnabás Tamás	AWQ77G	jandobarnabas@gmail.com

2026.03.09.

3. Analízis modell kidolgozása

3.1 Objektum katalógus

3.1.1 Game (Játék)

A szimuláció "dirigense". Ő indítja el és fejezi be a játékot, minden körben sorban megszólítja az összes többi objektumot – lépteti az időjárást, a járműveket, ellenőrzi a végfeltételt. Nála van nyilvántartva minden jármű és játékos.

3.1.2 Weather (Időjárás)

Minden körben "havazik": végigmegy az úthálózat összes felszíni sávján és növeli a hóvastagságot. Ő az egyetlen, aki tudja, hogy épp milyen intenzíven esik a hó.

3.1.3 Config (Konfiguráció)

A játék indítása előtt kitöltött "beállítólap". Tárolja, hogy hány körös legyen a játék és hány járművel induljon. Csak olvasható, a játék futása közben nem változik.

3.1.4 Scoreboard (Eredménytábla)

A játék végén "összeszámolja a pontokat". Összegyűjti a takarítók végső vagyonát és a buszvezetők teljesített fordulóit, majd ezekből előállít egy végeredményt.

3.1.5 Store (Bolt)

A takarítók "webshopja". Ellenőrzi, hogy a játékosnak van-e elég pénze, majd leveszi az összeget és "kiszállítja" a megvásárolt kotrófejet, nyersanyagot vagy új hókotrókat.

3.1.6 RoadNetwork (Úthálózat)

A város térképe és egyben az útvonalkereső motor. Nyilvántartja az összes csomópontot és útszakaszt, és ha egy jármű célba akar érni, tőle kérdezi meg a legrövidebb járható utat.

3.1.7 Road (Útszakasz)

Két csomópontot összekötő fizikai objektum, amely maga köré gyűjti az azonos irányú sávjait. Tudja, hogy felszíni út, híd vagy alagút-e, és segít megtalálni, hogy melyik sáv szomszédja melyiknek.

3.1.8 Route (Útvonal)

Egy konkrét útvonalterv, amelyet az útvonalkereső algoritmus állít elő. Sávok rendezett listáját tárolja, és menet közben "adagolja" a következő sávot a járműnek.

3.1.9 Lane (Sáv)

A gráf éle és egyben a hó-, jég- és baleseti állapot gazdája. Ő tudja, hogy rajta most mennyi hó és jég van, van-e baleset, és ebből kiszámolja, hogy egyáltalán járható-e, illetve milyen "nehéz" ráhajtani.

3.1.10 Node (Csomópont)

A gráf csúcsa, egy pont a városban ahol utak találkoznak. Önmagában csak azt tárolja, hogy hol van és mi a szerepe (kereszteződés, lakóhely, munkahely vagy végállomás).

3.1.11 Vehicles (Járművek)

Minden mozgó entitás közös őse. Tudja, hogy épp melyik sávon áll, hol tart a sávon belül, és minden körben megpróbál egy lépést tenni a célja felé. Ha akadályba ütközik, sávot vált vagy megáll.

3.1.12 Head (Kotrófej)

A hókotró "munkaeszköze". Minden fejféleség másképp kezeli a havat és a jeget: tolja, szórja, töri vagy olvasztja. Fogyaszt esetleg nyersanyagot a gazdájától, cserébe megtisztítja a sávot.

3.1.13 Player (Játékos)

Egy valódi emberi résztvevőt képvisel. Önmagában keveset tud – a tényleges cselekvések a szerepkörein keresztül valósulnak meg. Egy játékos egyszerre több szerepkört is betölthet.

3.1.14 Role (Szerepkör)

A játékos "munkaköri leírása". A takarítói szerepkör pénzt kezel és hókotrókat irányít, a buszvezető szerepkör útvonalat tervez és buszokat mozgat. Ugyanaz a játékos mindkét szerepkört felveheti egyszerre.

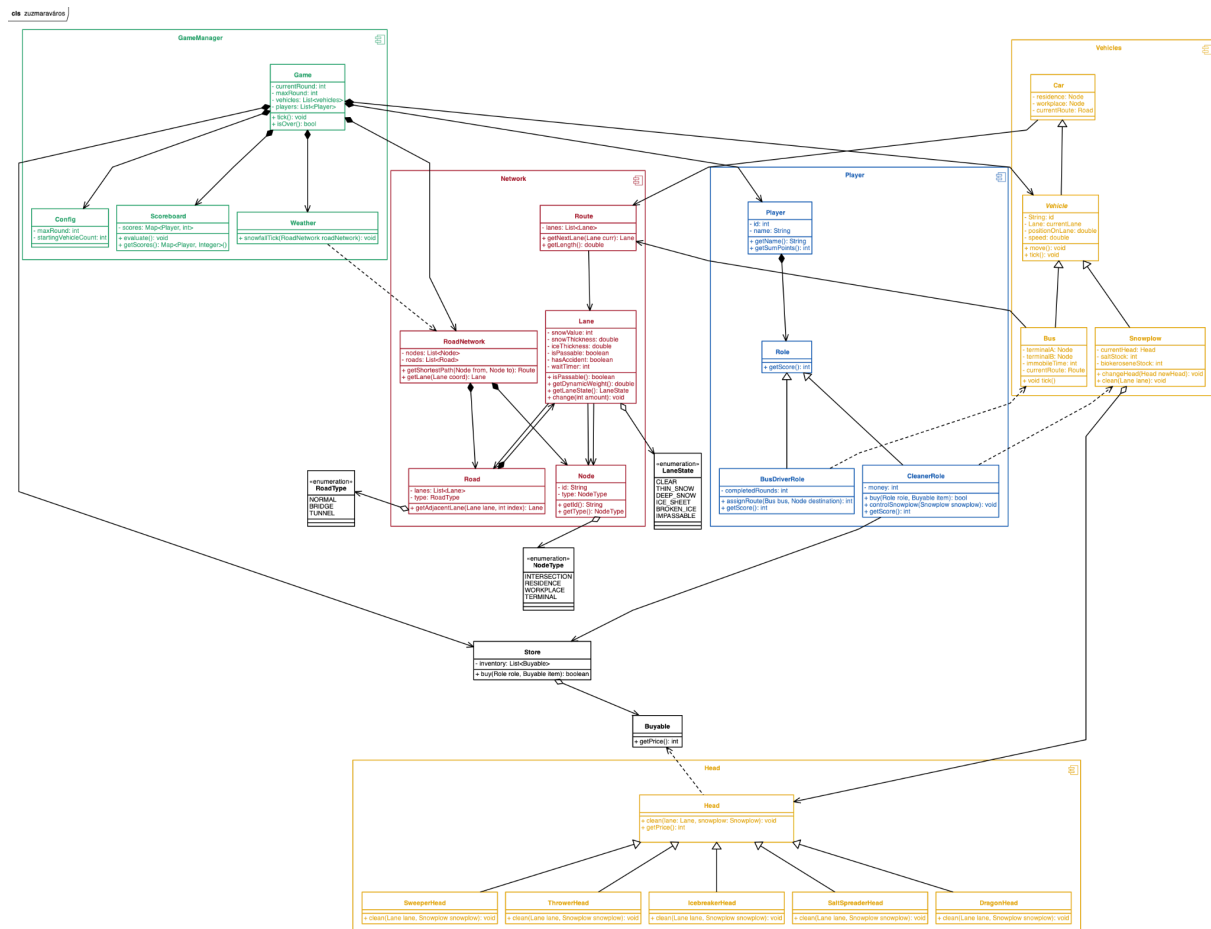
3.1.15 BusDriverRole (BuszvezetőSzerepkör)

A buszvezető felelős a buszok mozgásáért, útvonalak megtervezéséért és a fordulók teljesítéséért két végállomás között.

3.1.16 CleanerRole (TakarítóSzerepkör)

A takarító felelős a hó eltávolításáért, a hókotrók irányításáért és a hókotrókat érintő gazdasági döntésekért.

3.2 Statikus struktúra diagramok



3.3 Osztályok leírása

3.3.1 Bus

- Felelősség

A busz egy játékos által irányított tömegközlekedési jármű, amelynek célja meghatározott végállomások között sikeres fordulók teljesítése. Felelős a saját pozíciójának kezeléséért és az útvonalterv követéséért. Baleset esetén ideiglenesen mozgásképtelenné válik.

- Ősosztályok

Vehicle (Jármű)

- Interfészek

Nem valósít meg interfészt.

- Asszociációk

- **Route:** Az aktuálisan követett útvonalterv, amely meghatározza a haladási irányt

- **Attribútumok**
 - **terminalA: Node:** Az egyik végállomás
 - **terminalB: Node:** A másik végállomás
 - **immobileTime: int:** A baleset vagy elakadás miatti kényszerű várakozási idő
 - **currentRoute: Route:** Az aktuálisan követett útvonal
- **Metódusok**
 - **void tick():** Végrehajtja a busz mozgását az aktuális körben a sáv állapota alapján

3.3.2 BusDriverRole

- **Felelősség**

A BuszvezetőSzerep a Role leszármazottja. A buszvezető felelős a buszok mozgatásáért, útvonalak megtervezéséért és a fordulók teljesítéséért két végállomás között. A szerepkör pontszáma a sikeresen teljesített fordulókból adódik.

- **Össosztályok**

Role

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

- **BusDriverRole - Bus:** Az asszociáció túloldalán a Bus osztály áll. A kapcsolat célja, hogy a buszvezető irányíthassa a buszokat és teljesíthesse vele a fordulókat.

- **Attribútumok**

- **completedRounds: int** A buszvezető által teljesített fordulók száma.

- **Metódusok**

- **int assignRoute(Bus bus, Node destination):** A busz aktuális állapotából a cél csomópontba megtalálja a legrövidebb útvonalat.
- **int getScore():** A Role metódusának felüldefiniált metódusa, megadja a buszvezető szerepkör által szerzett pontszámot.

3.3.3 Buyable

- **Felelősség**

Egy interfész, amely minden megvásárolható elem számára közös viselkedést ír elő. Biztosítja, hogy minden elem rendelkezzen lekérdezhető vételárral a gazdasági műveletekhez.

- **Össosztályok**

Nincs őssosztálya.

- **Interfészek**

Ez maga egy interfész.

- **Asszociációk**

- **Store:** A megvásárolható elemeket a bolt kezeli és értékesíti.

- **Attribútumok**

Nincs attribútuma.

- **Metódusok**

- **int getPrice():** Visszaadja az adott megvásárolható elem aktuális árát.

3.3.4 Car

- **Felelősség**

A szimuláció civil járműveit képviseli, amelyek a lakóhelyük és munkahelyük között közlekednek a legrövidebb járható úton. Mozgásukkal tapossák a havat, ami jégpáncél kialakulásához vezethet

- **Össosztályok**

Vehicle (Jármű)

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

- **Route:** Az aktuálisan követett útvonalterv, amely meghatározza a haladási irányt

- **Attribútumok**

- **residence:** Kiindulási pont/lakóhely
- **workplace:** Célállomás/munkahely
- **currentRoute:** A Route objektum, amelyet az autó követ

- **Metódusok**

Nincs saját metódusa

3.3.5 CleanerRole

- **Felelősség**

A CleanerRole (TakarítóSzerepkör) a Role leszármazottja. A takarító felelős a hó eltávolításáért, a hókotrók irányításáért és a hókotrókat érintő gazdasági döntésekért. A szerepkör kezeli a játékos pénzét, vásárolhat a Boltban, és működteti a hókotrók fejeit.

- **Össosztályok**

Role

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

- **CleanerRole - Snowplow (Hókotró):** Az asszociáció túloldalán a Hókotró osztály áll. A kapcsolat célja, hogy a takarító irányíthassa a hókotrókat és takarítja vele az utakat.
- **CleanerRole – Store (Bolt):** A kapcsolat célja, hogy a takarító vásárolhasson nyersanyagot, új fejeket vagy hókotrókat.

- **Attribútumok**

- **money: int** A takarító jelenlegi vagyona.

- **Metódusok**

- **boolean buy(Role role, Buyable item):** A takarító a boltból a kotrófejek működéséhez szükséges üzemanyagokat, új kotrófejeket és hókotrókat vásárolhat.
- **void controlSnowplow(Snowplow snowplow):** A takarító irányítja a hókotrókat.
- **int getScore():** A Role metódusának felüldefiniált metódusa, megadja a takarító által szerzett pontszámot.

3.3.6 Config

- **Felelősség**

Ez az osztály tárolja a játék indulásához szükséges statikus paramétereket és alapbeállításokat. Segítségével konfigurálható a szimuláció hossza és a kezdeti járműállomány mérete.

- **Össosztályok**

Nincs össosztálya.

- **Interfészek**

Nincs interfész megvalósítva.

- **Asszociációk**

- **Game:** Inicializálási adatokat szolgáltat a központi vezérlőnek.

- **Attribútumok**

- **maxRound: int** A játék során elérhető maximális körszám.
- **startingVehicleCount: int** Meghatározza, hány járművel kezdődjön el a szimuláció.

- **Metódusok**

Nincs metódusa.

3.3.7 DragonHead

- **Felelősség**

Speciális kotrófej, amely gázturbinával azonnal elolvasztja a havat és a jeget a jármű előtt. Működéséhez biokerozint használ

- **Ősosztályok**

Head

- **Interfészek**

Buyable

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **void clean(Lane lane, Snowplow snowplow):** Ellenőrzi a hókotró biokerozin készletét, majd annak felhasználásával a sáv állapotát azonnal CLEAR értékre állítja

3.3.8 Game

- **Felelősség**

A Game osztály a szimuláció központi vezérlője, amely koordinálja a teljes játékmenetet, kezeli a körök számát, valamint nyilvántartja a résztvevő játékosokat és járműveket. Felelős a játéklógika léptetéséért és a befejezési feltételeket ellenőrzi.

- **Ősosztályok**

Nincs ősosztálya.

- **Interfészek**

Nincs megvalósított interfész.

- **Asszociációk**

- **Weather:** A játék és a környezeti hatások közötti kapcsolatot definiálja.
- **Config:** A játék inicializáláshoz szükséges beállításokat biztosítja.
- **Scoreboard:** A játék eredményeinek követésére szolgáló komponens.
- **Vehicle:** A szimulációban részt vevő járművek listája.
- **Player:** A játékban regisztrált játékosok gyűjteménye.

- **Attribútumok**

- **currentRound: int** Az aktuális szimulációs kör sorszámát tárolja.

- **maxRound: int** A játék maximális időtartamát határozza meg körökben.
 - **vehicles: List<Vehicle>** A rendszerben lévő járművek listája.
 - **players: List<Player>** A játékban résztvevő játékosok listája.
- **Metódusok**
 - **void tick():** Egy egységgel előre lépteti a játék állapotát és frissíti a belső logikát.
 - **boolean isOver():** Megvizsgálja, hogy a szimuláció elérte-e a maximális körszámot vagy véget ért-e.

3.3.9 Head

- **Felelősség**

A hókotrók munkaeszközeinek alaposztálya. Meghatározza a tisztítási folyamat interfészét és kezeli a nyersanyag-felhasználást

- **Ősosztályok**

Nincs

- **Interfészek**

Buyable (Megvásárolható)

- **Asszociációk**
 - **Snowplow:** A hókotró, amelyre a fej fel van szerelve

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**
 - **void clean(lane: Lane, snowplow: Snowplow):** Absztrakt művelet a sáv tisztítására, figyelembe véve a hókotró készleteit
 - **int getPrice():** Az interfészből származó metódus, visszaadja a fej árát

3.3.10 IcebreakerHead

- **Felelősség**

Speciális fej a jégpáncél fizikai feltörésére. A havat nem takarítja el, csak a jeget teszi takaríthatóvá a söprő vagy hányó fejek számára

- **Ősosztályok**

Head

- **Interfészek**

Buyable

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **void clean(Lane lane, Snowplow snowplow):** A sáv ICE_SHEET állapotát BROKEN_ICE állapotra módosítja

3.3.11 Lane

- **Felelősség**

A Lane osztály az úthálózat gráfjának egy irányított élet reprezentálja, amelyen a járművek közlekednek. Ez az osztály a "gazdája" az útszakasz fizikai állapotának: nyilvántartja a lehullott hó mennyiségét, a kialakult jégpáncél vastagságát és az esetleges balesetek tényét. Ezen adatok alapján felelős annak kiszámításáért, hogy a sáv az adott pillanatban járható-e, valamint meghatározza a sáv "dinamikus súlyát", amely jelzi az áthaladás nehézségét az útvonaltervező algoritmus számára.

- **Össztályok**

- **Interfészek**

- **Asszociációk**

source: Node – A sáv kiinduló csomópontja.

destination: Node – A sáv célállomása.

parentRoad: Road – Az az út objektum, amelyhez ez a sáv tartozik.

- **Attribútumok**

- **snowValue:** int – A sávon felhalmozódott hó mennyiségi mutatója.
- **snowThickness:** double – A hóréteg aktuális fizikai vastagsága.
- **iceThickness:** double – Az úton lévő jégpáncél vastagsága.
- **isPassable:** boolean – Logikai érték, amely megadja, hogy a sáv jelenleg járható-e a forgalom számára.
- **hasAccident:** boolean – Jelzi, ha a sávon baleset történt, ami akadályozza a haladást.
- **waitTimer:** int – Baleset vagy elakadás esetén a kényszerű várakozási időt mérő számláló.

- **Metódusok**

- **boolean isPassable():** Kiértékeli a sáv aktuális állapotát (hó, jég, baleset), és visszaadja, hogy a járművek áthajthatnak-e rajta.

- **double getDynamicWeight():** Kiszámítja a sáv súlyozását az útvonalkereséshez; a mélyebb hó vagy jegesedés növeli az értéket, így a járművek preferálják a tisztább szakaszokat.
- **LaneState getLaneState():** Visszaadja a sáv pillanatnyi minősítését egy felsorolási típus formájában (pl. CLEAR, THIN_SNOW, DEEP_SNOW, ICE_SHEET, IMPASSABLE).
- **void change(int amount):** Módosítja a hó vagy jég mennyiségét a sávon havazás vagy hókotrás hatására.

3.3.12 LaneState

· Felelősség

A LaneState felsorolás a sáv aktuális fizikai állapotát írja le. A különböző értékek a hó, jég és akadályok mértékét reprezentálják, amelyeket a járművek és a hókotrók működése során figyelembe kell venni. A felsorolás segíti a sáv járhatóságának és tisztítási szükségleteinek meghatározását.

· Össosztályok

Nincs osztály.

· Interfészek

Nem valósít meg interfészt.

· Asszociációk

Lane osztály attribútumként hivatkozik rá, hogy a típust reprezentálja.

· Attribútumok

Nincs attribútuma.

· Értékek:

- **CLEAR** - A sáv tiszta, akadálymentes.
- **THIN_SNOW** - Vékony hóréteg borítja.
- **DEEP_SNOW** - Vastag hóréteg borítja, nem lehet haladni.
- **ICE_SHEET** - A sáv jeges, csúszós.
- **BROKEN_ICE** - A sávon a jégtörő fej feltörte a havat, ezt el kell még söpörni.
- **IMPASSABLE** - A sáv járhatatlan (baleset miatt).

· Metódusok

Nincs metódusa.

3.3.13 Node

· Felelősség

A Node osztály az úthálózatot alkotó gráf csúcspontjait reprezentálja, ahol a város különböző útszakaszai találkoznak. Feladata a földrajzi vagy logikai pont azonosítása és annak típusának meghatározása. Tárolja, hogy az adott pont milyen szerepet tölt be a szimulációban: egyszerű kereszteződés, a civil autók kiinduló- vagy célállomása (lakóhely, munkahely), vagy a buszok számára kijelölt végállomás.

- **Összostályok**
- **Interfészek**
- **Asszociációk**
- **Attribútumok**
 - **id**: String – A csomópont egyedi azonosítója vagy neve.
 - **type**: NodeType – A csomópont típusát meghatározó felsorolásos érték (INTERSECTION, RESIDENCE, WORKPLACE, TERMINAL).
- **Metódusok**
 - **String getId()**: Visszaadja a csomópont egyedi azonosítóját.
 - **NodeType getType()**: Visszaadja a csomópont típusát, segítve az útvonaltervezőt a célállomások (pl. munkahelyek) azonosításában.

3.3.14 NodeType

- **Felelősség**

A NodeType felsorolás a csomópontok szerepét írja le az úthálózatban. Segít megkülönböztetni a különböző típusú helyszíneket, amelyeket az útvonaltervezés és a járművek célmeghatározása során használnak.

- **Összostályok**

Nincs összostálya.

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

Node osztály attribútumként hivatkozik rá, hogy a csomópont típusát reprezentálja.

- **Attribútumok**

Nincs attribútuma.

- **Értékek**
 - **INTERSECTION** – Közönséges kereszteződés.
 - **RESIDENCE** – Lakóhely, civil járművek kiinduló/célpontja.
 - **WORKPLACE** – Munkahely, civil járművek célpontja.
 - **TERMINAL** – Buszok végállomása.

- **Metódusok**

Nincs metódusa.

3.3.15 Player

- **Felelősség**

A Player egy valódi emberi résztvevőt reprezentál a rendszerben. A játékos a játék emberi irányítója, de önmagában kevés közvetlen műveletet végez, a tényleges interakciók a hozzá tartozó Role segítségével valósulnak meg. Egy játékos egyszerre több szerepkört is betölthet, és ezek a szerepkörök határozzák meg, hogy milyen járműveket irányíthat, milyen döntéseket hozhat, és hogyan szerez pontokat a játék során. A játékos stratégiai döntéseket hoz, amik hatással vannak a Zúzmaraváros állapotára, illetve a saját pontszámára.

- **Ősosztályok**

Nincs ősosztály.

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

- **Player - Role:** A kompozíció túloldalán a Role osztály áll. A kapcsolat célja, hogy a játékos a szerepkörein keresztül tudjon cselekedni a játékban. Egy játékos több szerepkört is tartalmazhat, és ezek határozzák meg, milyen járműveket irányíthat és milyen döntéseket hozhat.
- **Player - Game:** Az kompozíció túloldalán a Game osztály áll. A kapcsolat célja, hogy a Game nyilvántartsa az összes játékost. A Player résztvevője a Game-nek.

- **Attribútumok**

- **id: int** A játékos egyedi azonosítója.
- **name: String** A játékos neve.

- **Metódusok**

- **String getName():** A játékos nevét adja meg
- **int getSumPoints():** Összegzi a szerepkörök által szerzett pontokat.

3.3.16 Road

- **Felelősség**

A Road osztály két csomópontot összekötő fizikai útszakaszt reprezentál a játékteren. Feladata az egy irányba haladó sávok összefogása és koordinálása. Az osztály tárolja az út típusát (normál felszíni út, híd vagy alagút), amely meghatározza az adott szakaszra vonatkozó környezeti szabályokat, például azt, hogy az alagutakban nem eshet a hó. Emellett támogatja a járművek navigációját azáltal, hogy segít meghatározni a sávok közötti szomszédsági viszonyokat.

- **Össosztályok**
- **Interfészek**
- **Asszociációk**
 - **lanes:** List<Lane> – Az adott útszakaszhoz tartozó sávok listája.
 - **source:** Node – Az útszakasz kiinduló csomópontja.
 - **destination:** Node – Az útszakasz célállomását jelentő csomópont.
- **Attribútumok**
 - **lanes:** List<Lane> Az útszakaszt felépítő sávok gyűjteménye.
 - **type:** RoadType Az út típusát (NORMAL, BRIDGE, TUNNEL) meghatározó felsorolásos típus.
- **Metódusok**
 - **Lane getAdjacentLane(Lane lane, int index):** Megkeresi és visszaadja a paraméterként kapott sáv melletti szomszédos sávot a megadott index (irány) alapján. Ez a metódus teszi lehetővé a járművek számára a sávváltást akadályok vagy balesetek esetén.

3.3.17 RoadNetwork

- **Felelősség**

A RoadNetwork osztály felelős Zúzmaraváros teljes úthálózatának reprezentálásáért és kezeléséért. Ez az osztály működik a játék útvonalkereső motorjaként: nyilvántartja a város összes csomópontját és útszakaszát, valamint kiszámítja a járművek számára a céljukhoz vezető legrövidebb, aktuálisan járható utat. Figyelembe veszi az utak típusát (híd, alagút, felszíni út) és azok aktuális állapotát a tervezés során.

- **Össosztályok**
- **Interfészek**
- **Asszociációk**
 - **nodes:** List<Node> Az úthálózatot felépítő összes csomópont listája.
 - **roads:** List<Road> Az úthálózatot felépítő összes útszakasz (élek) listája.
- **Attribútumok**
 - **nodes:** List<Node> A városban található összes kereszteződés, lakóhely és munkahely gyűjteménye.
 - **roads:** List<Road> A csomópontokat összekötő útszakaszok, hidak és alagutak listája.
- **Metódusok**
 - **Route getShortestPath(Node from, Node to):** Meghatározza a két megadott csomópont közötti legrövidebb, akadálymentes útvonalat. A számítás során

figyelembe veszi a sávok járhatóságát (pl. elkerüli a mély havat vagy balesetet, ha van alternatíva).

- **Lane getLane(Lane coord):** Visszaadja a paraméterként megadott koordinátákhoz vagy azonosítóhoz tartozó konkrét sáv objektumot.

3.3.18 RoadType

- **Felelősség**

A RoadType felsorolás az útszakaszok típusát írja le. A különböző értékek meghatározzák, hogy az adott útszakaszra milyen környezeti szabályok vonatkoznak (pl. alagútban nem eshet a hó), és befolyásolják a járművek mozgását és a tisztítási folyamatokat.

- **Össosztályok**

Nincs össosztálya.

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

Road osztály attribútumként hivatkozik rá, hogy az útszakasz típusát reprezentálja.

- **Attribútumok**

Nincs attribútuma.

- **Értékek**

- **NORMAL** - Hagyományos felszíni út.
- **BRIDGE** - Híd, speciális környezeti viselkedéssel.
- **TUNNEL** - Alagút, ahol nem eshet a hó.

- **Metódusok**

Nincs metódusa.

3.3.19 Role

- **Felelősség**

A Role (Szerepkör) absztrakt osztály felel a járművek irányításáért. A játékosok különböző szerepkörökön keresztül irányíthatják a hozzájuk tartozó járműveket, végezhetnek további járművekhez kapcsolódó műveleteket. Egy játékos több szerepkört is felvehet egyszerre.

- **Össosztályok**

Nincs össosztálya.

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

- **Roles - Player (Játékos):** A kompozíció túloldalán a Player (Játékos) osztály áll. A kapcsolat célja, hogy a szerepkörökön keresztül a játékos tudjon cselekedni a játékban. Egy játékos több szerepkört is tartalmazhat, és ezek határozzák meg, milyen járműveket irányíthat és milyen döntéseket hozhat.

- **Attribútumok**

Nincs attribútum.

- **Metódusok**

- **int getScore():** megadja az adott szerepkör által szerzett pontszámot

3.3.20 Route

- **Felelősség**

A Route osztály egy konkrét útvonaltervet reprezentál, amelyet az úthálózat útvonalkereső algoritmus állít elő a járművek számára. Feladata a célba jutáshoz szükséges sávok rendezett listájának tárolása. A szimuláció során ez az osztály felelős azért, hogy a jármű haladása közben kiszolgálja ("adagolja") a következő sávot, amelyre a járműnek rá kell hajtania.

- **Össz osztályok**

- **Interfészek**

- **Asszociációk**

- **lanes:** List<Lane> – Az útvonalat felépítő sávok rendezett sorozata.

- **Attribútumok**

- **lanes:** List<Lane> – A jármű által bejárando sávok listája.

- **Metódusok**

- **Lane getNextLane(Lane curr):** Megkeresi a paraméterként kapott aktuális sávot az útvonalban, és visszaadja a soron következő sáv objektumot. Ezzel irányítja a járművet az útvonal mentén.
- **double getLength():** Visszaadja az útvonal teljes hosszát, amely az útvonalat alkotó sávok hosszának összege.

3.3.21 SaltSpreaderHead

- **Felelősség**

Sót juttat az útfelületre, ami megakadályozza a hó megmaradását és idővel felolvasztja a havat és a jeget. Működéséhez só nyersanyagot igényel

- **Össosztályok**

Head

- **Interfészek**

Buyable

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **void clean(Lane lane, Snowplow snowplow):** Csökkenti a hókotró sókészletét és alkalmazza a sószórást a sávon, ami elindítja a fokozatos olvadást

3.3.22 Scoreboard

- **Felelősség**

Az osztály feladata a játékosok teljesítményének folyamatos nyomon követése és az eredmények összesítése. Kezeli a pontszámok tárolását és biztosítja azok lekérdezhetőségét.

- **Össosztályok**

Nincs össosztálya.

- **Interfészek**

Nincs megvalósított interfész.

- **Asszociációk**

- **Game:** A játék menedzsment rétegének részeként együttműködik a vezérlővel.

- **Attribútumok**

- **scores: Map<Player, Integer>** Egy leképezés (map), amely a játékosokhoz rendeli a megszerzett pontszámokat.

- **Metódusok**

- **void evaluate():** Elvégzi a pontszámok aktuális állapot szerinti kiértékelését vagy frissítését.

- **Map<Player, Integer> getScores():** Visszaadja a pillanatnyi állást tartalmazó ponttáblázatot.

3.3.23 Snowplow

- **Felelősség**

Speciális jármű, amely az úthálózat tisztításáért felel. A takarító játékos irányítása alatt mozog, és a felszerelt kotrófej segítségével módosítja a sávok állapotát

- **Ősosztályok**

Vehicle (Jármű)

- **Interfészek**

Buyable (Megvásárolható)

- **Asszociációk**
 - **head:** A hókotróra szerelt aktuális munkaeszköz
 - **Cleanerrole:** A járművet birtokló és irányító takarító szerepkör
- **Attribútumok**
 - **currentHead: Head:** A járműre szerelt aktuális fej
 - **saltStock: int:** A sószóró fejhez tárolt sómennyiség
 - **biokeroseneStock: int:** A sárkány fejhez tárolt üzemanyag
- **Metódusok**
 - **void changeHead(Head newHead):** Lecseréli az aktuális kotrófejet egy másik típusra
 - **void clean(Lane lane):** Meghívja a felszerelt kotrófej tisztító funkcióját az aktuális sávra

3.3.24 Store

- **Felelősség**

A gazdasági alrendszer eleme, amely lehetővé teszi a megvásárolható tárgyak beszerzését a játékosok számára. Összeköti az elérhető kínálatot a vásárlási tranzakciókkal.

- **Ősosztályok**

Nincs ősosztálya.

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

- **Buyable:** A bolt készletében lévő megvásárolható elemekre utal.
- **Role:** A vásárlást kezdeményező szerepkörökkel áll kapcsolatban.
- **Attribútumok**
- **inventory: List<Buyable>** A boltban pillanatnyilag elérhető termékek (Buyable) listája.
- **Metódusok**
- **boolean buy(Role role, Buyable item):** Lebonyolít egy vásárlást egy adott szerepkör számára, és visszatér a sikerességével.

3.3.25 SweeperHead

- **Felelősség**

Mechanikus munkaeszköz, amely a havat közvetlenül a hókotró nyomvonala mellé, a szomszédos sávba tolja. A lefagyott jeget nem tudja eltávolítani, csak a havat és a már feltört jeget

- **Ősosztályok**

Head

- **Interfészek**

Buyable

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **void clean(Lane lane, Snowplow snowplow):** Ha a sáv nem jeges, a havat/feltört jeget áthelyezi a szomszédos sávba, a jelenlegi sávot pedig CLEAR állapotúvá teszi

3.3.26 ThrowerHead

- **Felelősség**

A söprő fejhez hasonlóan működik, de a havat nem a közvetlen szomszédos sávba, hanem annál távolabbra szórja. Kezdeti felszerelésként is választható

- **Ősosztályok**

Head

- **Interfészek**

Buyable

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **void clean(Lane lane, Snowplow snowplow):** eltávolítja a havat és a feltört jeget a sávból úgy, hogy azokat távoli területre juttatja

3.3.27 Vehicles

- **Felelősség**

Minden mozgó entitás közös absztrakt őse. Kezeli az alapvető mozgási logikát, a sávokon való elhelyezkedést és a sávváltás lehetőségét akadály esetén

- **Ősosztályok**

Nincs

- **Interfészek**

Nem valósít meg interfészt

- **Asszociációk**

- **Lane:** Az a sáv, amelyen a jármű jelenleg tartózkodik

- **Attribútumok**

- **id: String:** A jármű egyedi azonosítója
- **currentLane: Lane:** Az aktuális sáv referenciája
- **positionOnLane: double:** A jármű aktuális pozíciója az adott sávon belül
- **speed: double:** A jármű haladási sebessége

- **Metódusok**

- **void move():** A jármű mozgását végző belső logika
- **void tick():** A körönkénti léptetésért felelős metódus

3.3.28 Weather

· Felelősség

Az időjárási viszonyok szimulálásáért felelős osztály, amely dinamikusan módosítja a környezeti állapotokat. Elsődleges feladata a hóesés hatásának érvényesítése az úthálózaton.

· Ősosztályok

Nincs ősosztálya.

· Interfészek

Nincs megvalósított interfész.

· Asszociációk

- **RoadNetwork:** Közvetlenül módosítja az úthálózat elemeit (sávokat) a hóesés során.

· Attribútumok

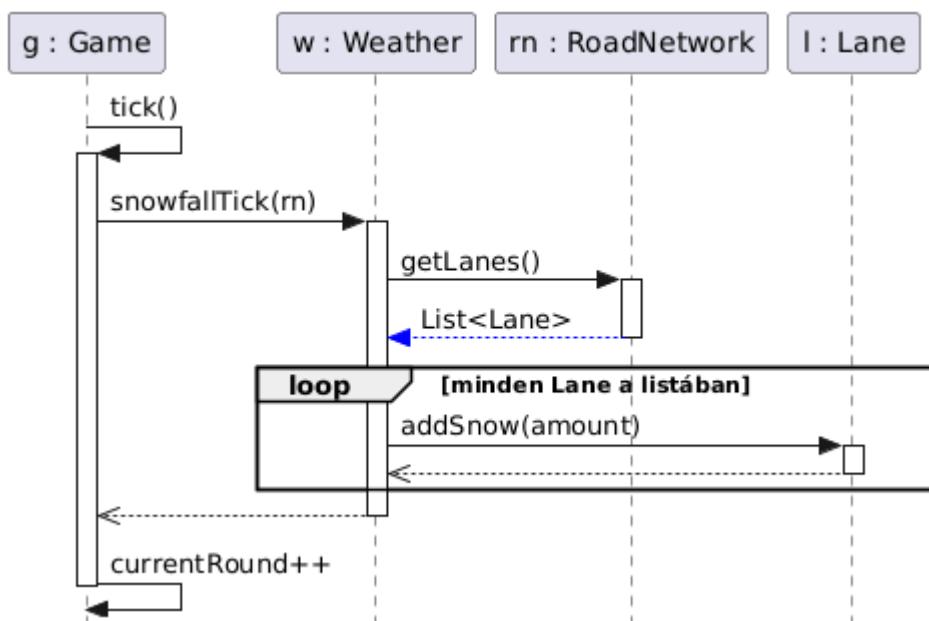
Nincs attribútuma.

· Metódusok

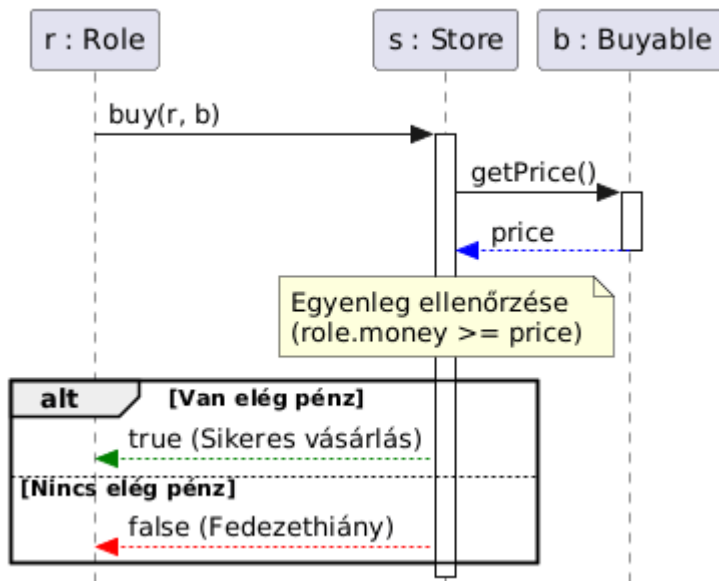
- **void snowfallTick(RoadNetwork roadNetwork):** Kiszámítja és alkalmazza a hóesés mértékét az úthálózat sávjaira az aktuális körben.

3.4 Szekvencia diagramok

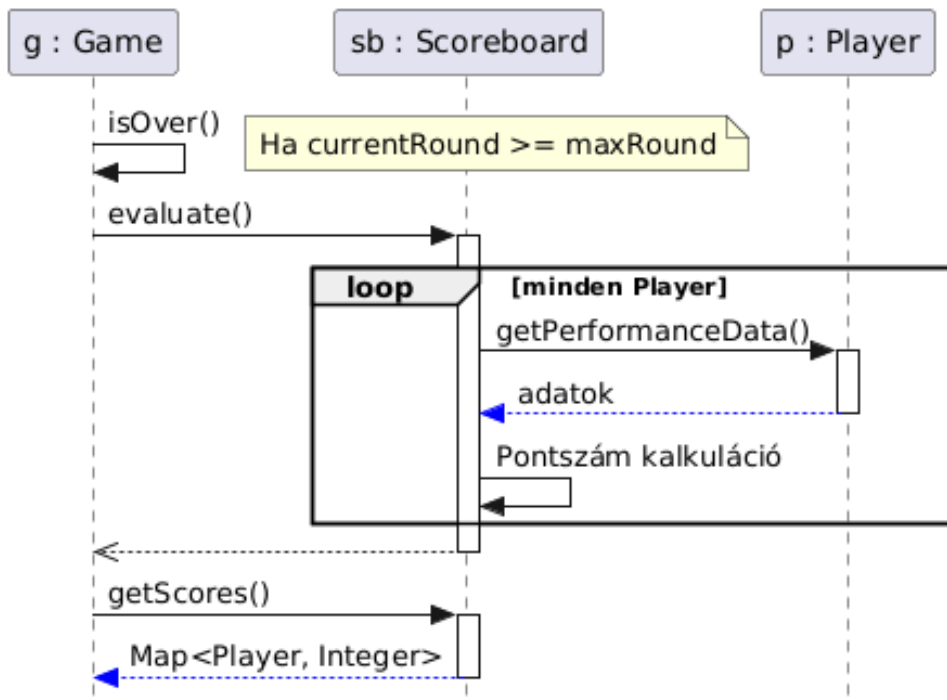
Körök és időjárás frissítése:



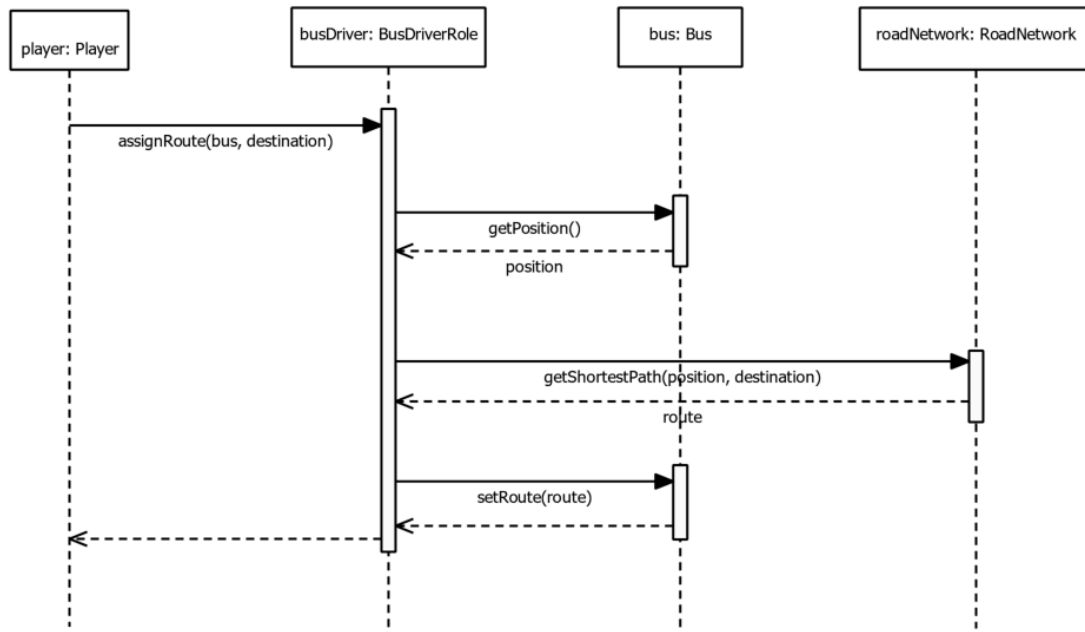
Vásárlási folyamat:



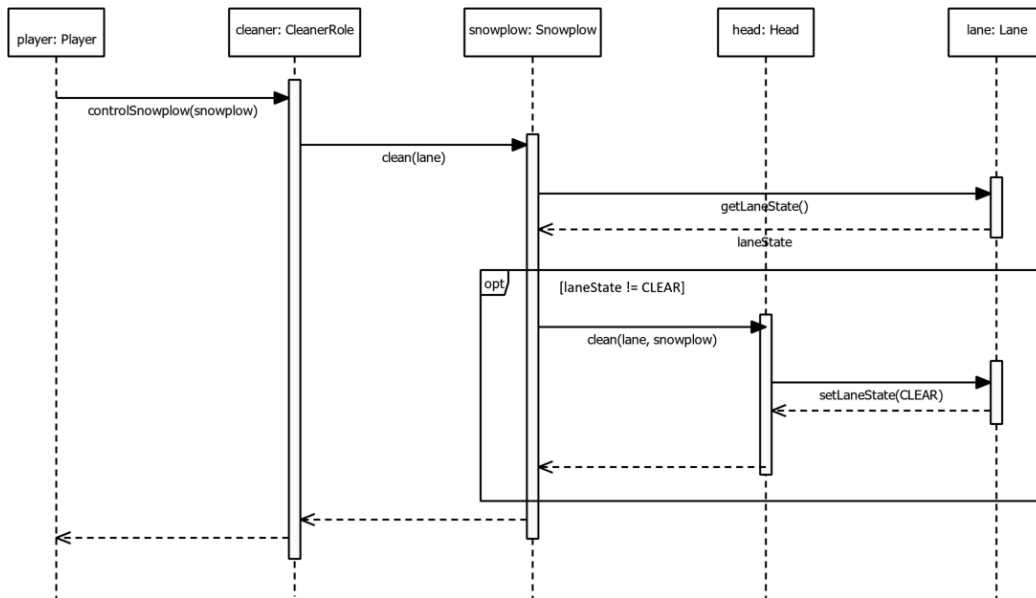
Játék vége és kiértékelés:



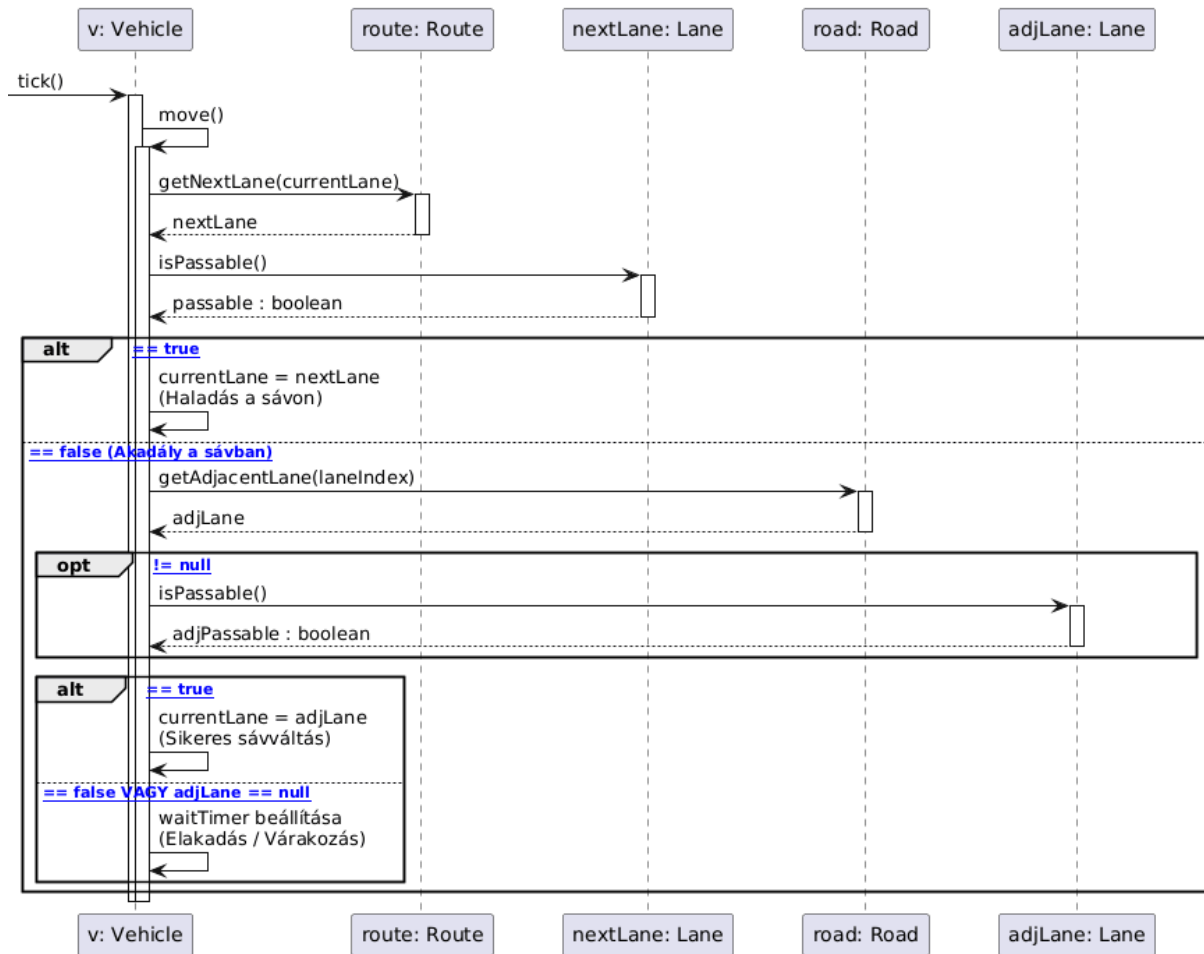
Busz útvonalának kijelölése:



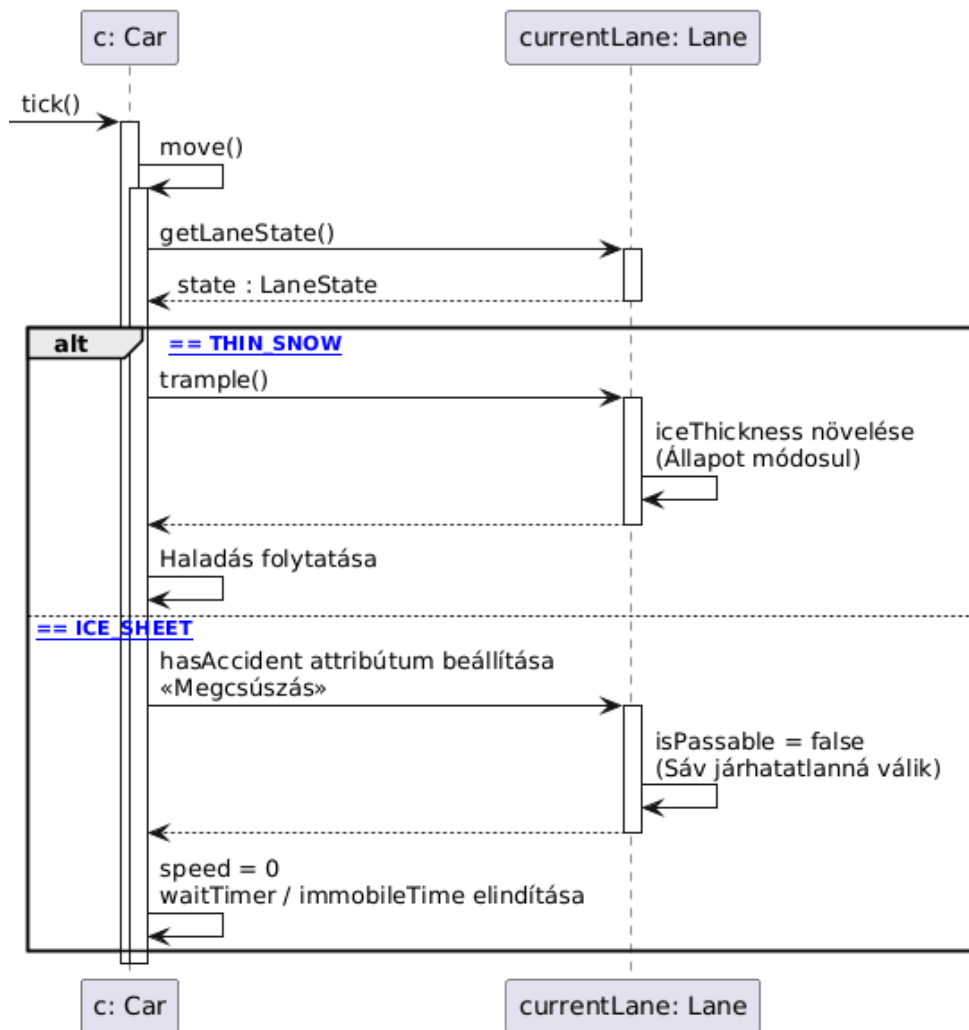
Takarítás hókotróval:



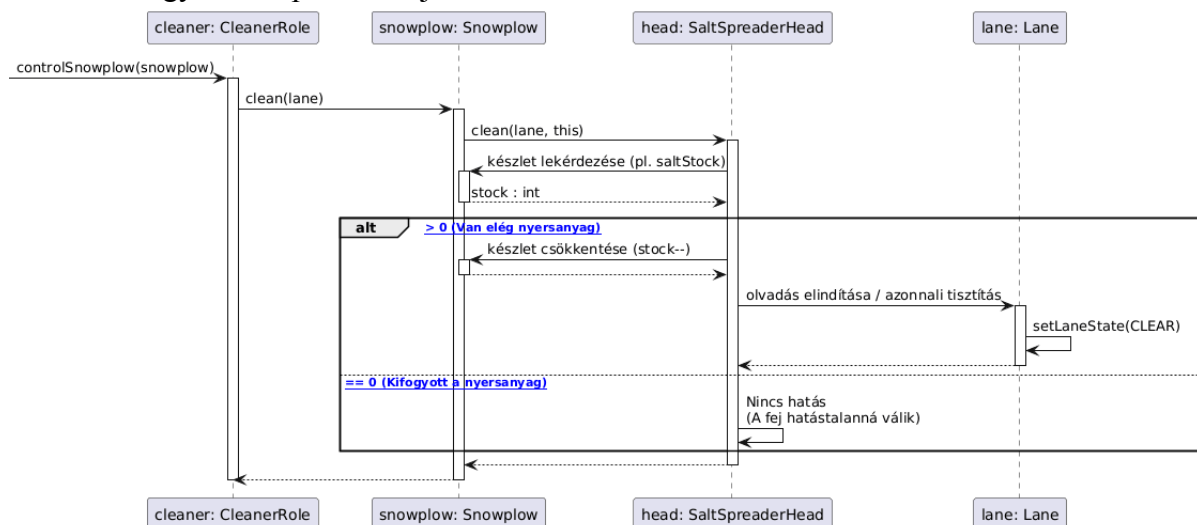
Járművek mozgása és sávváltása:



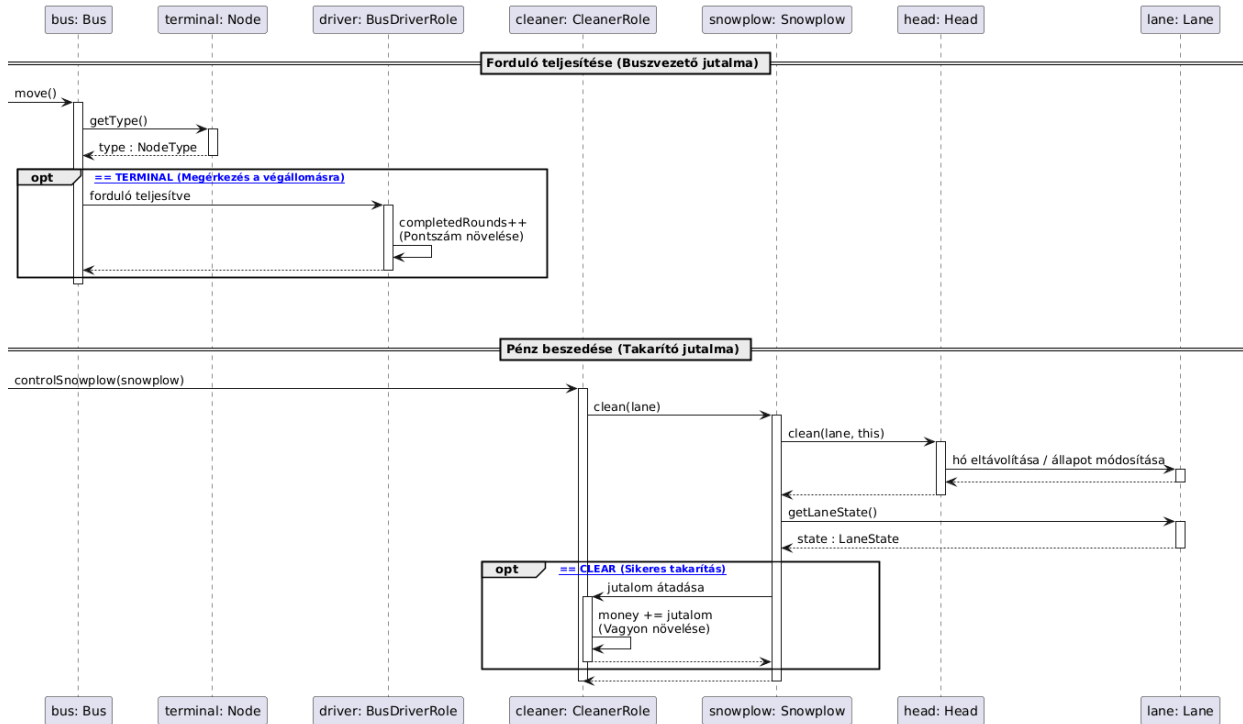
Hó taposása és baleset:



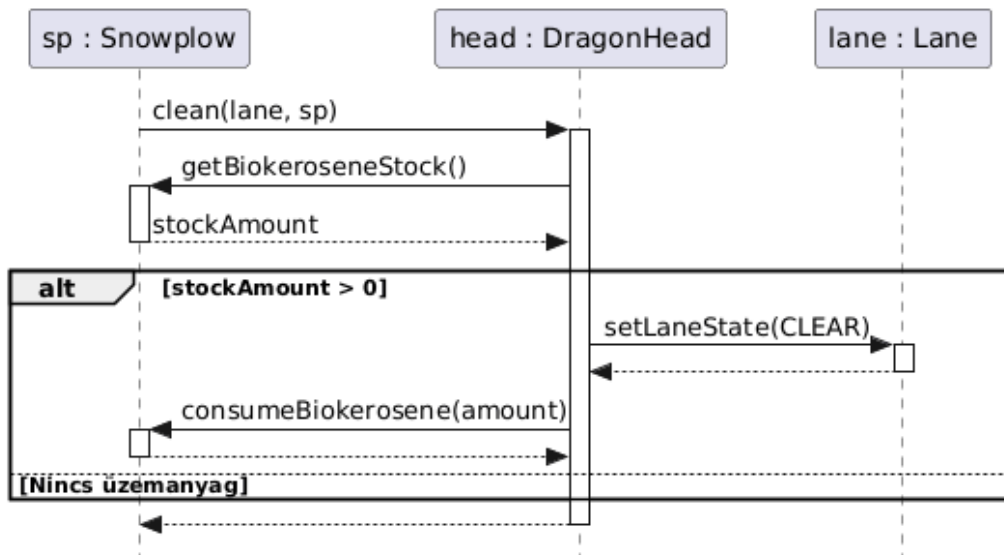
Erőforrás-fogyasztás speciális fejeknél:



Jutalmazási rendszer:

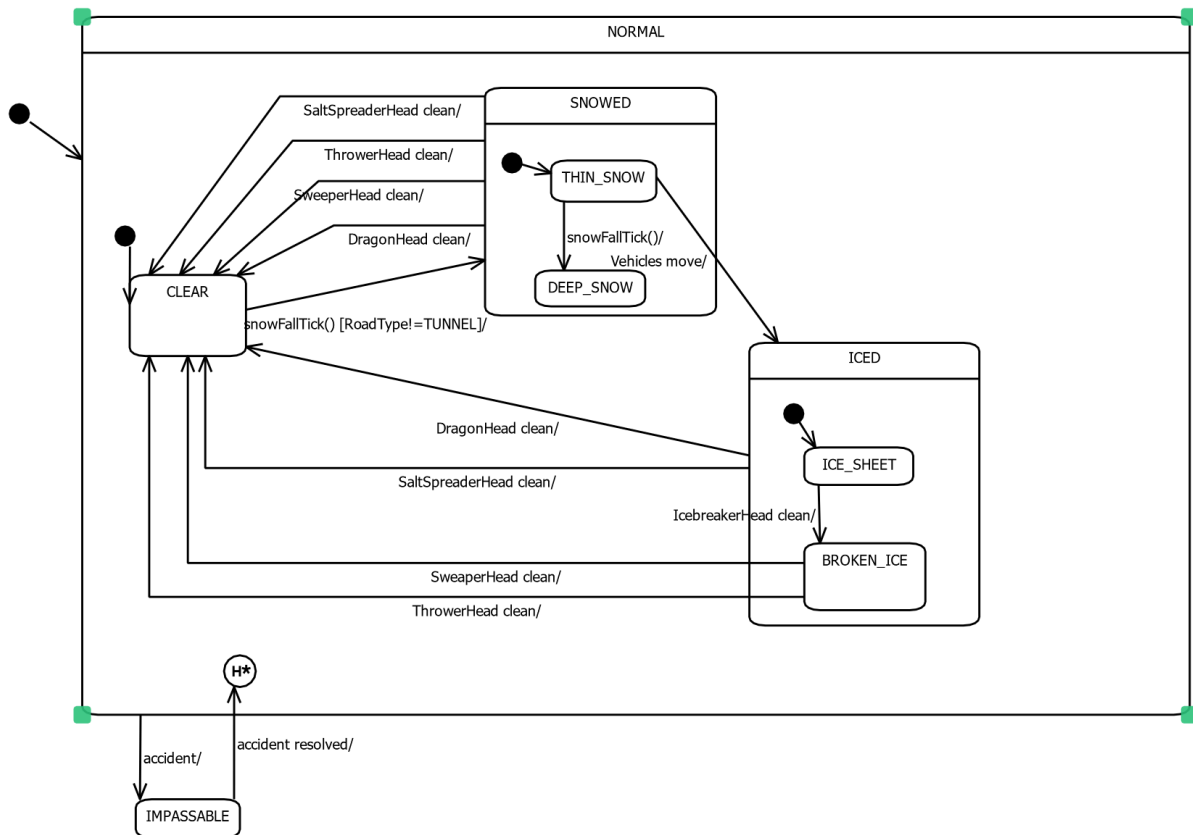


Nyersanyag-fogyasztás takarításkor:

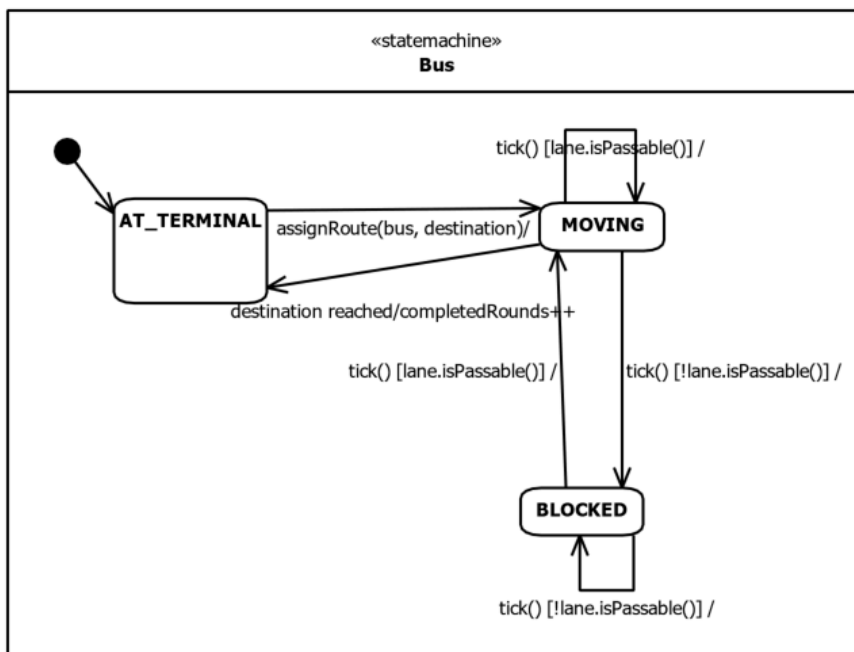


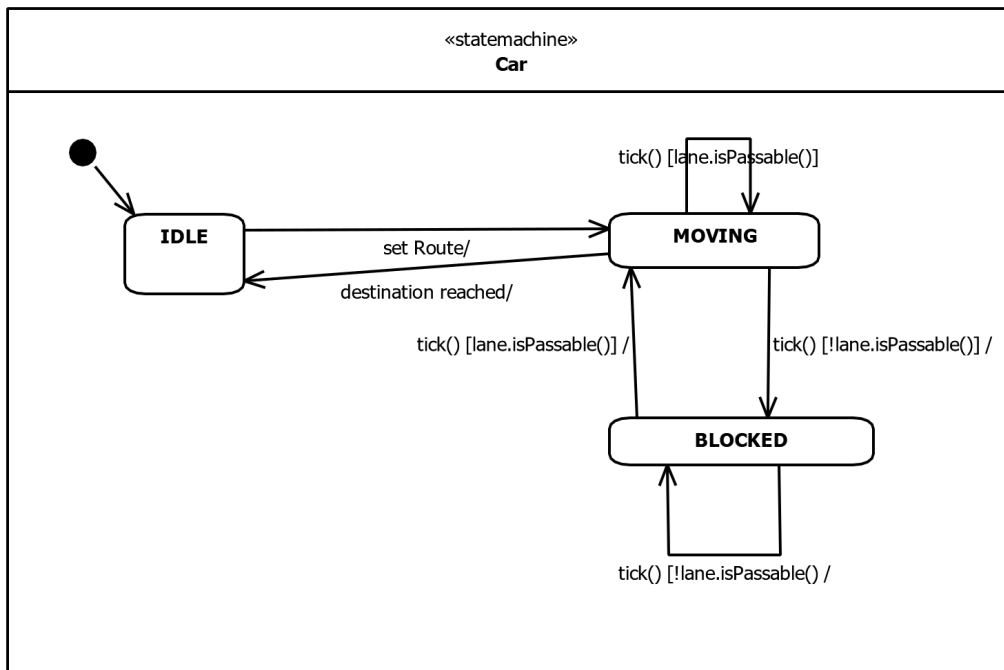
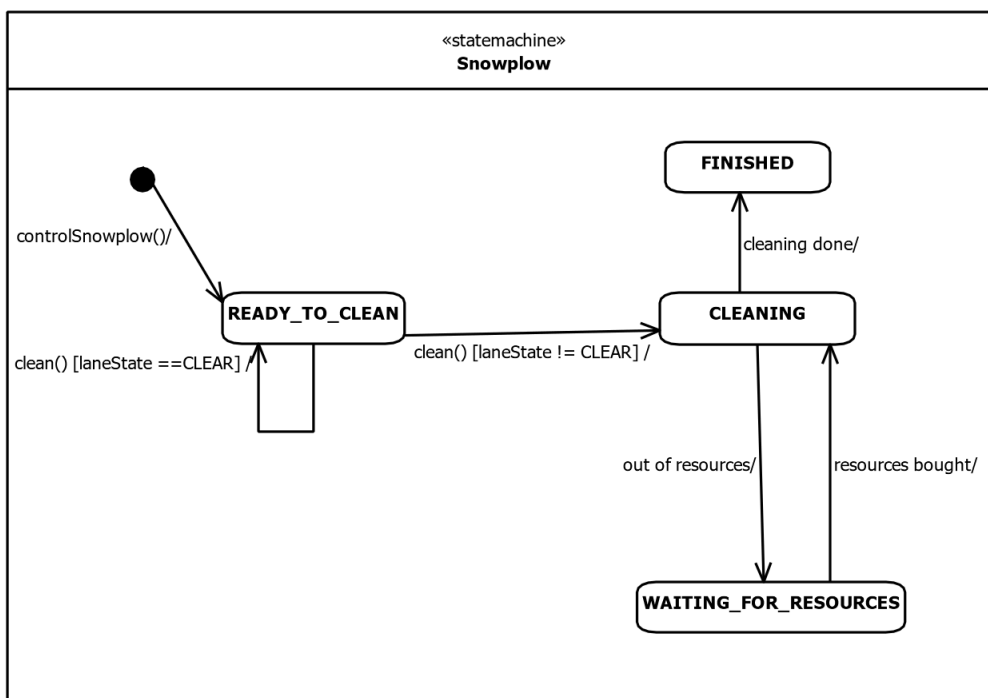
3.5 State-chartok

Út állapotát ábrázoló állapotgép:



Busz haladását ábrázoló állapotgép:



Autó haladását ábrázoló állapotgép:**Hókotó működését ábrázoló állapotgép:**

3.6 Napló

Kezdet	Időtartam	Résztevők	Leírás
2026. 03. 02. 20:00	1,5 óra	Czap Bocskai Tóth Jandó Dénes	Értekezlet az elkészítendő osztályokról
2026. 03. 04. 9:00	2 óra	Czap Bocskai Tóth Jandó Dénes	Értekezlet: heti tevékenységek kiosztása, osztályokról való elképzelések egyeztetése
2026. 03. 04.	3 óra	Dénes	Az értekezleten elhangzottak rendszerezése, osztály katalógus elkészítése
2026. 03. 05.	2 óra	Dénes	Osztálydiagram elkészítése
2026. 03. 06.	4 óra	Dénes	Úthálózat kidolgozása
2026. 03. 06.	6 óra	Bocskai	NodeType, RoadType, LaneState, Player, Role, BusDriverRole, CleanerRole leírása, a hozzájuk kapcsolódó szekvencia diagramok és az állapotgépek elkészítése
2026.03.06.	3 óra	Czap	RoadNetwork, Road, Route, Lane, Node osztályok leírásának kidolgozása.
2026. 03. 07.	2 óra	Tóth	Vehicle, Car, Bus, Snowplow, DragonHead, SweeperHead, ThrowerHead, IcebreakerHead, SaltSpreaderHead leírás elkészítése

2026. 03. 07.	1 óra	Tóth	Vehicle, Car, Bus, Snowplow, DragonHead, SweeperHead, ThrowerHead, IcebreakerHead, SaltSpreaderHead leírás elkészítésének befejezése
2026.03.07	3 óra	Jandó	Buyable, Config, Game, Scoreboard, Store, Weather osztályok leírásának kidolgozása.
2026. 03. 08.	2 óra	Dénes	Az osztálydiagram módosítása az analízis modell és a kidolgozott komponensek alapján.
2026.03.08.	4 óra	Czap	Járművek mozgása és sávváltása, hó taposása és baleset, erőforrás-fogyasztás speciális fejeknél, nyersanyag-fogyasztás és takarításkor szekvencia diagramok elkészítése.
2026.03.08	2 óra	Jandó	Körök és időjárás frissítése, Vásárlási folyamat, Játék vége és kiértékelés szekvencia diagramok elkészítése.

4. Analízis modell II.

23 – githappens

Konzulens:

Haragos Gergő Viktor

Csapattagok

Czap László	NS7V2T	czap.laszlo@edu.bme.hu
Bocskai Zsófia	UGSTW9	bocskaizsofi@gmail.com
Tóth Márton	K01YXA	toth.marton21@gmail.com
Dénes Péter István	PO6MVA	denespeter03tab@gmail.com
Jandó Barnabás Tamás	AWQ77G	jandobarnabas@gmail.com

2026.03.16.

3. Analízis modell kidolgozása

3.1 Objektum katalógus

3.1.1 Game (Játék)

A szimuláció "dirigense". Ő indítja el és fejezi be a játékot, minden körben sorban megszólítja az összes többi objektumot – lépteti az időjárást, a járműveket, ellenőrzi a végfeltételt. Nála van nyilvántartva minden jármű és játékos.

3.1.2 Weather (Időjárás)

Minden körben "havazik": végigmegy az úthálózat összes felszíni sávján és növeli a hóvastagságot. Ő az egyetlen, aki tudja, hogy épp milyen intenzíven esik a hó.

3.1.3 Config (Konfiguráció)

A játék indítása előtt kitöltött "beállítólap". Tárolja, hogy hány körös legyen a játék és hány járművel induljon. Csak olvasható, a játék futása közben nem változik.

3.1.4 Scoreboard (Eredménytábla)

A játék végén "összeszámolja a pontokat". Összegyűjti a takarítók végső vagyonát és a buszvezetők teljesített fordulót, majd ezekből előállít egy végeredményt.

3.1.5 Store (Bolt)

A takarítók "webshopja". Ellenőrzi, hogy a játékosnak van-e elég pénze, majd leveszi az összeget és "kiszállítja" a megvásárolt kotrófejet, nyersanyagot vagy új hókotrókat.

3.1.6 RoadNetwork (Úthálózat)

A város térképe és egyben az útvonalkereső motor. Nyilvántartja az összes csomópontot és útszakaszt, és ha egy jármű célba akar érni, tőle kérdezi meg a legrövidebb járható utat.

3.1.7 Road (Útszakasz)

Két csomópontot összekötő fizikai objektum, amely maga köré gyűjti az azonos irányú sávjait és segít megtalálni, hogy melyik sáv szomszédja melyiknek.

3.1.8 Route (Útvonal)

Egy konkrét útvonalterv, amelyet az útvonalkereső algoritmus állít elő. Sávkok rendezett listáját tárolja, és menet közben "adagolja" a következő sávot a járműnek.

3.1.9 Lane (Sáv)

A gráf éle és egyben a hó-, jég- és baleseti állapot gazdája.

3.1.10 Node (Csomópont)

A gráf csúcsa, egy pont a városban ahol utak találkoznak.

3.1.11 Vehicles (Járművek)

Minden mozgó entitás közös őse. Tudja, hogy épp melyik sávon áll, hol tart a sávon belül, és minden körben megpróbál egy lépést tenni a célja felé. Ha akadályba ütközik, sávot vált vagy megáll.

3.1.12 Head (Kotrófej)

A hókotró "munkaeszköze". Minden fejféléség másképp kezeli a havat és a jeget: tolja, szórja, töri vagy olvasztja. Fogyaszt esetleg nyersanyagot a gazdájától, cserébe megtisztítja a sávot.

3.1.13 Player (Játékos)

Egy valódi emberi résztvevőt képvisel. Önmagában keveset tud – a tényleges cselekvések a szerepköreik keresztül valósulnak meg. Egy játékos egyszerre több szerepkört is betölthet.

3.1.14 Role (Szerepkör)

A játékos "munkaköri leírása". A takarítói szerepkör pénzt kezel és hókotrókat irányít, a buszvezető szerepkör útvonalat tervez és buszokat mozgat. Ugyanaz a játékos mindkét szerepkört felveheti egyszerre.

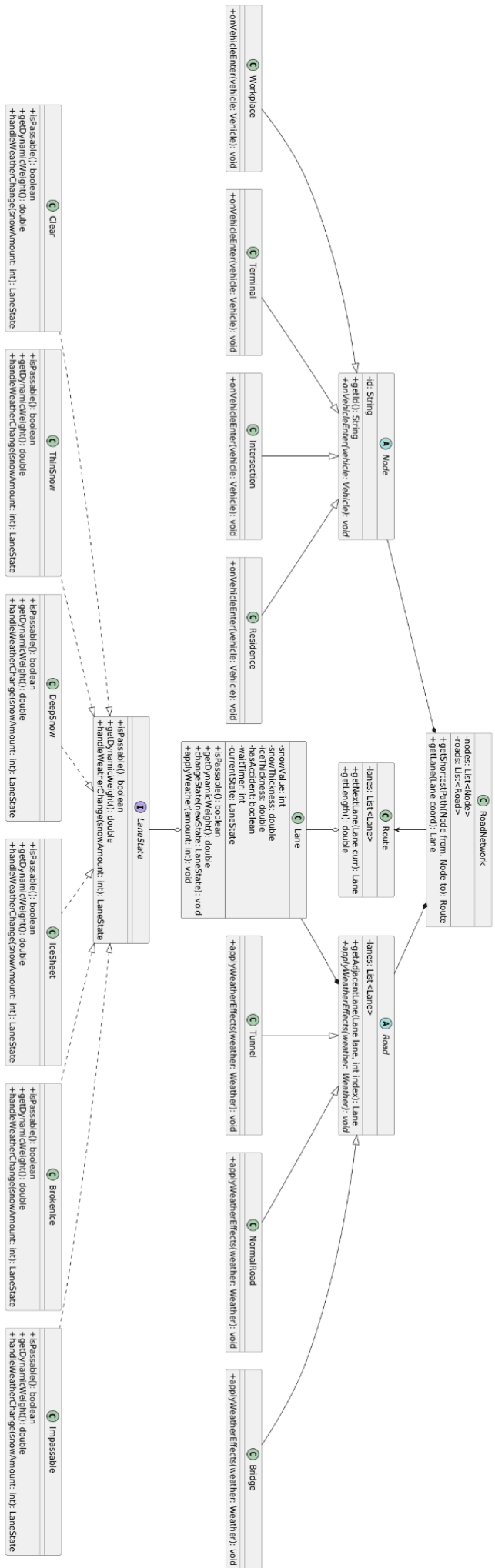
3.1.15 BusDriverRole (BuszvezetőSzerepkör)

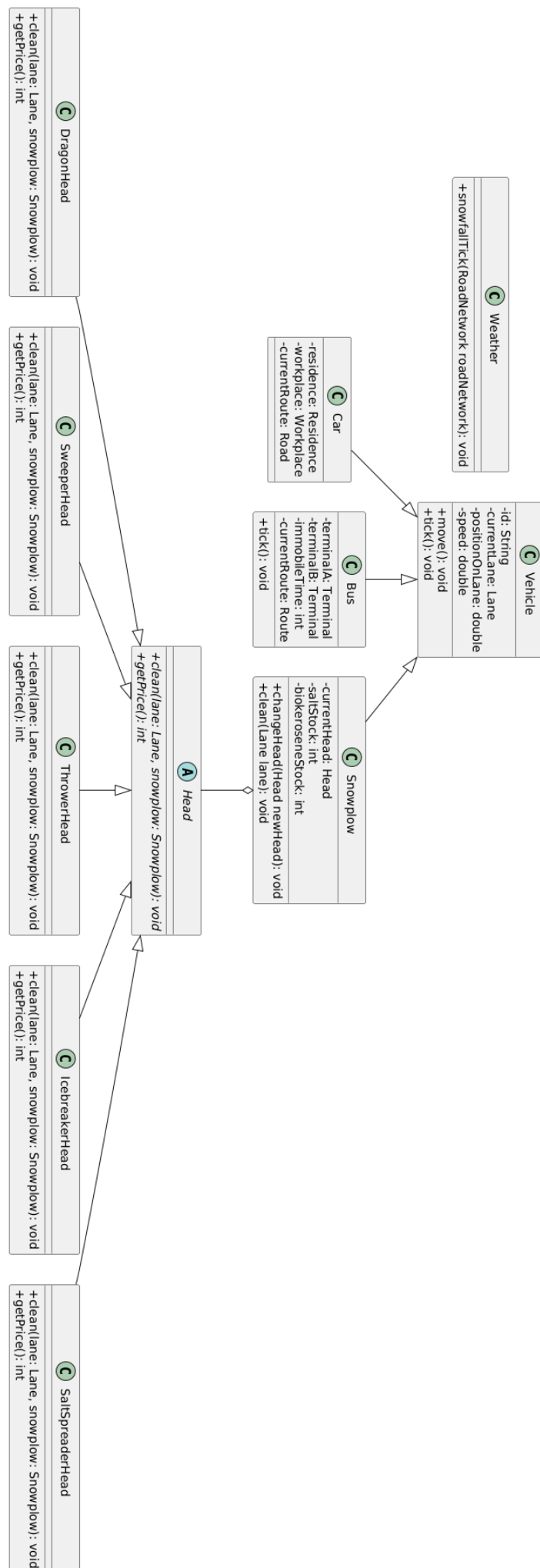
A buszvezető felelős a buszok mozgatásáért, útvonalak megtervezéséért és a fordulók teljesítéséért két végállomás között.

3.1.16 CleanerRole (TakarítóSzerepkör)

A takarító felelős a hó eltávolításáért, a hókotrók irányításáért és a hókotrókat érintő gazdasági döntésekért.

3.2 Statikus struktúra diagramok





3.3 Osztályok leírása

3.3.1 Bridge

- **Felelősség**

A Bridge osztály egy hidat reprezentál, ami egy speciális útként szolgál.

- **Ősosztályok**

Road

- **Interfészek**

Nincs

- **Asszociációk**

Nincs saját asszociációja.

- **Attribútumok**

Nincs saját attribútuma.

- **Metódusok**

- **void applyWeatherEffects(Weather weather):** Az időjárás hatását alkalmazza az útszakasz sávjaira.

3.3.2 BrokenIce

- **Felelősség**

Az osztály a sáv jégdarabokkal fedett (jégpáncél jégtörő fejjel feltörése utáni) állapotát reprezentálja. A söprő és hányó fejek eltávolítják ezt a réteget.

- **Ősosztályok**

Nincs ősosztálya.

- **Interfészek**

LaneState

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **boolean isPassable():** Az út járhatóságát lehet vele lekérdezni.
- **double getDynamicWeight():** Meghatározza az útvonaltervezés során használt dinamikus súlyt.
- **LaneState handleWeatherChange (int snowAmount):** Az időjárás változásának kezelése.

3.3.3 Bus

· Felelősség

A busz egy játékos által irányított tömegközlekedési jármű, amelynek célja meghatározott végállomások között sikeres fordulók teljesítése. Felelős a saját pozíciójának kezeléséért és az útvonalterv követéséért. Baleset esetén ideiglenesen mozgásképtelenné válik.

· Ősosztályok

Vehicle (Jármű)

· Interfészek

Nem valósít meg interfészt.

· Asszociációk

- **Route:** Az aktuálisan követett útvonalterv, amely meghatározza a haladási irányt

· Attribútumok

- **terminalA: Node:** Az egyik végállomás
- **terminalB: Node:** A másik végállomás
- **immobileTime: int:** A baleset vagy elakadás miatti kényszerű várakozási idő
- **currentRoute: Route:** Az aktuálisan követett útvonal

· Metódusok

- **void tick():** Végrehajtja a busz mozgását az aktuális körben a sáv állapota alapján

3.3.4 BusDriverRole

· Felelősség

A BuszvezetőSzerep a Role leszármazottja. A buszvezető felelős a buszok mozgatásáért, útvonalak megtervezéséért és a fordulók teljesítéséért két végállomás között. A szerepkör pontszáma a sikeresen teljesített fordulókból adódik.

· Ősosztályok

Role

· Interfészek

Nem valósít meg interfészt.

- **Asszociációk**
- **BusDriverRole - Bus:** Az asszociáció túloldalán a Bus osztály áll. A kapcsolat célja, hogy a buszvezető irányíthassa a buszokat és teljesíthesse vele a fordulókat.
- **Attribútumok**
- **completedRounds: int** A buszvezető által teljesített fordulók száma.
- **Metódusok**
- **int assignRoute(Bus bus, Node destination):** A busz aktuális állapotából a cél csomópontba megtalálja a legrövidebb útvonalat.
- **int getScore():** A Role metódusának felüldefiniált metódusa, megadja a buszvezető szerepkör által szerzett pontszámot.

3.3.5 Buyable

- **Felelősség**

Egy interfész, amely minden megvásárolható elem számára közös viselkedést ír elő. Biztosítja, hogy minden elem rendelkezzen lekérdezhető vételárral a gazdasági műveletekhez.

- **Ősosztályok**

Nincs ősosztálya.

- **Interfészek**

Ez maga egy interfész.

- **Asszociációk**
- **Store:** A megvásárolható elemeket a bolt kezeli és értékesíti.
- **Attribútumok**

Nincs attribútuma.

- **Metódusok**
- **int getPrice():** Visszaadja az adott megvásárolható elem aktuális árát.

3.3.6 Car

- **Felelősség**

A szimuláció civil járműveit képviseli, amelyek a lakóhelyük és munkahelyük között közlekednek a legrövidebb járható úton. Mozgásukkal tapossák a havat, ami jégpáncél kialakulásához vezethet

- **Össztályok**

Vehicle (Jármű)

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

- **Route:** Az aktuálisan követett útvonalterv, amely meghatározza a haladási irányt

- **Attribútumok**

- **residence:** Node Kiindulási pont/lakóhely
- **workplace:** Node Célállomás/munkahely
- **currentRoute:** Node A Route objektum, amelyet az autó követ

- **Metódusok**

Nincs saját metódusa

3.3.7 CleanerRole

- **Felelősség**

A CleanerRole (TakarítóSzerepkör) a Role leszármazottja. A takarító felelős a hó eltávolításáért, a hókotrók irányításáért és a hókotrókat érintő gazdasági döntésekért. A szerepkör kezeli a játékos pénzét, vásárolhat a Boltban, és működteti a hókotrók fejeit.

- **Össztályok**

Role

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

- **CleanerRole - Snowplow (Hókotró):** Az asszociáció túloldalán a Hókotró osztály áll. A kapcsolat célja, hogy a takarító irányíthassa a hókotrókat és takarítja vele az utakat.
- **CleanerRole – Store (Bolt):** A kapcsolat célja, hogy a takarító vásárolhasson nyersanyagot, új fejeket vagy hókotrókat.

- **Attribútumok**

- **money: int** A takarító jelenlegi vagyona.

- **Metódusok**

- **boolean buy(Role role, Buyable item):** A takarító a boltból a kotrófejek működéséhez szükséges üzemanyagot, új kotrófejeket és hókotrókat vásárolhat.

- **void controlSnowplow(Snowplow snowplow):** A takarító irányítja a hókotrót.
- **int getScore():** A Role metódusának felüldefiniált metódusa, megadja a takarító által szerzett pontszámot.

3.3.8 Clear

- **Felelősség**

A Clear osztály azt az állapotot modellezi, amikor az útsáv teljesen tiszta, nincs rajta hó vagy jég, így a járművek akadálytalanul közlekedhetnek rajta.

- **Össosztályok**

Nincs össosztálya.

- **Interfészek**

LaneState

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **boolean isPassable():** Az út járhatóságát lehet vele lekérdezni.
- **double getDynamicWeight():** Meghatározza az útvonaltervezés során használt dinamikus súlyt.
- **LaneState handleWeatherChange (int snowAmount):** Az időjárás változásának kezelése.

3.3.9 Config

- **Felelősség**

Ez az osztály tárolja a játék indulásához szükséges statikus paramétereket és alapbeállításokat. Segítségével konfigurálható a szimuláció hossza és a kezdeti járműállomány mérete.

- **Össosztályok**

Nincs össosztálya.

- **Interfészek**

Nincs interfész megvalósítva.

- **Asszociációk**
 - **Game:** Inicializálási adatokat szolgáltat a központi vezérlőnek.
- **Attribútumok**
 - **maxRound: int** A játék során elérhető maximális körszám.
 - **startingVehicleCount: int** Meghatározza, hány járművel kezdődjön el a szimuláció.
- **Metódusok**

Nincs metódusa.

3.3.10 DeepSnow

- **Felelősség**

A DeepSnow osztály azt az állapotot modellezi, amikor az útsávon jelentős mennyiségű hó halmozódott fel, ekkor a járművek elakadnak.

- **Össosztályok**

Nincs össosztálya.

- **Interfészek**

LaneState

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**
 - **boolean isPassable():** Az út járhatóságát lehet vele lekérdezni.
 - **double getDynamicWeight():** Meghatározza az útvonaltervezés során használt dinamikus súlyt.
 - **LaneState handleWeatherChange (int snowAmount):** Az időjárás változásának kezelése.

3.3.11 DragonHead

- **Felelősség**

Speciális kotrófej, amely gázturbinával azonnal elolvasztja a havat és a jeget a jármű előtt. Működéséhez biokerozint használ

- **Össztályok**

Head

- **Interfészek**

Buyable

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **void clean(Lane lane, Snowplow snowplow):** Ellenőrzi a hókotró biokerozin készletét, majd annak felhasználásával a sávot tisztítja.
- **int getPrice():** Az interfészből származó metódus, visszaadja a sárkányfej árát

3.3.12 Game

- **Felelősség**

A Game osztály a szimuláció központi vezérlője, amely koordinálja a teljes játékmenetet, kezeli a körök számát, valamint nyilvántartja a résztvevő játékosokat és járműveket. Felelős a játéklógika léptetéséért és a befejezési feltételeket ellenőrzi.

- **Össztályok**

Nincs osztálya.

- **Interfészek**

Nincs megvalósított interfész.

- **Asszociációk**

- **Weather:** A játék és a környezeti hatások közötti kapcsolatot definiálja.
- **Config:** A játék inicializáláshoz szükséges beállításokat biztosítja.
- **Scoreboard:** A játék eredményeinek követésére szolgáló komponens.
- **Vehicle:** A szimulációban részt vevő járművek listája.
- **Player:** A játékban regisztrált játékosok gyűjteménye.

- **Attribútumok**

- **currentRound: int** Az aktuális szimulációs kör sorszámát tárolja.

- **maxRound: int** A játék maximális időtartamát határozza meg körökben.
 - **vehicles: List<Vehicle>** A rendszerben lévő járművek listája.
 - **players: List<Player>** A játékban résztvevő játékosok listája.
- **Metódusok**
 - **void tick():** Egy egységgel előre lépteti a játék állapotát és frissíti a belső logikát.
 - **boolean isOver():** Megvizsgálja, hogy a szimuláció elérte-e a maximális körszámot vagy véget ért-e.

3.3.13 Head

- **Felelősség**

A hókotrók munkaeszközeinek alaposztálya. Meghatározza a tisztítási folyamat interfészét és kezeli a nyersanyag-felhasználást

- **Ősosztályok**

Nincs

- **Interfészek**

Buyable (Megvásárolható)

- **Asszociációk**
 - **Snowplow:** A hókotró, amelyre a fej fel van szerelve

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**
 - **void clean(lane: Lane, snowplow: Snowplow):** Absztrakt művelet a sáv tisztítására, figyelembe véve a hókotró készleteit
 - **int getPrice():** Az interfészből származó metódus, visszaadja a fej árát

3.3.14 IcebreakerHead

- **Felelősség**

Speciális fej a jégpáncél fizikai feltörésére. A havat nem takarítja el, csak a jeget teszi takaríthatóvá a söprő vagy hányó fejek számára

- **Ősosztályok**

Head

- **Interfészek**

Buyable

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**
 - **void clean(Lane lane, Snowplow snowplow)**: A sáv jégpáncél állapotát feltört jég állapotra módosítja.
 - **int getPrice()**: Az interfészből származó metódus, visszaadja a jégtörő fej árát

3.3.15 IceSheet

- **Felelősség**

Az osztály a sáv jégpáncéllal fedett állapotát reprezentálja. A járművek csúszkálnak rajta. Bizonyos hókotró fejek eltávolítják ezt a réteget.

- **Ősosztályok**

Nincs ősosztálya.

- **Interfészek**

LaneState

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**
 - **boolean isPassable()**: Az út járhatóságát lehet vele lekérdezni.
 - **double getDynamicWeight()**: Meghatározza az útvonaltervezés során használt dinamikus súlyt.
 - **LaneState handleWeatherChange (int snowAmount)**: Az időjárás változásának kezelése.

3.3.16 Impassable

- **Felelősség**

Az Impassable osztály célja annak modellezése, amikor egy útsáv teljesen járhatatlan baleset miatt.

- **Ősosztályok**

Nincs ősosztálya.

- **Interfészek**

LaneState

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **boolean isPassable():** Az út járhatóságát lehet vele lekérdezni.
- **double getDynamicWeight():** Meghatározza az útvonaltervezés során használt dinamikus súlyt.
- **LaneState handleWeatherChange (int snowAmount):** Az időjárás változásának kezelése.

3.3.17 Intersection

- **Felelősség**

Az Intersection osztály egy útkereszteződést modellez, ahol több útszakasz találkozik. Lehetővé teszi a járművek számára az egyik útról a másikra történő áthaladást.

- **Ősosztályok**

Node.

- **Interfészek**

Nincs.

- **Asszociációk**

Nincs saját asszociációja.

- **Attribútumok**

Nincs saját attribútuma.

- **Metódusok**

- **void onVehicleEnter (Vehicle vehicle):** A metódus feladata a jármű érkezésének kezelése.

3.3.18 Lane

· Felelősség

A Lane osztály az úthálózat gráfjának egy irányított élét reprezentálja, amelyen a járművek közlekednek. Ez az osztály a "gazdája" az útszakasz fizikai állapotának: nyilvántartja a lehullott hó mennyiségét, a kialakult jégpáncél vastagságát és az esetleges balesetek tényét. Ezen adatok alapján felelős annak kiszámításáért, hogy a sáv az adott pillanatban járható-e, valamint meghatározza a sáv "dinamikus súlyát", amely jelzi az áthaladás nehézségét az útvonaltervező algoritmus számára.

· Ősosztályok

Nincs.

· Interfészek

Nincs.

· Asszociációk

source: Node – A sáv kiinduló csomópontja.

destination: Node – A sáv célállomása.

parentRoad: Road – Az az út objektum, amelyhez ez a sáv tartozik.

state: LaneState – A sáv állapotát reprezentáló objektum.

· Attribútumok

- **snowValue:** int – A sávon felhalmozódott hó mennyiségi mutatója.
- **snowThickness:** double – A hóréteg aktuális fizikai vastagsága.
- **iceThickness:** double – Az úton lévő jégpáncél vastagsága.
- **isPassable:** boolean – Logikai érték, amely megadja, hogy a sáv jelenleg járható-e a forgalom számára.
- **hasAccident:** boolean – Jelzi, ha a sávon baleset történt, ami akadályozza a haladást.
- **waitTimer:** int – Baleset vagy elakadás esetén a kényszerű várakozási időt mérő számláló.

· Metódusok

- **boolean isPassable():** Kiértékeli a sáv aktuális állapotát (hó, jég, baleset), és visszaadja, hogy a járművek áthajthatnak-e rajta.
- **double getDynamicWeight():** Kiszámítja a sáv súlyozását az útvonalkereséshez; a mélyebb hó vagy jegesedés növeli az értéket, így a járművek preferálják a tisztább szakaszokat.
- **LaneState getLaneState():** Visszaadja a sáv leírására szolgáló objektumot.
- **void change(int amount):** Módosítja a hó vagy jég mennyiségét a sávon havazás vagy hókotrás hatására.

3.3.19 LaneState

- **Felelősség**

A LaneState interfész az útsáv lehetséges állapotainak közös viselkedését definiálja. Meghatározza azokat a metódusokat, amelyeket minden konkrét állapot osztálynak implementálnia kell.

- **Ősosztályok**

Nincs ősosztály.

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

Az interfészt a következő osztályok implementálják:

- Clear
- ThinSnow
- DeepSnow
- IceSheet
- BrokenIce
- Impassable

- **Attribútumok**

Nincs attribútuma.

- **Metódusok**

- **boolean isPassable()**: Az út járhatóságát lehet vele lekérdezni.
- **double getDynamicWeight()**: Meghatározza az útvonaltervezés során használt dinamikus súlyt.
- **LaneState handleWeatherChange (int snowAmount)**: Az időjárás változásának kezelése.

3.3.20 Node

- **Felelősség**

Az absztrakt Node osztály az úthálózatot alkotó gráf csúcspontjait reprezentálja, ahol a város különböző útszakaszai találkoznak. Feladata a földrajzi vagy logikai pont azonosítása.

- **Ősosztályok**

Nincs.

- **Interfészek**

Nincs.

- **Asszociációk**

A RoadNetwork osztály felhasználja.

- **Attribútumok**

- **id**: String – A csomópont egyedi azonosítója vagy neve.

- **Metódusok**

- **String getId()**: Visszaadja a csomópont egyedi azonosítóját.
- **void onVehicleEnter (Vehicle vehicle)**: A metódus feladata a jármű érkezésének kezelése.

3.3.21 NormalRoad

- **Felelősség**

A NormalRoad egy hagyományos útszakaszt modellez. Az időjárási hatások közvetlenül érvényesülnek rajta, így a sávok állapota a hó vagy jég mennyiségének megfelelően változhat.

- **Össztályok**

Road

- **Interfészek**

Nincs

- **Asszociációk**

Nincs saját asszociációja.

- **Attribútumok**

Nincs saját attribútuma.

- **Metódusok**

- **void applyWeatherEffects(Weather weather)**: Az időjárás hatását alkalmazza az útszakasz sávjaira.

3.3.22 Player

- **Felelősség**

A Player egy valódi emberi résztvevőt reprezentál a rendszerben. A játékos a játék emberi irányítója, de önmagában kevés közvetlen műveletet végez, a tényleges interakciók a hozzá tartozó Role segítségével valósulnak meg. Egy játékos egyszerre több szerepkört is betölthet, és ezek a szerepkörök határozzák meg, hogy milyen járműveket irányíthat, milyen döntéseket hozhat, és hogyan szerez pontokat a játék során. A játékos stratégiai döntéseket hoz, amik hatással vannak a Zúzmaraváros állapotára, illetve a saját pontszámára.

- **Össztályok**

Nincs őssosztály.

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

- **Player - Role:** A kompozíció túloldalán a Role osztály áll. A kapcsolat célja, hogy a játékos a szerepkörain keresztül tudjon cselekedni a játékban. Egy játékos több szerepkört is tartalmazhat, és ezek határozzák meg, milyen járműveket irányíthat és milyen döntéseket hozhat.
- **Player - Game:** Az kompozíció túloldalán a Game osztály áll. A kapcsolat célja, hogy a Game nyilvántartsa az összes játékost. A Player résztvevője a Game-nek.

- **Attribútumok**

- **id: int** A játékos egyedi azonosítója.
- **name: String** A játékos neve.

- **Metódusok**

- **String getName():** A játékos nevét adja meg
- **int getSumPoints():** Összegzi a szerepkörök által szerzett pontokat.

3.3.23 Residence

- **Felelősség**

A Residence osztály egy lakóhelyet modellez, ami a sofőröknek egyik úti céljául szolgál.

- **Őssosztályok**

Node.

- **Interfészek**

Nincs.

- **Asszociációk**

Nincs saját asszociációja.

- **Attribútumok**

Nincs saját attribútuma.

- **Metódusok**

- **void onVehicleEnter (Vehicle vehicle):** A metódus feladata a jármű érkezésének kezelése.

3.3.24 Road

- **Felelősség**

A Road absztrakt osztály két csomópontot összekötő fizikai útszakaszt reprezentál a játékteren. Feladata az egy irányba haladó sávok összefogása és koordinálása. Emellett támogatja a járművek navigációját azáltal, hogy segít meghatározni a sávok közötti szomszédsági viszonyokat.

- **Össosztályok**
- **Interfészek**
- **Asszociációk**
 - **lanes:** List<Lane> – Az adott útszakaszhoz tartozó sávok listája.
 - **source:** Node – Az útszakasz kiinduló csomópontja.
 - **destination:** Node – Az útszakasz célállomását jelentő csomópont.
- **Attribútumok**
 - **lanes:** List<Lane> Az útszakaszt felépítő sávok gyűjteménye.
- **Metódusok**
 - **Lane getAdjacentLane(Lane lane, int index):** Megkeresi és visszaadja a paraméterként kapott sáv melletti szomszédos sávot a megadott index (irány) alapján. Ez a metódus teszi lehetővé a járművek számára a sávváltást akadályok vagy balesetek esetén.
 - **void applyWeatherEffects(Weather weather):** Az időjárás hatását alkalmazza az útszakasz sávjaira.

3.3.25 RoadNetwork

- **Felelősség**

A RoadNetwork osztály felelős Zúzmaraváros teljes úthálózatának reprezentálásáért és kezeléséért. Ez az osztály működik a játék útvonalkereső motorjaként: nyilvántartja a város összes csomópontját és útszakaszát, valamint kiszámítja a járművek számára a céljukhoz vezető legrövidebb, aktuálisan járható utat. Figyelembe veszi az utak típusát (híd, alagút, felszíni út) és azok aktuális állapotát a tervezés során.

- **Össosztályok**
- **Interfészek**
- **Asszociációk**
 - **nodes:** List<Node> Az úthálózatot felépítő összes csomópont listája.
 - **roads:** List<Road> Az úthálózatot felépítő összes útszakasz (élek) listája.
- **Attribútumok**
 - **nodes:** List<Node> A városban található összes kereszteződés, lakóhely és munkahely gyűjteménye.
 - **roads:** List<Road> A csomópontokat összekötő útszakaszok, hidak és alagutak listája.

- **Metódusok**
- **Route getShortestPath(Node from, Node to):** Meghatározza a két megadott csomópont közötti legrövidebb, akadálymentes útvonalat. A számítás során figyelembe veszi a sávok járhatóságát (pl. elkerüli a mély havat vagy balesetet, ha van alternatíva).
- **Lane getLane(Lane coord):** Visszaadja a paraméterként megadott koordinátákhoz vagy azonosítóhoz tartozó konkrét sáv objektumot.

3.3.26 Role

- **Felelősség**

A Role (Szerepkör) absztrakt osztály felel a járművek irányításáért. A játékosok különböző szerepkörökön keresztül irányíthatják a hozzájuk tartozó járműveket, végezhetnek további járművekhez kapcsolódó műveleteket. Egy játékos több szerepkört is felvehet egyszerre.

- **Ősosztályok**

Nincs ősosztály.

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

- **Roles - Player (Játékos):** A kompozíció túloldalán a Player (Játékos) osztály áll. A kapcsolat célja, hogy a szerepkörökön keresztül a játékos tudjon cselekedni a játékban. Egy játékos több szerepkört is tartalmazhat, és ezek határozzák meg, milyen járműveket irányíthat és milyen döntéseket hozhat.

- **Attribútumok**

Nincs attribútum.

- **Metódusok**

- **int getScore():** megadja az adott szerepkör által szerzett pontszámot

3.3.27 Route

- **Felelősség**

A Route osztály egy konkrét útvonaltervet reprezentál, amelyet az úthálózat útvonalkereső algoritmus állít elő a járművek számára. Feladata a célba jutáshoz szükséges sávok rendezett listájának tárolása. A szimuláció során ez az osztály felelős azért, hogy a jármű haladása közben kiszolgálja ("adagolja") a következő sávot, amelyre a járműnek rá kell hajtania.

- **Ősosztályok**

- **Interfészek**
- **Asszociációk**
 - **lanes**: List<Lane> – Az útvonalat felépítő sávok rendezett sorozata.
- **Attribútumok**
 - **lanes**: List<Lane> – A jármű által bejárando sávok listája.
- **Metódusok**
 - **Lane getNextLane(Lane curr)**: Megkeresi a paraméterként kapott aktuális sávot az útvonalban, és visszaadja a soron következő sáv objektumot. Ezzel irányítja a járművet az útvonal mentén.
 - **double getLength()**: Visszaadja az útvonal teljes hosszát, amely az útvonalat alkotó sávok hosszának összege.

3.3.28 SaltSpreaderHead

- **Felelősség**

Sót juttat az útfelületre, ami megakadályozza a hó megmaradását és idővel felolvasztja a havat és a jeget. Működéséhez só nyersanyagot igényel

- **Ősosztályok**

Head

- **Interfészek**

Buyable

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**
 - **void clean(Lane lane, Snowplow snowplow)**: Csökkenti a hókotró sókészletét és alkalmazza a sószórást a sávon, ami elindítja a fokozatos olvadást
 - **int getPrice()**: Az interfészből származó metódus, visszaadja a sószóró fej árát

3.3.29 Scoreboard

- **Felelősség**

Az osztály feladata a játékosok teljesítményének folyamatos nyomon követése és az eredmények összesítése. Kezeli a pontszámok tárolását és biztosítja azok lekérdezhetőségét.

- **Ősosztályok**

Nincs ősosztály.

- **Interfészek**

Nincs megvalósított interfész.

- **Asszociációk**

- **Game:** A játék menedzsment rétegének részeként együttműködik a vezérlővel.

- **Attribútumok**

- **scores: Map<Player, Integer>** Egy leképezés (map), amely a játékosokhoz rendeli a megszerzett pontszámokat.

- **Metódusok**

- **void evaluate():** Elvégzi a pontszámok aktuális állapot szerinti kiértékelését vagy frissítését.
- **Map<Player, Integer> getScores():** Visszaadja a pillanatnyi állást tartalmazó ponttáblázatot.

3.3.30 Snowplow

- **Felelősség**

Speciális jármű, amely az úthálózat tisztításáért felel. A takarító játékos irányítása alatt mozog, és a felszerelt kotrófej segítségével módosítja a sávok állapotát

- **Ősosztályok**

Vehicle (Jármű)

- **Interfészek**

Buyable (Megvásárolható)

- **Asszociációk**

- **head:** A hókotróra szerelt aktuális munkaeszköz
- **Cleanerrole:** A járművet birtokló és irányító takarító szerepkör

- **Attribútumok**

- **currentHead: Head:** A járműre szerelt aktuális fej
- **saltStock: int:** A sószóró fejhez tárolt sómennyiség

- **biokeroseneStock: int:** A sárkány fejhez tárolt üzemanyag
- **Metódusok**
- **void changeHead(Head newHead):** Lecseréli az aktuális kotrófejet egy másik típusra
- **void clean(Lane lane):** Meghívja a felszerelt kotrófej tisztító funkcióját az aktuális sávra

3.3.31 Store

- **Felelősség**

A gazdasági alrendszer eleme, amely lehetővé teszi a megvásárolható tárgyak beszerzését a játékosok számára. Összeköti az elérhető kínálatot a vásárlási tranzakciókkal.

- **Ősosztályok**

Nincs ősosztálya.

- **Interfészek**

Nem valósít meg interfészt.

- **Asszociációk**

- **Buyable:** A bolt készletében lévő megvásárolható elemekre utal.
- **Role:** A vásárlást kezdeményező szerepkörökkel áll kapcsolatban.

- **Attribútumok**

- **inventory: List<Buyable>** A boltban pillanatnyilag elérhető termékek (Buyable) listája.

- **Metódusok**

- **boolean buy(Role role, Buyable item):** Lebonyolít egy vásárlást egy adott szerepkör számára, és visszatér a sikerességével.

3.3.32 SweeperHead

- **Felelősség**

Mechanikus munkaeszköz, amely a havat közvetlenül a hókotró nyomvonala mellé, a szomszédos sávba tolja. A lefagyott jeget nem tudja eltávolítani, csak a havat és a már feltört jeget

- **Ősosztályok**

Head

- **Interfészek**

Buyable

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **void clean(Lane lane, Snowplow snowplow):** Ha a sáv nem jeges, a havat/feltört jeget áthelyezi a szomszédos sávba, ezzel a jelenlegi sávot megtisztítja.
- **int getPrice():** Az interfészből származó metódus, visszaadja a söprő fej árát

3.3.33 Terminal

- **Felelősség**

A Terminal osztály egy busz végállomását modellezi. A buszvezetők feladata a buszok végállomásai között közlekedni.

- **Ősosztályok**

Node.

- **Interfészek**

Nincs.

- **Asszociációk**

Nincs saját asszociációja.

- **Attribútumok**

Nincs saját attribútuma.

- **Metódusok**

- **void onVehicleEnter (Vehicle vehicle):** A metódus feladata a jármű érkezésének kezelése.

3.3.34 ThinSnow

- **Felelősség**

A ThinSnow osztály a sáv vékony hórétegű állapotát reprezentálja. A járművek lassabban haladnak rajta, de a sáv még járható. A hókotró fejek könnyen eltávolítják ezt a réteget.

- **Ősosztályok**

Nincs ősosztálya.

- **Interfészek**

LaneState

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **boolean isPassable()**: Az út járhatóságát lehet vele lekérdezni.
- **double getDynamicWeight()**: Meghatározza az útvonaltervezés során használt dinamikus súlyt.
- **LaneState handleWeatherChange (int snowAmount)**: Az időjárási viszonyok változásának kezelése.

3.3.35 ThrowerHead

- **Felelősség**

A söprő fejhez hasonlóan működik, de a havat nem a közvetlen szomszédos sávba, hanem annál távolabbra szórja. Kezdeti felszerelésként is választható

- **Ősosztályok**

Head

- **Interfészek**

Buyable

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **void clean(Lane lane, Snowplow snowplow)**: eltávolítja a havat és a feltört jeget a sávból úgy, hogy azokat távoli területre juttatja

- **int getPrice():** Az interfészből származó metódus, visszaadja a hányó fej árát

3.3.36 Tunnel

- **Felelősség**

A Tunnel osztály egy alagutat modellez. Az alagutakban az időjárás hatása korlátozott, ezért a hó vagy jég általában nem befolyásolja az útsávok állapotát.

- **Ősosztályok**

Road

- **Interfészek**

Nincs

- **Asszociációk**

Nincs saját asszociációja.

- **Attribútumok**

Nincs saját attribútuma.

- **Metódusok**

- **void applyWeatherEffects(Weather weather):** Az időjárás hatását alkalmazza az útszakasz sávjaira.

3.3.37 Vehicles

- **Felelősség**

Minden mozgó entitás közös absztrakt őse. Kezeli az alapvető mozgási logikát, a sávokon való elhelyezkedést és a sávváltás lehetőségét akadály esetén

- **Ősosztályok**

Nincs

- **Interfészek**

Nem valósít meg interfészt

- **Asszociációk**

- **Lane:** Az a sáv, amelyen a jármű jelenleg tartózkodik

- **Attribútumok**

- **id: String:** A jármű egyedi azonosítója
- **currentLane: Lane:** Az aktuális sáv referenciája
- **positionOnLane: double:** A jármű aktuális pozíciója az adott sávon belül

- **speed: double:** A jármű haladási sebessége
- **Metódusok**
- **void move():** A jármű mozgását végző belső logika
- **void tick():** A körönkénti léptetésért felelős metódus

3.3.38 Weather

- **Felelősség**

Az időjárási viszonyok szimulálásáért felelős osztály, amely dinamikusan módosítja a környezeti állapotokat. Elsődleges feladata a hóesés hatásának érvényesítése az úthálózaton.

- **Ősosztályok**

Nincs ősosztálya.

- **Interfészek**

Nincs megvalósított interfész.

- **Asszociációk**

- **RoadNetwork:** Közvetlenül módosítja az úthálózat elemeit (sávokat) a hóesés során.

- **Attribútumok**

Nincs attribútuma.

- **Metódusok**

- **void snowfallTick(RoadNetwork roadNetwork):** Kiszámítja és alkalmazza a hóesés mértékét az úthálózat sávjaira az aktuális körben.

3.3.39 Workplace

- **Felelősség**

Az Workplace osztály egy munkahelyet modellez, ami a sofőrök egyik úticéljaként szolgál.

- **Ősosztályok**

Node.

- **Interfészek**

Nincs.

- **Asszociációk**

Nincs saját asszociációja.

- **Attribútumok**

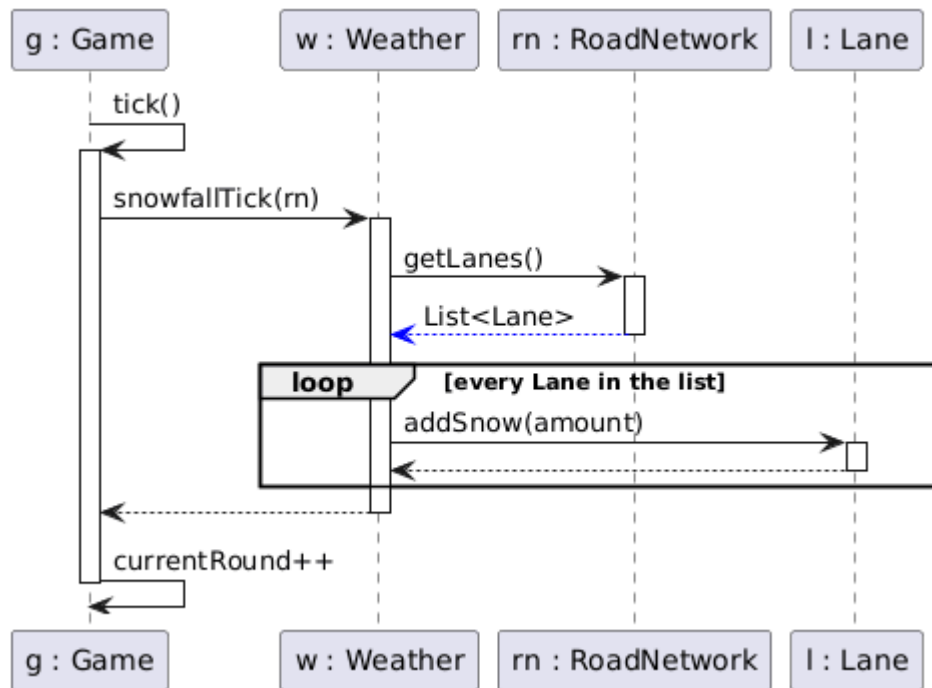
Nincs saját attribútuma.

- **Metódusok**

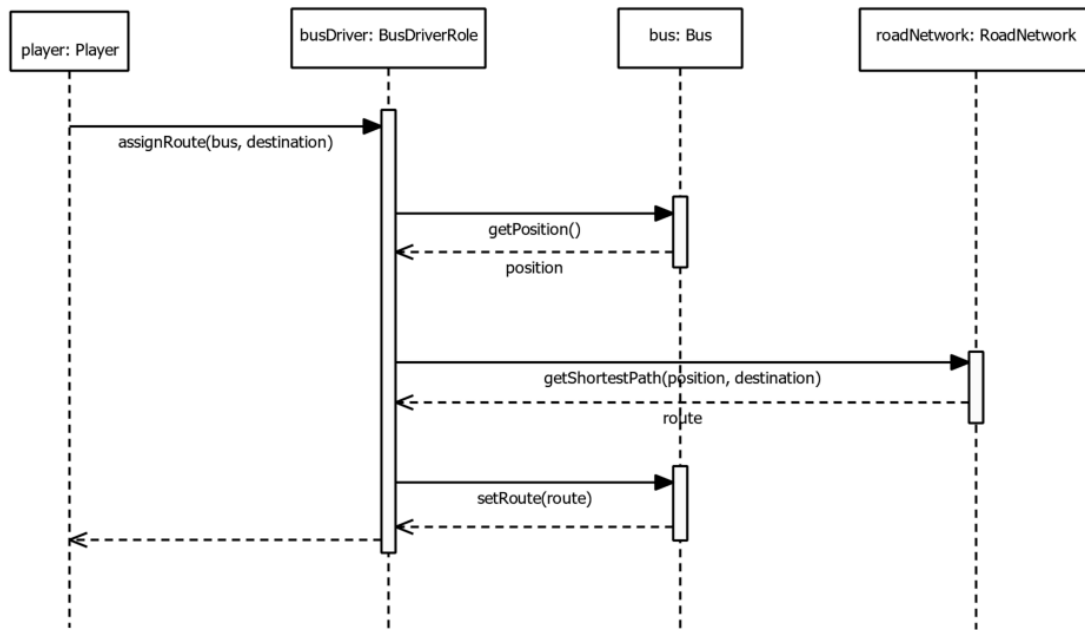
- **void onVehicleEnter (Vehicle vehicle):** A metódus feladata a jármű érkezésének kezelése.

3.4 Szekvencia diagramok

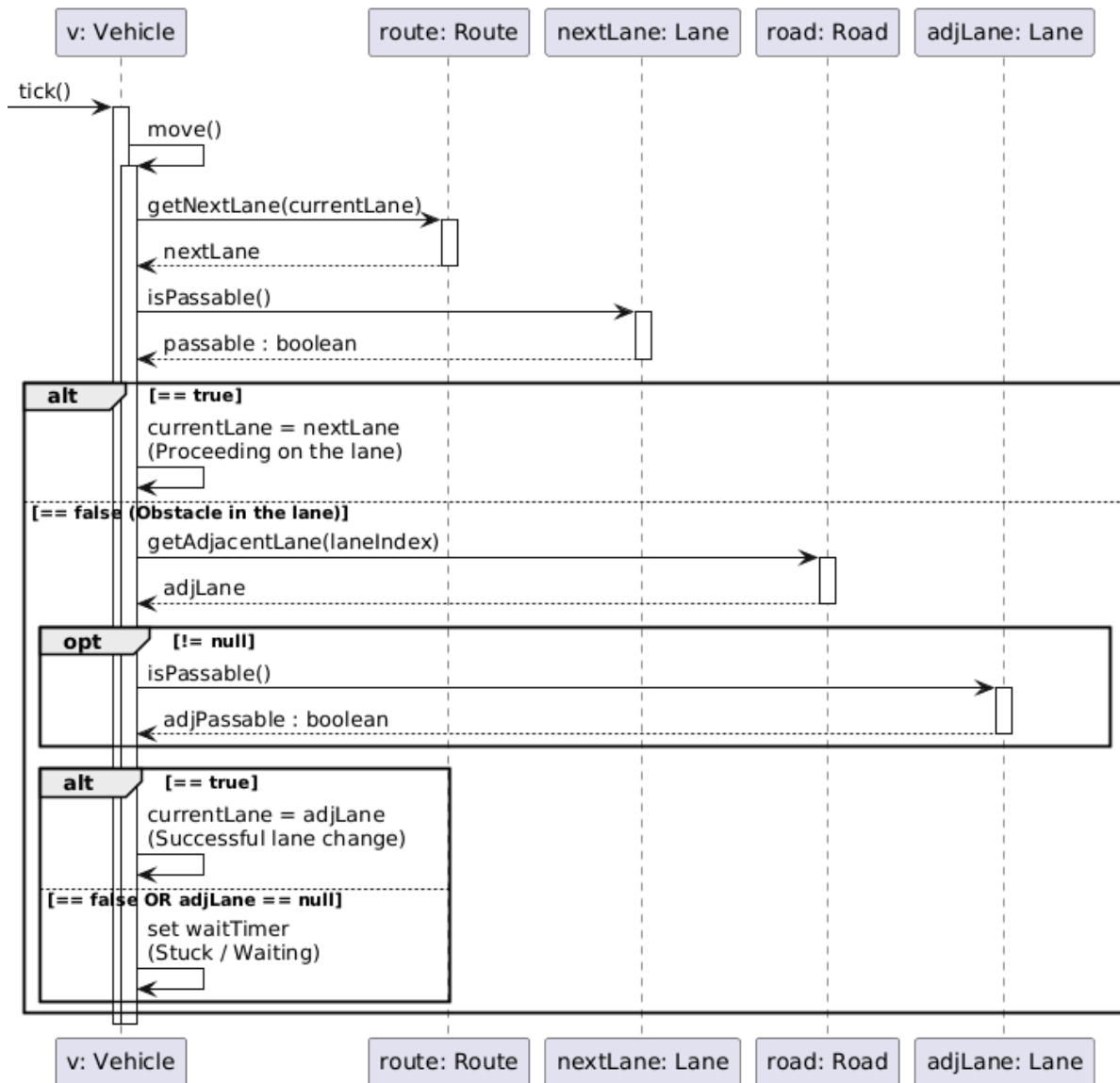
Körök és időjárás frissítése:



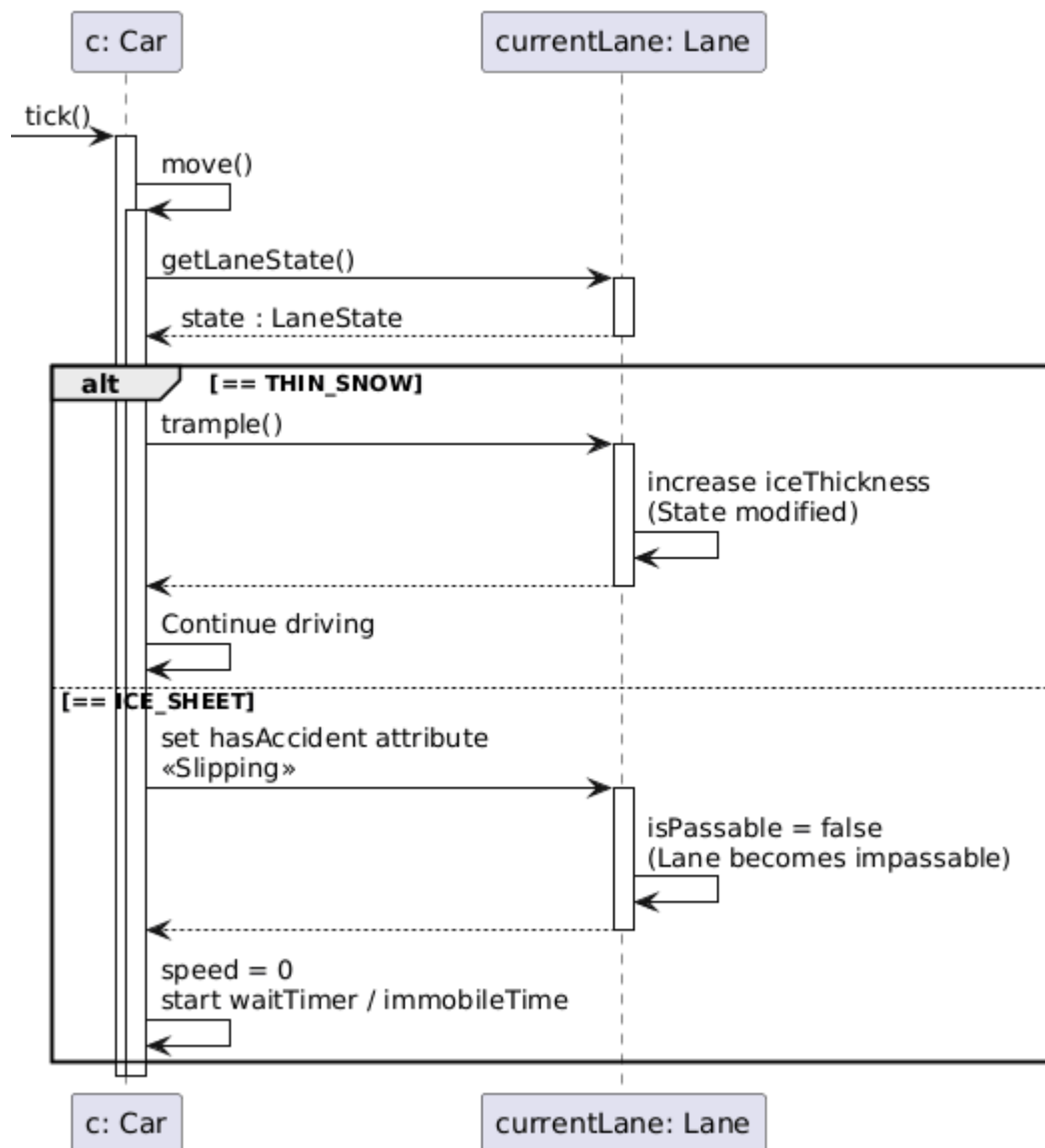
Busz útvonalának kijelölése:



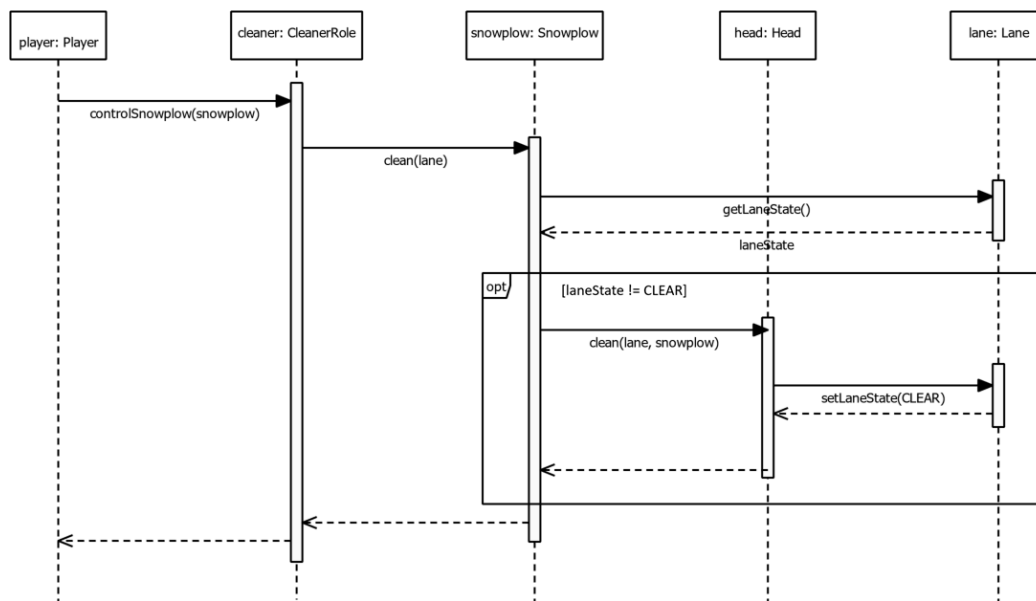
Járművek mozgása és sávváltása:



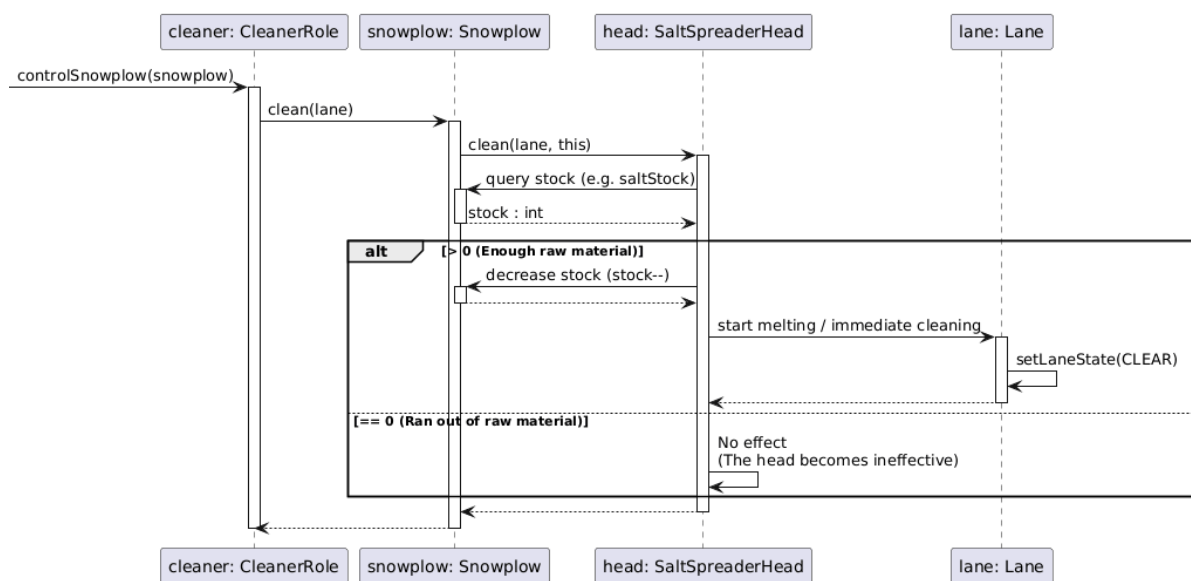
Hó taposása és baleset:



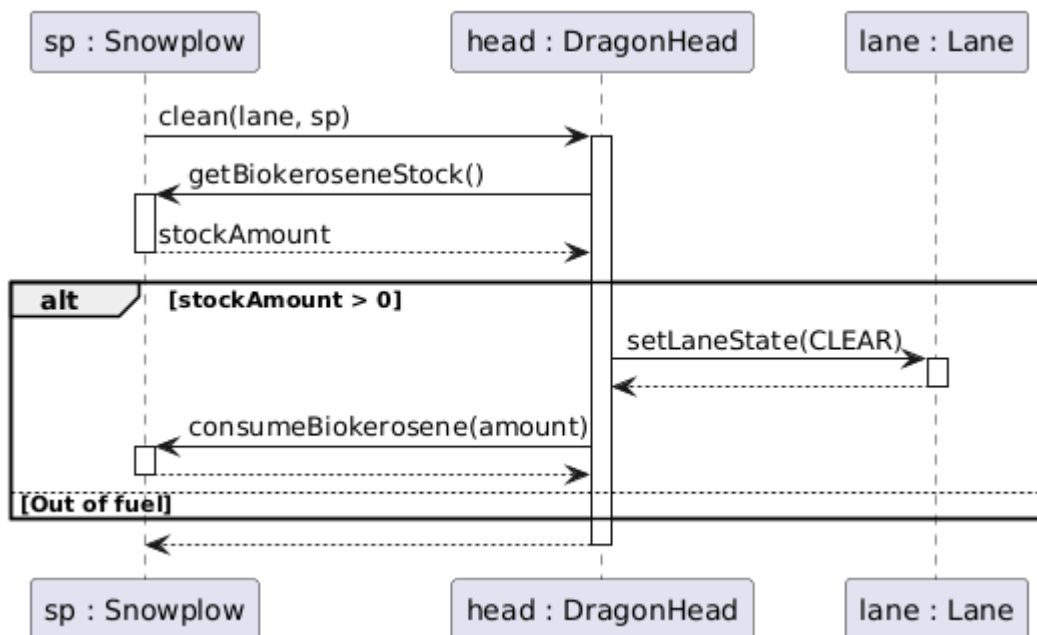
Takarítás hókotróval:



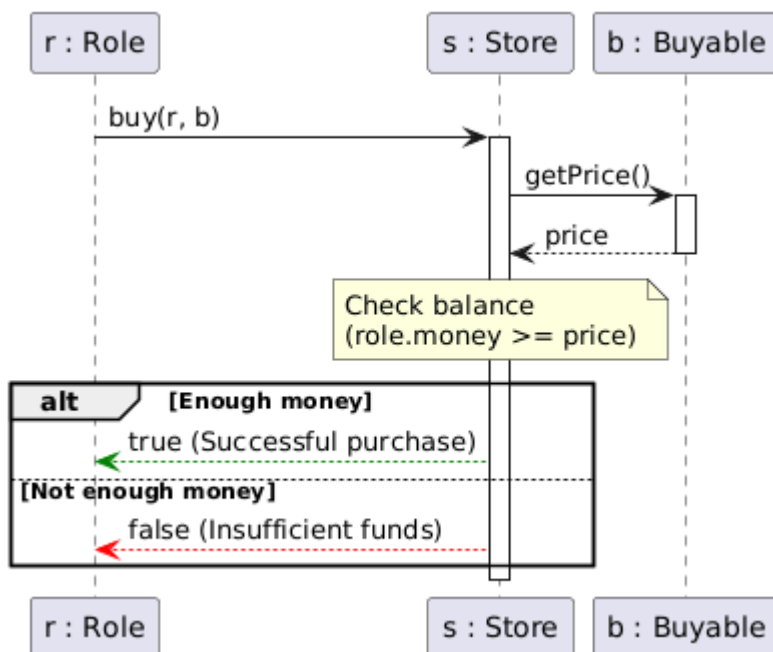
Erőforrás-fogyasztás speciális fejeknél:



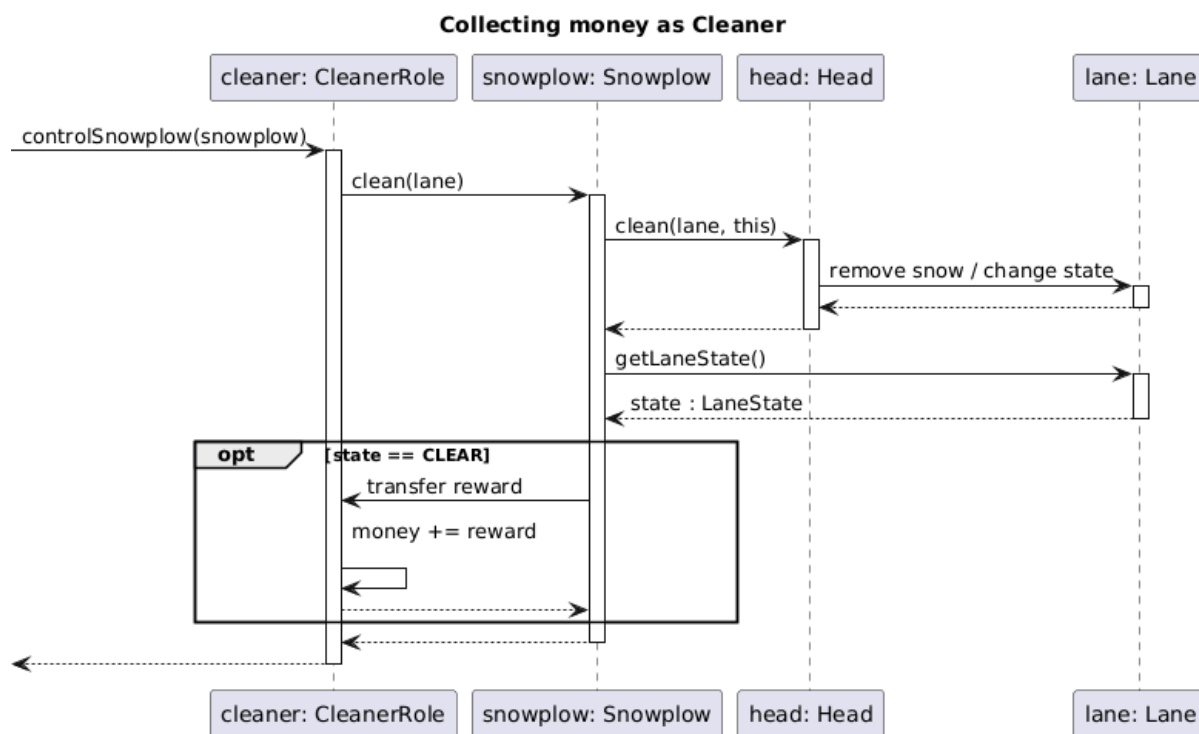
Nyersanyag-fogyasztás takarításakor:



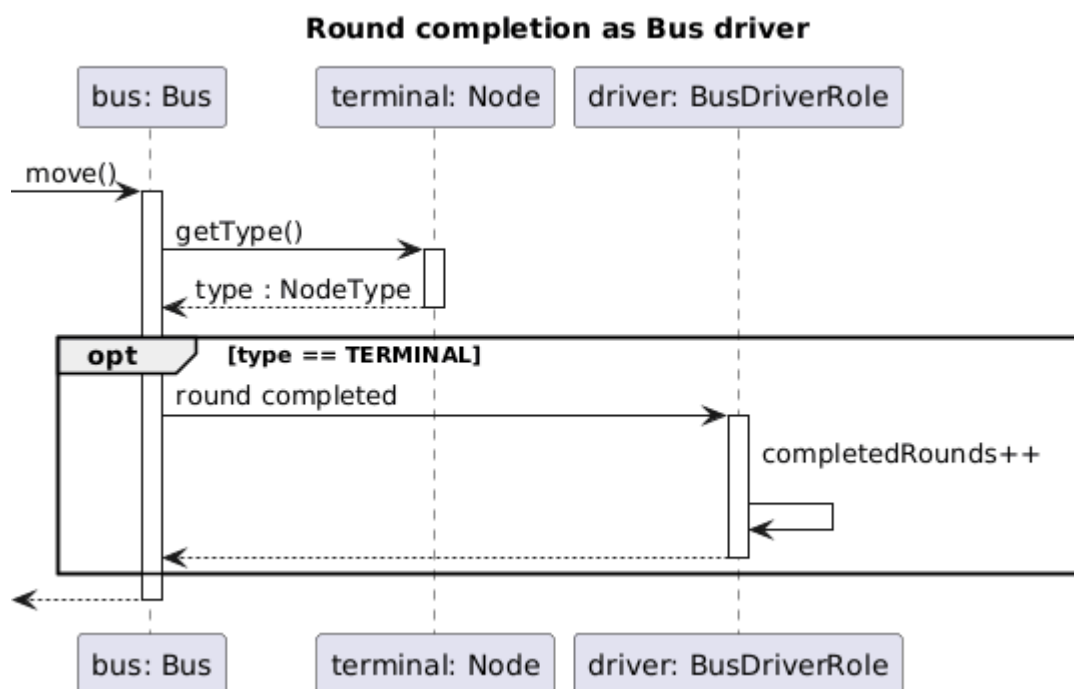
Vásárlási folyamat:



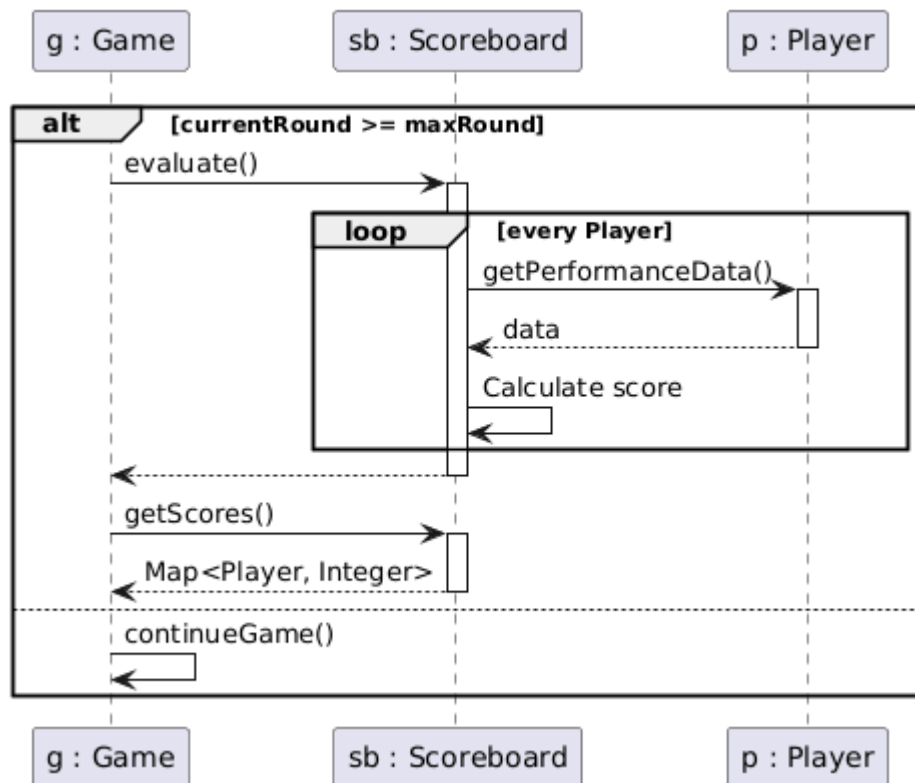
Hókotró jutalmazása:



Buszsofőr kiértékelése:

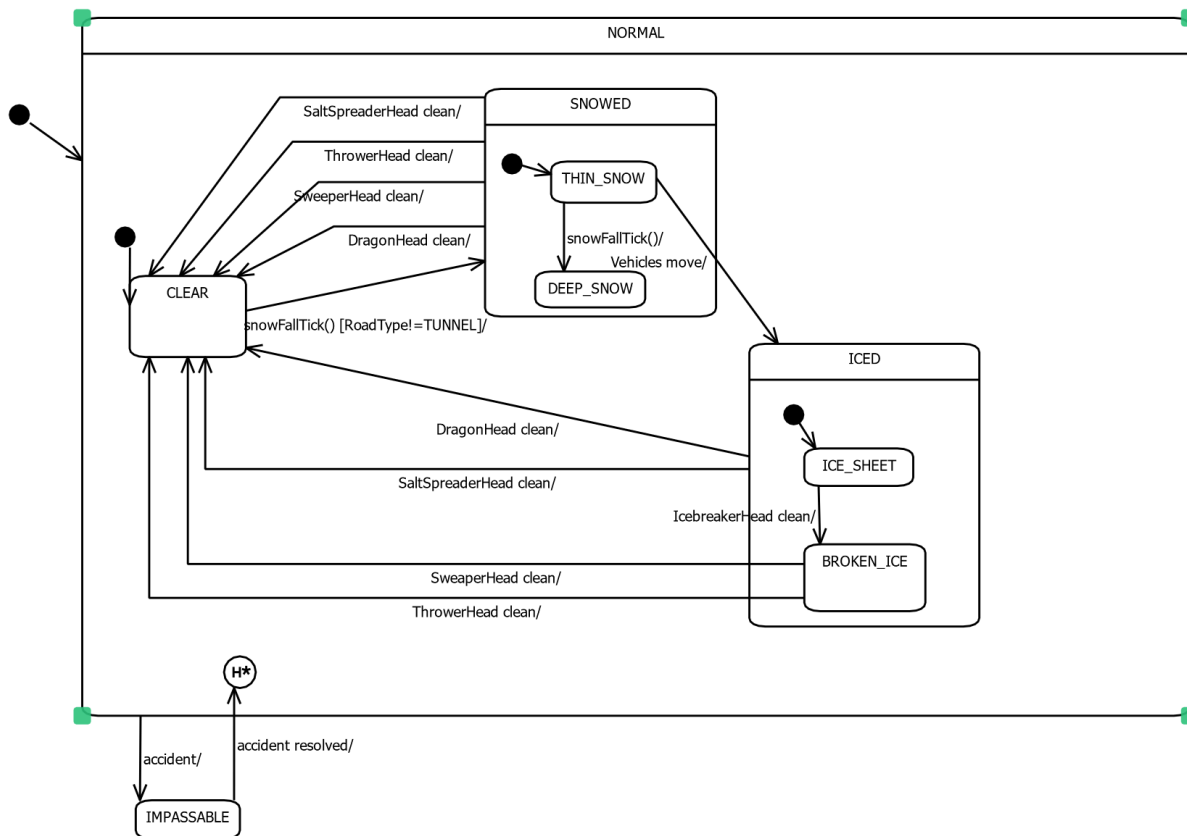


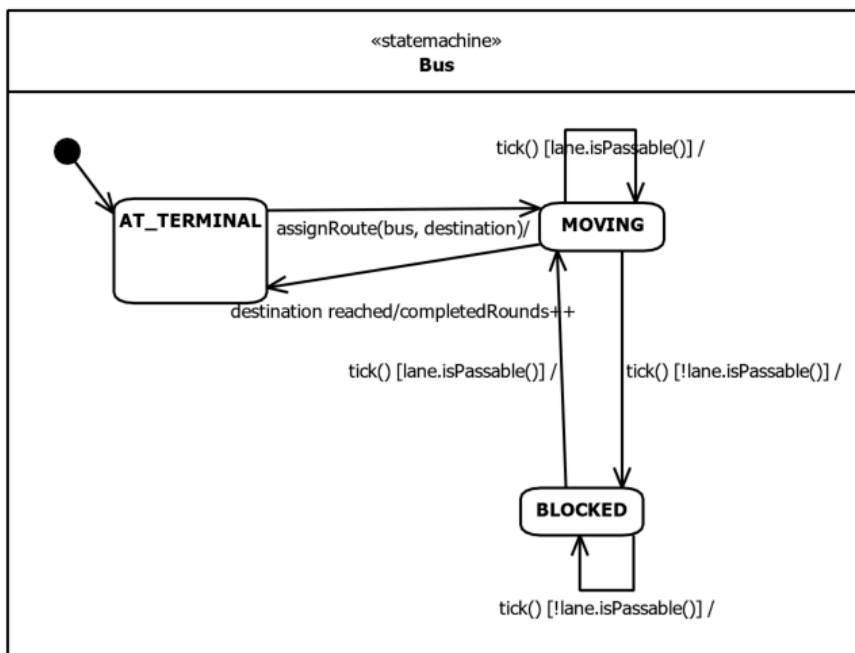
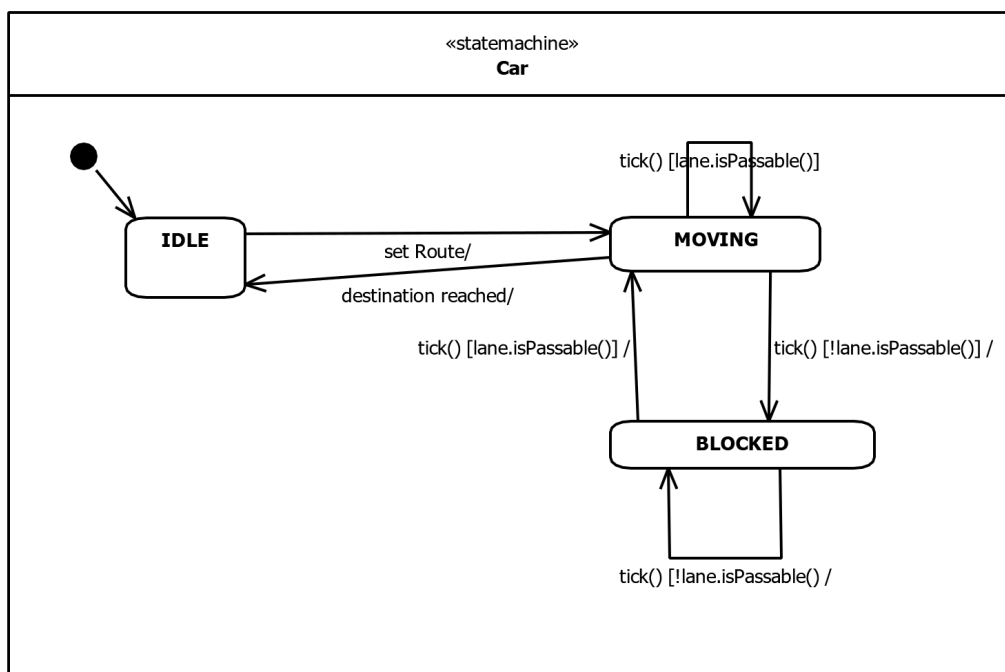
Játék vége és kiértékelés:

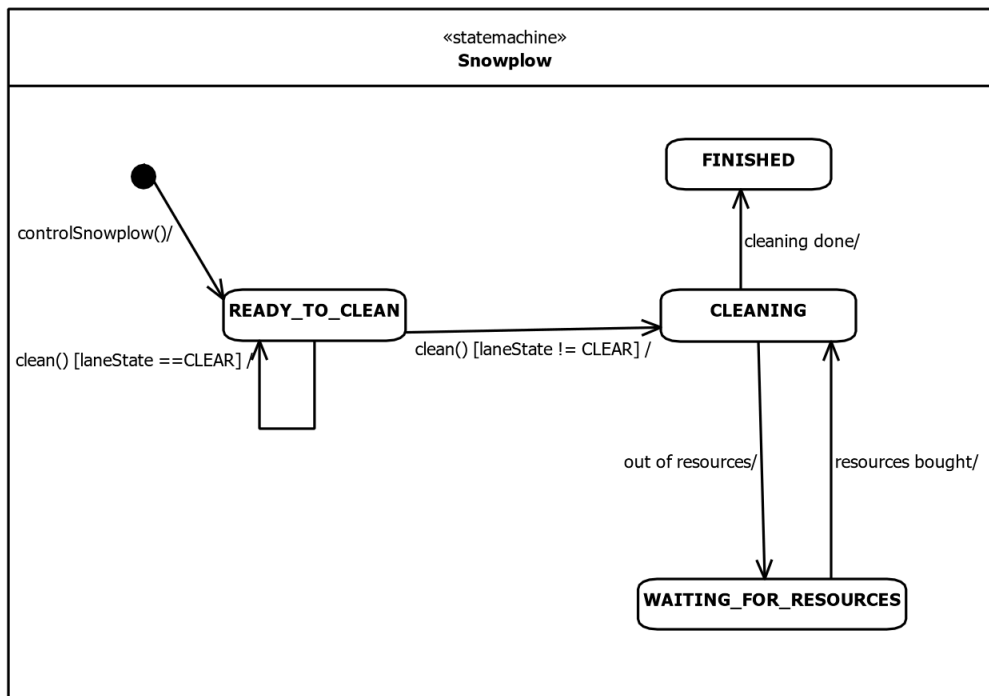


3.5 State-chartok

Út állapotát ábrázoló állapotgép:



Busz haladását ábrázoló állapotgép:**Autó haladását ábrázoló állapotgép:**

Hókotó működését ábrázoló állapotgép:

3.6 Napló

Kezdet	Időtartam	Résztevők	Leírás
2026. 03. 11.	2 óra	Czap Bocskai Tóth Jandó Dénes	Értekezlet az elkészítendő módosításokról
2026. 03. 12.	4 óra	Dénes	Diagramok szerkesztése és javítása, osztályok újragondolása
2026. 03. 12.	4 óra	Czap	Uml szerkesztése, feldarabolása
2026. 03. 14.	2 óra	Bocskai	Új osztályok leírása
2026. 03. 14.	2 óra	Czap	Szekvencia diagram szerkesztése, javítása
2026. 03. 15.	1 óra	Tóth	Javítások átnézése

5. Skeleton tervezése.

23 – githappens

Konzulens:

Haragos Gergő Viktor

Csapattagok

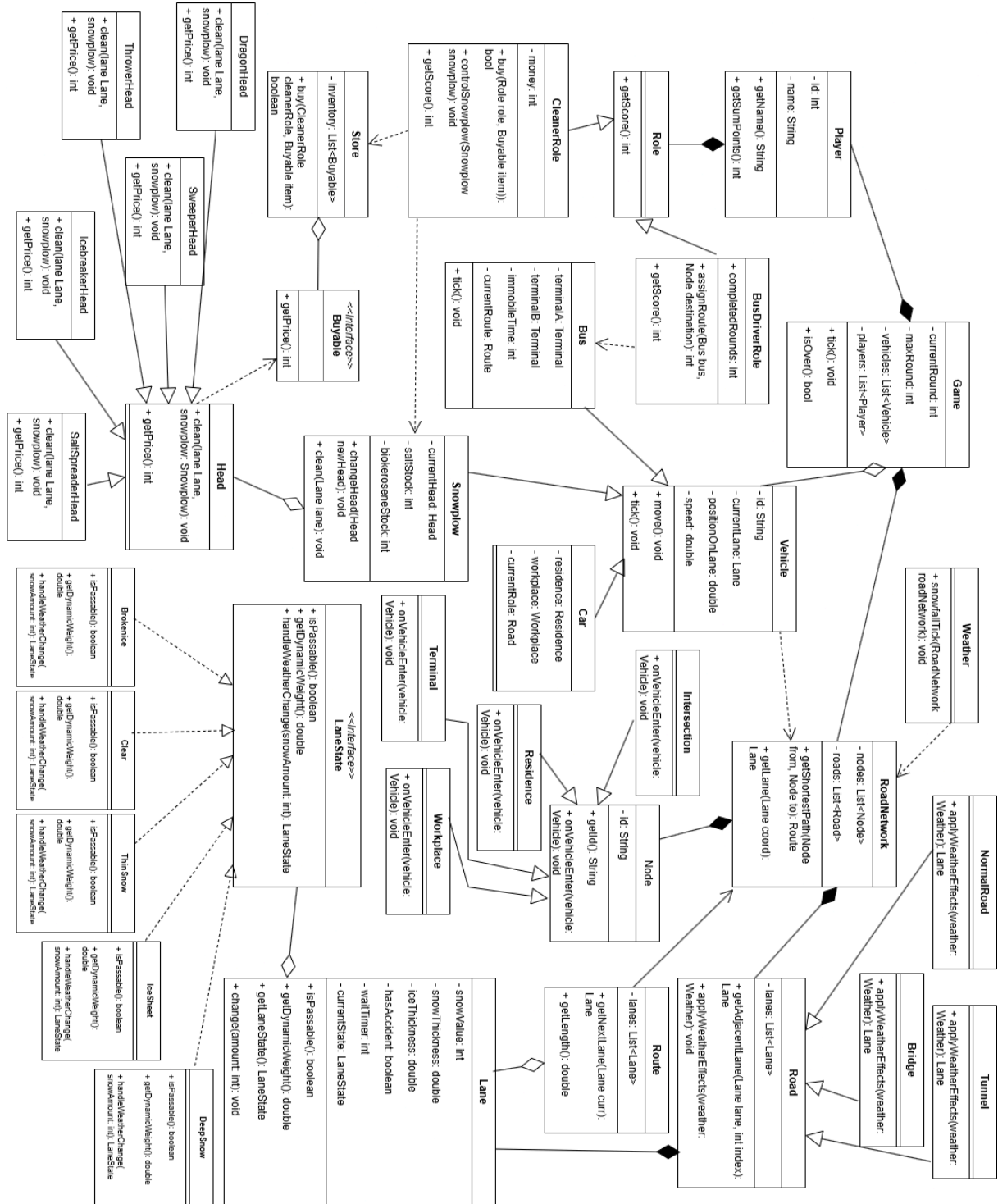
Czap László	NS7V2T	czap.laszlo@edu.bme.hu
Bocskai Zsófia	UGSTW9	bocskaizsofi@gmail.com
Tóth Márton	K01YXA	toth.marton21@gmail.com
Dénes Péter István	PO6MVA	denespeter03tab@gmail.com
Jandó Barnabás Tamás	AWQ77G	jandobarnabas@gmail.com

2026.03.22.

5. Szkeleton tervezése

5.1 Javítások

5.1.1 Osztálydiagram



5.1.2 Osztályok leírása

5.1.2.1 Road

- **Felelősség**

A Road absztrakt osztály két csomópontot összekötő fizikai útszakaszt reprezentál a játékkeren. Feladata az egy irányba haladó sávok összefogása és koordinálása. Emellett támogatja a járművek navigációját azáltal, hogy segít meghatározni a sávok közötti szomszédsági viszonyokat.

- **Össosztályok**

- **Interfészek**

- **Asszociációk**

- **lanes:** List<Lane> – Az adott útszakaszhoz tartozó sávok listája.
- **source:** Node – Az útszakasz kiinduló csomópontja.
- **destination:** Node – Az útszakasz célállomását jelentő csomópont.

- **Attribútumok**

- **lanes:** List<Lane> Az útszakaszt felépítő sávok gyűjteménye.

- **Metódusok**

6. **Lane getAdjacentLane(Lane lane, int index):** Megkeresi a paraméterként kapott sáv melletti szomszédos sávot a megadott index alapján. **A metódus lekérdezéskor ellenőrzi a szomszédos sáv járhatóságát, és csak akkor adja vissza a sávot, ha az akadálymentes.** Ez a metódus teszi lehetővé a járművek számára a sávváltást akadályok vagy balesetek esetén.
7. **void applyWeatherEffects(Weather weather):** Az időjárás hatását alkalmazza az útszakasz sávjaira.

5.1.2.2 Route

- **Felelősség**

A Route osztály egy konkrét útvonaltervet reprezentál, amelyet az úthálózat útvonalkereső algoritmusa állít elő a járművek számára. Feladata a célba jutáshoz szükséges sávok rendezett listájának tárolása. A szimuláció során ez az osztály felelős azért, hogy a jármű haladása közben kiszolgálja ("adagolja") a következő sávot, amelyre a járműnek rá kell hajtania.

- **Össosztályok**

- **Interfészek**

- **Asszociációk**

- **lanes:** List<Lane> – Az útvonalat felépítő sávok rendezett sorozata.

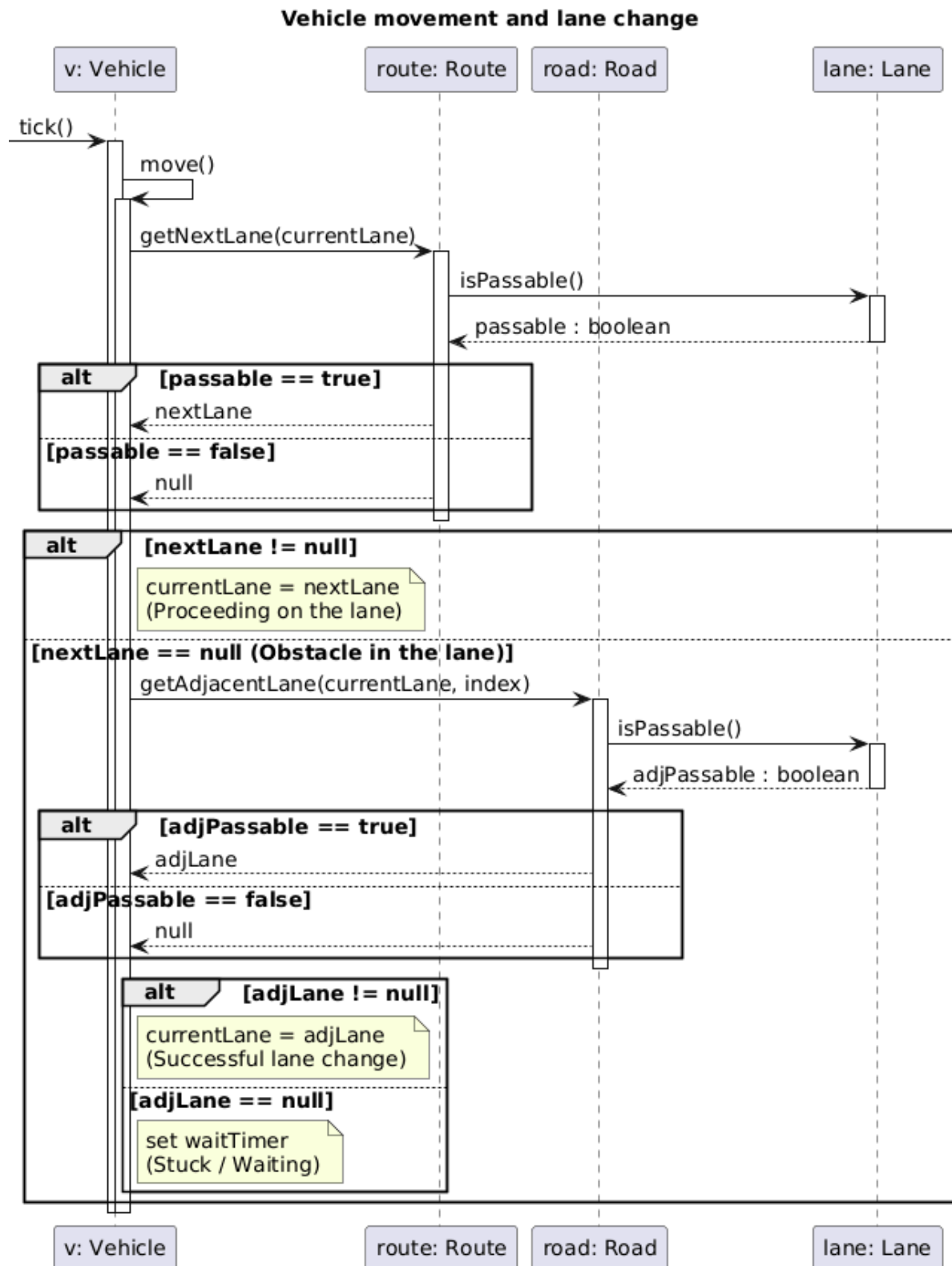
- **Attribútumok**

- **lanes:** List<Lane> – A jármű által bejárandó sávok listája.

- **Metódusok**
- **Lane getNextLane(Lane curr):** Megkeresi a paraméterként kapott aktuális sávot az útvonalban, és visszaadja a soron következő sáv objektumot. **A metódus belsőleg ellenőrzi a sáv járhatóságát is, és ha akadály van rajta, null értékkel tér vissza.** Ezzel irányítja a járművet az útvonal mentén.
- **double getLength():** Visszaadja az útvonal teljes hosszát, amely az útvonalat alkotó sávok hosszának összege.

5.1.3 Szekvencia diagram

Járművek mozgása és sávváltása:



7.1 A szkeleton modell valóságos use-case-ei

7.1.1 Use-case diagram



7.1.2 Use-case leírások

Use-case neve	1. Játék indítása, inicializáció teszt
Rövid leírás	A teszteset ellenőrzi, hogy a játék indulásakor minden szükséges objektum helyesen jön létre: buszok, buszvezetők, autók, hókotrók és takarítók.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: A játék még nem indult el, nincsenek se járművek, se szereplők, se havazás 2. A játékot elindítja a Game.start() 3. Létrejönnek a buszvezetők és hozzájuk tartozó buszok alaphelyzetben 4. Létrejönnek a takarítók és hozzájuk tartozó hókotrók alaphelyzetben 5. Létrejön a civil autók listája

Use-case neve	2. Havazás hatása az útállapotra teszt
Rövid leírás	A teszteset ellenőrzi, hogy a Weather osztály havazás eseménye hogyan rontja le egy adott sáv állapotát.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. A Weather objektum meghívja a snowfallTick(roadNetwork) metódust. 2. A RoadNetwork végigiterál az utakon és sávokon, majd meghívja az applyWeatherEffects() műveletet. 3. A Lane meghívja a jelenlegi állapotán (LaneState) a handleWeatherChange(snowAmount) függvényt. 4. Döntés: mennyi hó esett és mi a jelenlegi állapot? -Ha a sáv Clear volt, és kevés hó esett, az új állapot ThinSnow lesz. -Ha a sáv ThinSnow volt, az új állapot DeepSnow lehet. 5. A sáv állapota frissül a changeState(newState) meghívásával.

Use-case neve	3. Alagút időjárás-mentessége teszt
Rövid leírás	A teszteset ellenőrzi, hogy a havazás (Weather.snowfallTick()) nincs hatással az alagút (Tunnel) típusú utakra.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: Egy Tunnel típusú útszakasz sávja Clear állapotú. 2. A Weather meghívja a snowfallTick(roadNetwork) metódust. 3. A Tunnel megkapja az applyWeatherEffect(weather) hívást. 4. A Tunnel felülírja az alapértelmezett viselkedést, így a benne lévő sáv állapota Clear marad, a hómennyiség nem nő.

Use-case neve	4. Jármű behajtása kereszteződésbe teszt
Rövid leírás	A tesztet ellenőrzi, hogy a jármű megfelelően halad át egy csomóponton..
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: A jármű elérte az aktuális Lane végét. 2. A jármű meghívja a csomóponton az onVehicleEnter(vehicle) metódust. 3. A csomópont meghatározza, hogy a jármű melyik sávon folytathatja az útját. 4. A jármű currentLane attribútuma beállítódik az új sávra. 5. A jármű folytatja útját az új szakaszon.

Use-case neve	5. Jármű sávváltása teszt
Rövid leírás	A tesztet ellenőrzi, hogy a jármű helyesen vált sávot, ha a szomszédos Lane járható.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltételek: - a jármű az L1 sávon halad (currentLane = L1) - L1 sávval szomszédos L2 sáv - L2 sáv járható 2. A jármű megkapja a „vált sávot L2-re” utasítást 3. A jármű ellenőrzi L2 állapotát 4. Döntés: az L2 sáv 1. járható 2. nem járható 5. Ha járható, a jármű az L2 sávba kerül (currentLane = L1) 6. positionOnLane átkerül az L2 megfelelő pontjára 7. Ha nem járható, a jármű az L1 sávban marad

Use-case neve	6. Hókotró haladása teszt
Rövid leírás	A tesztet ellenőrzi, hogy a hókotró helyesen halad Lane-eken.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. A takarító szerepkör mozgatja a hókotró (Snowplow.move()) 2. positionOnLane nő

Use-case neve	7. Takarítás söprő fejjel teszt
Rövid leírás	A tesztet ellenőrzi, hogy a hókotró söprő fejjel megfelelően takarítja az utat.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: A hókotró aktuális feje a söprő fej 2. A CleanerRole a hókotró a takarítani kívánt útszakaszra irányítja.

	3. Döntés: az útszakasz 1. havas (vékony vagy mély hóréteg) 2. jégpáncél 3. feltört jég 4. Ha az útszakasz jégpáncél, a sörpő fej nem tudja eltakarítani. 5. Ha az útszakasz hó vagy feltört jég, a söprő fej oldalra tolja a havat, közvetlenül a hókotró nyomvonalára mellé, azaz a szomszédos sávba vagy az út mellé 6. A CleanerRole megkapja a jutalmat (CleanerRole.money értéke megnő a jutalom összegével) 7. A sörpő fej tiszta útszakaszt hagy maga után
--	---

Use-case neve	8. Takarítás hányó fejjel teszt
Rövid leírás	A tesztet ellenőrzi, hogy a hókotró hányó fejjel megfelelően takarítja az utat.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: A hókotró aktuális feje a hányó fej 2. A CleanerRole a hókotró a takarítani kívánt útszakaszra irányítja. 3. Döntés: az útszakasz 1. havas (vékony vagy mély hóréteg) 2. jégpáncél 3. feltört jég 4. Ha az útszakasz jégpáncél, a hányó fej nem tudja eltakarítani. 5. Ha az útszakasz hó vagy feltört jég, a hányó fej oldalra, de messzire szórja 6. A CleanerRole megkapja a jutalmat (CleanerRole.money értéke megnő a jutalom összegével) 7. A hányó fej tiszta útszakaszt hagy maga után

Use-case neve	9. Takarítás jégtörő fejjel teszt
Rövid leírás	A tesztet ellenőrzi, hogy a hókotró jégtörő fejjel megfelelően takarítja az utat.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: A hókotró aktuális feje a jégtörő fej 2. A CleanerRole a hókotró a takarítani kívánt útszakaszra irányítja. 3. Döntés: az útszakasz 1. havas (vékony vagy mély hóréteg) 2. jégpáncél 3. feltört jég 4. Ha az útszakasz havas vagy feltört jég, a jégtörő fej nem tudja eltakarítani. 5. Ha az útszakasz jégpáncél, a jégtörő fej feltöri a jeget 6. A jégtörő fej feltört jeges útszakaszt hagy maga után

Use-case neve	10. Takarítás sószóró fejjel teszt
Rövid leírás	A tesztet ellenőrzi, hogy a hókotró sószóró fejjel megfelelően takarítja az utat.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: A hókotró aktuális feje a sószóró fej 2. A CleanerRole a hókotró a takarítani kívánt útszakaszra irányítja. 3. Döntés: van megfelelő mennyiségű só a hókotróban 1. nem 2. igen 4. Ha nincs elég só, a sószóró fej hatástalanná válik, nem változtat az úton 5. Ha van elég só, a sószóró fej beszórja sóval az utat 6. A só elolvasztja a havat vagy jeget 7. A CleanerRole megkapja a jutalmat (CleanerRole.money értéke megnő a jutalom összegével) 8. A sószóró fej tiszta útszakaszt hagy maga után

Use-case neve	11. Takarítás sárkány fejjel teszt
Rövid leírás	A tesztet ellenőrzi, hogy a hókotró sárkány fejjel megfelelően takarítja az utat.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: A hókotró aktuális feje a sárkány fej 2. A CleanerRole a hókotró a takarítani kívánt útszakaszra irányítja. 3. Döntés: van megfelelő mennyiségű biokerozin a hókotróban 1. nem 2. igen 4. Ha nincs elég biokerozin, a sárkány fej hatástalanná válik, nem változtat az úton 5. Ha van elég biokerozin, a sárkány fej gázturbinával elolvasztja a havat vagy jeget 6. A CleanerRole megkapja a jutalmat (CleanerRole.money értéke megnő a jutalom összegével) 7. A sárkány fej tiszta útszakaszt hagy maga után

Use-case neve	12. Sikeres hókotró vásárlás teszt
Rövid leírás	A tesztet ellenőrzi, hogy a CleanerRole megfelelően vásárol hókotró a Store-ból, ha van elég pénze.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: $\text{CleanerRole.money} \geq \text{item.getPrice}()$ (a CleanerRole-nak van elég pénze a hókotró megvételéhez)

	2. A CleanerRole buy(cleanerRole, item) műveletet hív a Store-on 3. A Store ellenőrzi a CleanerRole pénzét 4. A Store levonja a hókotró árát a CleanerRole pénzéből 5. A CleanerRole listájához hozzáadódik az újabb hókotró
--	---

Use-case neve	13. Sikeres kotrófej vásárlás teszt
Rövid leírás	A tesztet ellenőrzi, hogy a CleanerRole megfelelően vásárol kotrófejet (söprő, hányó, jégtörő, sósórozó, sárkány fejet) a Store-ból, ha van elég pénze.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: CleanerRole.money $\geq$ item.getPrice() (a CleanerRole-nak van elég pénze a fej megvételéhez) 2. A CleanerRole buy(cleanerRole, item) műveletet hív a Store-on 3. A Store ellenőrzi a CleanerRole pénzét 4. A Store levonja a fej árát a CleanerRole pénzéből 5. A CleanerRole megkapja az új fejet

Use-case neve	14. Sikeres biokerozin vásárlás teszt
Rövid leírás	A tesztet ellenőrzi, hogy a CleanerRole megfelelően vásárol biokerozint a Store-ból, ha van elég pénze.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: CleanerRole.money $\geq$ item.getPrice() (a CleanerRole-nak van elég pénze az áru megvételéhez) 2. A CleanerRole buy(cleanerRole, item) műveletet hív a Store-on 3. A Store ellenőrzi a CleanerRole pénzét 4. A Store levonja a biokerozin árát a CleanerRole pénzéből 5. A CleanerRole hókotrójának bokeroseneStock értéke a megvásárolt biokerozin mennyiségével nő

Use-case neve	15. Sikeres só vásárlás teszt
Rövid leírás	A tesztet ellenőrzi, hogy a CleanerRole megfelelően vásárol a Store-ból, ha van elég pénze.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: CleanerRole.money $\geq$ item.getPrice() (a CleanerRole-nak van elég pénze a só megvételéhez) 2. A CleanerRole buy(cleanerRole, item) műveletet hív a Store-on 3. A Store ellenőrzi a CleanerRole pénzét 4. A Store levonja a só árát a CleanerRole pénzéből 5. A CleanerRole hókotrójának saltStock értéke a megvásárolt só mennyiségével nő

Use-case neve	16. Sikertelen vásárlás teszt
Rövid leírás	A tesztet ellenőrzi, hogy a CleanerRole nem tud vásárolni a Store-ból, ha nincs elég pénze.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: CleanerRole.money < item.getPrice() (a CleanerRole-nak nincs elég pénze az áru megvételéhez) 2. A CleanerRole buy(cleanerRole, item) műveletet hív a Store-on 3. A Store ellenőrzi a CleanerRole pénzét 4. A Store visszautasítja a vásárlást 5. A CleanerRole pénze nem változik, a vásárlás sikertelen

Use-case neve	17. Kotrófej cseréje teszt
Rövid leírás	A tesztet ellenőrzi, hogy sikeresen cseréli a CleanerRole szerepkör a kívánt hókotró takarító fejét.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. A takarító kiválasztja a hókotrót (Snowplow) és a kívánt új fejet (newHead) a meglévő fejek közül 2. A CleanerRole szerepkör meghívja a fej cseréje műveletet az adott hókotrón (Snowplow.changeHead(newHead)) 3. A hókotróról lekerül a régi fej és az új fej jelenik meg rajta (a Snowplow.currentHead értéke newHead-re változik)

Use-case neve	18. Nyersanyag kifogyás teszt
Rövid leírás	A speciális fej működése megszűnik nyersanyag hiányában.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. A hókotró használ egy speciális fejet 2. Ellenőrzi a készletet 3. Ha 0: • A clean() hatástalan 4. A játékos dönt: • Fejet cserél • Vagy vásárol

Use-case neve	19. Pontszerzés takarítással teszt
Rövid leírás	A takarító pénzt kap minden megtisztított sávért..
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. A hókotró megtisztít egy sávot 2. A rendszer felismeri a tisztítást 3. A takarító pénze nő 4. A pontszám frissül

Use-case neve	20. Busz útvonalhoz rendelése teszt
----------------------	-------------------------------------

Rövid leírás	A teszteset ellenőrzi, hogy a BusDriverRole sikeresen hozzá tud rendelni egy célállomást és a legrövidebb útvonalat a buszhoz.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: Adott egy Bus és egy célállomás (Node destination). 2. A BusDriverRole meghívja az assignRoute(bus, destination) műveletet. 3. A rendszer a RoadNetwork.getShortestPath(from, to) segítségével lekéri a legrövidebb útvonalat (Route). 4. A Bus currentRoute attribútuma beállítódik a kiszámolt útvonalra. 5. A Bus elindul a cél felé, a tick() hatására a move() lefut.

Use-case neve	21. Busz közlekedés teszt
Rövid leírás	A busz a kijelölt útvonal mentén halad, figyelembe véve az út állapotát és az esetleges akadályokat.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. A játék tick-et léptet. 2. A busz lekéri a következő sávot a Route-ból. 3. A busz ellenőrzi a sáv járhatóságát. 4. Ha járható → továbbhalad. 5. Ha nem járható → sávváltást próbál. 6. Ha nem lehetséges → megáll és vár. 7. Baleset esetén immobileTime aktiválódik.

Use-case neve	22. Busz forduló teljesítése teszt
Rövid leírás	A teszteset ellenőrzi, hogy a busz helyesen növeli a fordulószámot, amikor eléri a végállomást.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltételek: - Bus TerminalA → TerminalB irányba halad - Bus.positionOnLane = laneLength 2. Bus.move() 3. Bus felismeri a végállomást 4. BusDriverRole.completedRounds értéke nő. 5. Bus új útvonalat talál vissza a TerminalA-ba 6. Bus elindul a TerminalA-ba

Use-case neve	23. Autó haladása járható sávban teszt
Rövid leírás	A teszteset ellenőrzi, hogy a Car képes-e az útvonalon következő sávba lépni, ha az járható állapotban van.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: A Car előtt lévő következő Lane állapota járható (pl. Clear állapotú, így az isPassable() == true). 2. A Car meghívja a saját move() metódusát a tick() hatására.

	3. A jármű lekérdezi a következő sáv adatait. 4. A rendszer ellenőrzi a következő sáv isPassable() értékét. 5. Mivel az érték true, a Car positionOnLane és currentLane értéke frissül. 6. Az autó sikeresen továbbhalad.
--	--

Use-case neve	24. Autó elakadása járhatatlan úton teszt
Rövid leírás	A tesztet ellenőrzi, hogy az autó megáll-e, ha az előtte lévő sáv járhatatlanná válik (DeepSnow, baleset).
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: A Car előtt lévő következő Lane állapota járhatatlan (isPassable() == false). 2. A Car meghívja a move() metódust a tick() hatására. 3. A jármű lekérdezi a következő sáv isPassable() értékét. 4. Mivel az érték false, a Car nem lép át az új sávra. 5. Az autó sebessége 0-ra csökken, várakozni kezd (esetleg a waitTimer megnő).

Use-case neve	25. Autó célba érése teszt
Rövid leírás	A tesztet ellenőrzi, hogy mi történik, amikor egy civil autó sikeresen eléri a célállomását (lakóhely vagy munkahely).
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: A Car az útvonala legutolsó sávjának a végére ér. 2. A Car meghívja az onVehicleEnter(vehicle) metódust a cél csomóponton. 3. A csomópont felismeri, hogy az autó a céljához ért. 4. Az autó kikerül a forgalomból (eltűnik a sávból, esetleg ideiglenesen inaktívvá válik).

Use-case neve	26. Autó útvonalának újratervezése akadály esetén
Rövid leírás	A tesztet ellenőrzi, hogy ha a legrövidebb út járhatatlanná válik (baleset vagy mély hó miatt), az autó képes-e alternatív útvonalat keresni.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: A Car halad a currentRoute mentén. 2. A következő sáv állapota járhatatlanná változik. 3. A Car észleli az akadályt, és új útvonalat kér a getShortestPath() segítségével. 4. A RoadNetwork figyelembe veszi a megnövekedett dinamikus súlyokat és az akadályt, majd egy alternatív útvonalat ad vissza. 5. A Car ezen az új útvonalon folytatja az útját.

Use-case neve	27. Jégpáncél kialakulása teszt
----------------------	---------------------------------

Rövid leírás	A teszteset ellenőrzi, hogy ha kellő számú autó halad át egy vékony hóval (ThinSnow) borított sávok, akkor a sáv állapota jégpáncéllá (IceSheet) változik-e.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: Adott egy sáv ThinSnow állapottal 2. Egy Car objektum meghívja a move() metódust és áthalad a sávon. 3. A sáv belső számlálója (ami a taposást méri) növekszik. 4. További autók haladnak át a sávon, amíg a taposási határértéket el nem érik. 5. A sáv állapota automatikusan frissül ThinSnow-ról IceSheet-re. 6. Az útszakasz járható marad, de csúszóssá válik (balesetveszély megnő).

Use-case neve	28. Ütközés: autó-autó teszt
Rövid leírás	A teszteset ellenőrzi, hogy két autó ütközése esetén a Lane járhatatlanná válik.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltételek: - A és B autó csúszás miatt összeütköznek 2. A.move() 3. B.move() 4. Ütközés detektálása 5. Lane.hasAccident = true 6. Lane-hez tartozó LaneState Impassable 7. Mindkét autó megáll

Use-case neve	29. Ütközés: autó-busz teszt
Rövid leírás	A teszteset ellenőrzi, hogy autó és busz ütközése esetén a busz mozgásképtelenné válik.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltételek: - A autó és B busz csúszás miatt összeütköznek 2. A.move() 3. B.move() 4. Ütközés detektálása 5. Lane.hasAccident = true 6. Lane-hez tartozó LaneState Impassable 7. Az autó megáll 8. Bus.immobileTime beállítása, a busz mozgásképtelen adott ideig

Use-case neve	30. Ütközés: busz-busz teszt
Rövid leírás	A teszteset ellenőrzi, hogy két busz ütközése esetén mindkettő mozgásképtelenné válik.

Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltételek: - A és B busz csúszás miatt összeütköznek 2. A.move() 3. B.move() 4. Ütközés detektálása 5. Lane.hasAccident = true 6. Lane-hez tartozó LaneState Impassable 7. Mindkét busz immobileTime értéket kap, mozgásképtelen adott ideig

Use-case neve	31. Busz mozgásképtelenség megszűnése teszt
Rövid leírás	A tesztet ellenőrzi, hogy egy balesetet szenvedett busz a megadott idő letelte után újra képes-e elindulni.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Előfeltétel: A Bus immobileTime értéke nagyobb, mint 0 egy korábbi ütközés miatt. 2. A Game meghívja a Bus.tick() metódust. 3. A Bus nem mozdul, de az immobileTime értéke csökken eggyel. 4. Ezt folyamatosan ismételjük addig, amíg az immobileTime eléri a 0-t. 5. A következő tick() híváskor a Bus újra végrehajtja a move() metódust, és elindul.

Use-case neve	32. Játék kiértékelése teszt
Rövid leírás	A tesztet ellenőrzi, hogy a játék végén helyesen számolódnak ki a buszvezetők és takarítók pontjai.
Aktorok	Tesztelő, Skeleton
Forgatókönyv	1. Game.end() meghívódik 2. Buszvezető szerepkörök pontszámát kiszámolja a turnCount értékekből 3. Takarító szerepkörök pontszámát kiszámolja a vásárlásokból 4. A játékosok szerepköreinek pontszámait egyesíti 5. A játék lezárul

Use-case neve	33. Útvonalkereső algoritmus, legrövidebb út tesztje
Rövid leírás	A tesztet tisztán azt ellenőrzi, hogy a RoadNetwork.getShortestPath(from, to) metódus valóban a legkisebb, legoptimálisabb útvonalat adja-e vissza egy olyan hálózatban, ahol az egyik rövidebb út járhatatlan (pl. baleset vagy mély hó miatt).
Aktorok	Tesztelő, Skeleton

Forgatókönyv	1. Előfeltétel: Adott egy úthálózat A és B végponttal, amihez két útvonal is vezet: • Útvonal 1: Fizikailag rövidebb, de az egyik sávja járhatatlan (isPassable() == false baleset miatt). • Útvonal 2: Fizikailag hosszabb (több sávból áll), de minden sávja tiszta (Clear állapotú, alacsony dinamikus súly). 2. A Tesztelő meghívja a RoadNetwork.getShortestPath(A, B) metódust. 3. A RoadNetwork gráfkereső algoritmus kiértékeli a sávok dinamikus súlyait és járhatóságát. 4. A rendszer felismeri, hogy az Útvonal 1 járhatatlan (vagy végtelenül nagy a súlya). 5. A metódus visszatérési értéként (Route) a tiszta, de hosszabb Útvonal 2-t adja vissza. 6. A Tesztelő ellenőrzi, hogy a kapott Route valóban az Útvonal 2 sávjait tartalmazza-e.
---------------------	---

7.2 A szkeleton kezelői felületének terve, dialógusok

A szkeleton program terminál jellegű kezelői felülettel rendelkezik. A felhasználó, mint tesztelő, előre definiált parancsokat adhat meg, amelyek egy-egy teszt szekvenciát indítanak el. A cél a rendszer működésének ellenőrzése a szekvencia diagramok alapján.

A program indításakor az alábbi szöveg jelenik meg:

Snowplow Skeleton TestProgram

For help type help and press enter after the > mark.

For exit type exit after the > mark.

1. Segítség kérése

>help

A parancs hatására egy leírás jelenik meg, amely ismerteti az elérhető parancsokat és azok használatát:

- Write commands after the > mark.
- help: Lists all available commands and their usage.
- ls: Lists all available test cases with their numbers.
- run test «test_number»: Runs the selected test sequence.
- exit: Terminates the Snowplow Skeleton TestProgram.

2. Tesztek listázása

>ls

1. Játék indítása, inicializáló teszt
2. Havazás hatása az útállapotra teszt
3. Alagút időjárás-mentessége teszt
4. Jármű behajtása kereszteződésbe teszt

5. Jármű sávváltás teszt
6. Hókotró haladása teszt
7. Takarítás söprő fejjel teszt
8. Takarítás hányó fejjel teszt
9. Takarítás jégtörő fejjel teszt
10. Takarítás sószóró fejjel teszt
11. Takarítás sárkány fejjel teszt
12. Sikeres hókotró vásárlás teszt
13. Sikeres kotrófej vásárlás teszt
14. Sikeres biokerozin vásárlás teszt
15. Sikeres só vásárlás teszt
16. Sikertelen vásárlás teszt
17. Kotrófej cseréje teszt
18. Nyersanyag kifogyás teszt
19. Pontszerzés takarítással teszt
20. Busz útvonalhoz rendelés teszt
21. Busz közlekedés teszt
22. Busz forduló teljesítése teszt
23. Autó haladása járható sávban teszt
24. Autó elakadása járhatatlan úton teszt
25. Autó célba érése teszt
26. Autó útvonalának újratervezése akadály esetén
27. Jégpáncél kialakulása teszt
28. Ütközés: autó-autó teszt
29. Ütközés: autó-busz teszt
30. Ütközés: busz-busz teszt
31. Busz mozgásképtelenség megszűnése teszt
32. Játék kiértékelése teszt
33. Útvonalkereső algoritmus, legrövidebb út tesztje

3. run test «teszt_száma»

> run test 4

A parancs hatására a kiválasztott tesztszekvencia végrehajtódik.

A kimenet a szekvenciadiagramoknak megfelelő formában jelenik meg, ahol:

- >>> *jelöli a metódushívást,*
- <<< *jelöli a visszatérést,*
- [STATE] *jelöli az állapotváltozást*

Példa kimenet:

```
>>>[Vehicle : Car].changeLane(targetLane)
>>>[Lane : targetLane].isPassable()
<<<return true
>>>[Vehicle : Car].setCurrentLane(targetLane)
[STATE] Vehicle.currentLane = targetLane
<<<return success
```

Döntési pontok kezelése

A tesztek futása során a program bizonyos pontokon döntést kérhet a tesztelőtől. Ilyenkor az alábbi formátum jelenik meg:

Decision: A sáv járható?

1: Igen

2: Nem

>>

A tesztelő a >> jel után adhatja meg a választ:

>> 1

Ez esetben az „igen” ág hajtódik végre.

Példa teljes dialógus:

```
> run test 2
```

```
>>>[Weather].snowfall()
>>>[Road].applySnow()
>>>[Lane].updateState()
<<<return ThinSnow
[STATE] Lane.state = ThinSnow
<<<return done
```

[RESULT] Teszt sikeresen lefutott.

4. exit

```
>exit
```

A parancs hatására a program leáll.

A felület célja

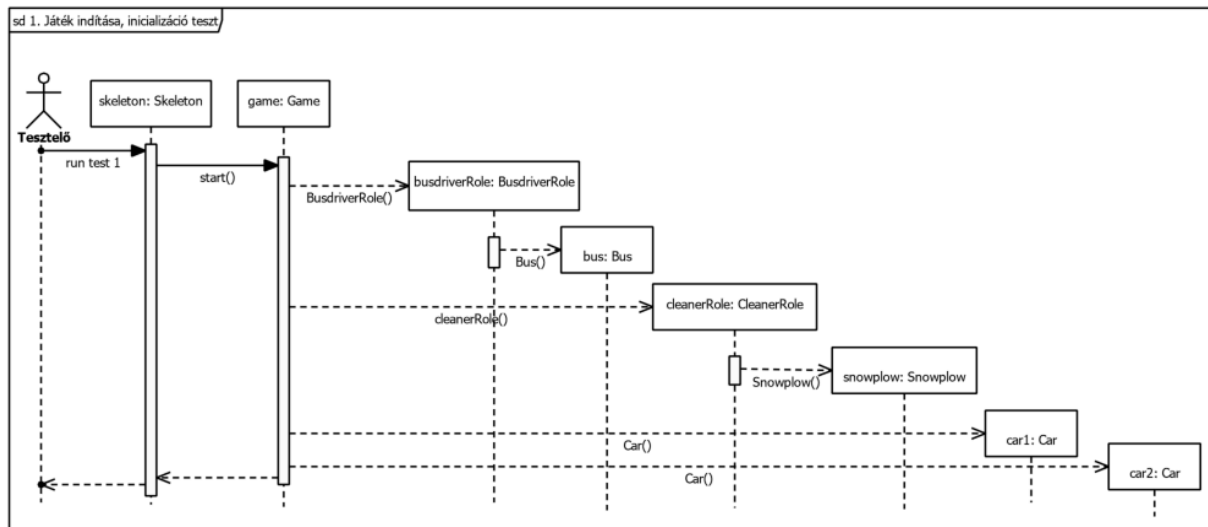
A kezelői felület célja, hogy a tesztszekvenciák végrehajtása során a rendszer működése részletesen és követhető módon jelenjen meg. A metódushívások, visszatérések és állapotváltozások formátuma lehetővé teszi a szekvenciadiagramokkal való közvetlen összehasonlítást.

Megjegyzés a szekvenciák és inicializálás kapcsolatáról

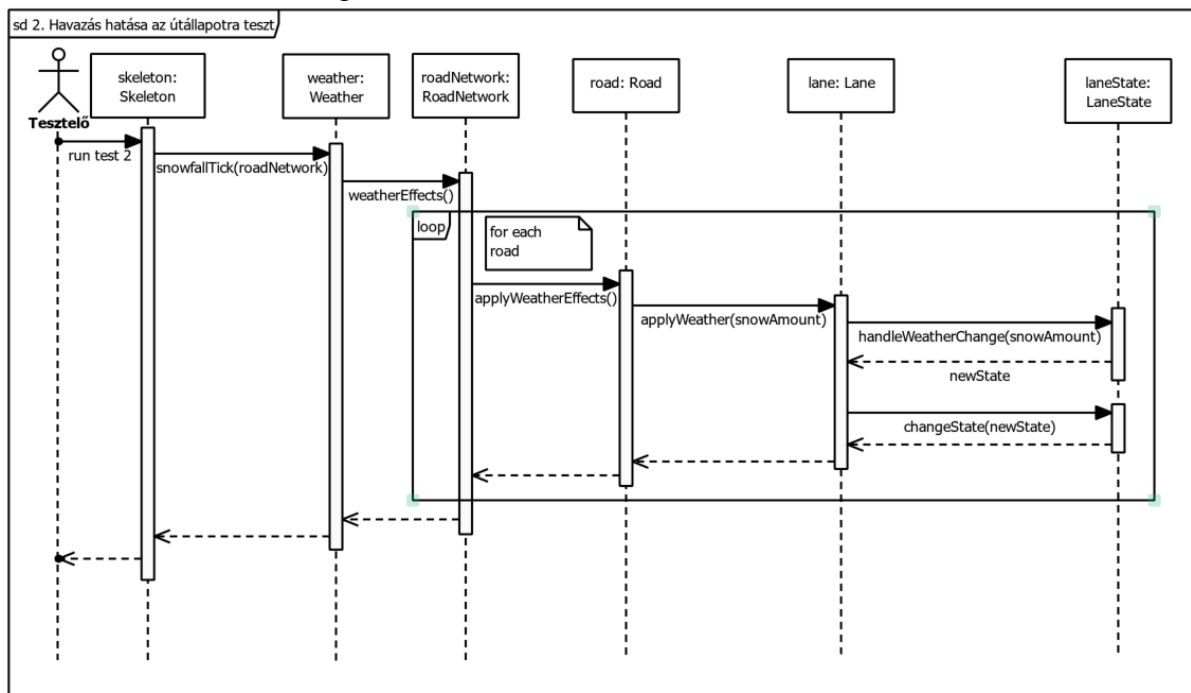
Minden tesztszekvencia egy előre definiált állapotból indul. Egyes tesztek közös inicializáló kommunikációs diagramot használhatnak, azokban minden esetben egyértelműen meg van adva, hogy mely tesztszekvencia mely inicializáló diagramhoz tartozik.

7.3 Szekvencia diagramok a belső működésre

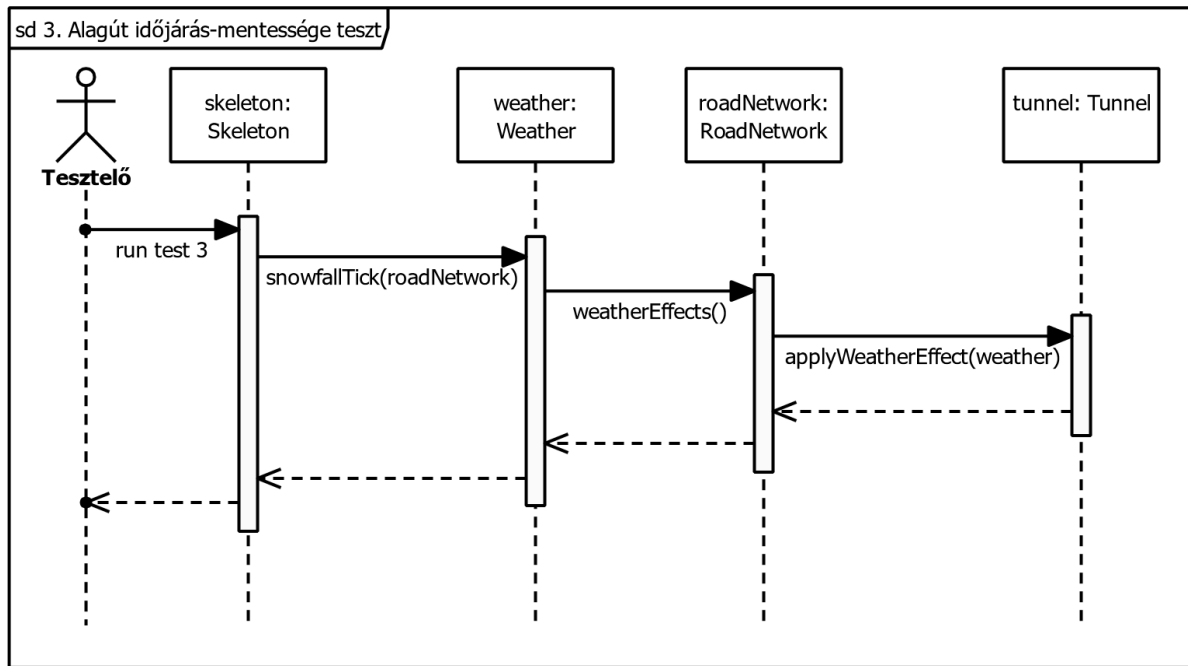
1. Játék indítása, inicializáció teszt



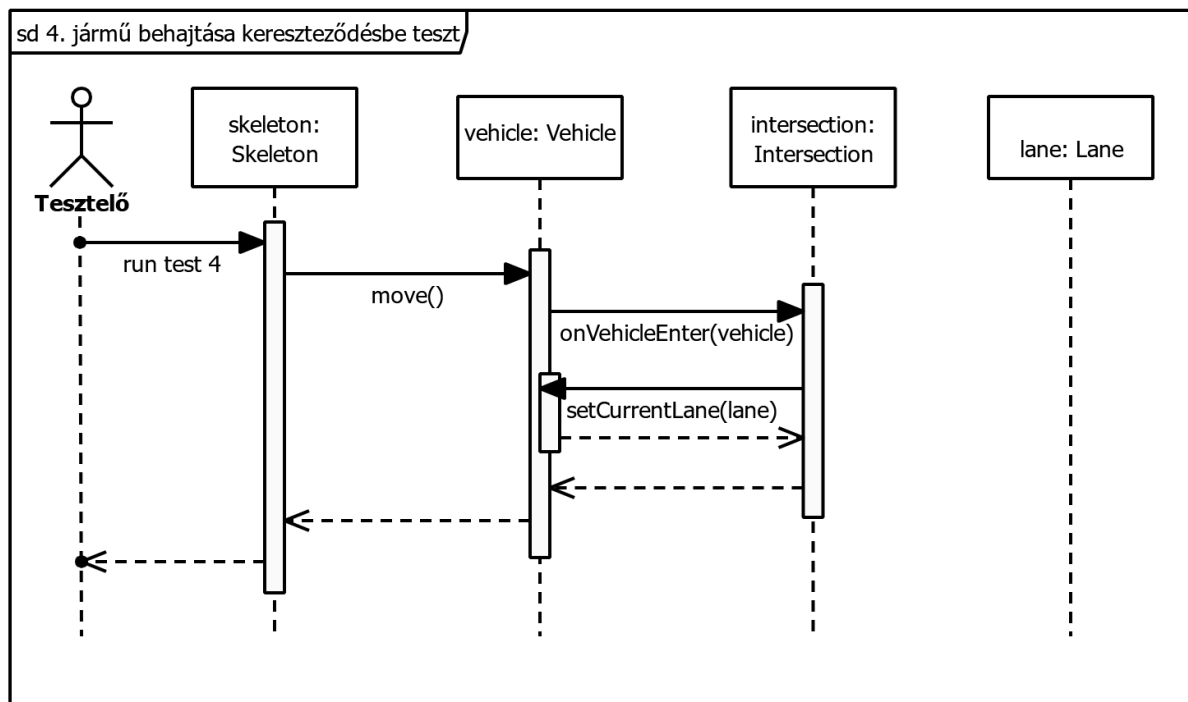
2. Havazás hatása az útállapotra teszt



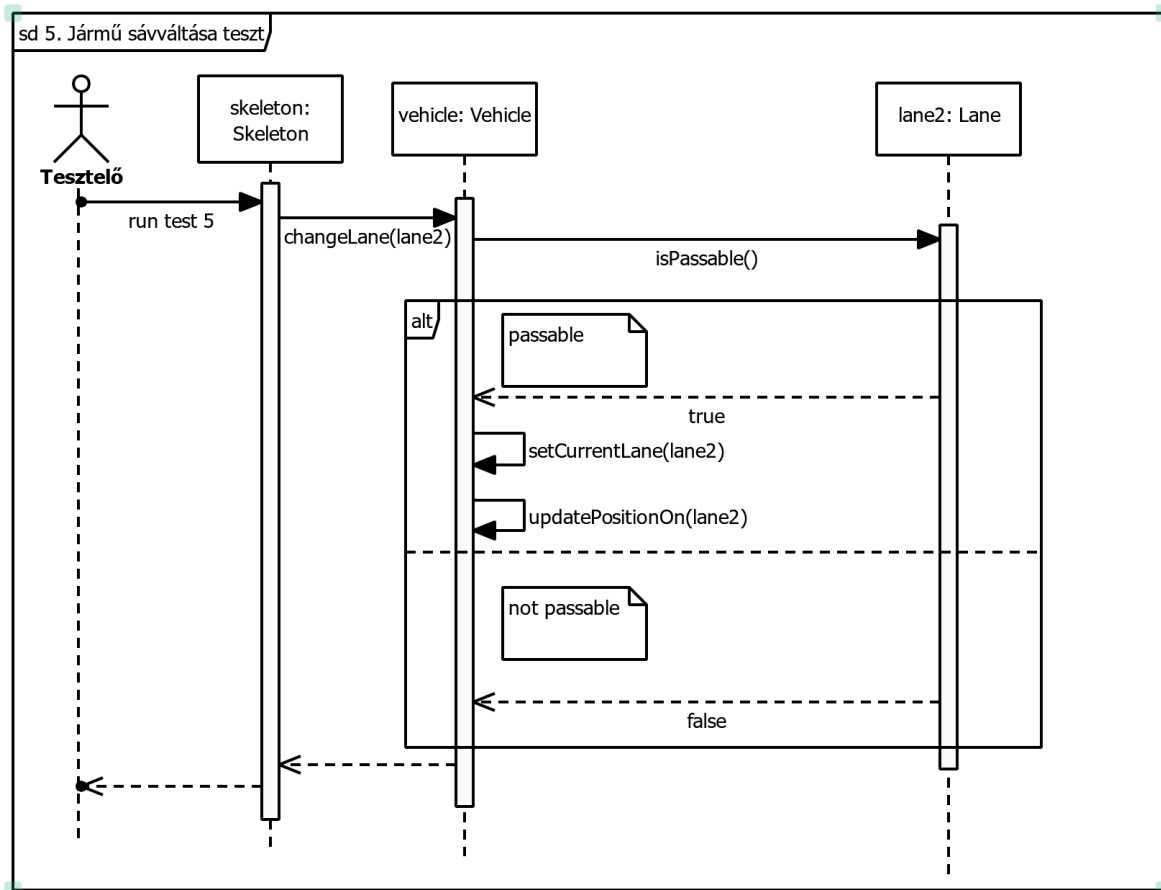
3. Alagút időjárás-mentessége teszt



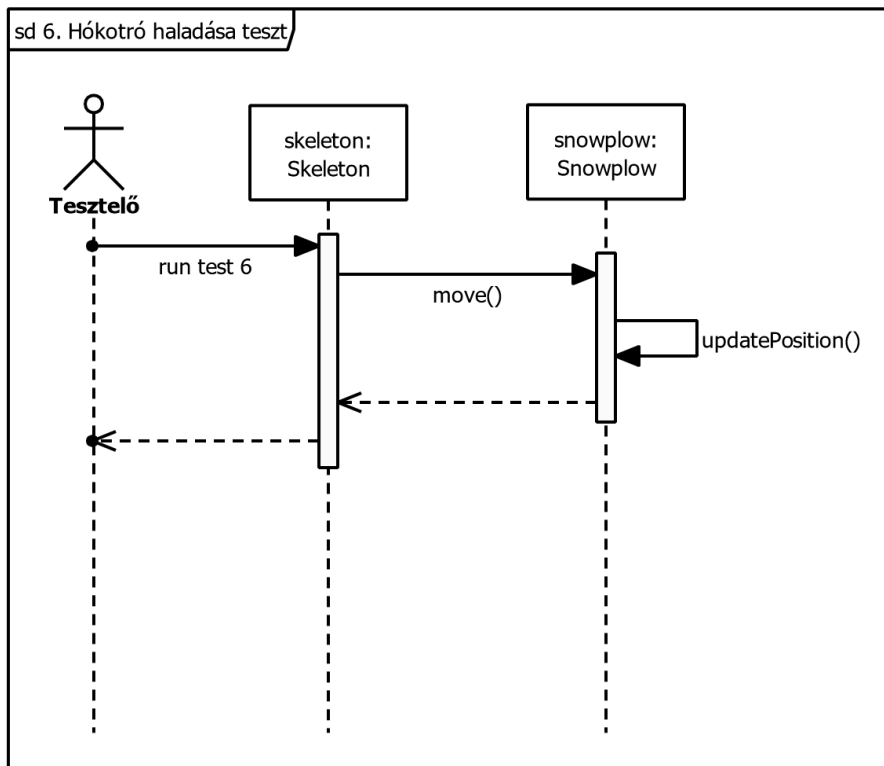
4. Jármű behajtása kereszteződésbe teszt



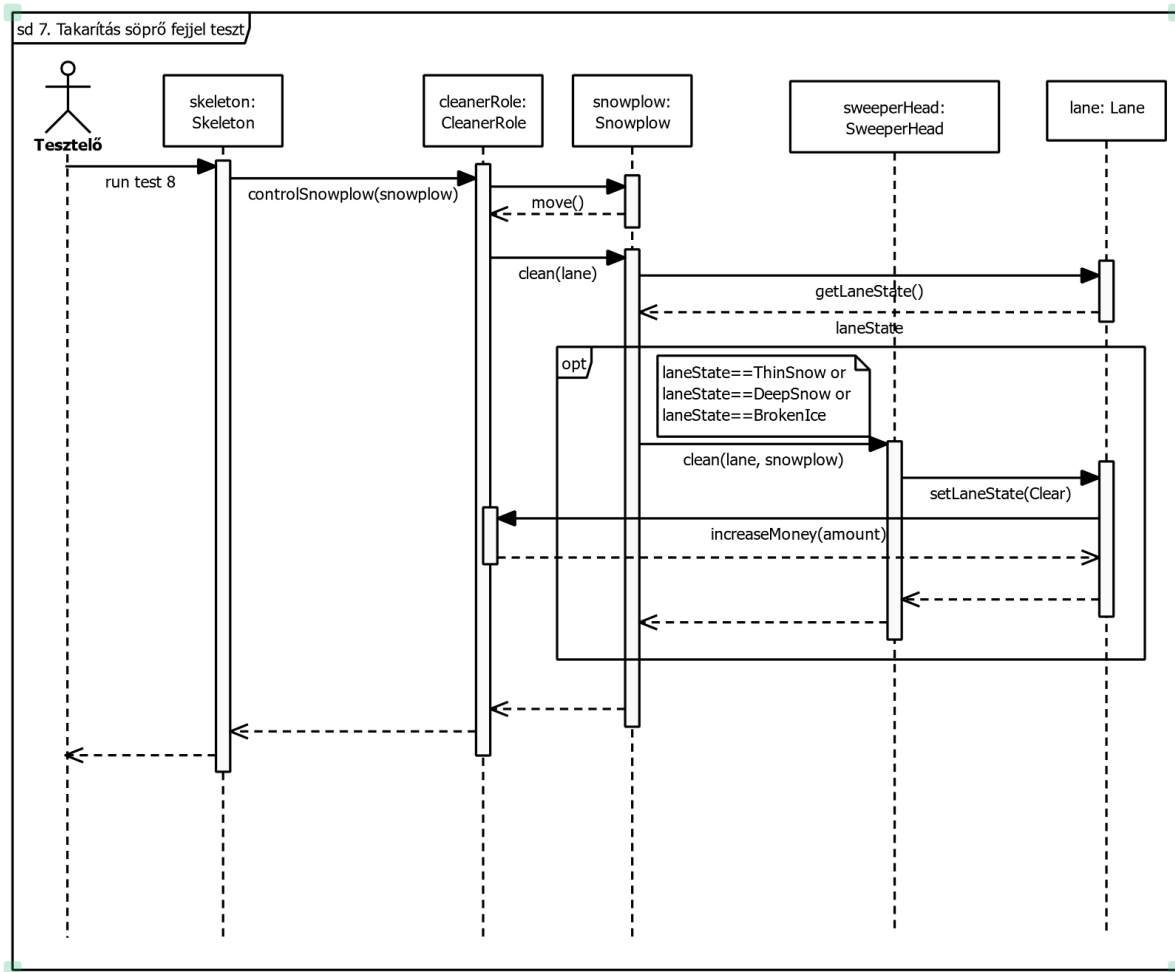
5. Jármű sávváltása teszt



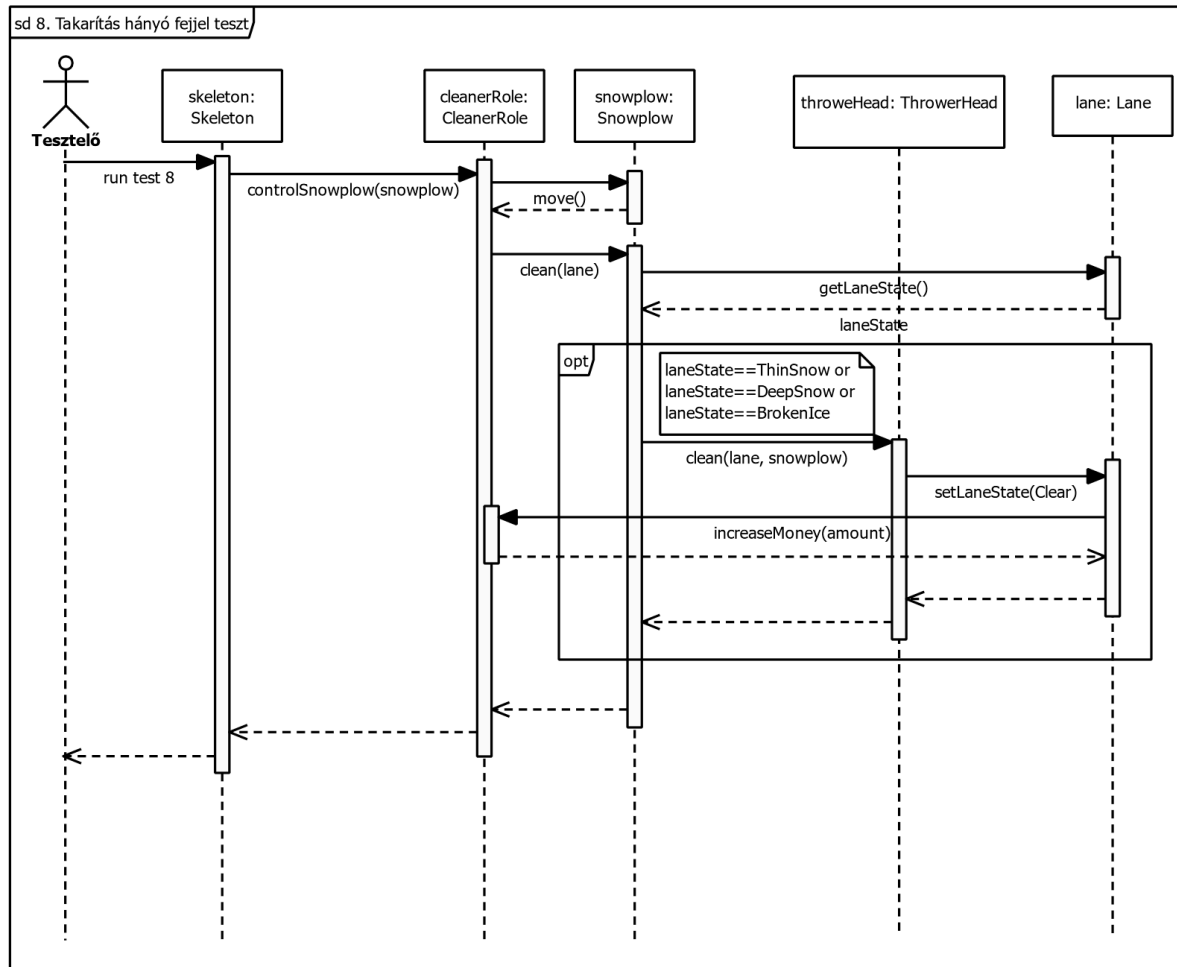
6. Hókotró haladása teszt



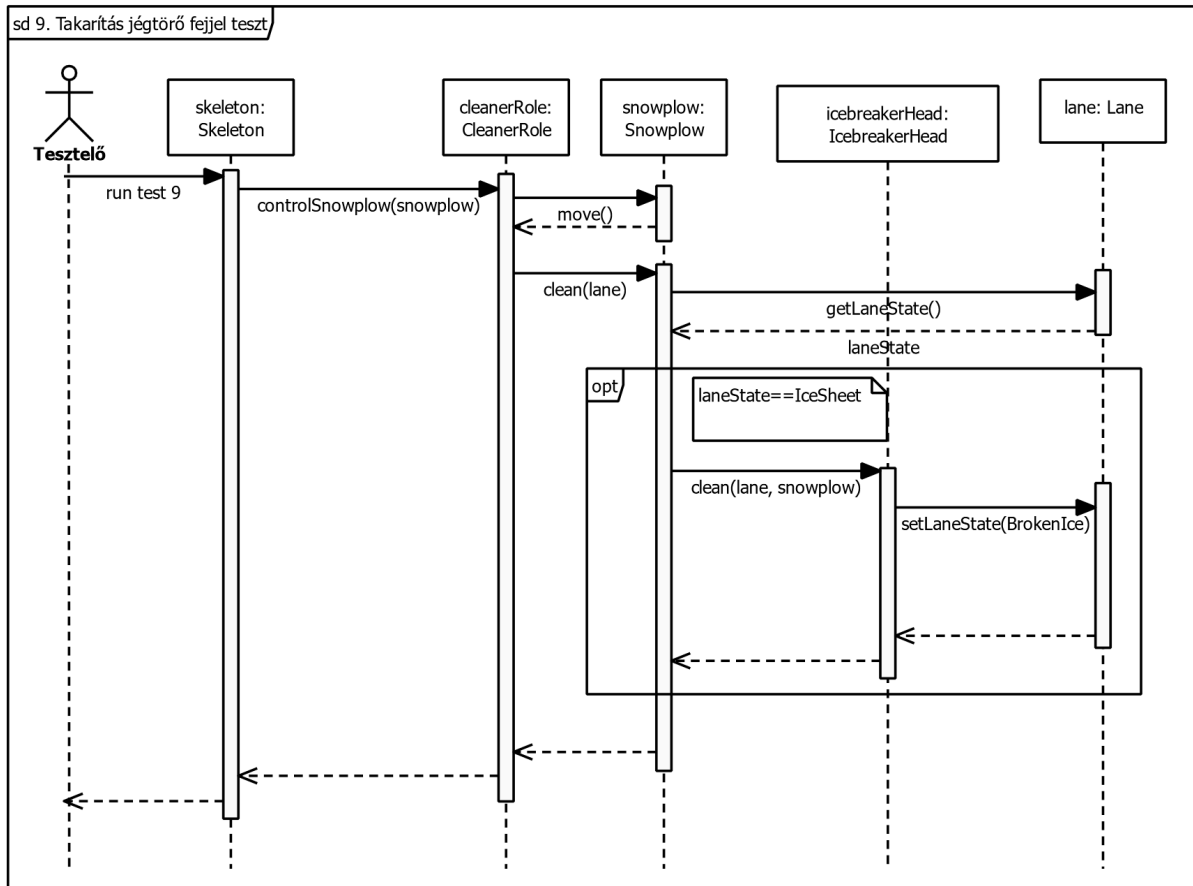
7. Takarítás söprő fejjel teszt



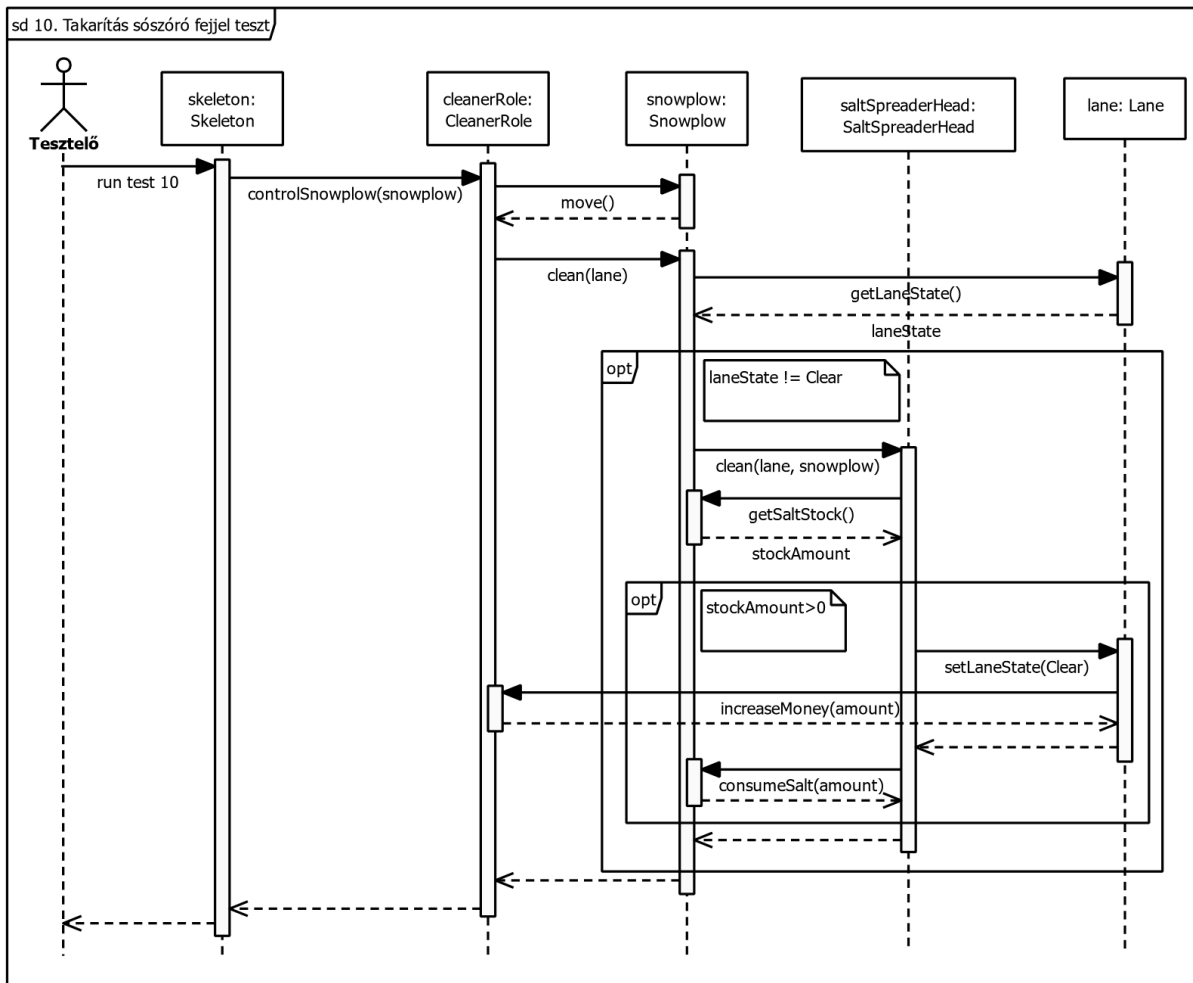
8. Takarítás hányó fejjel teszt



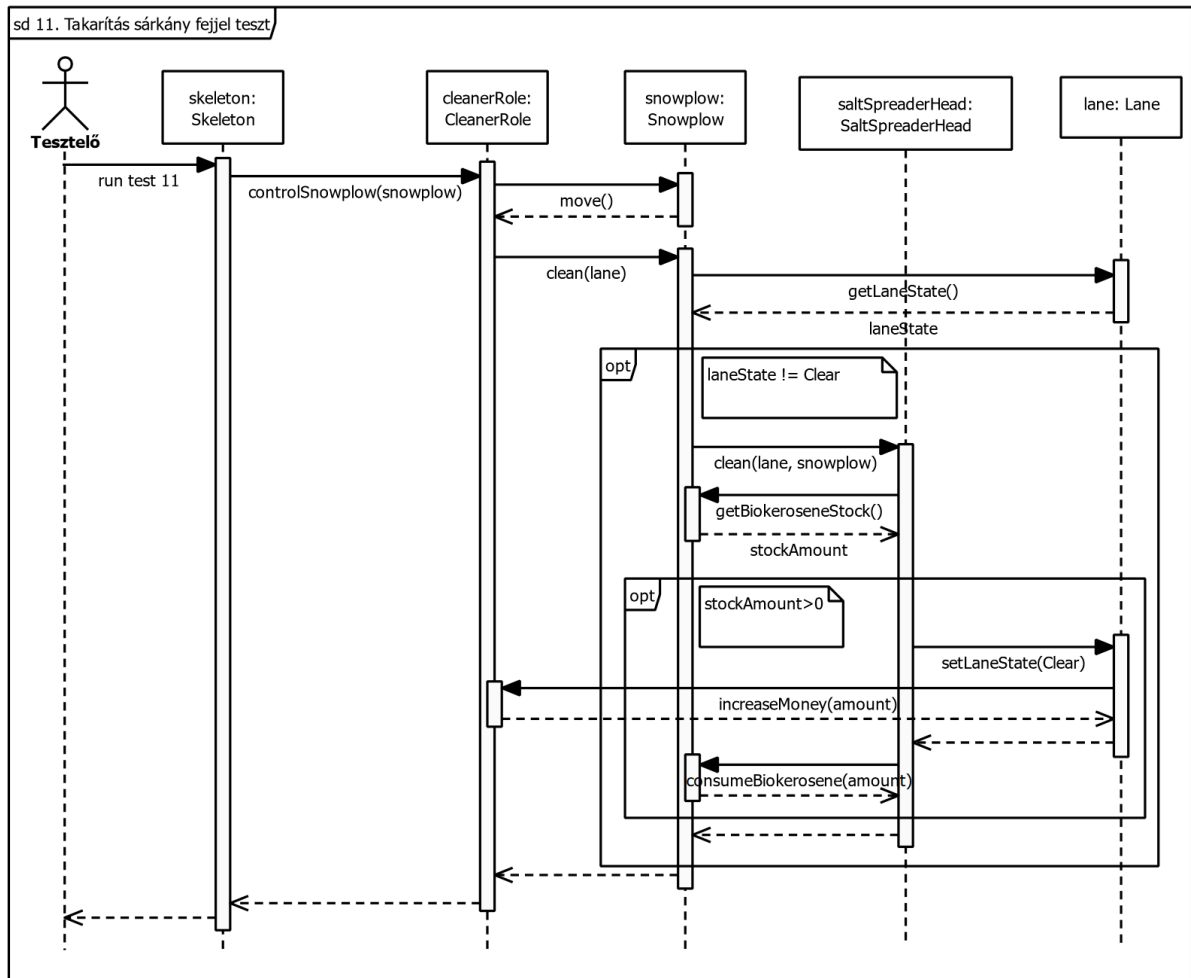
9. Takarítás jégtörő fejjel teszt



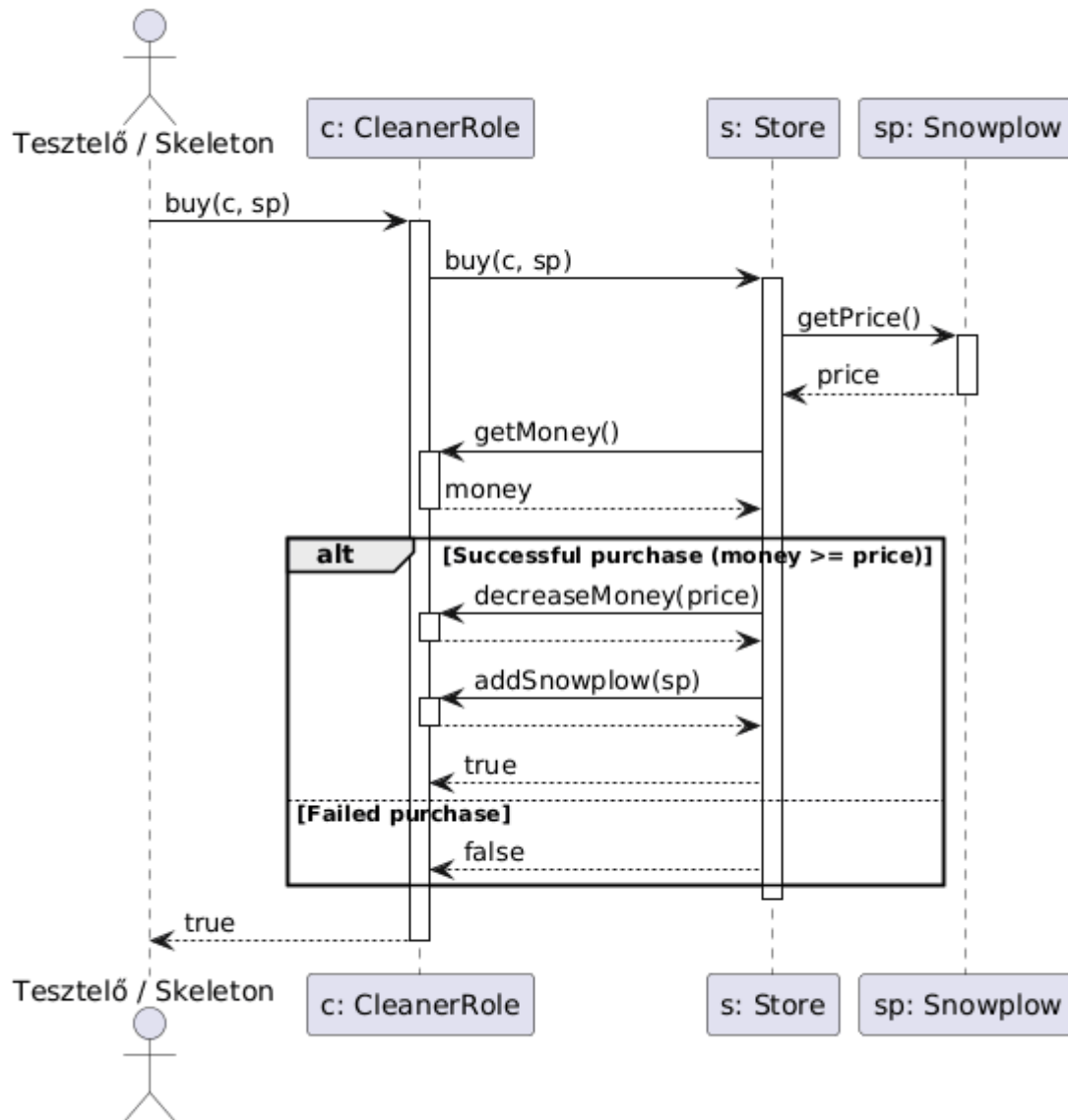
10. Takarítás sósóró fejjel teszt



11. Takarítás sárkány fejjel teszt

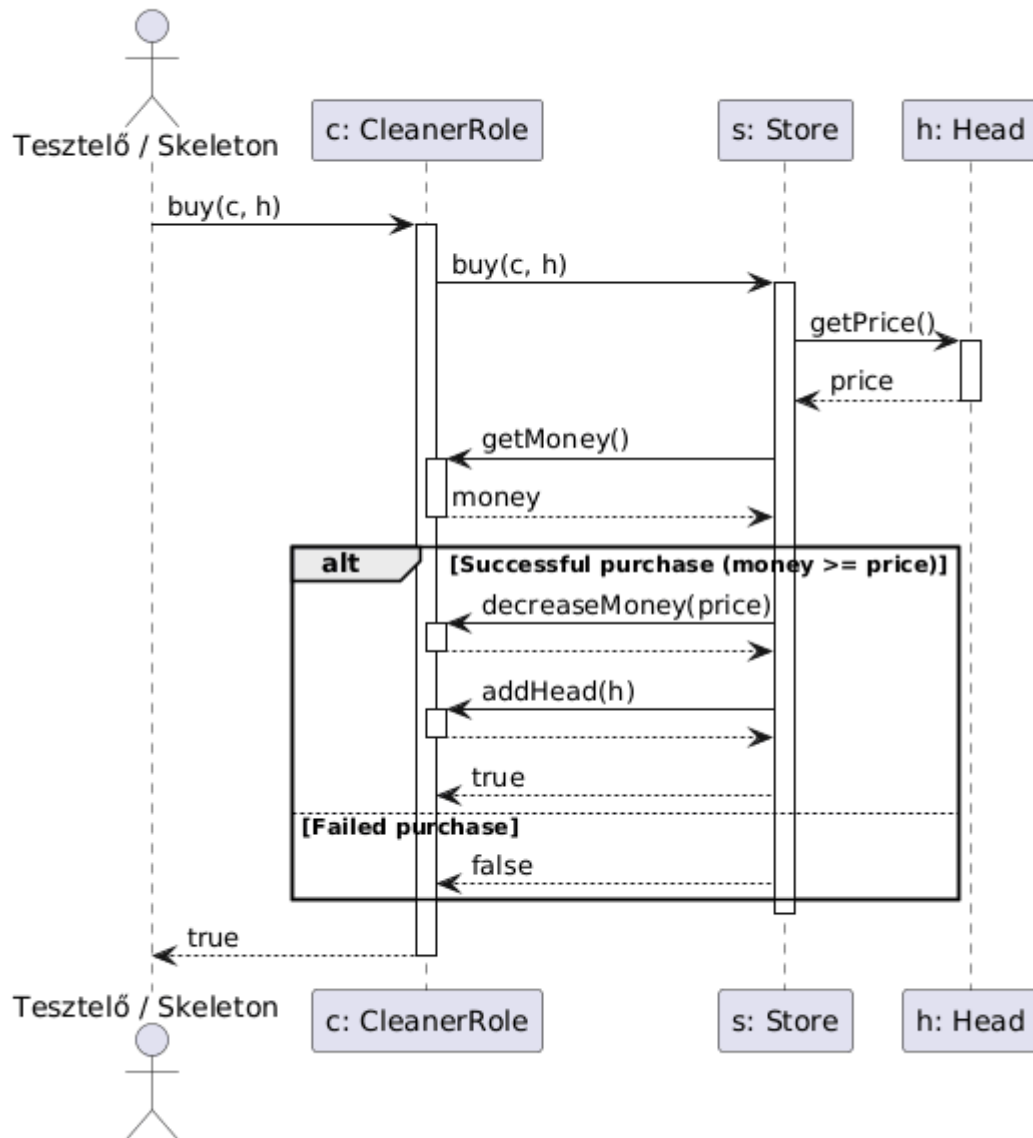


12. Sikeres hókotró vásárlás teszt

12. Sikeres hókotró vásárlás teszt

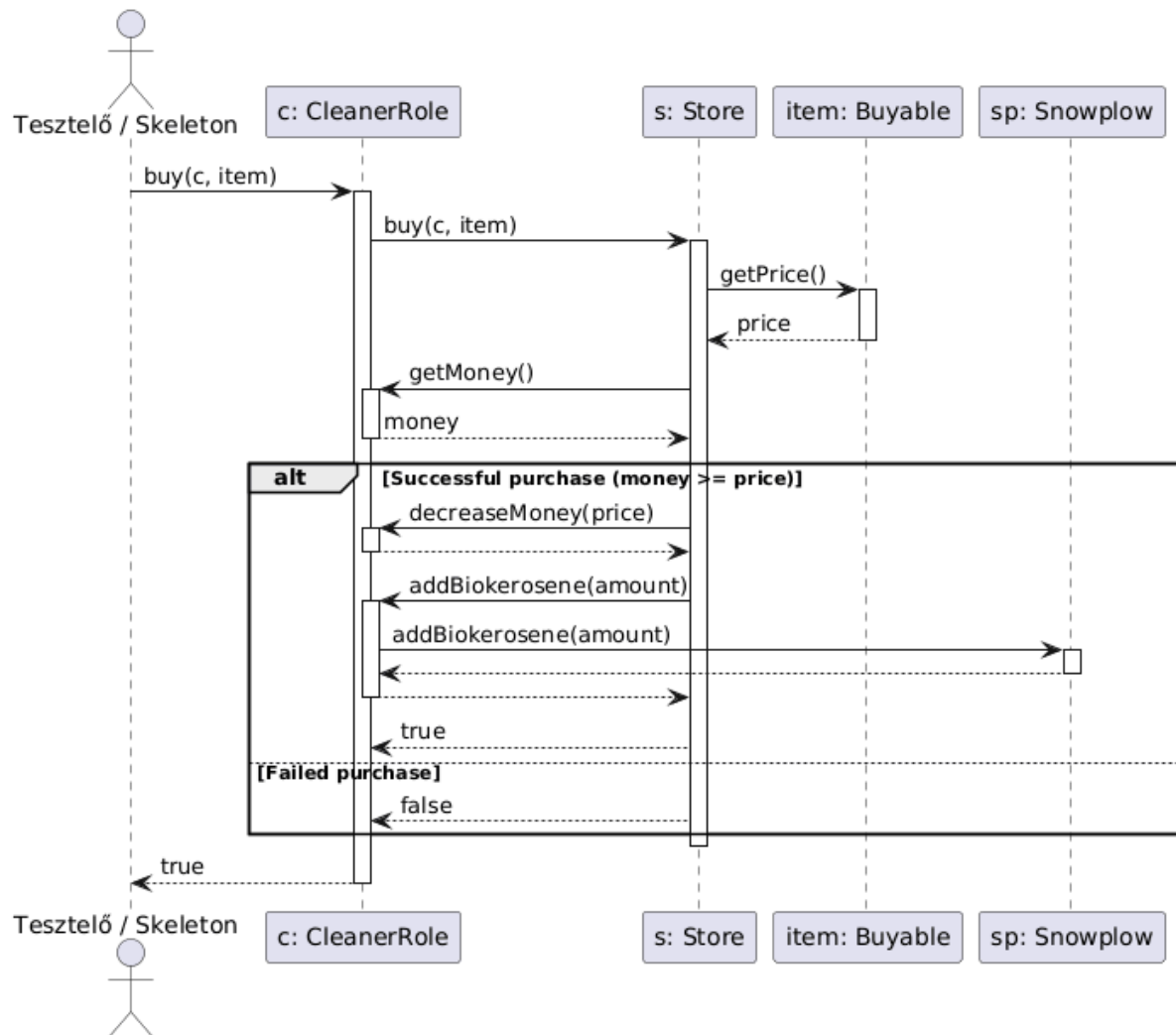
13. Sikeres kotrófej vásárlás teszt

13. Sikeres kotrófej vásárlás teszt



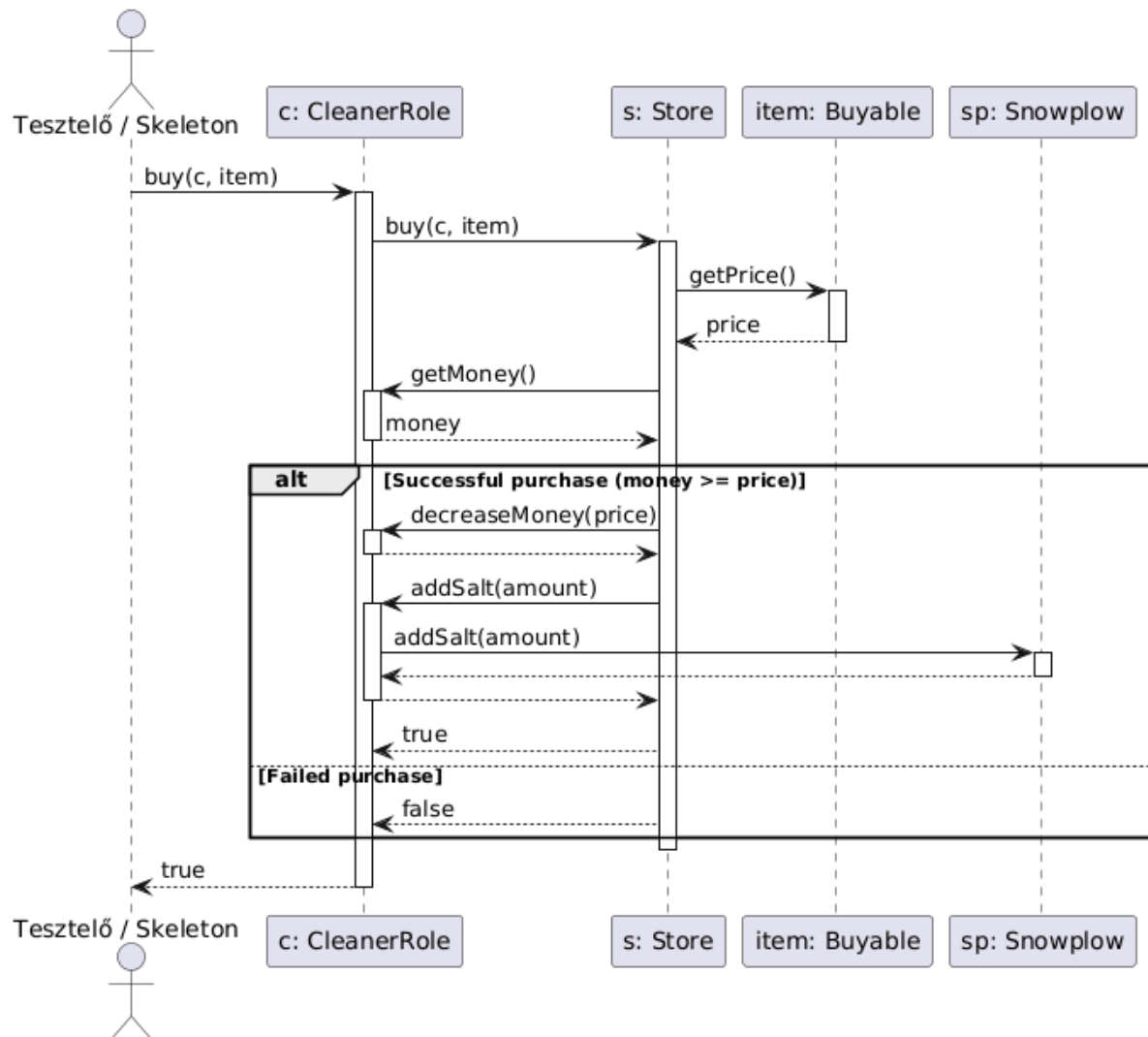
14. Sikeres biokerozin vásárlás teszt

14. Sikeres biokerozin vásárlás teszt



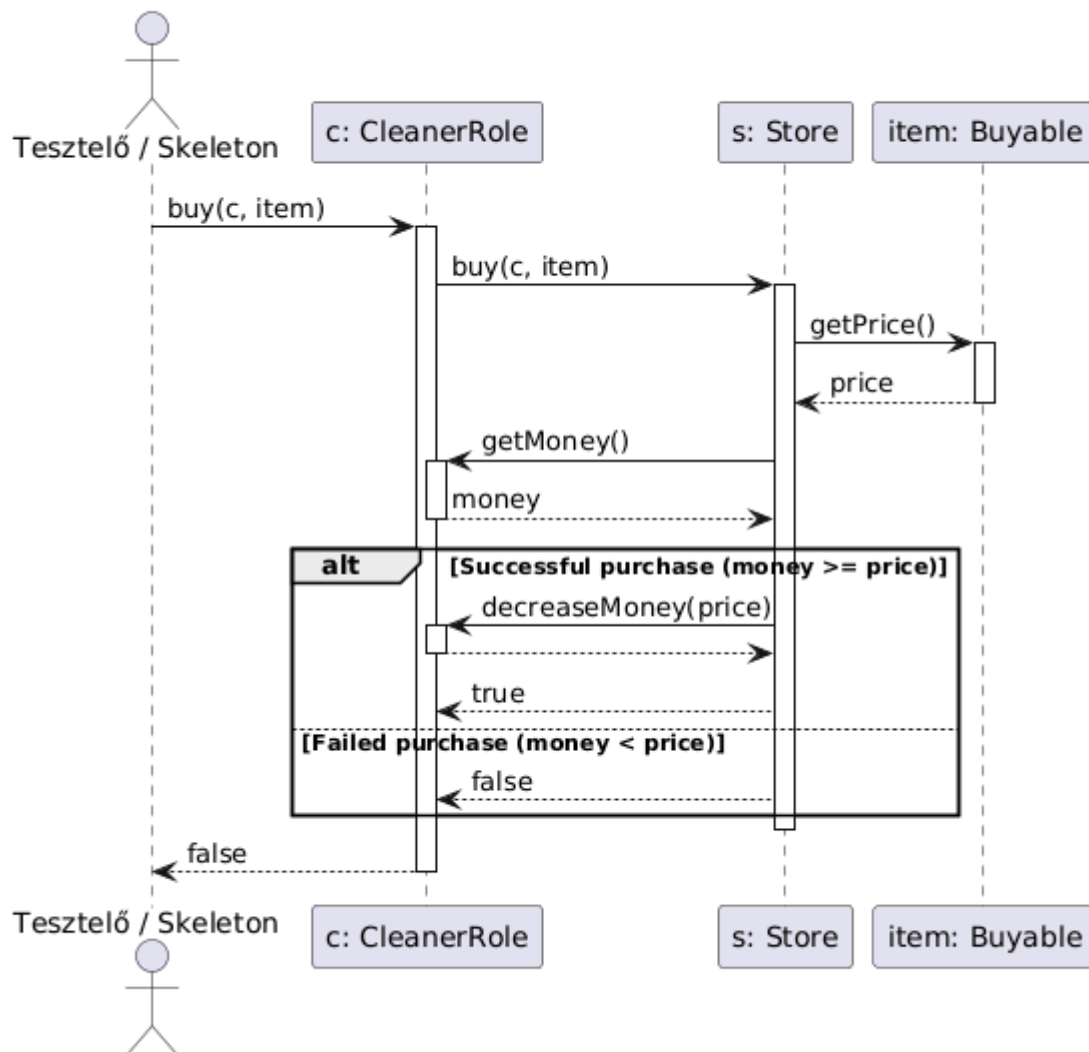
15. Sikeres hó vásárlás teszt

15. Sikeres só vásárlás teszt



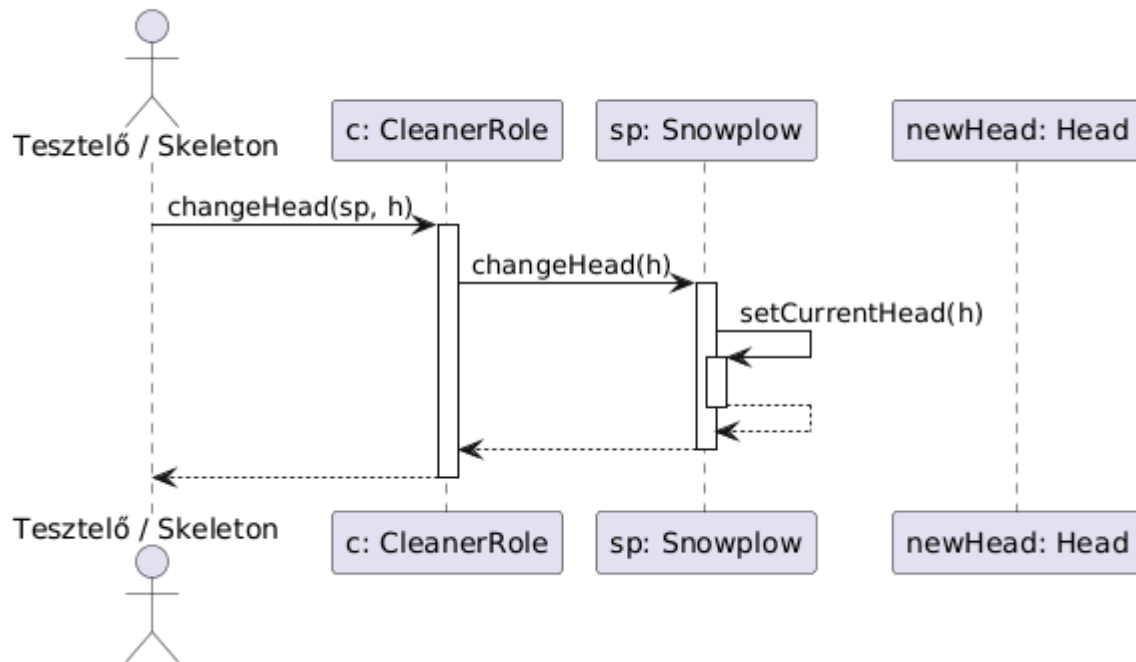
16. Sikertelen vásárlás teszt

16. Sikertelen vásárlás teszt



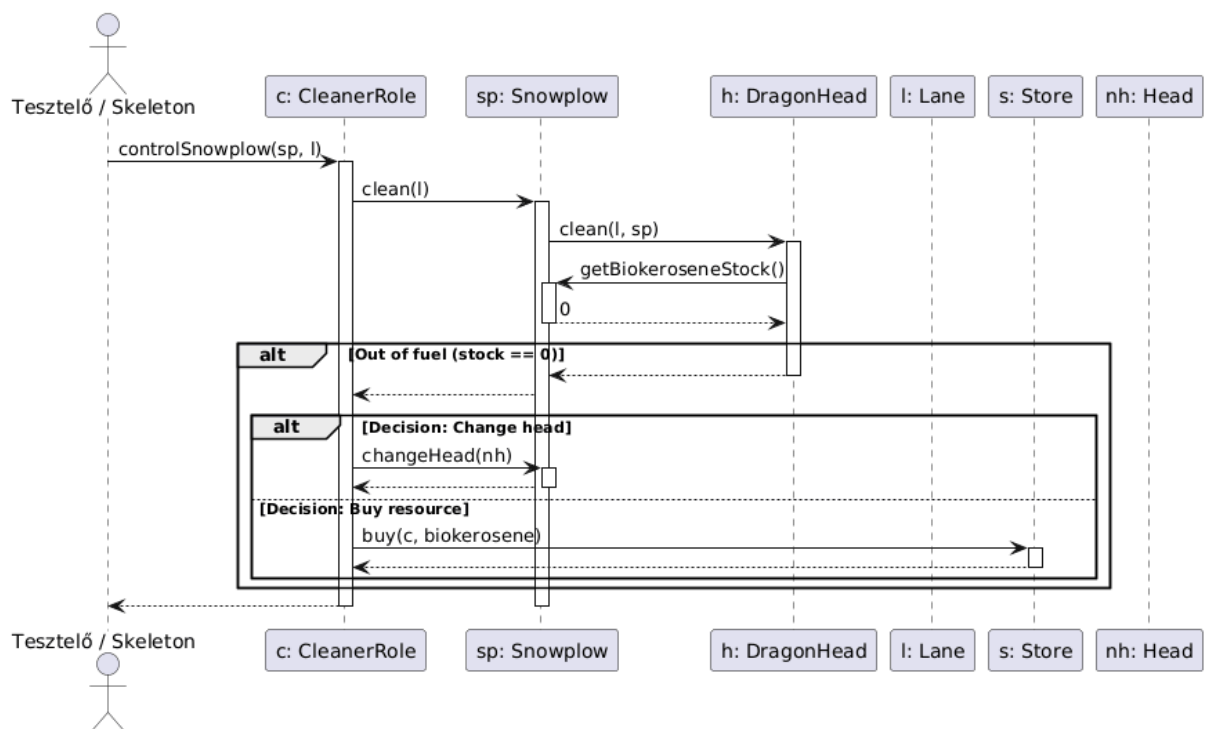
17. Kotrófej cseréje teszt

17. Kotrófej cseréje teszt

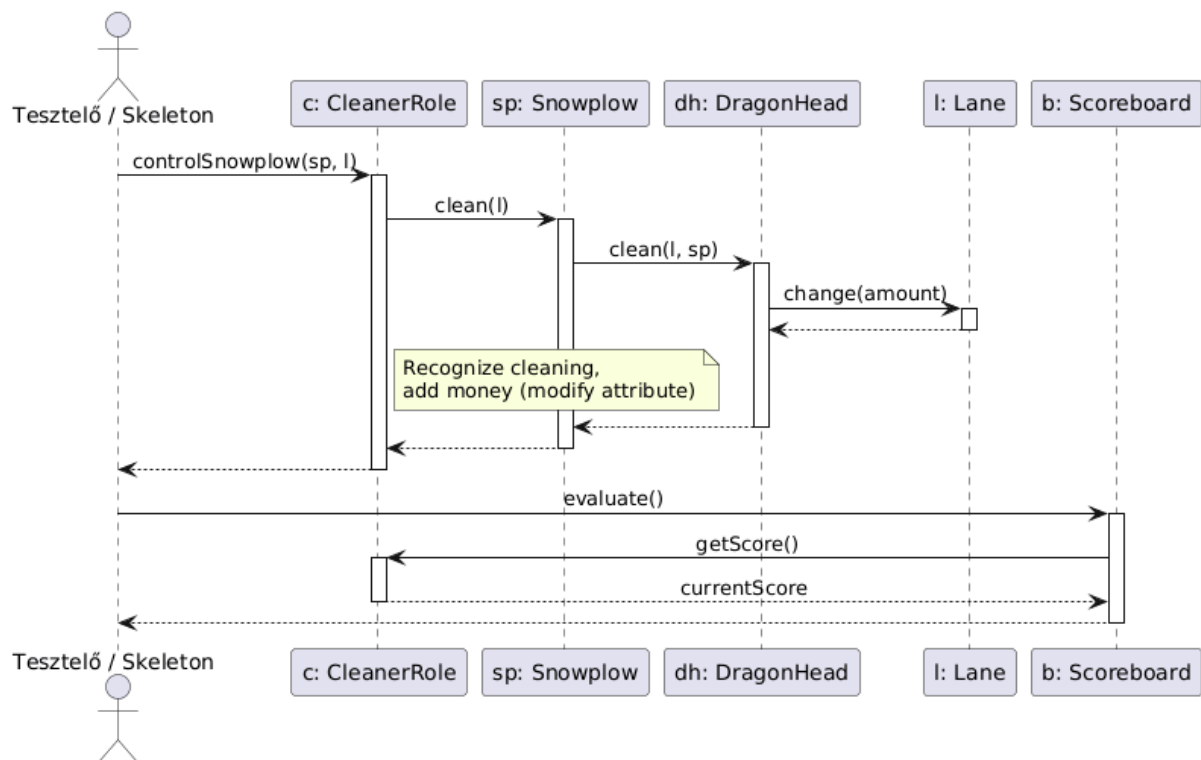
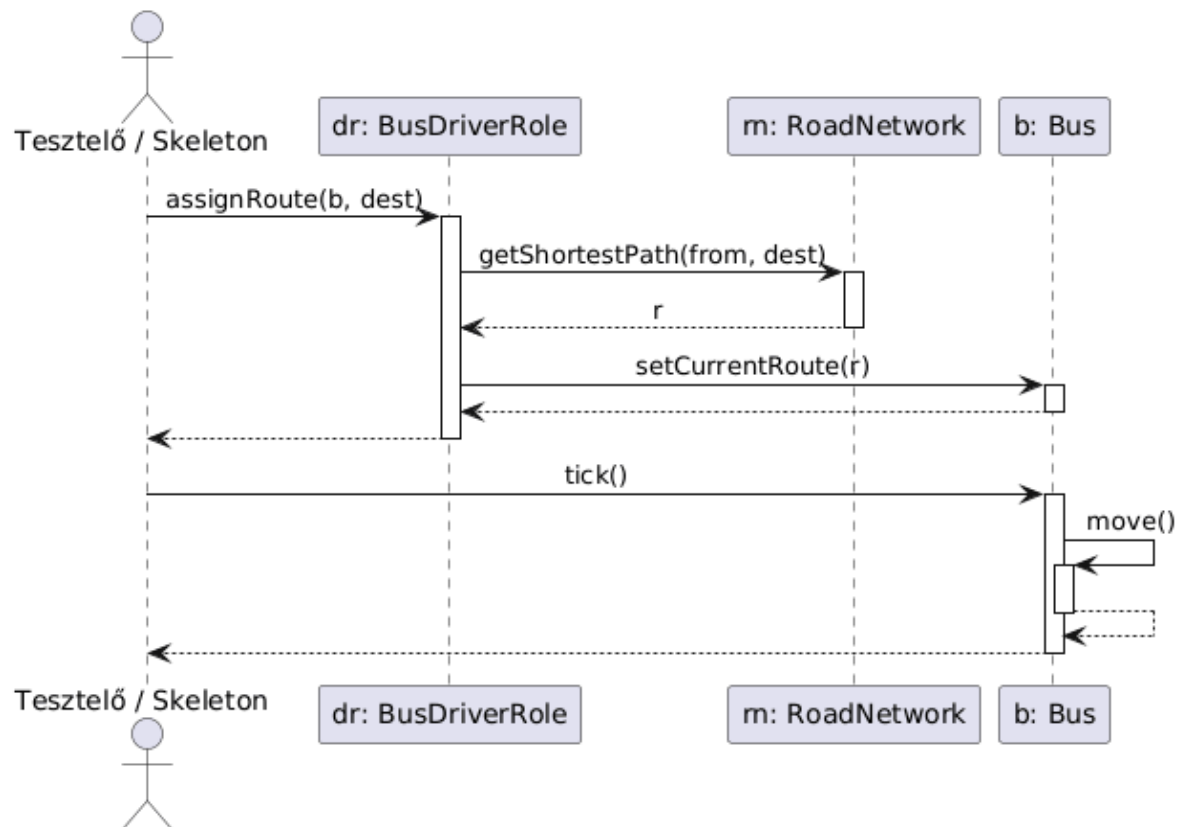


18. Nyersanyag kifogyás teszt

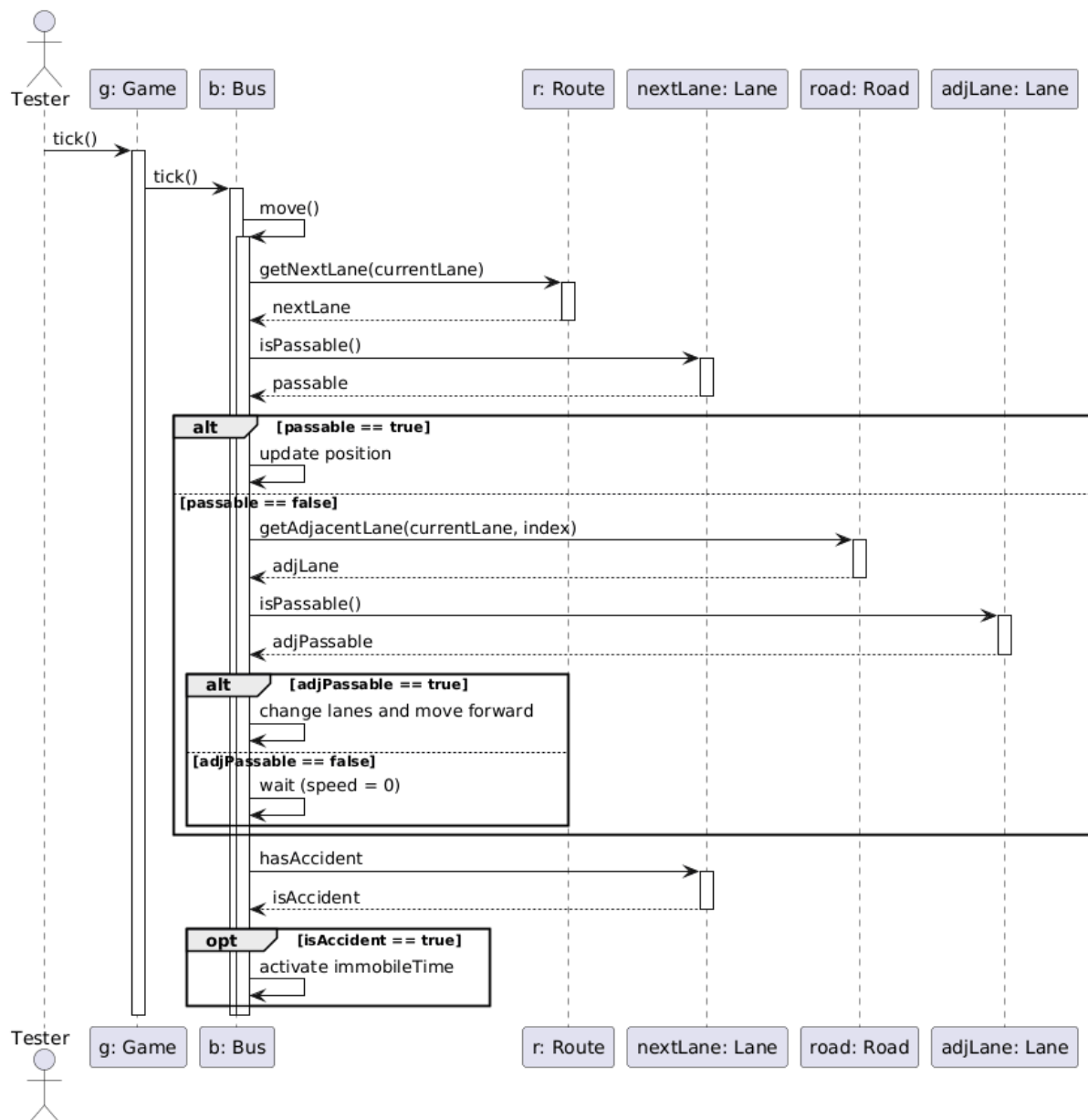
18. Nyersanyag kifogyás teszt



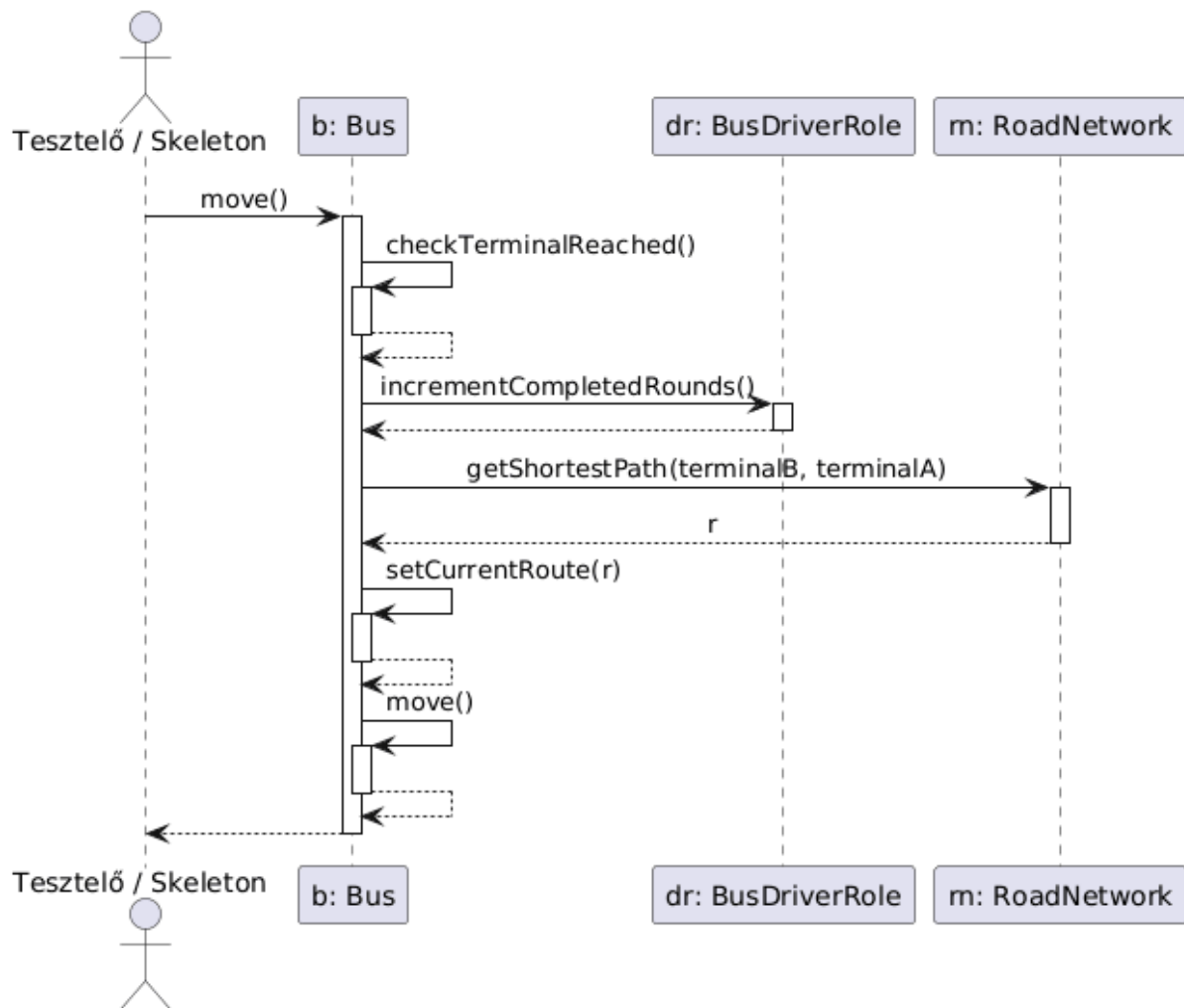
19. Pontszerzés takarítással teszt

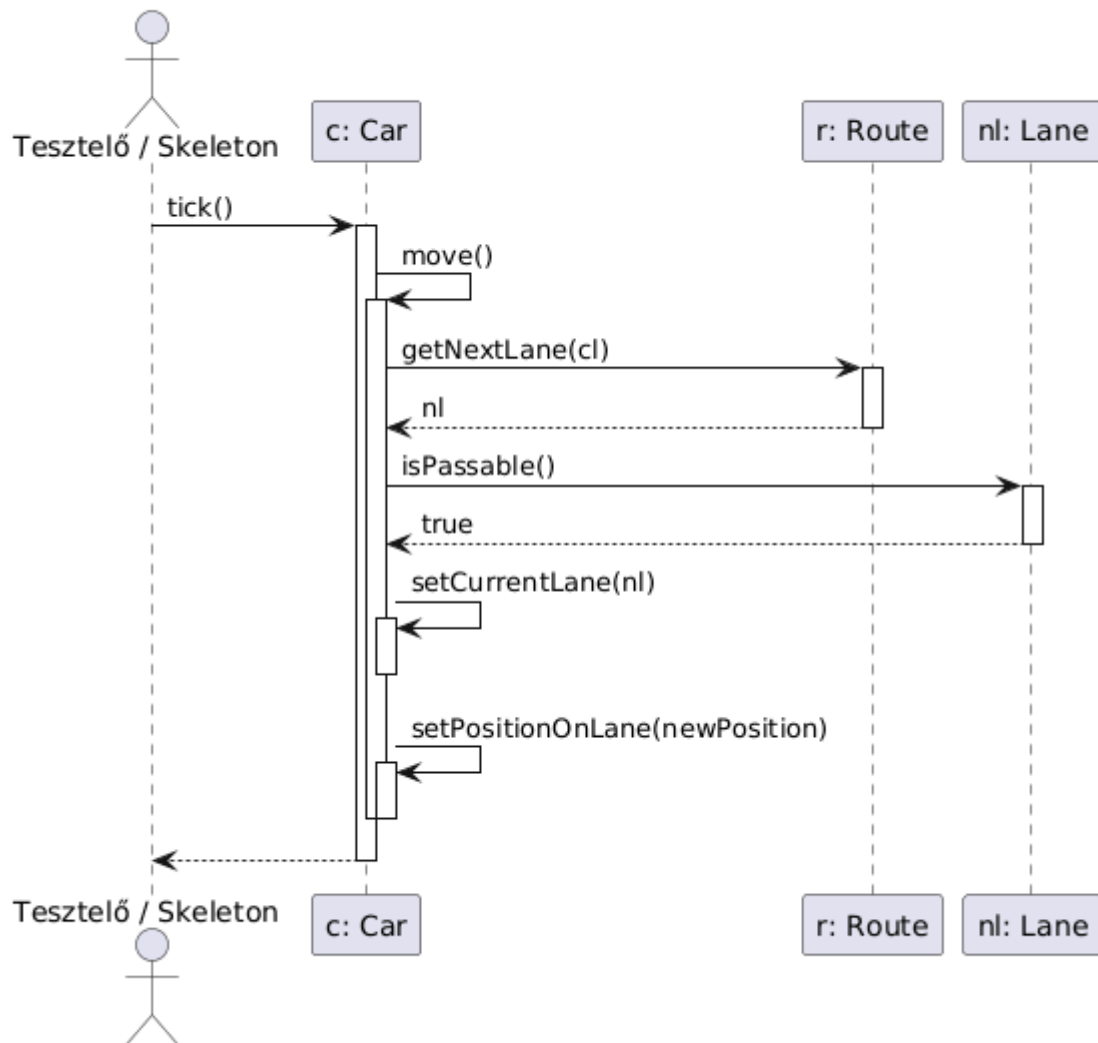
19. Pontszerzés takarítással teszt**20. Busz útvonalhoz rendelése teszt****20. Busz útvonalhoz rendelése teszt****21. Busz közlekedés teszt**

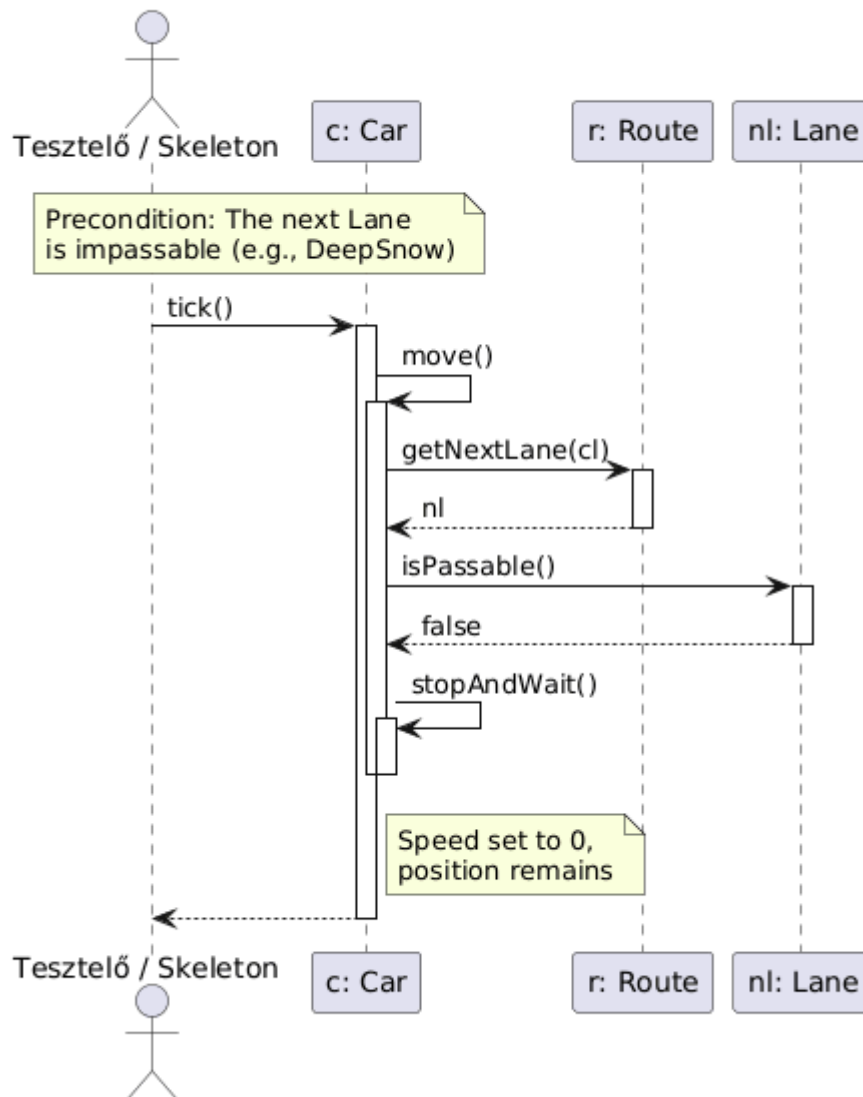
21. Busz közlekedés teszt



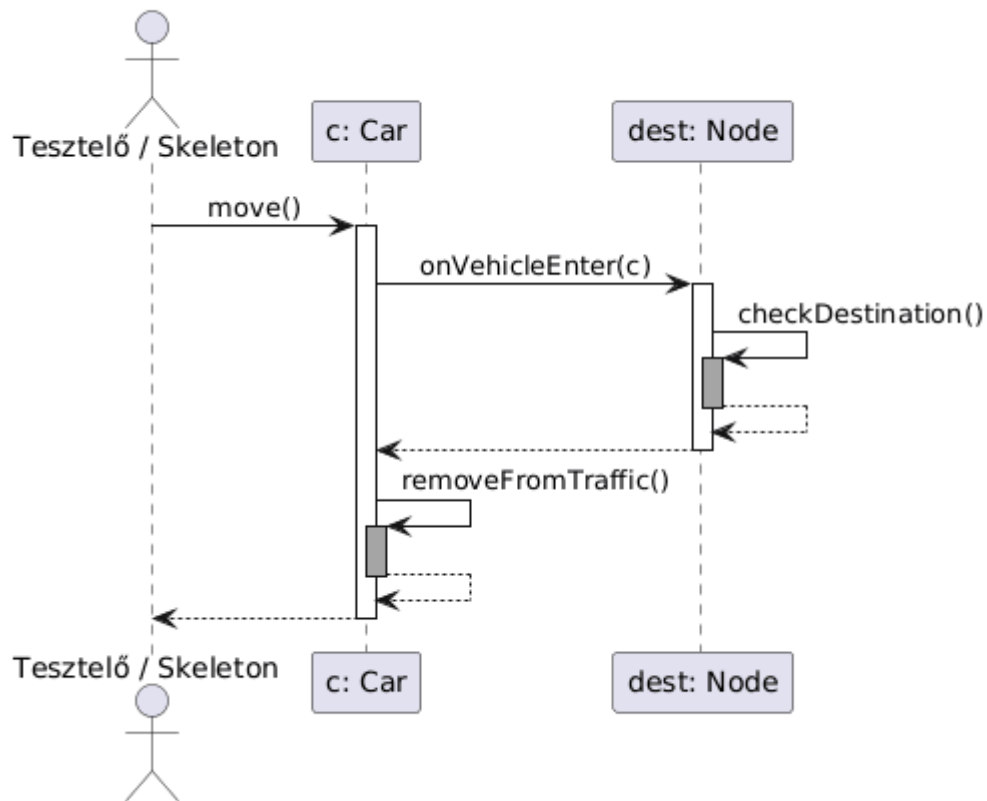
22. Busz forduló teljesítése teszt

22. Busz forduló teljesítése teszt**23. Autó haladása járható sávban teszt**

23. Autó haladása járható sávban teszt**24. Autó elakadása járhatatlan úton teszt**

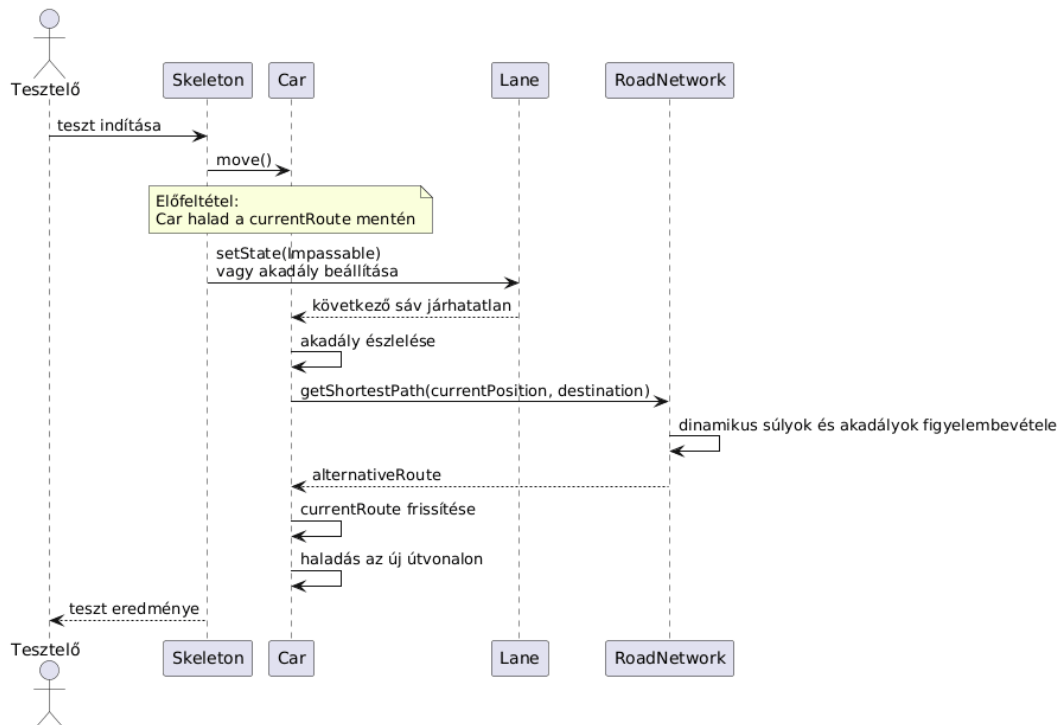
24. Autó elakadása járhatatlan úton teszt**25. Autó célba érése teszt**

25. Autó célba érése teszt

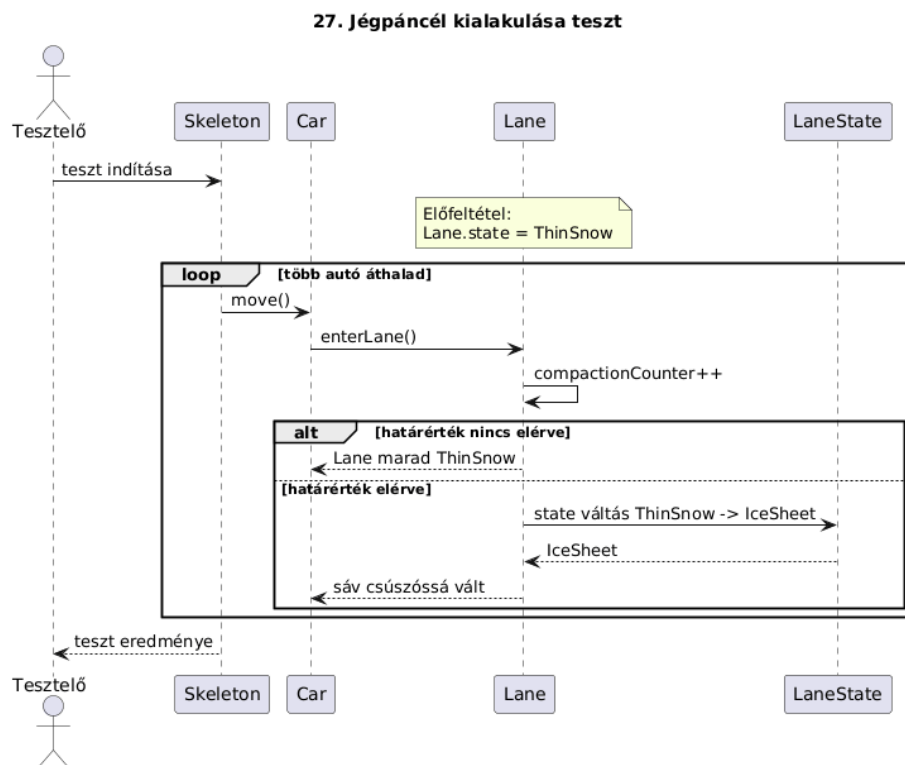


26. Autó útvonalának újratervezése akadály esetén

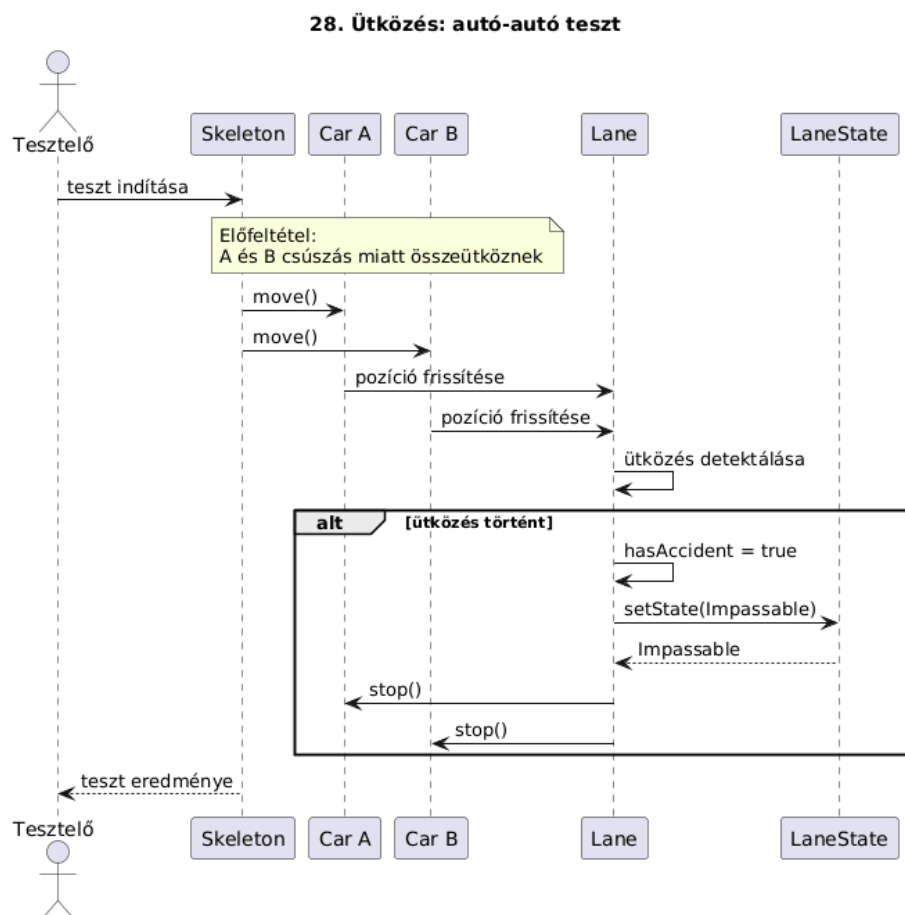
26. Autó útvonalának újratervezése akadály esetén



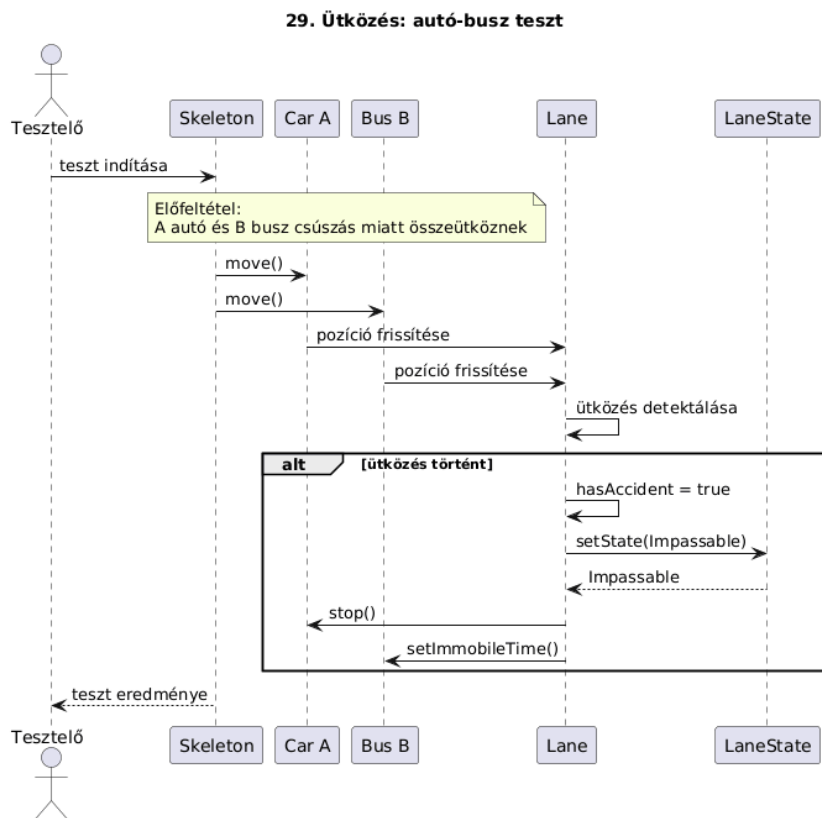
27. Jégpáncél kialakulása teszt



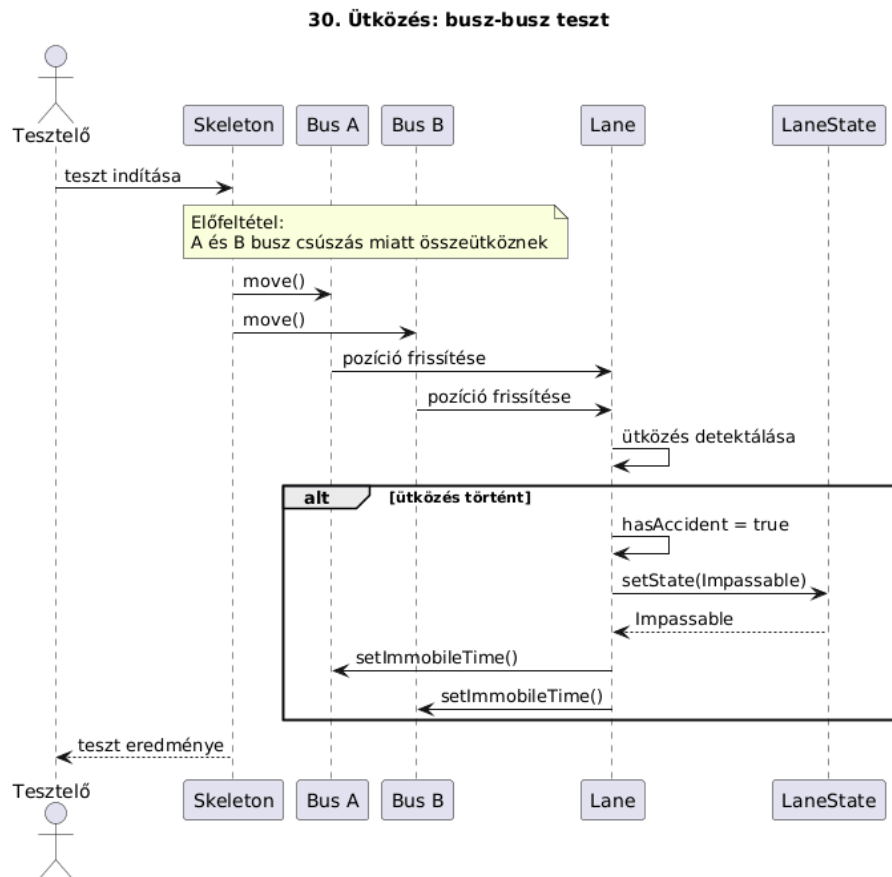
28. Ütközés: autó-autó teszt



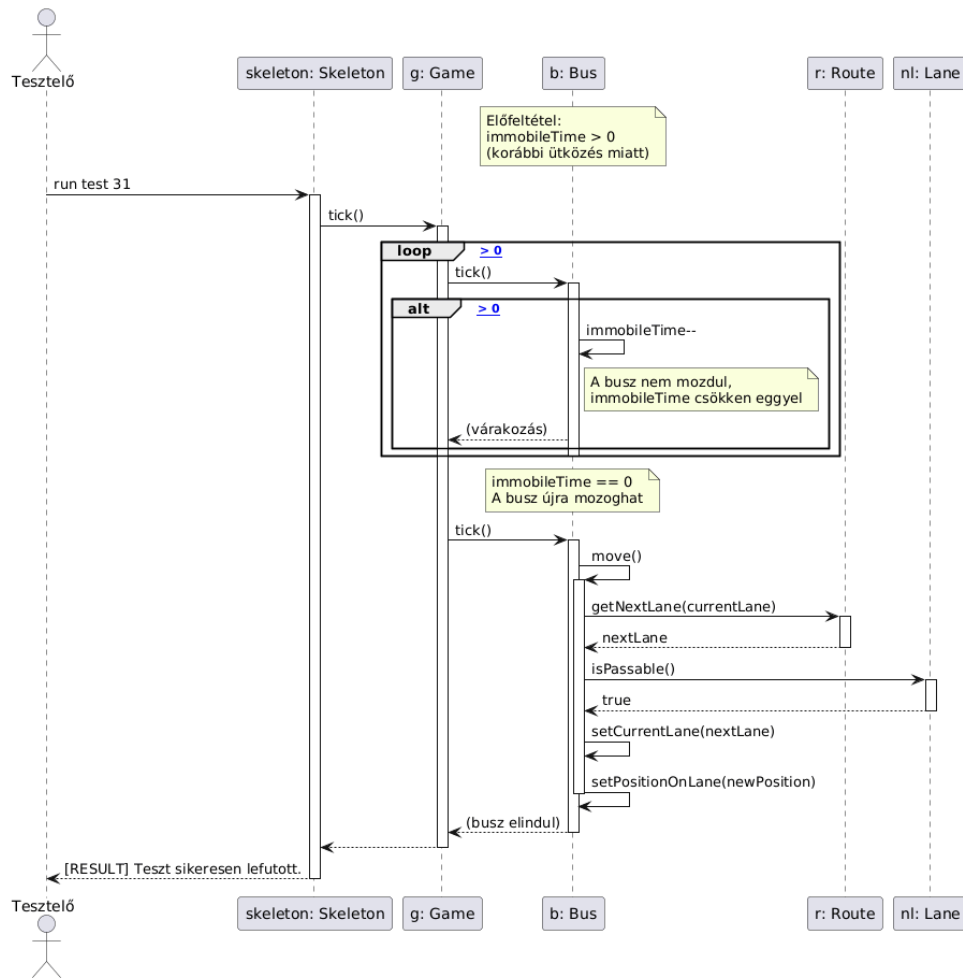
29. Ütközés: autó-busz teszt



30. Ütközés: busz-busz teszt



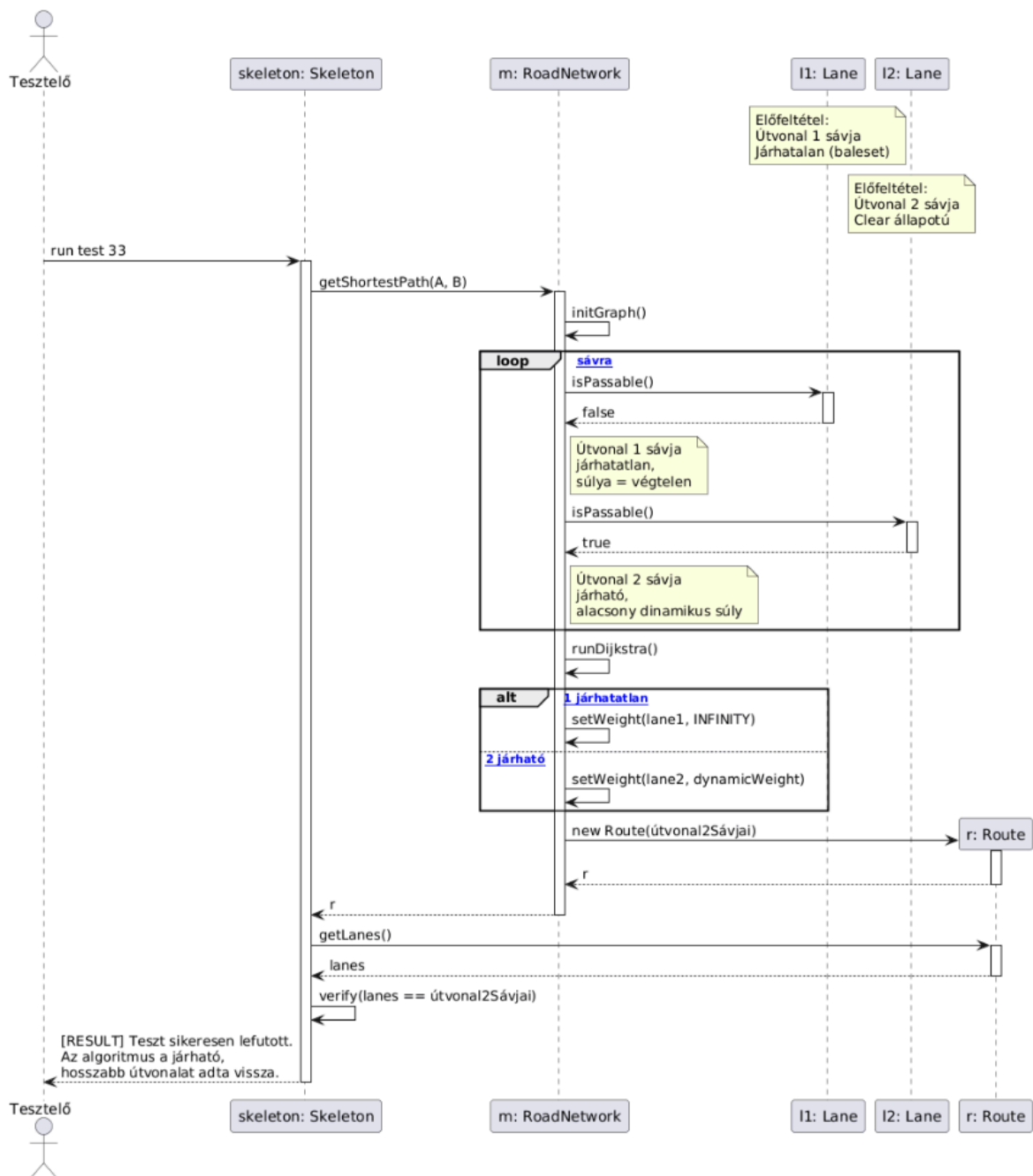
31. Busz mozgásképtelenség megszűnése teszt



32. Játék kiértékelése teszt

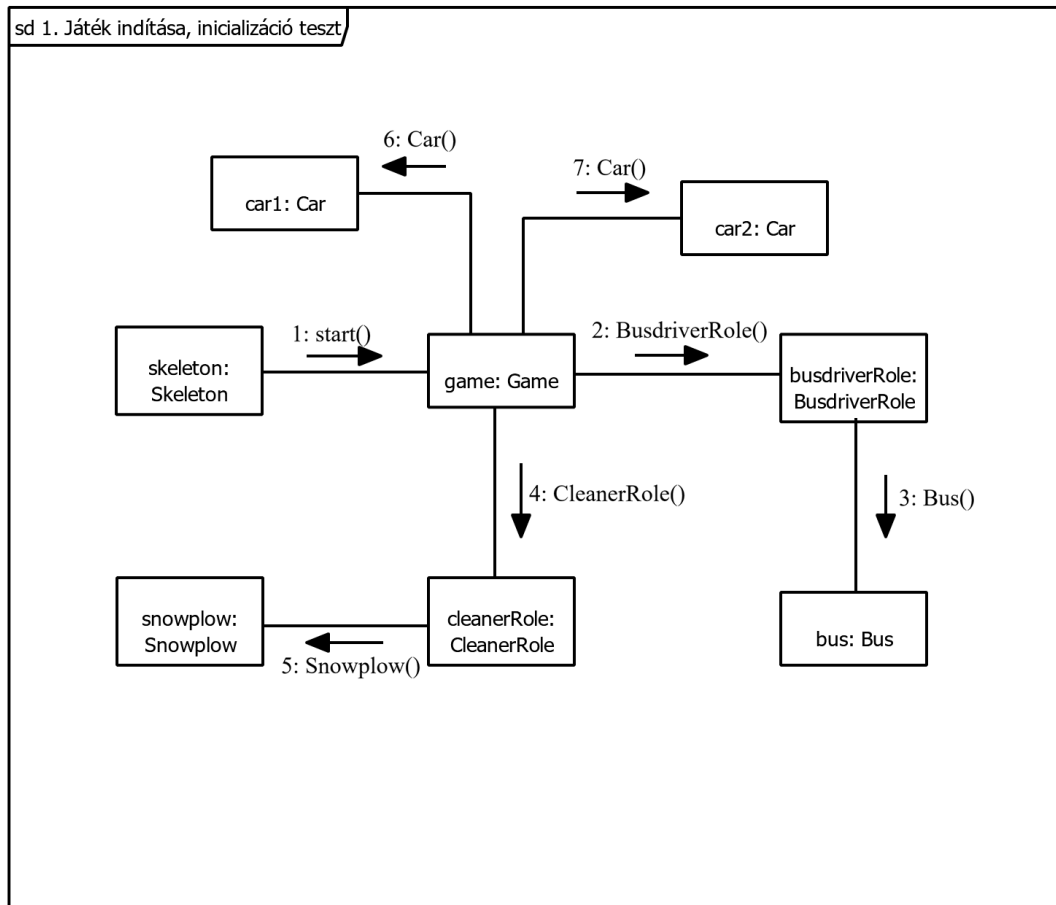


33. Útvonalkereső algoritmus, legrövidebb út

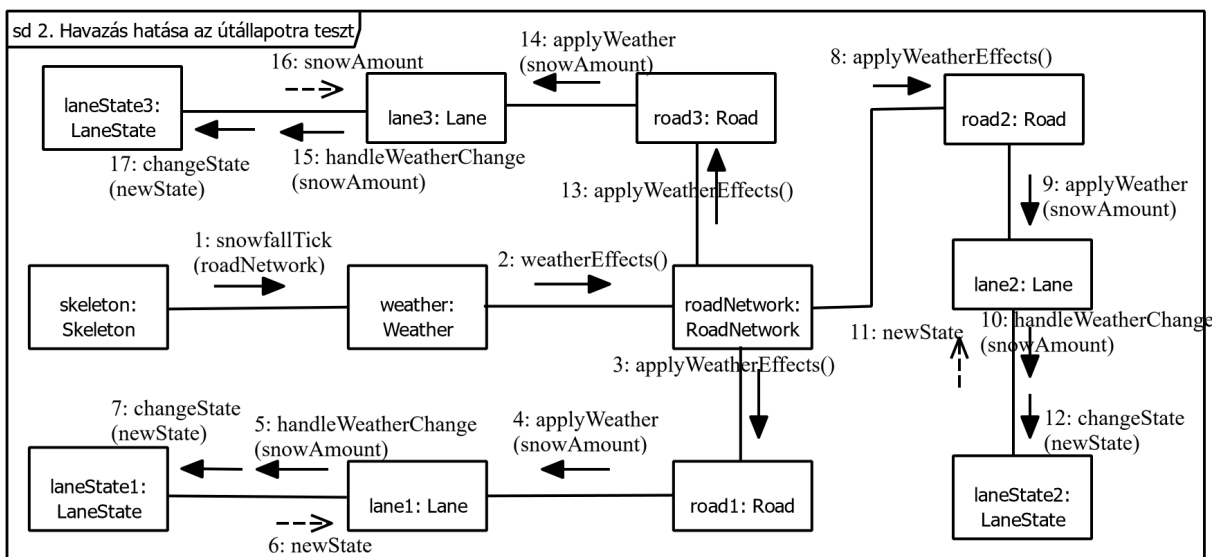


7.4 Kommunikációs diagramok

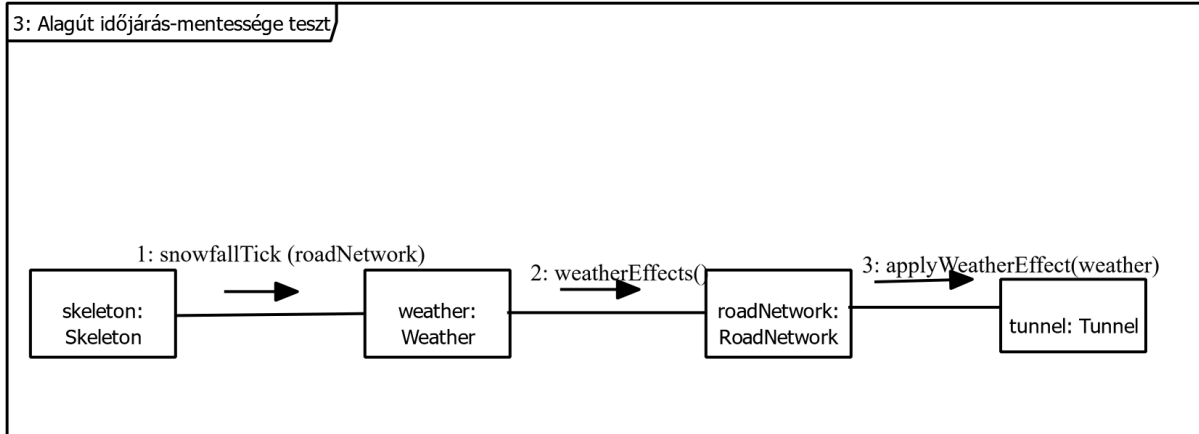
1. Játék indítása, inicializációs teszt



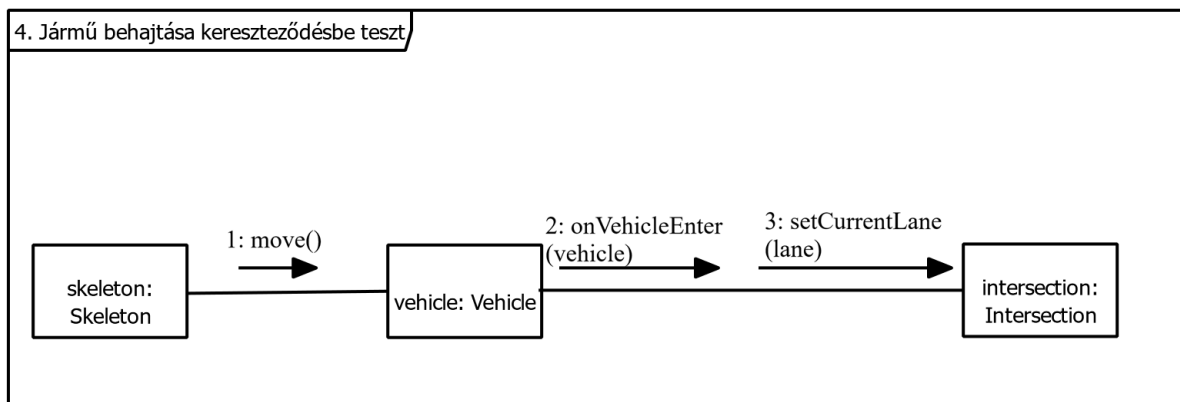
2. Havazás hatása az útállapotra teszt



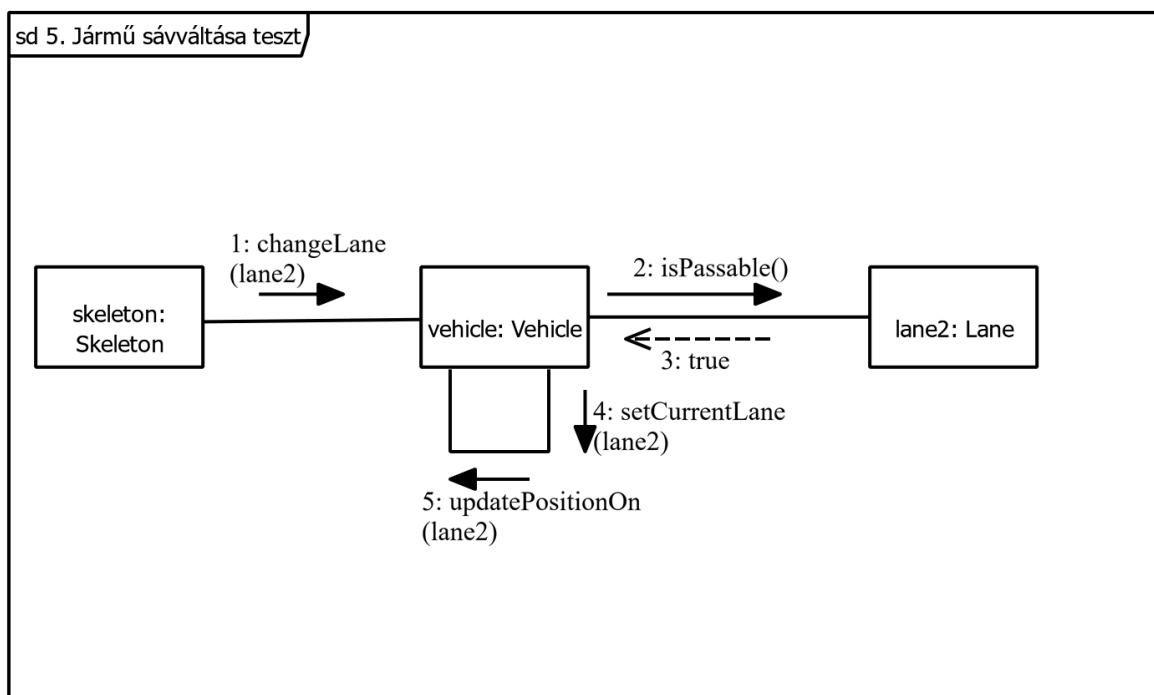
3. Alagút időjárás-mentessége teszt



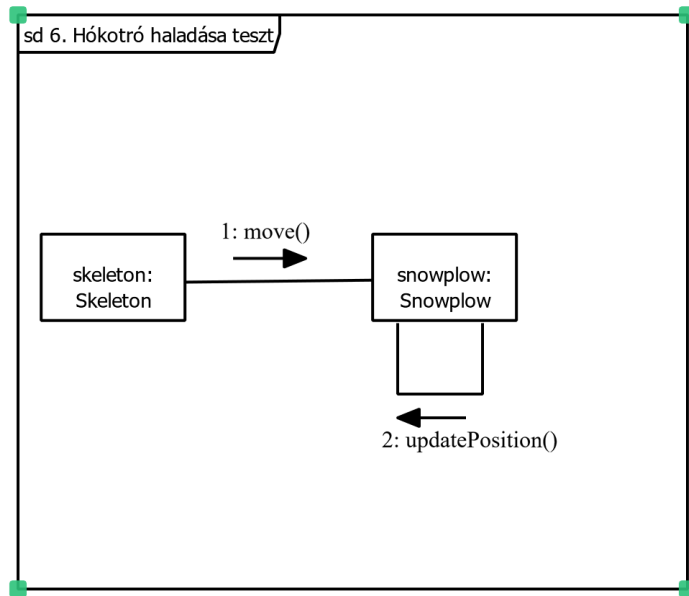
4. Jármű behajtása kereszteződésbe teszt



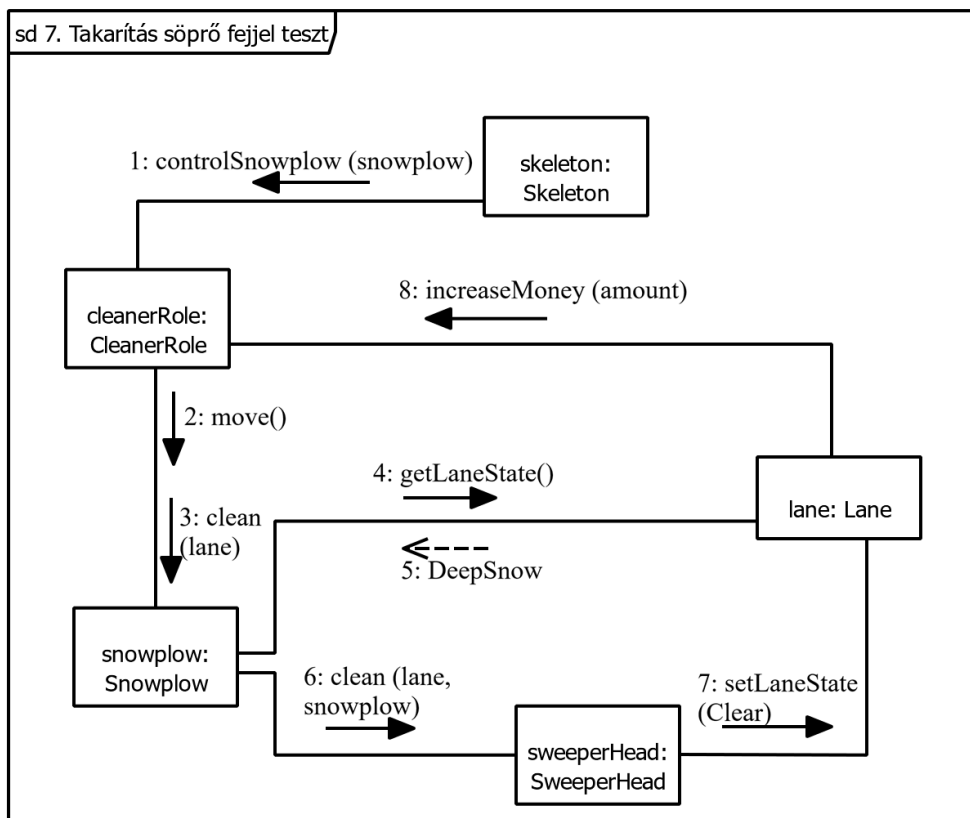
5. Jármű sávváltása teszt



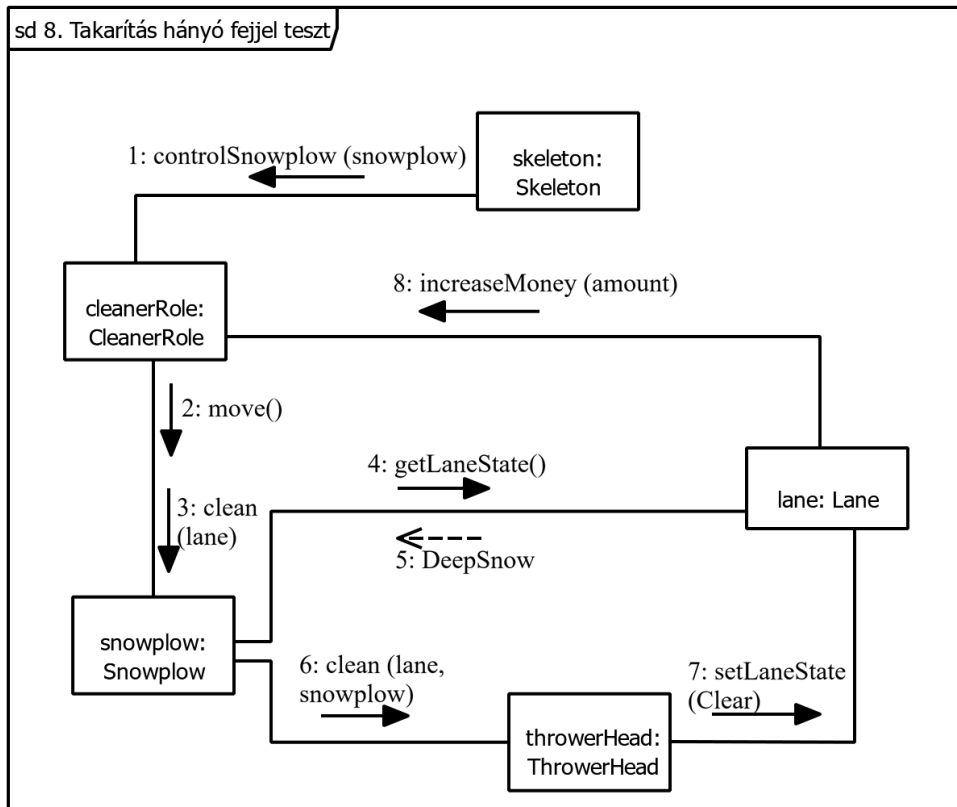
6. Hókotró haladása teszt



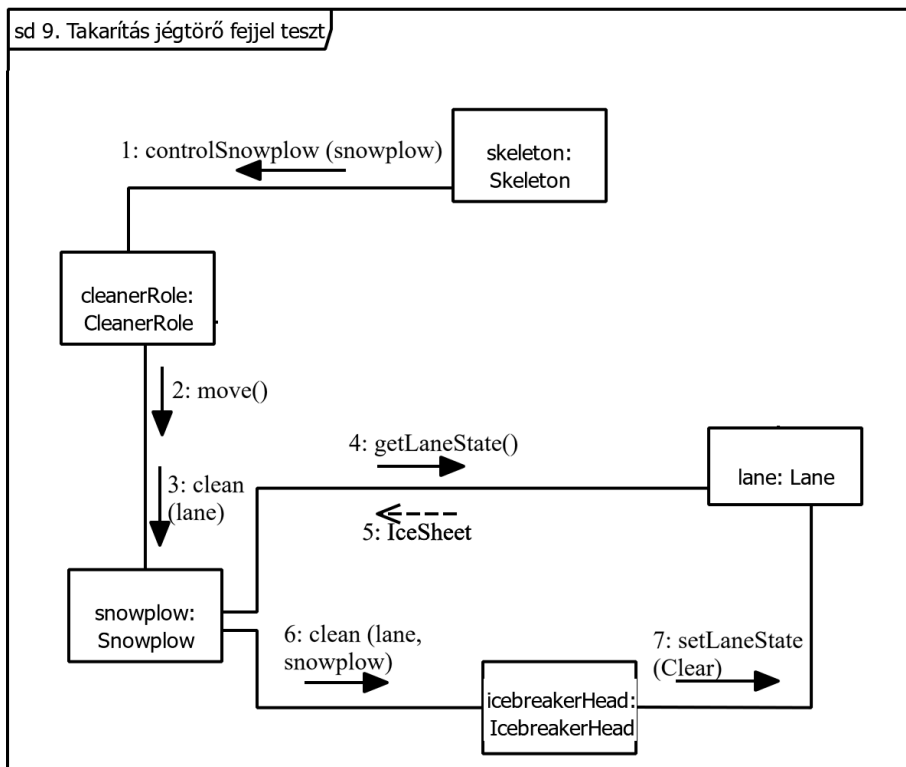
7. Takarítás söprő fejjel teszt



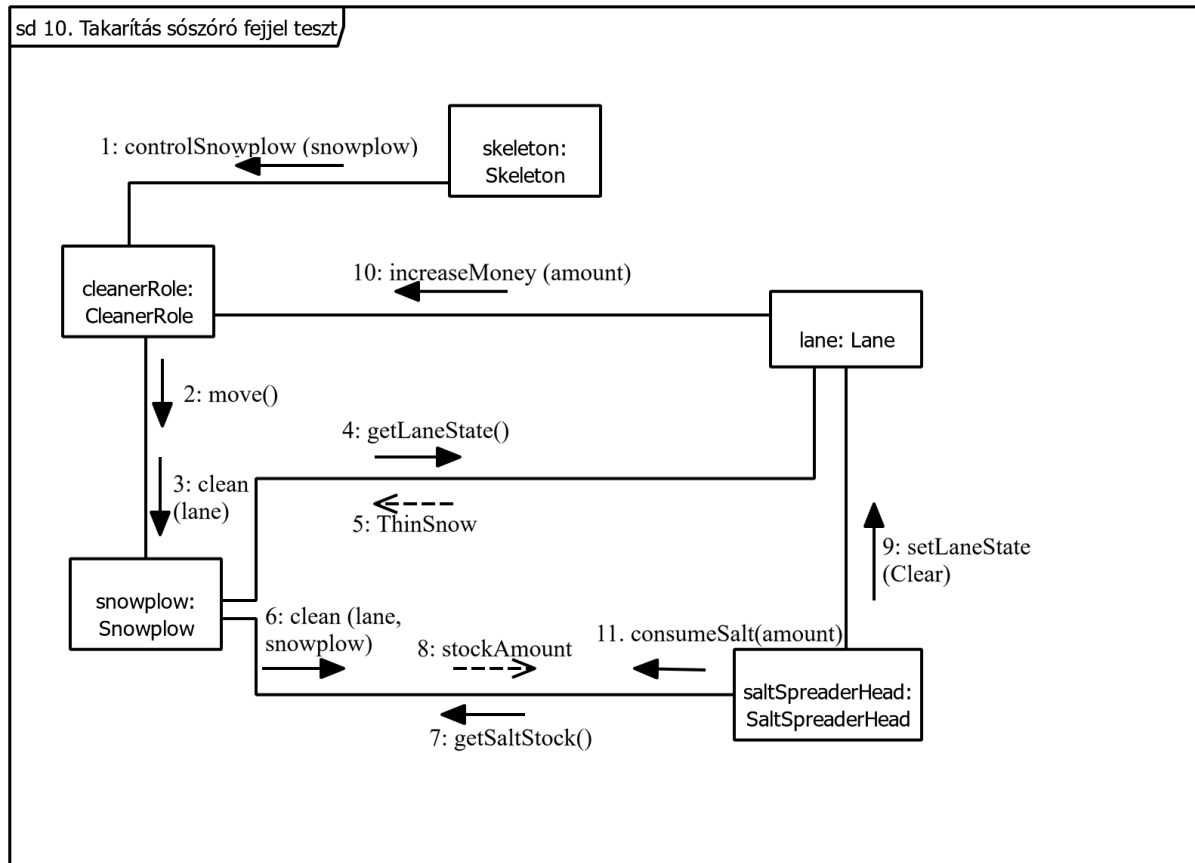
8. Takarítás hányó fejjel teszt



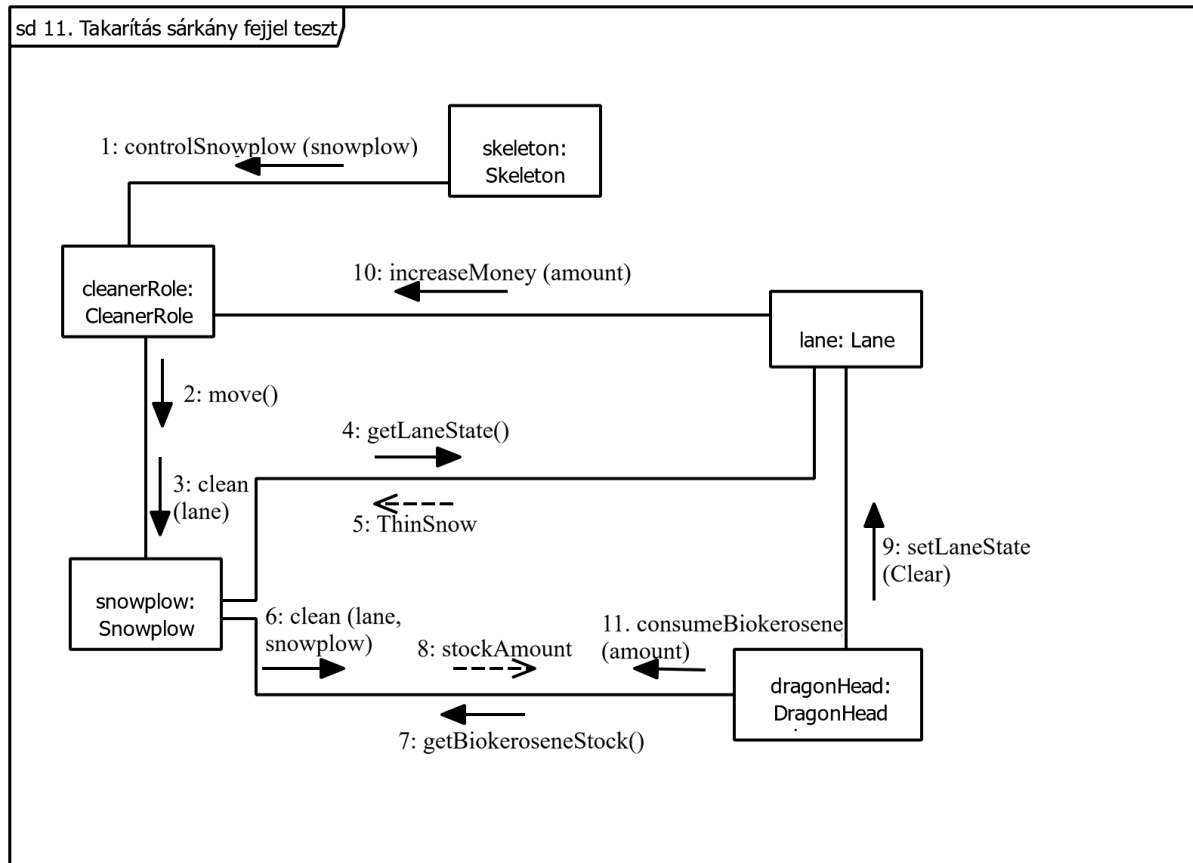
9. Takarítás jégtörő fejjel



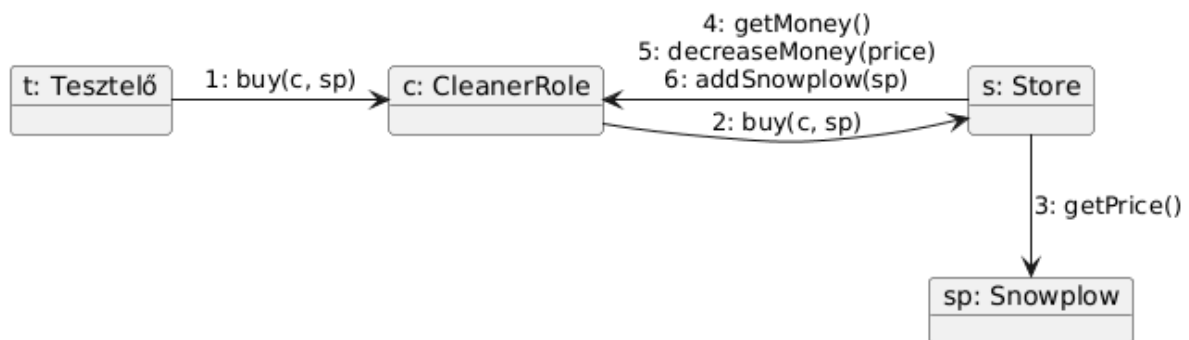
10. Takarítás sósószóró fejjel teszt



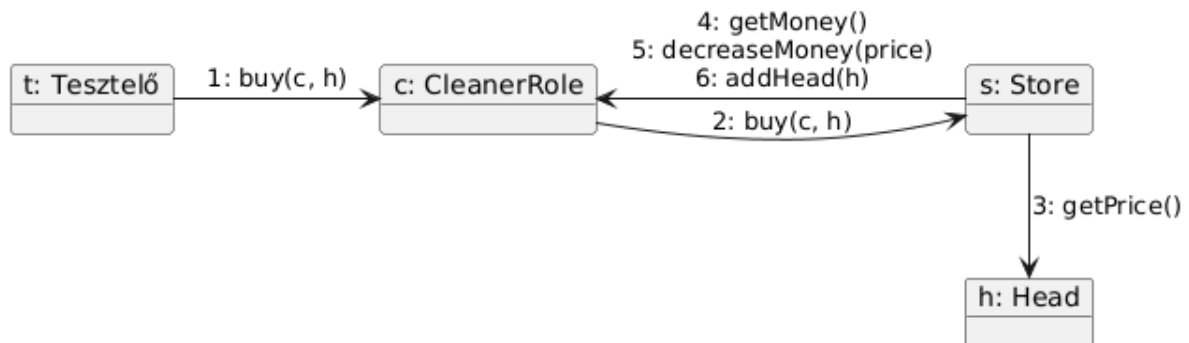
11. Takarítás sárkány fejjel teszt



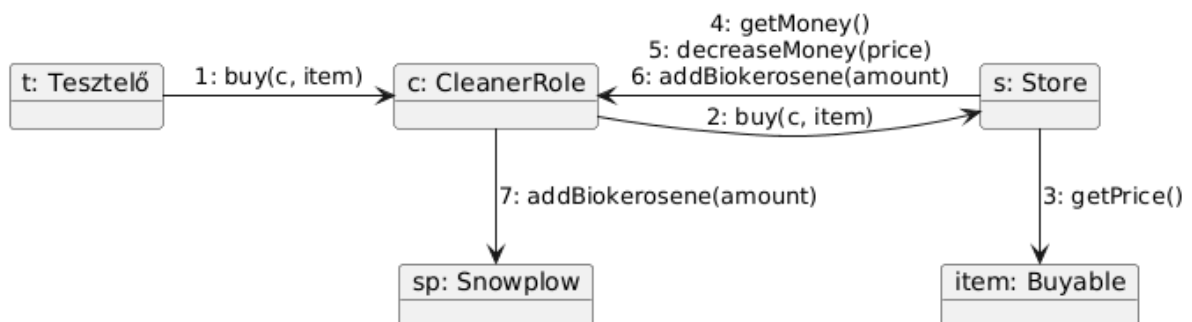
12. Sikeres hókotró vásárlás teszt

12. Sikeres hókotró vásárlás teszt

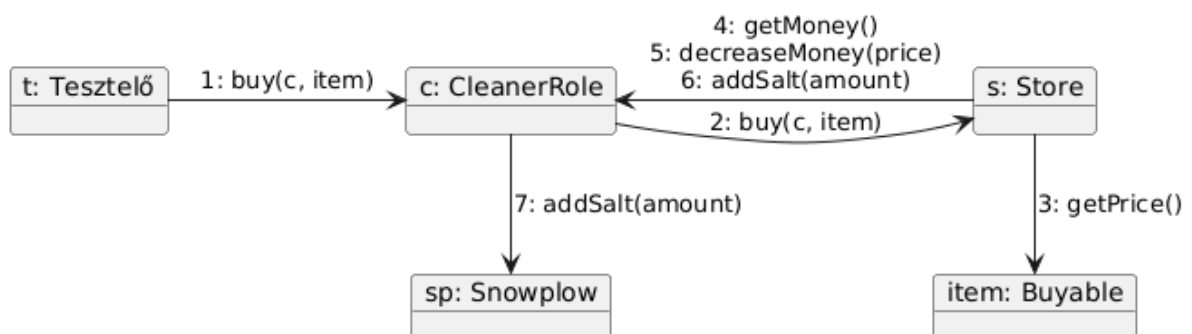
13. Sikeres kotrófej vásárlás teszt

13. Sikeres kotrófej vásárlás teszt

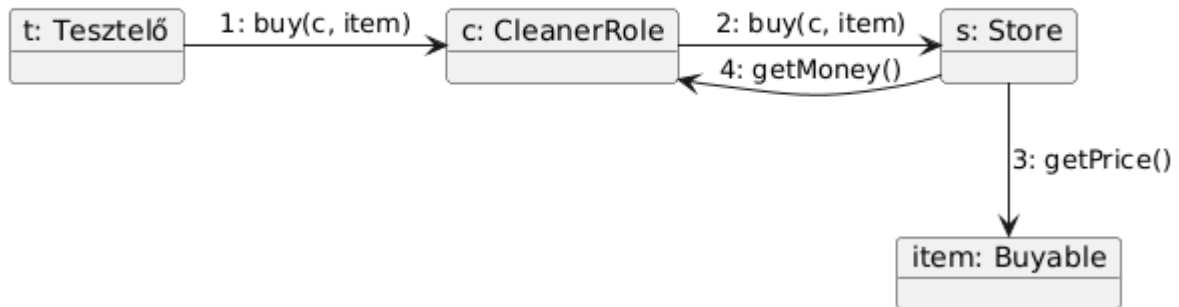
14. Sikeres biokerozin vásárlás teszt

14. Sikeres biokerozin vásárlás teszt

15. Sikeres só vásárlás teszt

15. Sikeres só vásárlás teszt

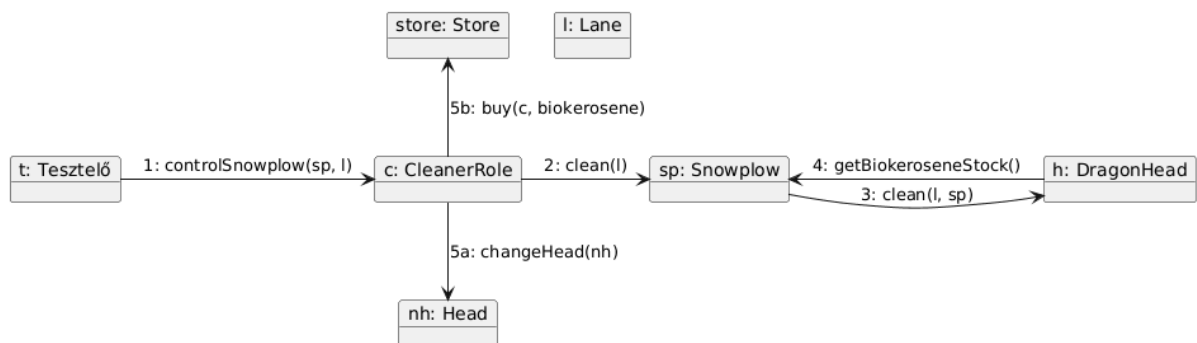
16. Sikertelen vásárlás teszt

16. Sikertelen vásárlás teszt

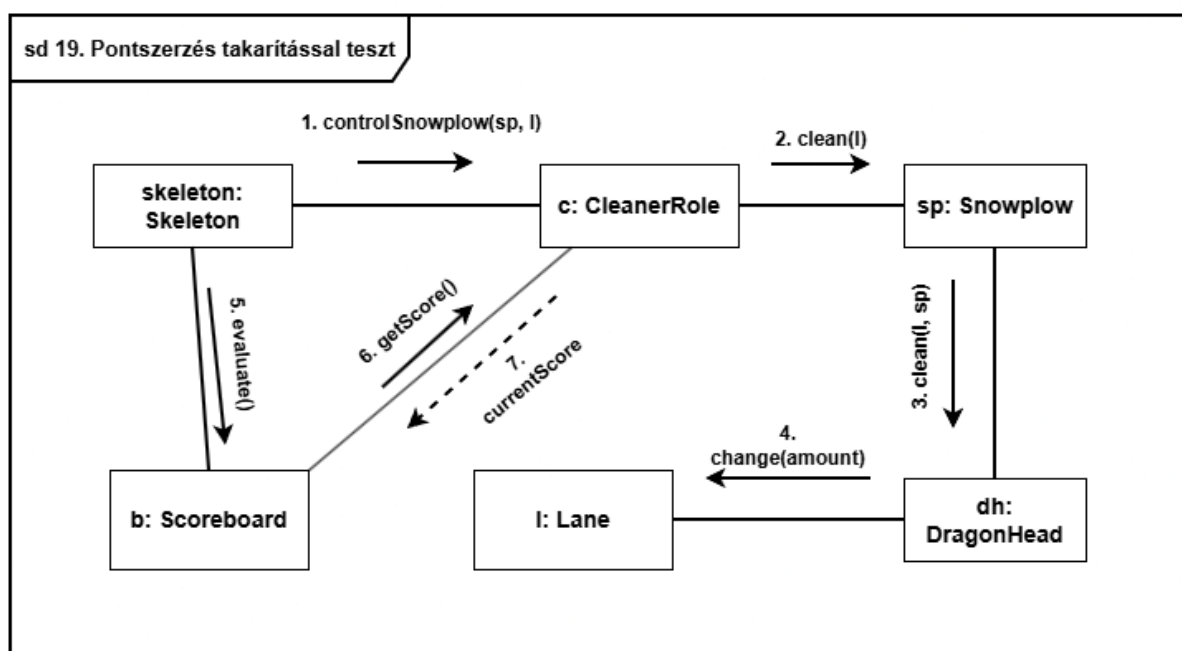
17. Kotrófej cseréje teszt

17. Kotrófej cseréje teszt

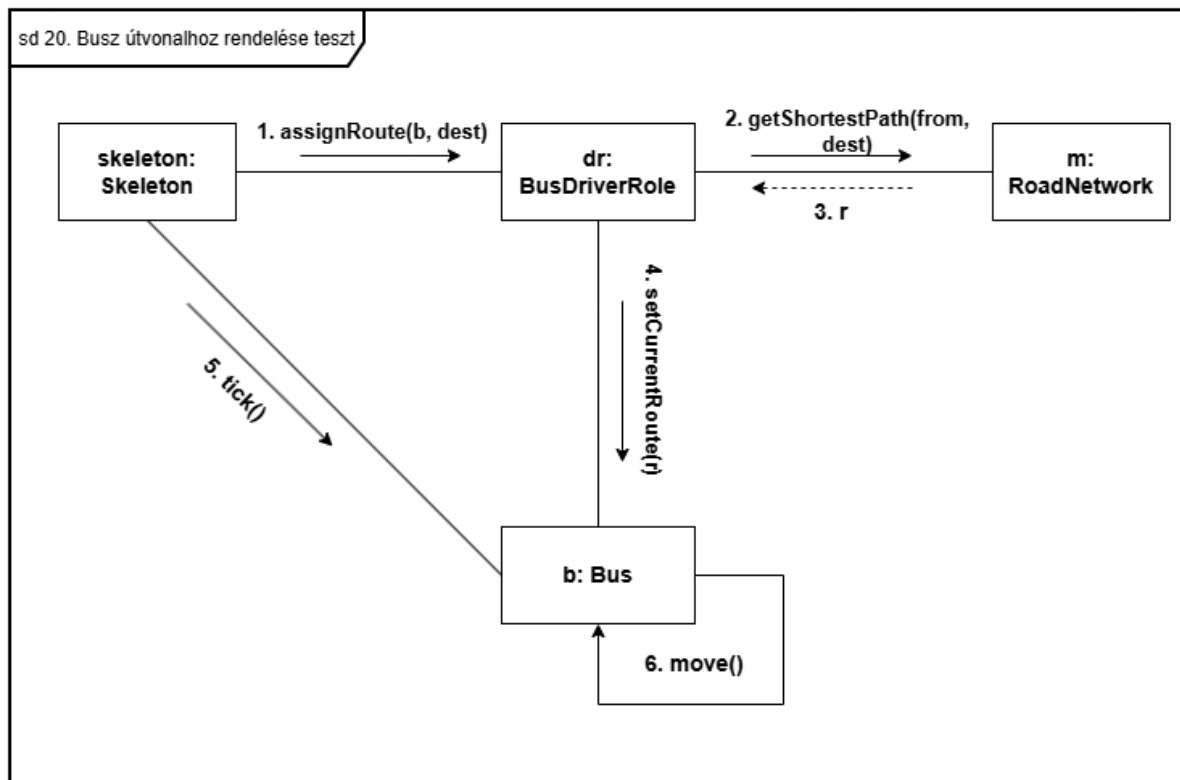
18. Nyersanyag kifogyás teszt

18. Nyersanyag kifogyás teszt

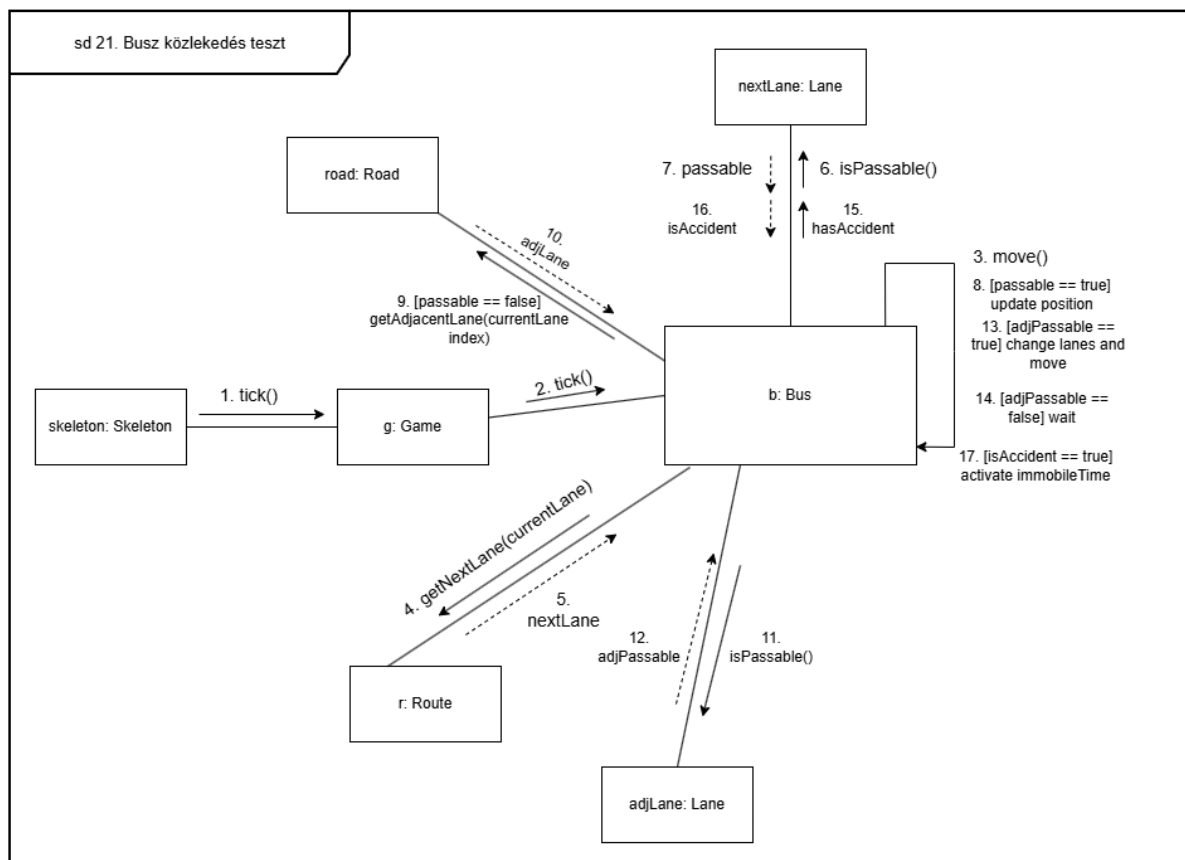
19. Pontszerzés takarítással teszt



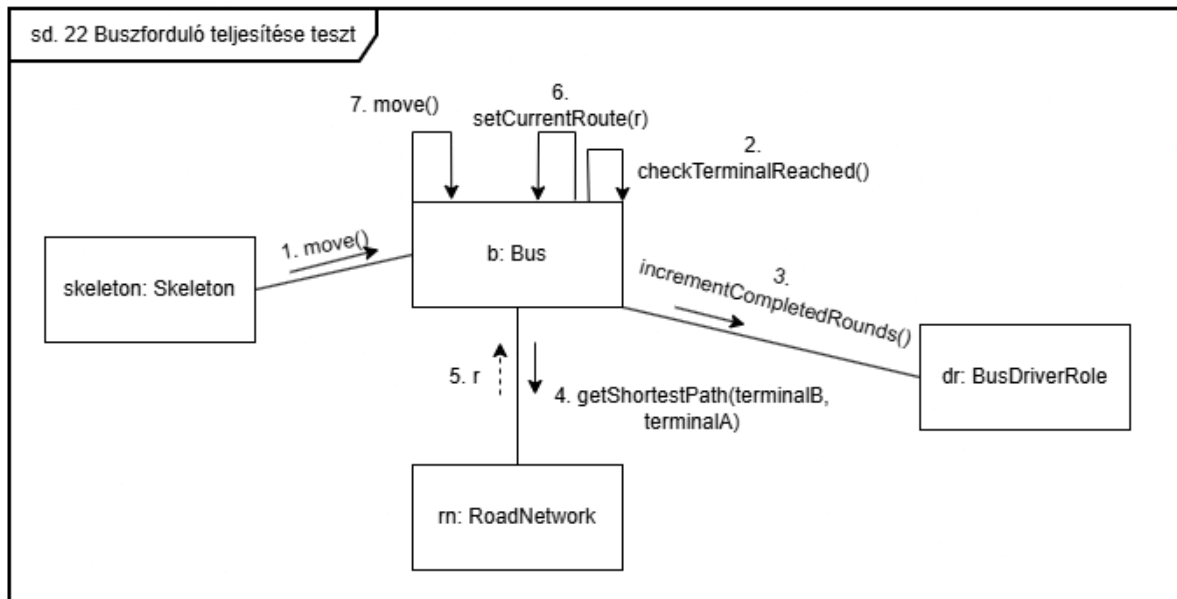
20. Busz útvonalhoz rendelése teszt



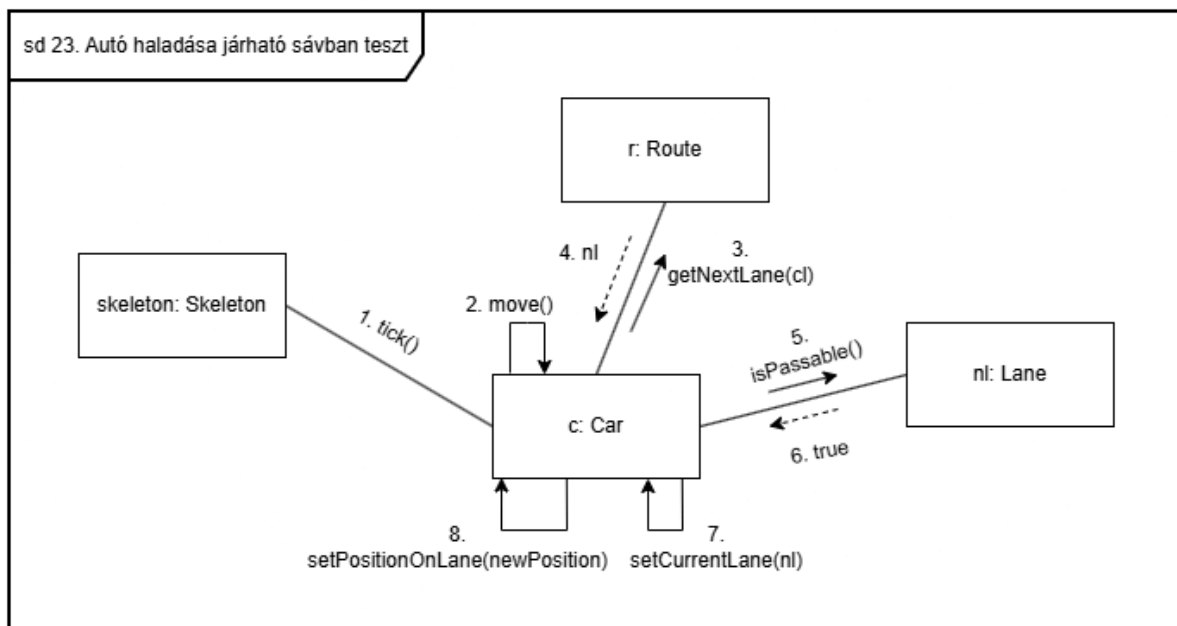
21. Busz közlekedés teszt



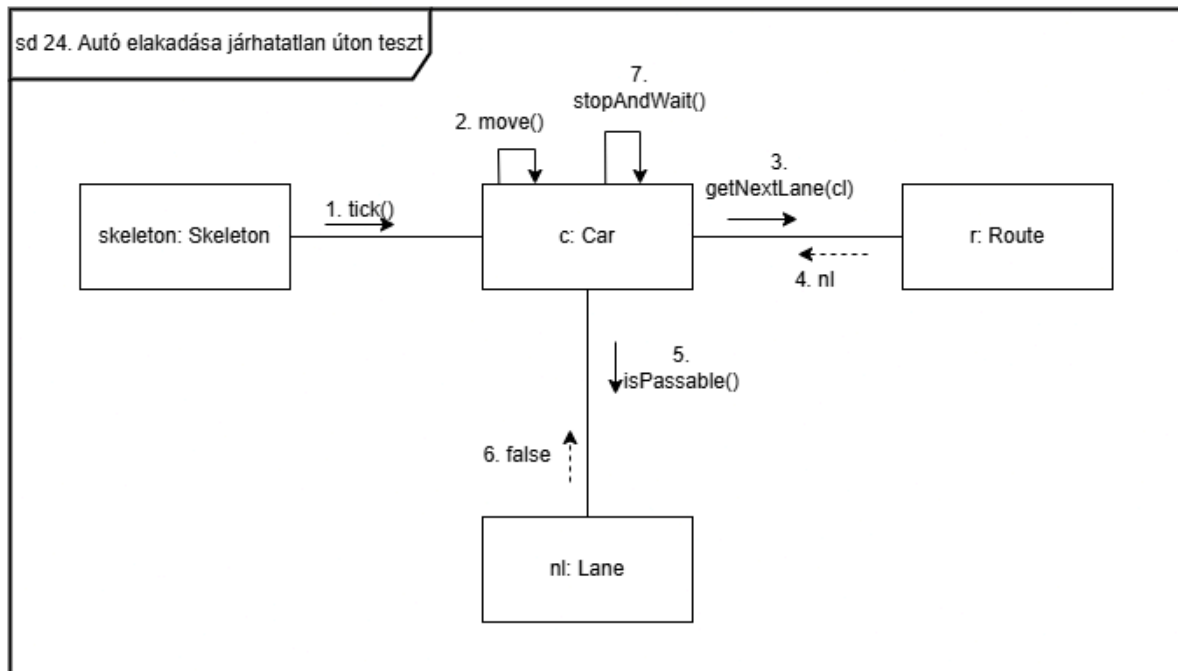
22. Buszforduló teljesítése teszt



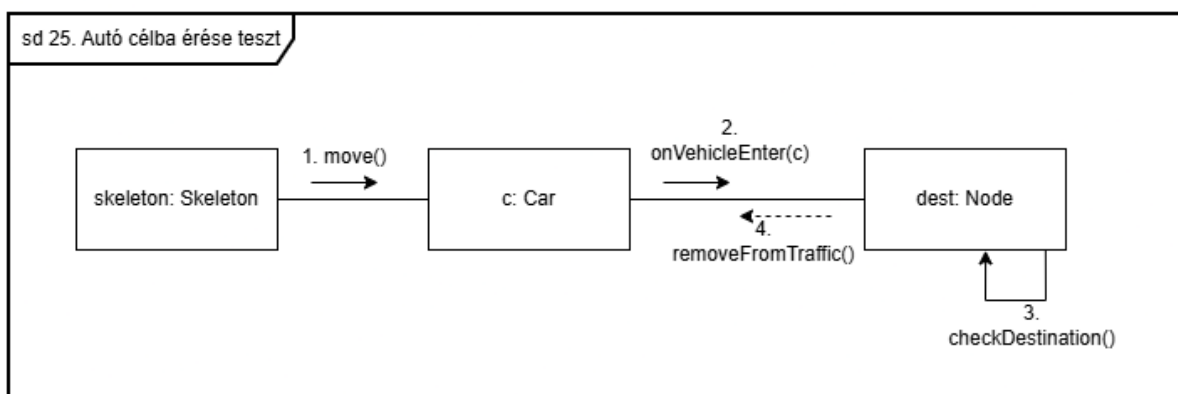
23. Autó haladása járható sávban teszt



24. Autó elakadása járhatatlan úton teszt

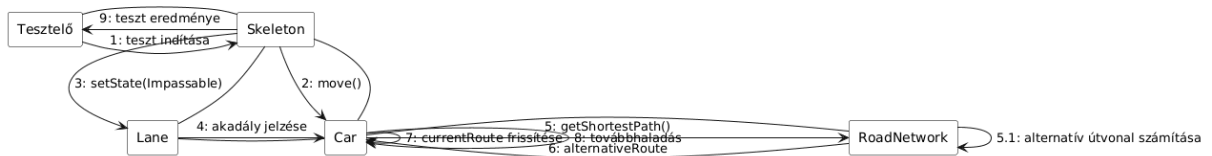


25. Autó célba érése teszt



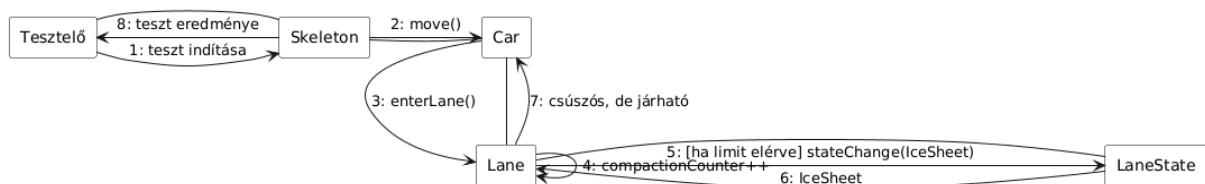
26. Autó útvonalának újratervezése akadály esetén

26. Autó útvonalának újratervezése akadály esetén

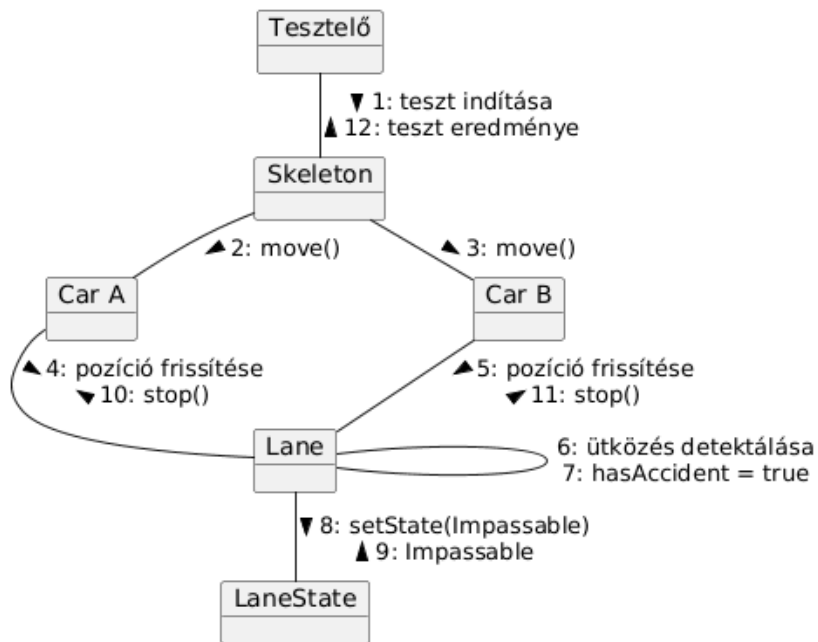


27. Jégpáncél kialakulása teszt

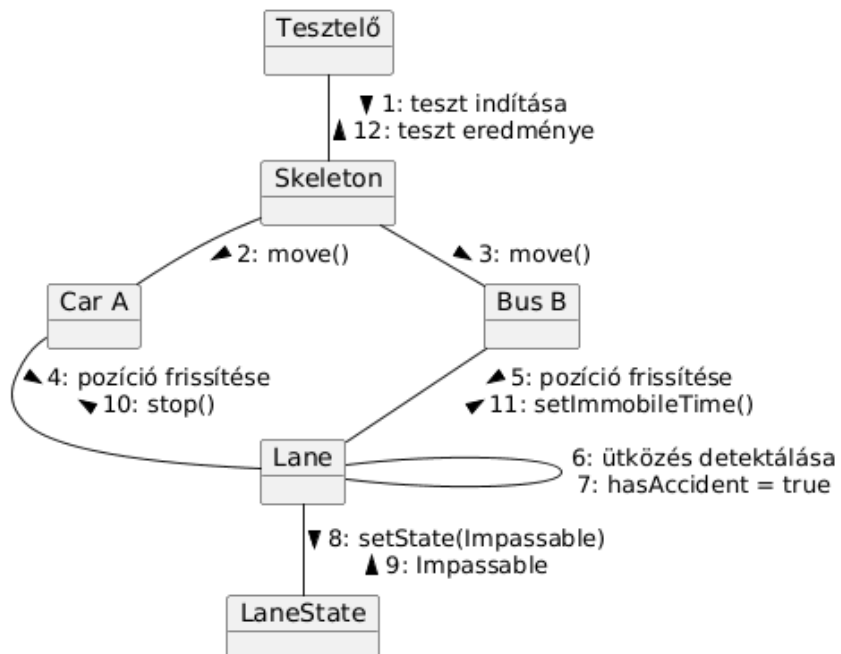
27. Jégpáncél kialakulása teszt



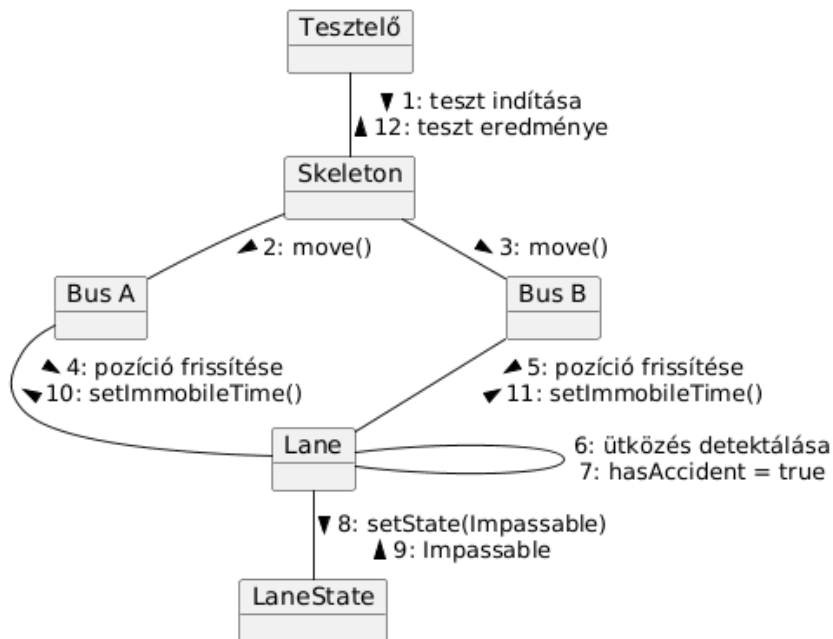
28. Ütközés: autó-autó teszt

28. Ütközés: autó-autó teszt

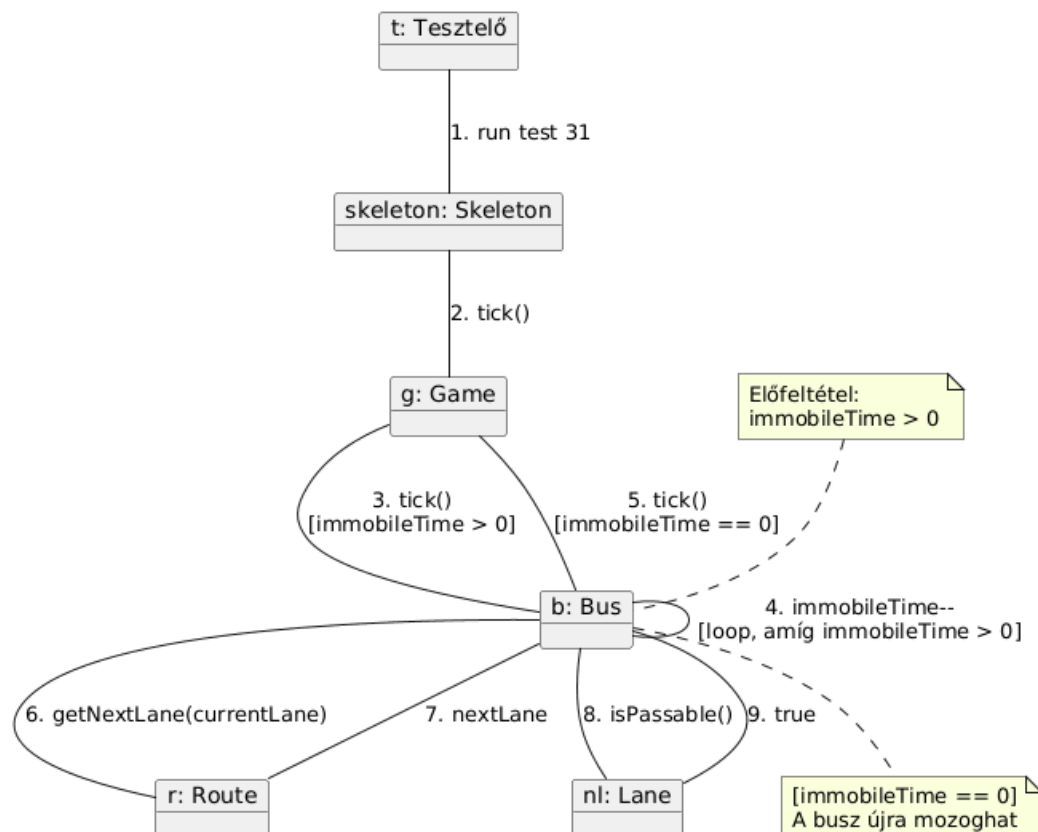
29. Ütközés: autó-busz teszt

29. Ütközés: autó-busz teszt

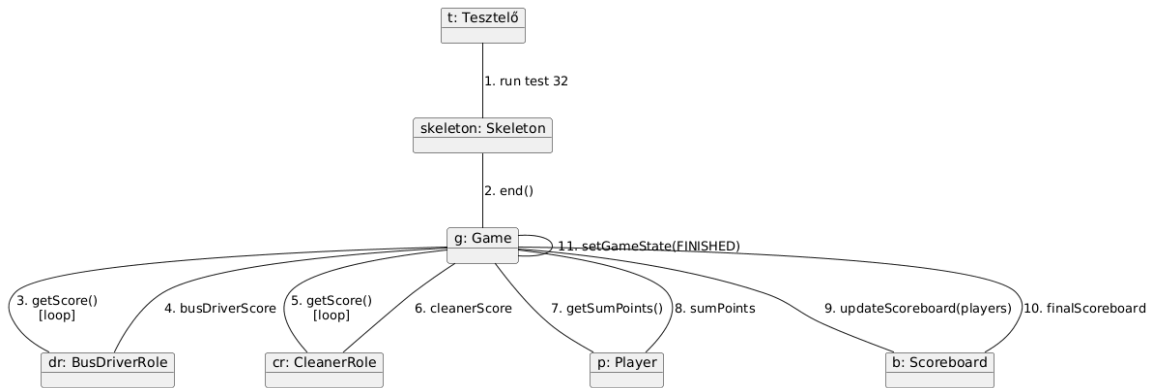
30. Ütközés: busz-busz teszt

30. Ütközés: busz-busz teszt

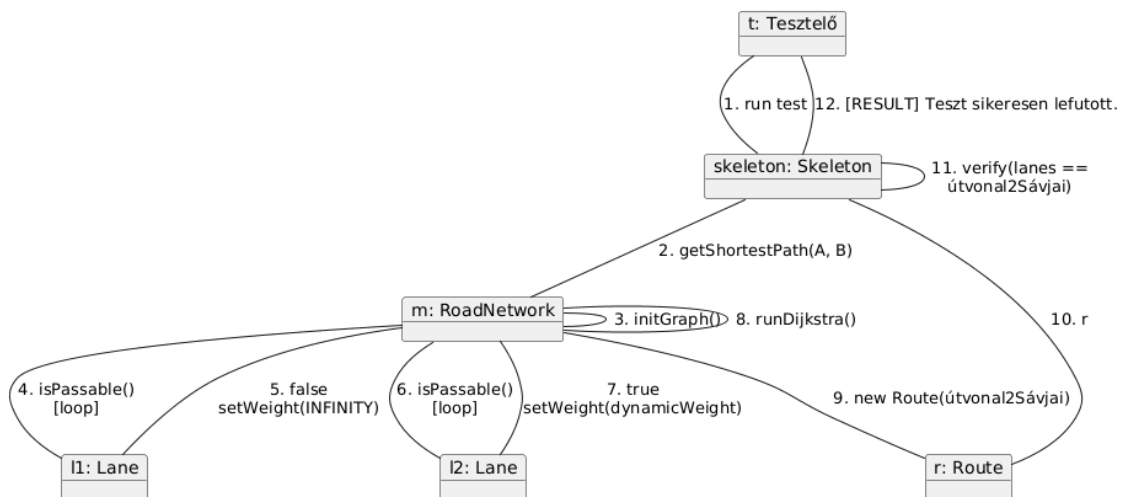
31. Busz mozgásképtelenség



32. Játék kiértékelése



33. Útvonalkereső algoritmus, legrövidebb út



7.5 Napló

Kezdet	Időtartam	Résztevők	Leírás
2026.03.18. 20:30	1 óra	Czap Bocskai Tóth Jandó Dénes	Értekezlet
2026.03.19.	1 óra	Bocskai	Use-case leírások
2026.03.19.	1 óra	Tóth	Use-case leírások
2026.03.19.	1 óra	Czap	Use-case leírások
2026.03.19.	1 óra	Dénes	Use-case leírások
2026.03.20.	3 óra	Bocskai	1. -11. use-case leíráshoz tartozó szekvenciadiagram elkészítése
2026.03.20.	2 óra	Czap	Jármű sávváltás szekvencia és uml diagram javítása, osztályleírás helyesbítése
2026.03.20.	2 óra	Jandó	Use-case leírások
2026.03.21.	2 óra	Jandó	12-18 use-case leírásokhoz tartozó szekvenciadiagram elkészítése
2026.03.21.	2 óra	Czap	19-25. use-case leírásokhoz tartozó szekvenciadiagram elkészítése
2026.03.21.	3 óra	Bocskai	1-11. use-case leírásokhoz tartozó kommunikációs diagramok elkészítése
2026. 03.21.	2 óra	Tóth	7.2 A szkeleton kezelői felületének terve, dialógusok elkészítése
2026.03.22.	2 óra	Jandó	12-18 use case-hez tartozó kommunikációs diagramok elkészítése.
2026.03.22.	3 óra	Czap	19-25. use-case leírásokhoz tartozó kommunikációs

			diagramok elkészítése
2026.03.22.	3 óra	Tóth	26-30. use-case leírásokhoz tartozó szekvencia és kommunikációs diagramok elkészítése
2026.03.22.	3 óra	Dénes	Use case és kommunikációs diagramok kidolgozása

6. Skeleton beadás.

23 – githappens

Konzulens:

Haragos Gergő Viktor

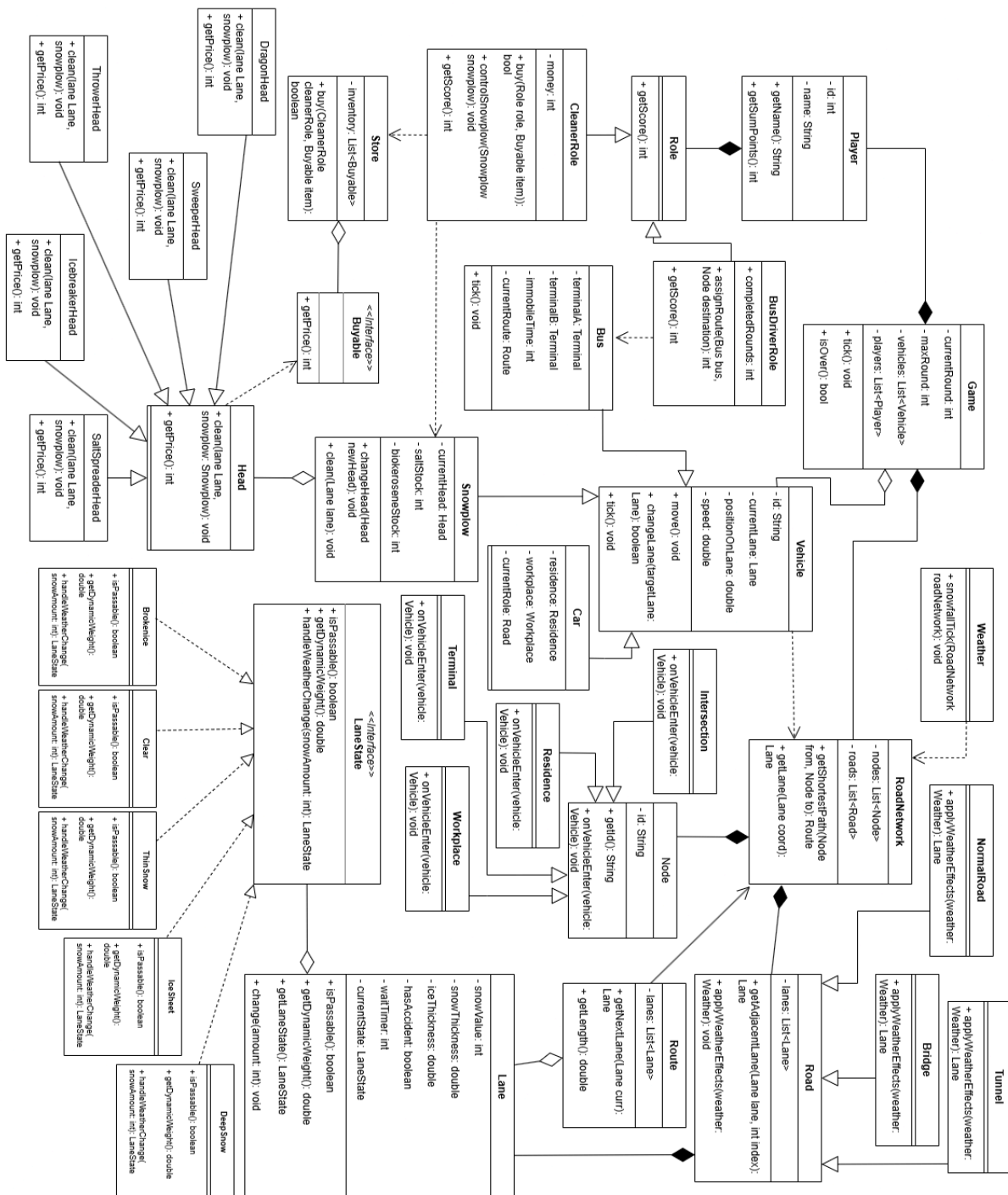
Csapattagok

Czap László	NS7V2T	czap.laszlo@edu.bme.hu
Bocskai Zsófia	UGSTW9	bocskaizsofi@gmail.com
Tóth Márton	K01YXA	toth.marton21@gmail.com
Dénes Péter István	PO6MVA	denespeter03tab@gmail.com
Jandó Barnabás Tamás	AWQ77G	jandobarnabas@gmail.com

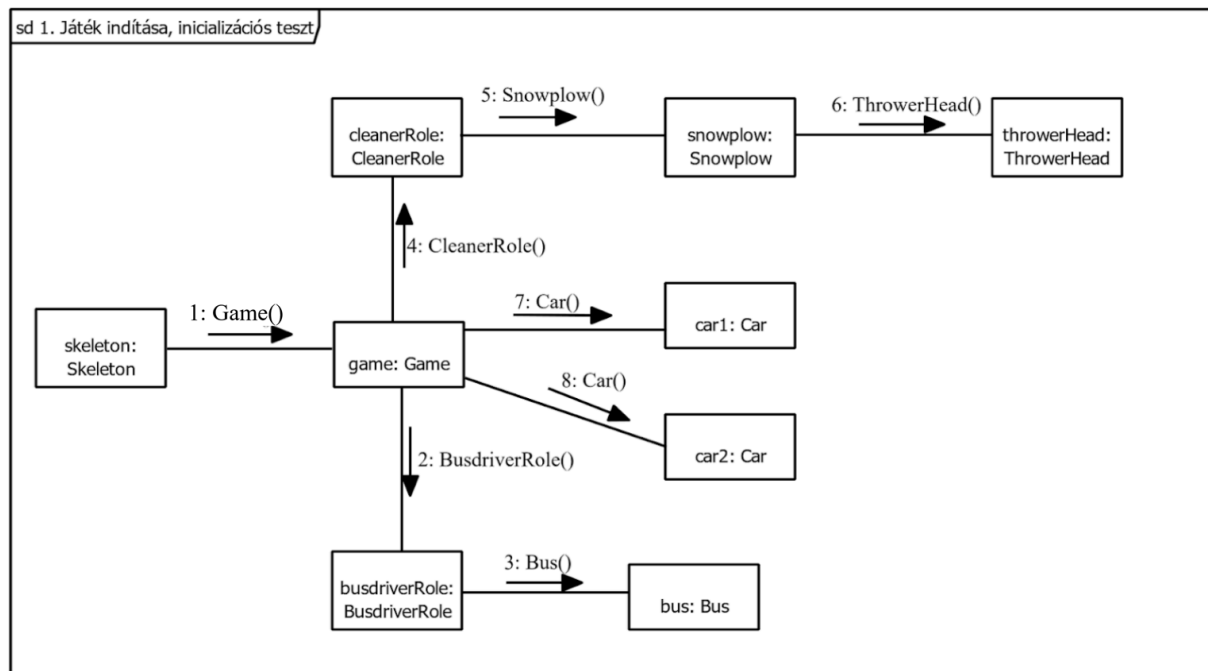
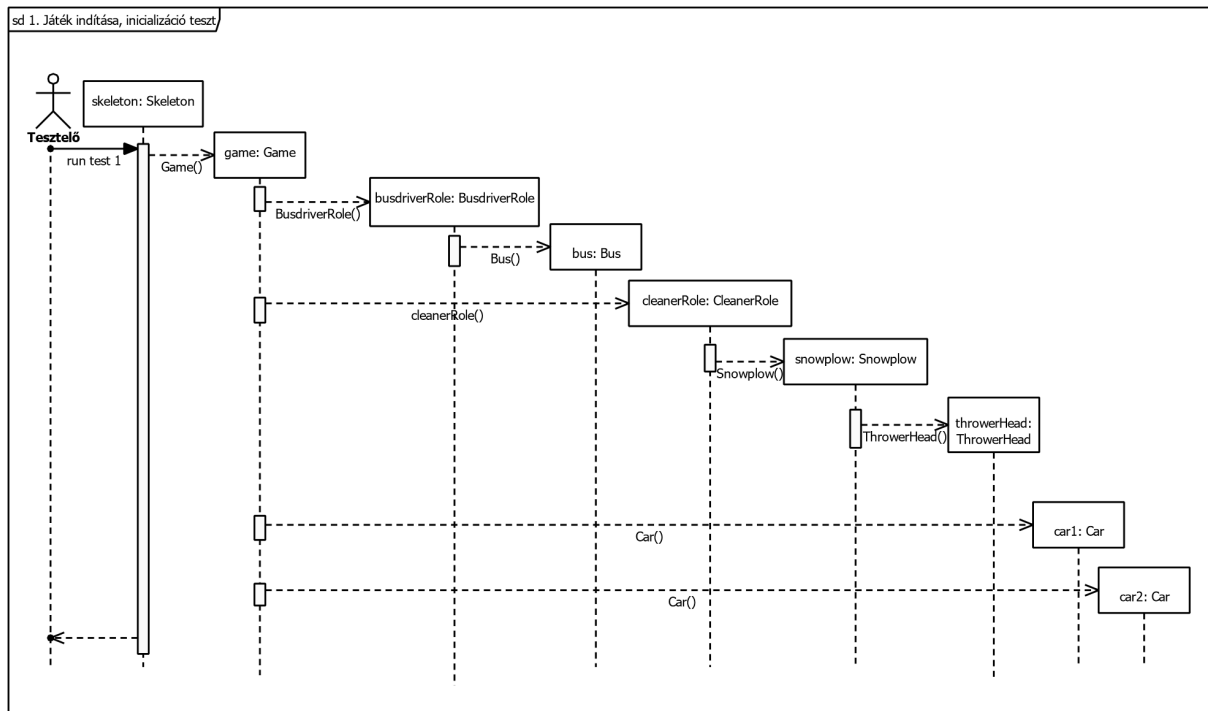
6. Szkeleton beadás

6.1 Javítások

1. Uml diagram javítása



2. Játék indítása, inicializációs teszt



6.2 Fordítási és futtatási útmutató

6.2.1 Fájllista

Fájl neve	Méret	Keletkezés ideje	Tartalom
test_runner.bat	998 byte	2026.03.28.	Windows batch script, amely lefordítja az összes Java fájlt és futtat 32 tesztet,

			majd összehasonlítja az assertion fájlokkal.
SnowplowSkeletonTestProgram.java	11710 byte	2026.03.29.	Fő tesztprogram osztály interaktív CLI menüvel
Bridge.java	610 byte	2026.03.29.	Road típus hidakhoz, időjárás hatás kezelése.
Brokenice.java	1182 byte	2026.03.29.	Lane állapot törött jéghez.
Bus.java	5327 byte	2026.03.30.	Közlekedési jármű, Vehicle-ből származik, terminál végpontokkal és útvonalkezeléssel.
BusdriverRole.java	1692 byte	2026.03.30.	Szerep buszsofőröknek, útvonal- és pontszámkezelés.
Buyable.java	356 byte	2026.03.29.	Interfész vásárolható tárgyakhoz.
Car.java	4403 byte	2026.03.30.	Személygépkocsi osztály, lakhely és munkahely végpontokkal.
CleanerRole.java	4149 byte	2026.03.29.	Szerep takarítónak, hókotrók, pénz és vásárlások kezelése.
Clear.java	1352 byte	2026.03.29.	Lane állapot tiszta utakhoz.
DeepSnow.java	1721 byte	2026.03.29.	Lane állapot mély hóhoz.
DragonHead.java	1723 byte	2026.03.29.	Hókotrórészt, amely biokerószint használ jég olvasztásához.
Game.java	2746 byte	2026.03.29.	Központi játékvezérlő, körök, járművek és játékosok kezelése.
Head.java	525 byte	2026.03.29.	Absztrakt bázisosztály hókotrórészekhez.
IcebreakerHead.java	1031 byte	2026.03.29.	Hókotrórészt jégtöréshez.
IceSheet.java	1564 byte	2026.03.29.	Lane állapot jégtakaróhoz.
Intersection.java	942 byte	2026.03.29.	Node típus útkereszteződésekhez.
Lane.java	3232 byte	2026.03.29.	Egyetlen út sáv állapotkezeléssel és időjárás hatásokkal.
LaneState.java	789 byte	2026.03.29.	Interfész sávállapotok definiálásához.
Node.java	787 byte	2026.03.30.	Bázisosztály hálózati node-okhoz.
NormalRoad.java	845 byte	2026.03.29.	Normál út típus normál időjárási hatásokkal.
Player.java	2094 byte	2026.03.29.	Emberi játékos osztály több szereppel és pontszám-számítással.
Residence.java	664 byte	2026.03.29.	Node típus lakóövezetekhez.

Road.java	532 byte	2026.03.29.	Absztrakt bázisosztály út típusokhoz.
RoadNetwork.java	1061 byte	2026.03.29.	Hálózat topológia és legrövidebbút-számítás kezelése.
Role.java	494 byte	2026.03.29.	Absztrakt bázisosztály játékos szerepekhez.
Route.java	957 byte	2026.03.29.	Számított út sávokon keresztül.
SaltSpreaderHead.java	1079 byte	2026.03.29.	Hókotrórészt szószóróhoz.
Skeleton.java	3095 byte	2026.03.29.	Központi naplózó utility, kimenetformázás és inputkezelés.
Snowplow.java	3256 byte	2026.03.29.	Hókotró jármű cserélhető részekkel és erőforráskezeléssel.
Store.java	1844 byte	2026.03.30.	Bolt vásárlások kezelése pénzzel és készlettel.
SweeperHead.java	1333 byte	2026.03.29.	Hókotrórészt hósepréshez.
Terminal.java	825 byte	2026.03.30.	Node típus buszterminálokhoz.
ThinSnow.java	1325 byte	2026.03.29.	Lane állapot vékony hóhoz.
ThrowerHead.java	1187 byte	2026.03.29.	Hókotrórészt hó eldobásához.
Tunnel.java	606 byte	2026.03.29.	Út típus alagutakhoz.
Vehicle.java	3077 byte	2026.03.29.	Absztrakt bázisosztály járművekhez.
Weather.java	692 byte	2026.03.29.	Időjárási rendszer hőzáró események kezelésére.
Workplace.java	975 byte	2026.03.30.	Node típus munkahelyekhez.
Test1.java	787 byte	2026.03.29.	Játék indítása, inicializáció teszt
Test2.java	851 byte	2026.03.29.	Havazás hatása az útállapotra teszt
Test3.java	932 byte	2026.03.29.	Alagút időjárás-mentessége teszt
Test4.java	894 byte	2026.03.29.	Jármű behajtása kereszteződésbe teszt
Test5.java	686 byte	2026.03.29.	Jármű sávváltása teszt
Test6.java	637 byte	2026.03.29.	Hókotró haladása teszt
Test7.java	954 byte	2026.03.29.	Takarítás sörpő fejjel teszt
Test8.java	1001 byte	2026.03.29.	Takarítás hányó fejjel teszt
Test9.java	992 byte	2026.03.29.	Takarítás jégtörő fejjel teszt
Test10.java	746 byte	2026.03.29.	Takarítás szószóró fejjel teszt
Test11.java	720 byte	2026.03.29.	Takarítás sárkány fejjel teszt
Test13.java	1876 byte	2026.03.29.	Sikeres kotrófej vásárlás teszt
Test14.java	2255 byte	2026.03.30.	Sikeres biokerozin vásárlás teszt
Test15.java	2379 byte	2026.03.29.	Sikeres só vásárlás teszt

Test16.java	2231 byte	2026.03.30.	Sikertelen vásárlás teszt
Test17.java	957 byte	2026.03.29.	Kotrófej cseréje teszt
Test18.java	2585 byte	2026.03.29.	Nyersanyag kifogyás teszt
Test19.java	936 byte	2026.03.29.	Pontszerzés takarítással teszt
Test20.java	894 byte	2026.03.29.	Busz útvonalhoz rendelése teszt
Test21.java	589 byte	2026.03.29.	Busz közlekedés teszt
Test22.java	954 byte	2026.03.30.	Busz forduló teljesítése teszt
Test23.java	1187 byte	2026.03.29.	Autó haladása járható sávban teszt
Test24.java	1069 byte	2026.03.30.	Autó elakadása járhatatlan úton teszt
Test25.java	455 byte	2026.03.30.	Autó célba érése teszt
Test26.java	2210 byte	2026.03.30.	Autó útvonalának újratervezése akadály esetén
Test27.java	1533 byte	2026.03.29.	Jégpáncél kialakulása teszt
Test28.java	1659 byte	2026.03.30.	Ütközés: autó-autó teszt
Test29.java	1879 byte	2026.03.30.	Ütközés: autó-busz teszt
Test30.java	1949 byte	2026.03.30.	Ütközés: busz-busz teszt
Test31.java	1951 byte	2026.03.30.	Busz mozgásképtelenség megszűnése teszt
Test32.java	1606 byte	2026.03.30.	Játék kiértékelése teszt
Test33.java	2155 byte	2026.03.30.	Útvonalkereső algoritmus, legrövidebb út tesztje
TestCase.java	681 byte	2026.03.29.	Általános teszteset-vázlat definiál, amelyből a konkrét tesztek származnak.
test1_assert.txt	422 byte	2026.03.29.	Várt kimenet az 1. teszthez.
test2_assert.txt	489 byte	2026.03.29.	Várt kimenet az 2. teszthez.
test3_assert.txt	196 byte	2026.03.29.	Várt kimenet az 3. teszthez.
test4_assert.txt	231 byte	2026.03.29.	Várt kimenet az 4. teszthez.
test5_assert.txt	341 byte	2026.03.29.	Várt kimenet az 5. teszthez.
test6_assert.txt	257 byte	2026.03.29.	Várt kimenet az 6. teszthez.
test7_assert.txt	870 byte	2026.03.29.	Várt kimenet az 7. teszthez.
test8_assert.txt	1002 byte	2026.03.29.	Várt kimenet az 8. teszthez.
test9_assert.txt	899 byte	2026.03.29.	Várt kimenet az 9. teszthez.
test10_assert.txt	936 byte	2026.03.29.	Várt kimenet az 10. teszthez.
test11_assert.txt	1003 byte	2026.03.29.	Várt kimenet az 11. teszthez.
test13_assert.txt	793 byte	2026.03.29.	Várt kimenet az 13. teszthez.
test14_assert.txt	1103 byte	2026.03.29.	Várt kimenet az 14. teszthez.
test15_assert.txt	829 byte	2026.03.29.	Várt kimenet az 15. teszthez.
test16_assert.txt	680 byte	2026.03.30.	Várt kimenet az 16. teszthez.
test17_assert.txt	559 byte	2026.03.29.	Várt kimenet az 17. teszthez.
test18_assert.txt	1268 byte	2026.03.29.	Várt kimenet az 18. teszthez.
test19_assert.txt	857 byte	2026.03.30.	Várt kimenet az 19. teszthez.
test20_assert.txt	792 byte	2026.03.30.	Várt kimenet az 20. teszthez.
test21_assert.txt	793 byte	2026.03.30.	Várt kimenet az 21. teszthez.
test22_assert.txt	983 byte	2026.03.30.	Várt kimenet az 22. teszthez.

test23_assert.txt	506 byte	2026.03.30.	Várt kimenet az 23. teszthez.
test24_assert.txt	575 byte	2026.03.30.	Várt kimenet az 24. teszthez.
test25_assert.txt	230 byte	2026.03.30.	Várt kimenet az 25. teszthez.
test26_assert.txt	725 byte	2026.03.30.	Várt kimenet az 26. teszthez.
test27_assert.txt	591 byte	2026.03.30.	Várt kimenet az 27. teszthez.
test28_assert.txt	566 byte	2026.03.30.	Várt kimenet az 28. teszthez.
test29_assert.txt	1034 byte	2026.03.30.	Várt kimenet az 29. teszthez.
test30_assert.txt	1594 byte	2026.03.30.	Várt kimenet az 30. teszthez.
test31_assert.txt	568 byte	2026.03.30.	Várt kimenet az 31. teszthez.
test32_assert.txt	463 byte	2026.03.30.	Várt kimenet az 32. teszthez.
test33_assert.txt	336 byte	2026.03.30.	Várt kimenet az 33. teszthez.
test1_in.txt	0 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test2_in.txt	1 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test3_in.txt	0 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test4_in.txt	0 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test5_in.txt	1 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test6_in.txt	0 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test7_in.txt	1 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test8_in.txt	16 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test9_in.txt	16 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test10_in.txt	17 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test11_in.txt	23 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test13_in.txt	1 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.

test14_in.txt	1 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test15_in.txt	1 byte	2026.03.30.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test16_in.txt	1 byte	2026.03.30.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test17_in.txt	0 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test18_in.txt	1 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test19_in.txt	0 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test20_in.txt	4 byte	2026.03.30.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test21_in.txt	4 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test22_in.txt	4 byte	2026.03.30.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test23_in.txt	1 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test24_in.txt	1 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test25_in.txt	0 byte	2026.03.29.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test26_in.txt	4 byte	2026.03.30.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test27_in.txt	0 byte	2026.03.30.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test28_in.txt	4 byte	2026.03.30.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test29_in.txt	10 byte	2026.03.30.	Tartalmazza az előre meghatározott felhasználói válaszokat.

test30_in.txt	16 byte	2026.03.30.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test31_in.txt	0 byte	2026.03.30.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test32_in.txt	0 byte	2026.03.30.	Tartalmazza az előre meghatározott felhasználói válaszokat.
test33_in.txt	0 byte	2026.03.30.	Tartalmazza az előre meghatározott felhasználói válaszokat.

6.2.2 Fordítás

A program futtatása parancssorból történik. A helyes működéshez, relatív útvonalak feloldásához a felhasználónak először be kell lépnie a skeleton mappába, majd onnan fordíthatja és indíthatja a tesztprogramot.

Git bash:

- `mkdir -p skeleton/bin`
- `javac -d bin src/*.java tests/TestCase.java tests/*.java SnowplowSkeletonTestProgram.java`

Windows PowerShell:

- `mkdir -p skeleton\bin`
- `javac -d bin src/*.java tests/*.java SnowplowSkeletonTestProgram.java`

6.2.3 Futtatás

Git bash:

- `java -cp bin skeleton.SnowplowSkeletonTestProgram`
- Automatizált futtatás: `./test_runner.bat` terminálból vagy dupla kattintással fájlra

Windows PowerShell:

- `java -cp bin skeleton.SnowplowSkeletonTestProgram`
- Automatizált futtatás: `./test_runner.bat` terminálból vagy dupla kattintással fájlra

6.3 Értékelés

Tag neve	Tag neptun	Munka százalékban
Czap László	NS7V2T	20%
Bocskai Zsófia	UGSTW9	20%
Tóth Márton	K01YXA	20%
Dénes Péter István	PO6MVA	20%
Jandó Barnabás Tamás	AWQ77G	20%

6.4 Napló

Kezdet	Időtartam	Résztevők	Leírás
2026.03.25. 21:00	2 óra	Czap Bocskai Tóth Jandó Dénes	Értekezlet.
2026.03.26.	2 óra	Tóth	Skeleton futtató program megírása
2026.03.26.	6 óra	Czap	Batch teszt futtató megírása, teszt be és kimenet összehasonlításának kezelése.
2026.03.27.	2 óra	Tóth	Szükséges skeleton osztályok megírása
2026.03.27.	3 óra	Dénes	Skeleton osztályok megírása
2026.03.27.	3 óra	Czap	Úthálózathoz kapcsolódó osztályok megírása.
2026.03.27.	2 óra	Bocskai	Player, Role, BusdriverRole, CleanerRole osztályok megírása
2026.03.27.	3 óra	Tóth	8-10 teszt megírása
2026.03.28.	4 óra	Dénes	tesztek és skeleton
2026.03.28.	3 óra	Bocskai	1-7 teszt megírásának elkezdése
2026.03.28.	4 óra	Czap	19-25 tesztek megkezdése
2026.03.29.	2 óra	Czap Bocskai Tóth Jandó Dénes	Értekezlet.
2026.03.29.	6 óra	Bocskai	1-7 teszt befejezése
2026.03.29.	6 óra	Tóth	Tesztek javítása
2026.03.29.	6 óra	Dénes	tesztek és skeleton
2026.03.29.	6 óra	Czap	Tesztek javítása, befejezése. Futás szépítése.
2026.03.29.	6 óra	Jandó	11-18 tesztek implementálása
2026.03.30.	1 óra	Tóth	Dokumentáció írása

7. Prototípus koncepciója

23 – githappens

Konzulens:

Haragos Gergő Viktor

Csapattagok

Czap László	NS7V2T	czap.laszlo@edu.bme.hu
Bocskai Zsófia	UGSTW9	bocskaizsofi@gmail.com
Tóth Márton	K01YXA	toth.marton21@gmail.com
Dénes Péter István	PO6MVA	denespeter03tab@gmail.com
Jandó Barnabás Tamás	AWQ77G	jandobarnabas@gmail.com

2026.03.22.

Speciális kotrófej, amely zúzott követ (zuzalékot) szór a jármű előtt. Működéséhez zúzott kő szükséges, ebből csak véges mennyiség fér el egyidőben a hókotrón és a boltból (Store) lehet megvásárolni.

- **Össosztályok**

Head

- **Interfészek**

Buyable

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **void clean(Lane lane, Snowplow snowplow):** Ellenőrzi a hókotró zúzott kő készletét, majd annak felhasználásával a sávot tisztítja.
- **int getPrice():** Az interfészből származó metódus, visszaadja a zúzott követ szóró fej árát

Gravel

- **Felelősség**

Az osztály a sáv zúzott kővel fedett állapotát reprezentálja. A kiszórt zuzalék a jég csúszósságát megszünteti, de a söprő és hányó fej a zuzalékot ugyanúgy eltakarítja, mint a havat. A lángszóró, a jégtörő és a só a zuzalékra nem hat. A zuzalék nem tömörödik. Ha hó esik rá, akkor a hó egy idő után befedi, a hó a szokott módon letaposható és így lefagyott jéggé válik.

- **Össosztályok**

Nincs össosztálya.

- **Interfészek**

LaneState

- **Asszociációk**

Nincs saját asszociációja

- **Attribútumok**

Nincs saját attribútuma

- **Metódusok**

- **boolean isPassable()**: Az út járhatóságát lehet vele lekérdezni.
- **double getDynamicWeight()**: Meghatározza az útvonaltervezés során használt dinamikus súlyt.
- **LaneState handleWeatherChange (int snowAmount)**: Az időjárás változásának kezelése.

Snowplow

Új attribútummal bővült:

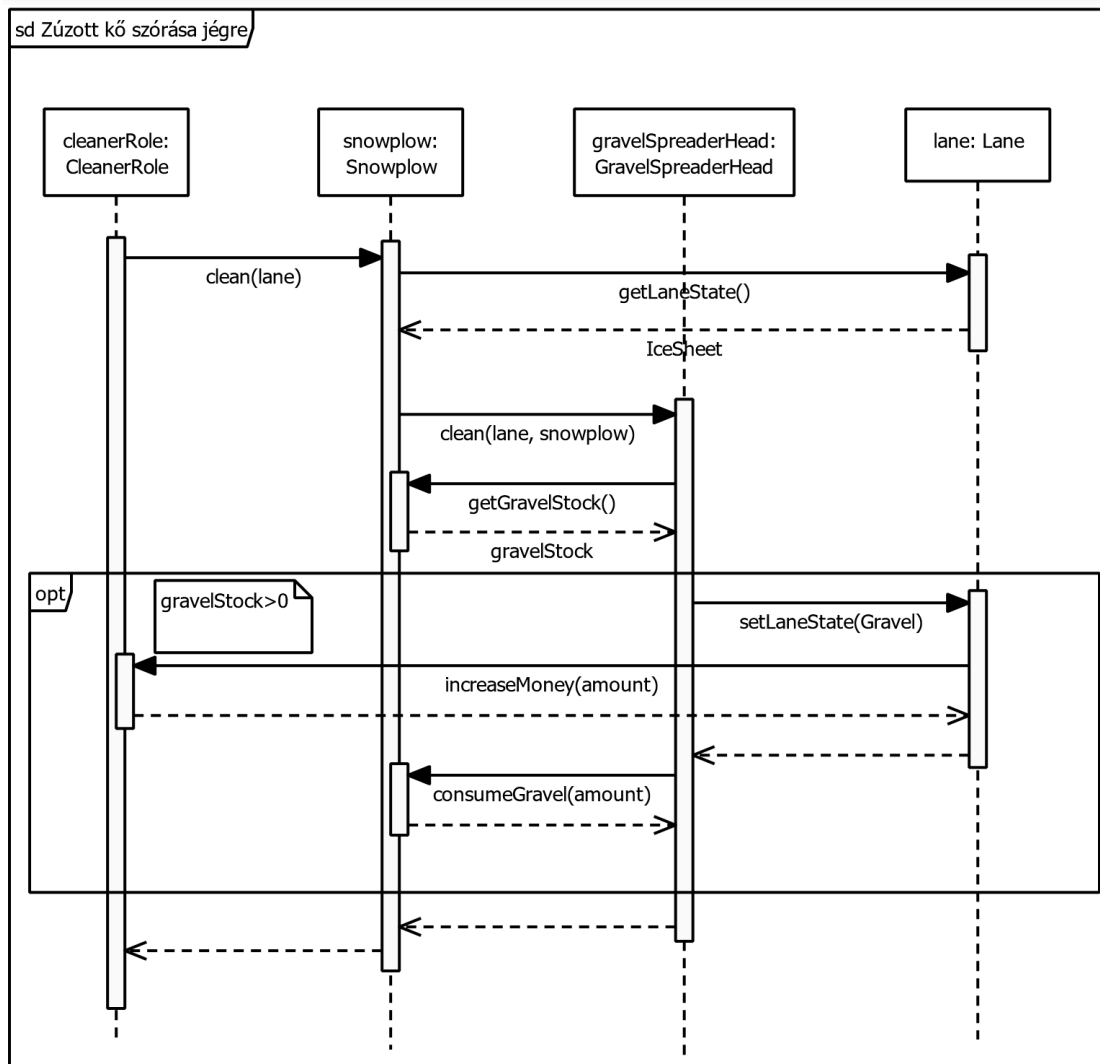
- **gravelStock: int** - A hókotróban tárolt zúzott kő mennyiség

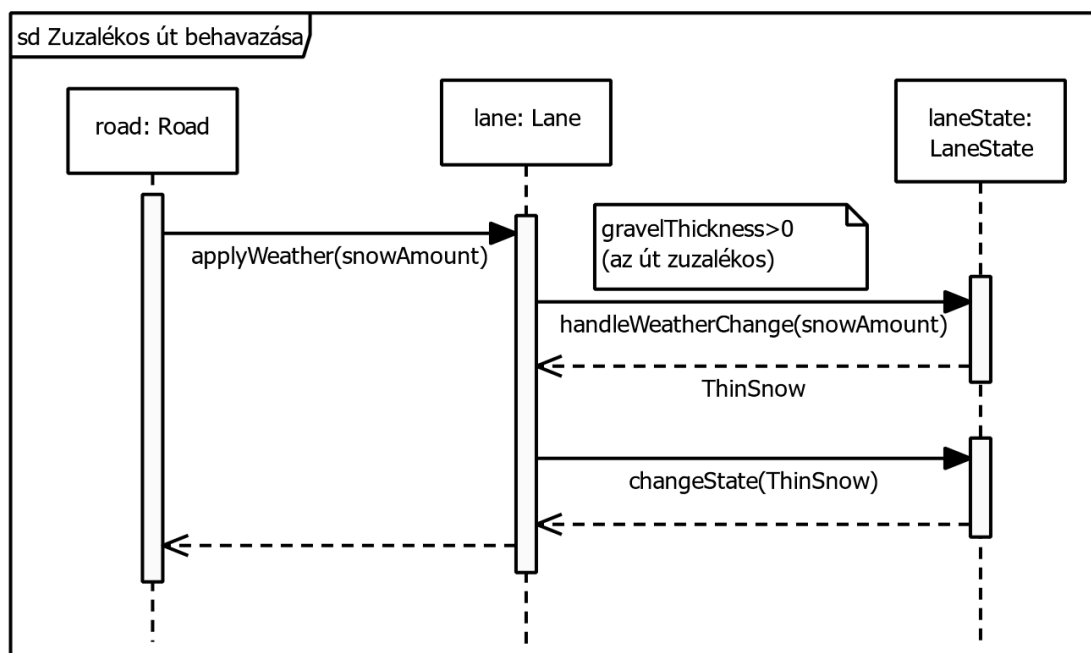
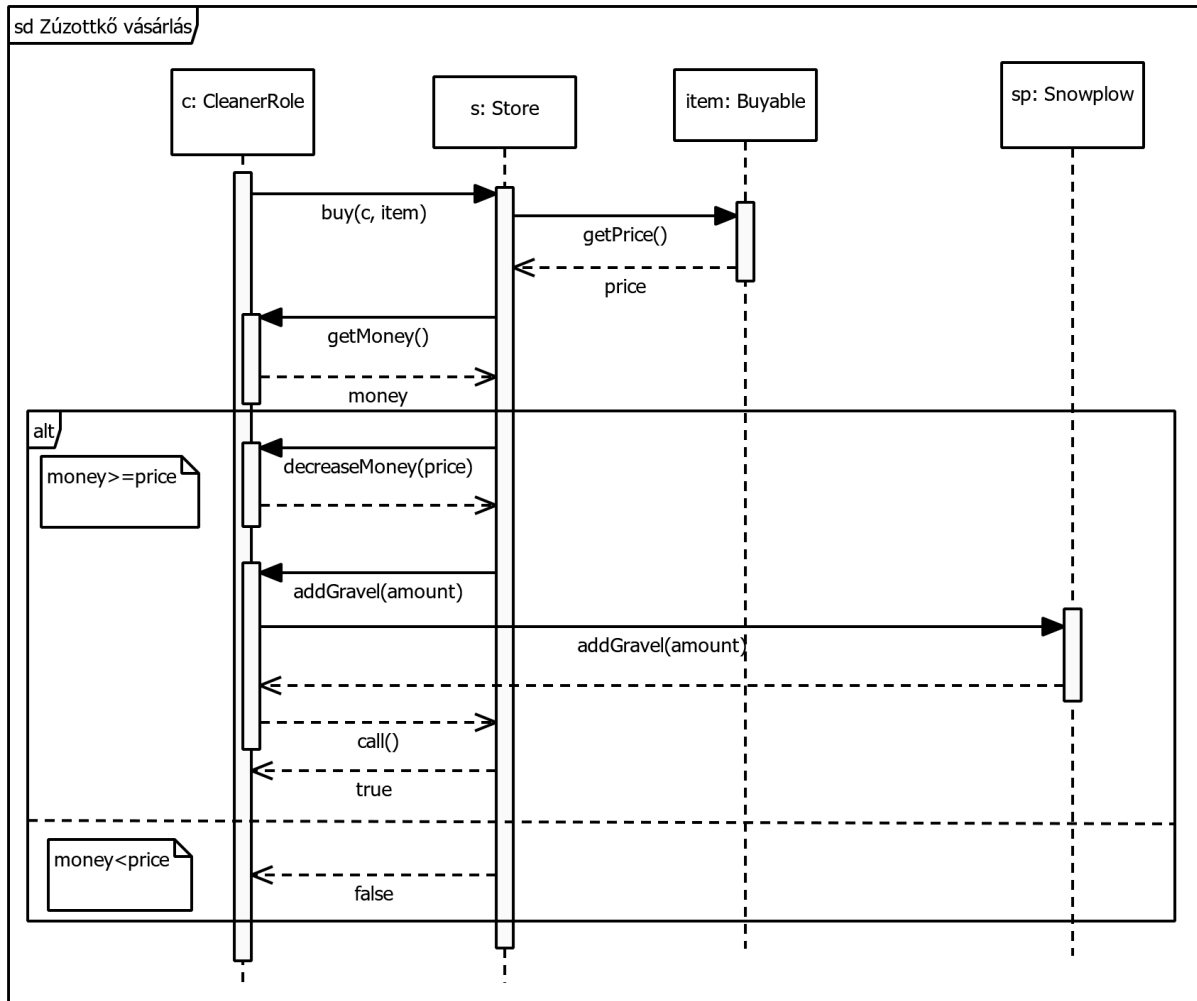
Lane

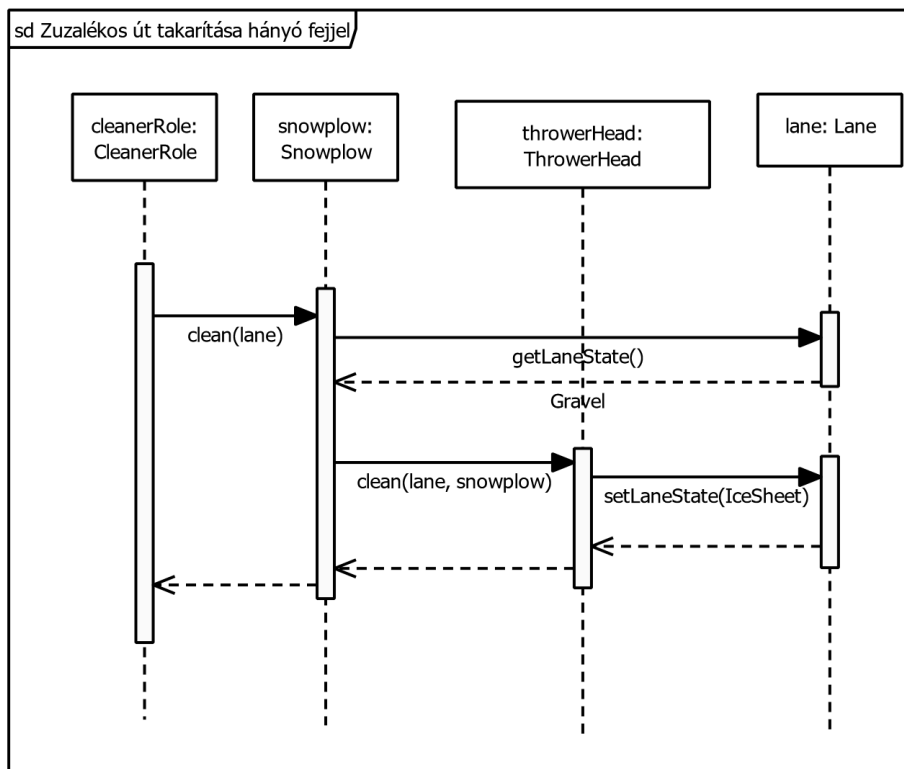
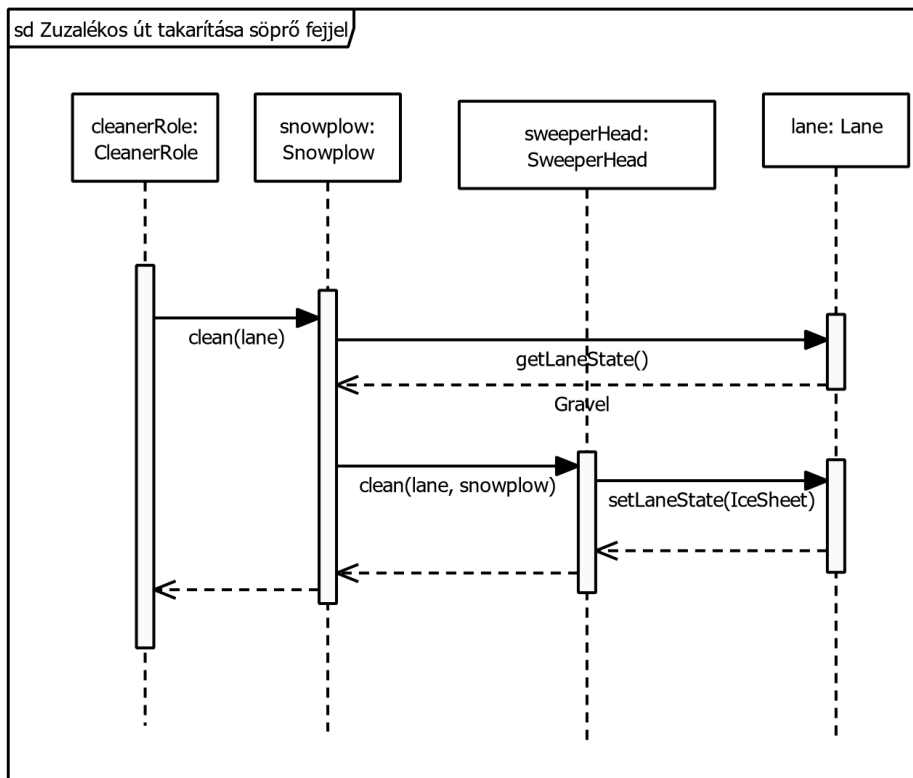
Új attribútummal bővült:

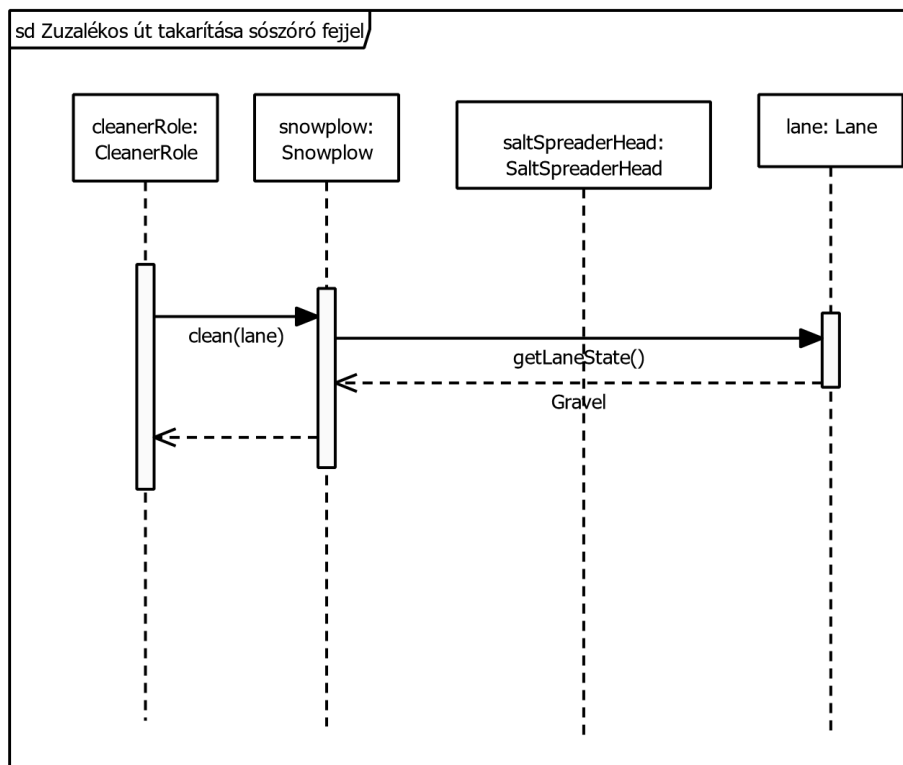
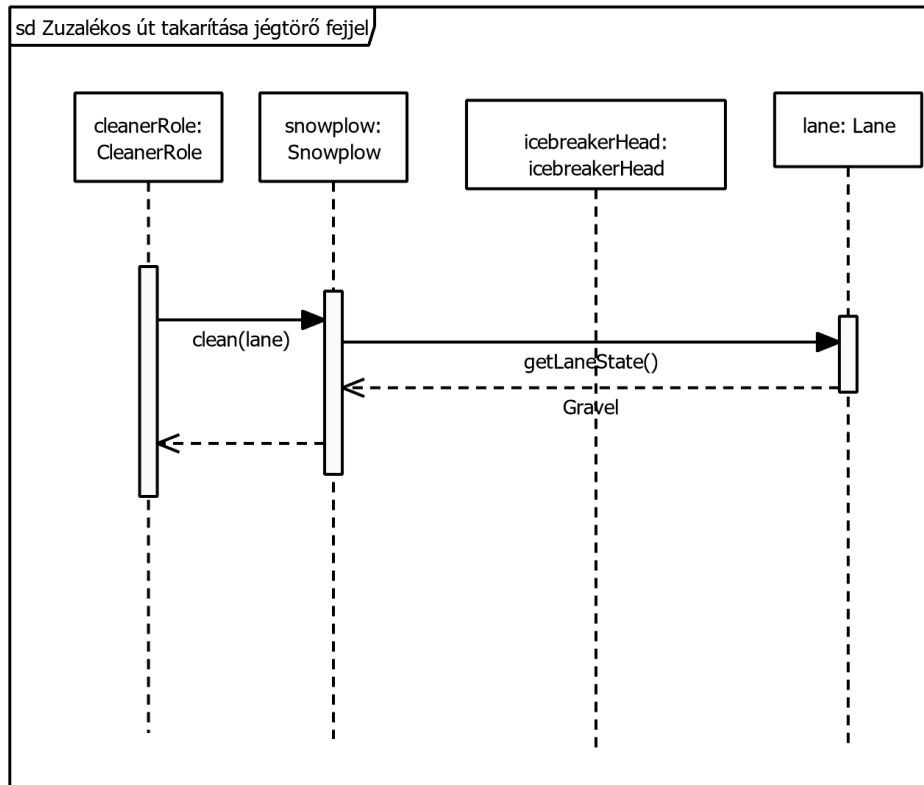
- **gravelThickness: double** - A zúzott kő aktuális fizikai vastagsága

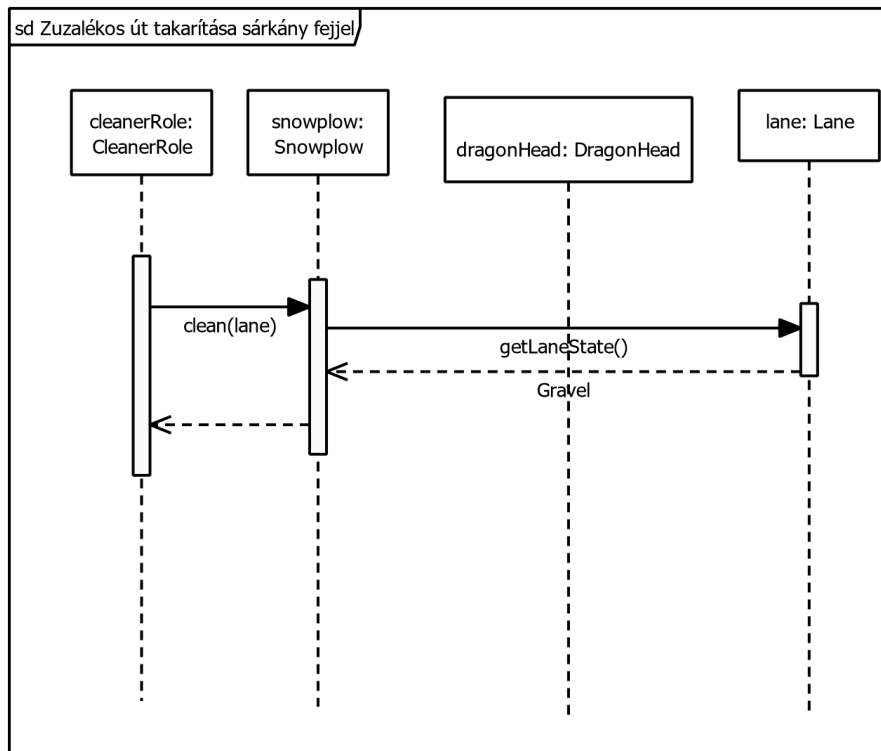
7.0.3 Szekvencia-diagramok











7.1 Prototípus interface-definíciója

7.1.1 Az interfész általános leírása

Az interfész egy konzolos be és kiviteli felületet valósít meg. A parancsok bevitele a konzolon vagy a szöveges fájlokba írt szkript segítségével történhet.

7.1.2 Bemeneti nyelv

A prototípus a tesztelhetőség és az automatizálhatóság érdekében egy egyszerű, soronként értelmezett, szöveges parancsnyelvet használ. A bemenet érkezik interaktívan a szabványos bemenetről (konzol), vagy kötegelt (batch) feldolgozás céljából előre megírt szöveges fájlokból (tesztszkriptekből).

Minden sor egy parancsot és annak szóközzel elválasztott paramétereit tartalmazza. A beépített parancsok csupa kisbetűvel írandók.

load <fájlnev>

Leírás: Betölti és inicializálja a pálya állapotát, a szereplőket és a játékmenetet a megadott konfigurációs fájlból. A sikeres betöltés alapfeltétele a tesztesetek futtatásának.

Opciók:

random <on|off>

Leírás: A determinisztikus működés és a reprodukálható tesztelés érdekében a véletlenszám-generátor alapú események (pl. váratlan havazás, szélvihar) ki- és bekapcsolására szolgál.

exit

Leírás: Leállítja a prototípus futását.

mozgas <jármű_id> <cél_mező_id>

Leírás: A megadott járművet a szomszédos célmezőre mozgatja. A rendszer a háttérben ellenőrzi a lépés érvényességét (szomszédosság, járhatóság, elegendő üzemanyag vagy mozgáspont).

takarit <jármű_id>

Leírás: A hókotró az aktuális mezőn elvégzi a tisztítást a jelenleg aktív, felszerelt kotrófej típusának megfelelően (pl. hajtja végre a hó vagy jég vastagságának csökkentését).

fej_csere <jármű_id> <fej_típus>

Leírás: A jármű aktív fejének lecserélése a raktárban/készletben lévő másik eszközre.

Opciók: sweeper, thrower, icebreaker, saltspreader, dragonhead, gravelspreader

vasarol <játékos_id> <tárgy_id> <menyiség>

Leírás: A játékos a boltban a rendelkezésére álló erőforrások (pénz/pont) terhére tárgyat vagy nyersanyagot (pl. só, kavics, új kotrófej) vásárol.

pass

Leírás: A soron lévő entitás vagy játékos cselekvés nélkül átadja a körét (várakozás).

stat <entitás_id>

Leírás: Részletes állapotinformációt ír ki a szabványos kimenetre a megadott entitásról. Jármű esetén pl. üzemanyagszint, felszerelt fejek; mező esetén hó- és jégvastagság, járhatóság.

list <kategória>

Leírás: Kilistázza a kategóriába tartozó összes elem azonosítóját és rövid státuszát.

Opciók: vehicles, roads, players.

7.1.3 Kimeneti nyelv

A prototípus kimeneti nyelve szöveges formátumú. A kimenet elsődleges célja, hogy a futtatott tesztesetek eredményei összehasonlíthatóak az elvárt kimenetekkel, akár automatizált módon is.

A program kimenete alapértelmezett kimenete a konzolra történik, de fájlba is átirányítható.

A véletlenszerűséget ki lehet kapcsolni.

[Objektum neve] [Paraméter] : [Régi érték] -> [Új érték]

Pályaképet megadó szintaxis és tesztelés menete

A tesztek automatizált futtatása a megadott parancsok és három dedikált szöveges fájl (arrange.txt, act.txt, assert.txt) segítségével történik.

A tesztekhez szükséges kezdőállapotot (a pályaképet, az úthálózatot és a járműveket) az arrange.txt-ben állítjuk össze a rendszerépítő parancsok segítségével. A teszt futtatása során az adott teszthez kapcsolódó act.txt-ben megadott parancsokat futtatjuk le. Ez a fájl felelős a pálya betöltéséért, a cselekvések végrehajtásáért és az állapot kimentéséért.

Példa a tesztelés menetére:

arrange.txt: (Kezdőállapot felépítése)

```
add_road -n "road_1" -snow 5 -ice 0
add_road -n "road_2" -snow 3 -ice 0
connect "road_1" -n "road_2"
add_plow -i 1 -n "plow_1" -pos "road_1"
```

act.txt: (A tesztlépések végrehajtása)

```
load "arrange.txt"
mozgas "plow_1" "road_2"
save "output.txt"
```

Konzol kimenet:

[plow_1] [position] : [road_1] -> [road_2]

Értékelés és validáció: A megadott példában az arrange.txt-ben felépített pályán az act.txt-nek megfelelően átmozgatjuk a "road_1" útszakaszról a "road_2" útszakaszra a hókotrót, és kimentjük az output.txt-be a játék állapotát.

A save parancs serializálással menti ki a játék teljes aktuális állapotát. Ezt az eredményfájlt (output.txt) összevetve a fejlesztők által előre megadott assert.txt-vel (amely a hibátlan lefutás utáni elvárt állapotot tartalmazza) egyértelműen eldönthető, hogy a teszt sikeres volt-e.

7.2 Összes részletes use-case

Use-case neve	1. Játék indítása
Rövid leírás	A felhasználó új játékot indít a főmenüből. A rendszer létrehozza az úthálózatot, az alap járműveket, a játékosokat, valamint inicializálja a kezdő időjárási és sávállapotokat.
Aktorok	Játékos, Rendszer
Forgatókönyv	1. A játékos koadja a <i>load <fájlnév></i> parancsot. 2. A rendszer betölti a konfigurációs fájlt. 3. A rendszer inicializálja: ○ a pályát ○ a csomópontokat, ○ az útszakaszokat ○ a járműveket ○ a játékosokat 4. A rendszer visszajelzést ír a konzolra az inicializálás sikerességéről.

Use-case neve	2. Hókotró mozgatása és út tisztítása
Rövid leírás	A takarító szerepkörű játékos kiválasztja a hókotrót, elmozdítja egy útszakaszra, majd az aktuális felszereléssel megtisztítja a sávot.
Aktorok	Takarító játékos, Rendszer
Forgatókönyv	1. A játékos kiadja a <i>mozgas snowplow_1 ut_12</i> parancsot. 2. A rendszer ellenőrzi a lépés érvényességét. 3. A hókotró áthelyezésre kerül. 4. A játékos kiadja a <i>takarit snowplow_1</i> parancsot. 5. A rendszer a kotrófej alapján módosítja a sáv állapotát. 6. A rendszer kiírja az eredményt a konzolra.

Use-case neve	3. Kotrófej cseréje
Rövid leírás	A takarító játékos lecseréli a hókotró aktuális fejét egy másik típusra, hogy más úállapotot is kezelni tudjon.
Aktorok	Takarító játékos, Rendszer
Forgatókönyv	1. A játékos kiadja a <i>fej_csere snowplow_1 icebreaker</i> parancsot. 2. A rendszer lecseréli a hókotró aktuális fejét. 3. A rendszer visszajelzést ír a konzolra.

Use-case neve	4. Vásárlás a boltban
Rövid leírás	A takarító játékos megnyitja a bolt panelt, kiválaszt egy megvásárolható eszközt vagy nyersanyagot, majd végrehajtja a vásárlást.
Aktorok	Takarító játékos, Rendszer, Bolt
Forgatókönyv	1. A játékos a kiadja a vasarol salt 3 parancsot. 2. A rendszer ellenőrzi: ○ van-e elég pénz, ○ a termék megvásárolható-e. 3. Siker esetén a rendszer levonja az összeget és hozzáadja a készlethez.

Use-case neve	5. Busz útvonalának automatikus kijelölése és teljesítése
Rövid leírás	A rendszer automatikusan kiválaszt két terminált, amelyek között a busznak közlekednie kell. A játékos feladata a busz mozgatása és az útvonal teljesítése. Siker esetén új útvonal generálódik, és a jutalom a megtett távolsággal arányos.
Aktorok	Buszsofőr játékos, Rendszer
Forgatókönyv	1. A rendszer a kör elején automatikusan kiválaszt két terminált. 2. A rendszer kiszámít egy útvonalat a két pont között.

	3. A rendszer eltárolja az útvonalat a buszhoz. 4. A játékos kiadja a <i>mozgas bus_1</i> <i><következő_szakaszh></i> parancsot. 5. A rendszer ellenőrzi, hogy a mozgás megfelel-e az útvonalnak és az út állapotának. 6. A busz a következő útszakaszra lép. 7. A 4–6. lépések ismétlődnek, amíg a busz el nem éri a célt. 8. A rendszer kiszámítja a megtett távolságot. 9. A rendszer növeli a játékos pontszámát a távolság alapján. 10. A rendszer kiírja a teljesítés eredményét a konzolra. 11. A rendszer automatikusan új útvonalat generál.
--	---

Use-case neve	6. Busz közlekedtetése forduló teljesítéséhez
Rövid leírás	A buszsofőr a buszt a kijelölt útvonalon végigvezeti, miközben a rendszer figyeli az útállapotot, az esetleges akadályokat és a forduló teljesítését.
Aktorok	Buszsofőr játékos, Rendszer
Forgatókönyv	1. A játékos kiadja a <i>mozgas bus_1</i> <i><következő_szakaszh></i> parancsot. 2. A rendszer ellenőrzi: ○ a szakasz szomszédos-e ○ a szakasz járható-e 3. Ha a feltételek teljesülnek, a busz áthelyezésre kerül. 4. A rendszer frissíti a busz aktuális pozícióját. 5. A rendszer kiírja az új pozíciót a konzolra. 6. A folyamat ismétlődő minden körben.

Use-case neve	7. Autók automatikus közlekedése lakhely és munkahely között
Rövid leírás	Az autók a rendszer által vezérelve automatikusan közlekednek a lakhelyük és munkahelyük között, a játékos közvetlen beavatkozása nélkül.
Aktorok	Rendszer

Forgatókönyv	1. A rendszer létrehozza az autókat a játék inicializálásakor. 2. Minden autóhoz tartozik egy lakhely és egy munkahely. 3. A rendszer kiszámítja az útvonalat. 4. A rendszer minden körben végrehajtja: ○ ellenőrzi a következő útszakaszt, ○ ha járható, az autó továbblép. 5. A rendszer frissíti az autók pozícióját. 6. Ha az autó eléri a célját, a rendszer új útvonalat rendel hozzá. 7. A rendszer kiírja a mozgás eredményét a konzolra.
---------------------	---

Use-case neve	8. Útvonal újratervezése akadály esetén
Rövid leírás	Az autó eredeti útvonala használhatatlanná válik, ezért a rendszer új útvonalat számol.
Aktorok	Rendszer
Forgatókönyv	1. A jármű haladása során egy útszakasz állapota megváltozik. 2. A rendszer érzékeli, hogy a következő szakasz nem járható. 3. A rendszer kiírja a figyelmeztetést a konzolra. 4. A rendszer új útvonal keresését indítja el. 5. A rendszer meghatározza az új legrövidebb járható útvonalat. 6. Ha talál megfelelő útvonalat: ○ frissíti a jármű útvonalát ○ kiírja az új útvonalat 7. A jármű az új útvonalon halad tovább.

Use-case neve	9. Ütközések kezelése automatikus és játékos által vezérelt járművek között
Rövid leírás	A rendszer kezeli az ütközéseket, amelyek létrejöhetnek automatikusan közlekedő autók között, illetve játékos által irányított busz és más járművek között.
Aktorok	Rendszer, Buszsofőr játékos

Forgatókönyv	1. Egy jármű mozgási parancsot hajt végre (<i>mozgas</i>). 2. A rendszer meghatározza a célpozíciót. 3. A rendszer ellenőrzi, hogy a célpozíció foglalt-e. A) Automatikus ütközés (autó-autó): 4. Két autó ugyanarra a sávra lépne. 5. A rendszer ütközést detektál. 6. A rendszer a sávot járhatatlanná teszi. 7. A rendszer kiírja az eseményt a konzolra. B) Játékos által okozott ütközés (busz): 8. A játékos a buszt foglalt sávra mozgatná. 9. A rendszer ütközést detektál. 10. A rendszer megállítja a buszt. 11. A rendszer büntetést alkalmazhat (opcionális). 12. A rendszer kiírja az eseményt a konzolra.
---------------------	--

Use-case neve	10. Autó és busz elakadása mély hóban
Rövid leírás	Ha a hóvastagság túl nagy, az autó és a busz nem tud továbbhaladni.
Aktorok	Rendszer, Autó, Buszsofőr
Forgatókönyv	1. Az autó, busz a következő útszakaszra lépne. 2. A rendszer ellenőrzi a hóvastagságot. 3. Ha mély hó, a jármű elakad. 4. A Naplópanel rögzíti az eseményt. 5. Az út megtisztítása után az elakadás megszűnik.

Use-case neve	11. Fej hatástalanná válása készlet hiányában
----------------------	---

Rövid leírás	A zuzalékot szóró, sósóró és sárkány fej működése leáll, ha elfogy a zuzalék, só vagy a biokerozin.
Aktorok	Rendszer, Takarító játékos
Forgatókönyv	1. A játékos takarítást indít vagy folytatja a megkezdett takarítást a zuzalékot szóró, sósóró vagy sárkány fejjel. 2. A rendszer ellenőrzi a készletet. 3. Ha a készlet 0, a fej nem fejt ki hatást. 4. A Naplópanel figyelmeztetést ír ki. 5. A játékosnak a továbbiakban 2 lehetősége van a sikeres takarításra: fejcserét kezdeményez vagy vásárol a Boltból, ezzel feltölti a készletet.

Use-case neve	12. Játék kiértékelése
Rövid leírás	A maximális körszám elérésekor a rendszer kiszámítja és szövegesen kilistázza a játékosok pontszámait.
Aktorok	Rendszer, Játékos
Forgatókönyv	1. A Rendszer érzékeli a körlimit elérését ($\text{currentRound} \geq \text{maxRound}$). 2. A Rendszer meghívja az <code>evaluate()</code> metódust a pontok összesítéséhez. 3. A Rendszer a Konzolra írja a végeredményt: "Játék vége! Pontszámok: Játékos1: 120 pont, Játékos2: 85 pont." 4. A Rendszer parancsot vár a kilépéshez vagy a főmenübe való visszatéréshez.

Use-case neve	13. Sáv állapotának lekérdezése
Rövid leírás	A játékos egy parancs segítségével lekéri egy konkrét sáv fizikai adatait a takarítás megtervezéséhez.
Aktorok	Játékos, Rendszer
Forgatókönyv	1. A Játékos kiadja az inspect <lane_id> parancsot. 2. A Rendszer lekéri a Lane objektum attribútumait (snowThickness, iceThickness, isPassable). 3. A Rendszer kiszámítja és megjeleníti a dinamikus súlyt (getDynamicWeight). 4. A Konzol megjeleníti az adatokat: "Sáv: L5"

Use-case neve	14. Jeges út érdesítése zúzalékkal
Rövid leírás	A hókotró a zúzalékszóró fejjel megszünteti az út csúszósságát.
Aktorok	Takarító játékos, Rendszer
Forgatókönyv	1. A Játékos felszereli a GravelSpreaderHead-et a hókotróra: fej_csere gravelspreader. 2. A Játékos a jeges sávra mozgatja a járművet és kiadja a clean parancsot. 3. A Rendszer ellenőrzi, hogy van-e zúzalék (gravelStock > 0). 4. A Rendszer a sáv állapotát GRAVEL típusúra módosítja, ami megszünteti a csúszósságot.

	5. A Konzol kiírja: "A sáv felszórva zúzalékkal, a csúszásveszély megszűnt."
--	--

Use-case neve	15. Vékony hóréteg eltakarítása SweeperHead-del
Rövid leírás	A hókotró a seprű fejjel (SweeperHead) letakarítja a vékony hóréteget az útról.
Aktorok	Takarító játékos, Rendszer
Forgatókönyv	1. A Játékos felszereli a SweeperHead-et a hókotróra: fej_csere sweeper. A Játékos a vékony hóval borított sávra mozgatja a járművet és kiadja a clean parancsot. A Rendszer a sáv állapotát Clear típusúra módosítja, ami járhatóvá teszi az utat. A Konzol kiírja: "A vékony hóréteg lesöpörve, az út tiszta."

Use-case neve	16. Vastag hóréteg eltakarítása ThrowerHead-del
Rövid leírás	A hókotró a hómaró fejjel (ThrowerHead) eltávolítja a vastag, mély havat az útról.
Aktorok	Takarító játékos, Rendszer
Forgatókönyv	1. A Játékos felszereli a ThrowerHead-et a hókotróra: fej_csere thrower. 2. A Játékos a mély hóval (DeepSnow) borított sávra mozgatja a járművet és kiadja a clean parancsot.

	3.A Rendszer a sáv állapotát Clear típusúra módosítja, megszüntetve az akadályt. 4.A Konzol kiírja: "A vastag hóréteg eltisztítva, az út megtisztítva.".
--	--

Use-case neve	17. Jégpáncél feltörése jégtörővel
Rövid leírás	A hókotró a jégtörő fejjel (IcebreakerHead) feltöri a balesetveszélyes, összefüggő jégpáncélt.
Aktorok	Takarító játékos, Rendszer
Forgatókönyv	1.A Játékos felszereli az IcebreakerHead-et a hókotróra fej_csere icebreaker. 2.A Játékos a jeges (IceSheet) sávra mozgatja a járművet és kiadja a clean parancsot. 3.A Rendszer a sáv állapotát feltört jég (BrokenIce) típusúra módosítja, ami javítja a tapadást az összefüggő jéghez képest. 4.A Konzol kiírja: "A jégpáncél feltörve, az út tapadása javult.".

Use-case neve	18. Jég felolvasztása sószórással
Rövid leírás	A hókotró a sószóró fejjel (SaltSpreaderHead) felolvasztja a jeget, megakadályozva a csúszást.
Aktorok	Takarító játékos, Rendszer
Forgatókönyv	1.A Játékos felszereli a SaltSpreaderHead-et a hókotróra: fej_csere saltspreader. 2.A Játékos a jeges sávra mozgatja a járművet és kiadja a clean parancsot.

	3.A Rendszer ellenőrzi, hogy van-e elegendő sókészlet a járművön (saltStock > 0). 4.A Rendszer a sáv állapotát Clear típusúra módosítja, a jég elolvad. 5.A Konzol kiírja: "A sáv felsózva, a jég elolvadt, az út tiszta.".
--	---

Use-case neve	19. Hó és jég leolvasztása sárkányfejjel
Rövid leírás	A hókotró a sárkányfejjel (DragonHead) intenzív hővel azonnal leolvasztja a havat és a jeget az útról.
Aktorok	Takarító játékos, Rendszer
Forgatókönyv	1.A Játékos felszereli a DragonHead-et a hókotróra: fej_csere dragonhead. 2.A Játékos a hóval vagy jéggel borított sávra mozgatja a járművet és kiadja a clean parancsot. 3.A Rendszer ellenőrzi, hogy van-e elegendő üzemanyag/biokerozin a járművön (biokeroseneStock > 0). 4.A Rendszer a sáv állapotát azonnal Clear típusúra módosítja, a hó/jég elpárolog. 5.A Konzol kiírja: "A sáv lángszóróval megtisztítva, az út teljesen tiszta.".

7.3 Tesztelési terv

Teszt-eset neve	1. Játék indítása
Rövid leírás	Osztályok: Game, Player, Role, BusdriverRole, CleanerRole, RoadNetwork, Node, Road, Lane, Vehicle, Weather, Skeleton

	Funkcionalitás: • A játék inicializálásának folyamatát teszteli a főmenüből indulva. • Ellenőrzi, hogy a rendszer megfelelően létrehozza a pályát, csomópontokat, útszakaszokat, járműveket, játékosokat, valamint a kezdő állapotokat. Aktor: Játékos, Rendszer
Teszt célja	Forgatókönyv: • A játékos a Főmenü felületen kiválasztja az „Új játék indítása” opciót. • A rendszer megjeleníti a játékbeállítási panelt. • A játékos megadja az induló paramétereket, például a játékosok számát vagy a kezdő konfigurációt. • A játékos a „Start” gombra kattint, vagy kiadja a new_game parancsot. • A rendszer inicializálja: ○ a pályát, ○ a csomópontokat, ○ az útszakaszokat, ○ a járműveket, ○ a játékoszerepeket, ○ a kezdő időjárási és sávállapotokat. • A rendszer megnyitja a Játéktér nézetet. • A Naplópanel megjeleníti az inicializáció sikerességét.

Teszt-eset neve	2. Hókotró mozgatása és út tisztítása
Rövid leírás	Osztályok: Snowplow, Vehicle, Lane, LaneState, Head, SweeperHead, ThrowerHead, IcebreakerHead, SaltSpreaderHead, DragonHead, CleanerRole, RoadNetwork, Skeleton Funkcionalitás: • A hókotró mozgatása és a különböző kotrófejekkel végzett út tisztításának kezelése, valamint a sáv állapotának módosítása. Aktor: Takarító játékos, Rendszer
Teszt célja	Forgatókönyv: • A játékos a Játékosinformációs panelen kiválasztja az aktív takarítót. • A Játéktér nézetben kijelöli a hókotrót. • A rendszer kiemeli a hókotrót és megjeleníti az elérhető műveleteket az Akciópanelen.

	• A játékos a „Mozgás” műveletet választja, vagy kiadja a mozgás snowplow_1 ut_12 parancsot. • A rendszer ellenőrzi, hogy a kijelölt útszakasz elérhető-e. • A hókotró a kiválasztott útszakaszra kerül. • A játékos az Akciópanelen a „Takarítás” műveletet választja, vagy kiadja a takarít snowplow_1 sweeper parancsot. • A rendszer az aktív kotrófej típusának megfelelően módosítja a sáv állapotát. • A Játéktér frissül, és az útszakasz új állapota látható lesz. • A Naplópanel rögzíti az akció eredményét. • Ha a művelet pontot ér, a Játékosinformációs panelen frissül a pontszám.
--	---

Teszt-eset neve	3. Kotrófej cseréje
Rövid leírás	Osztályok: Snowplow, Head, SweeperHead, ThrowerHead, IcebreakerHead, SaltSpreaderHead, DragonHead, CleanerRole, Skeleton Funkcionalitás: • A hókotró felszerelésének dinamikus cseréje és az aktuális kotrófej frissítése. Aktor: Takarító játékos, Rendszer
Teszt célja	Forgatókönyv: • A játékos a Játéktér nézetben kiválasztja a hókotró. • A rendszer megjeleníti az aktuális felszerelést a Játékosinformációs panelen. • A játékos az Akciópanelen a „Fejcsere” opciót választja. • A rendszer megnyitja a Felszerelésválasztó ablakot. • A játékos kiválasztja az új fejtípust, például: ○ sweeper, ○ thrower, ○ icebreaker, ○ saltspreader, ○ dragonhead, ○ gravelspreader. • A játékos megerősíti a választást, vagy kiadja a fej_csere snowplow_1 icebreaker parancsot. • A rendszer lecseréli a felszerelést. • A Játékosinformációs panel frissül.

	• A Naplópanel rögzíti a sikeres fejcserét.
--	---

Teszt-eset neve	4. Vásárlás a boltban
Rövid leírás	Osztályok: Store, Buyable, CleanerRole, Player, Snowplow, Skeleton Funkcionalitás: • A bolt működése, vásárlások végrehajtása, pénz levonása és a megvásárolt elemek készletbe helyezése. Aktor: Takarító játékos, Rendszer, Bolt
Teszt célja	Forgatókönyv: • A játékos kiválasztja a boltot az Akciópanelen, a „Bolt megnyitása” opciót használva. • A rendszer megnyitja a Bolt panelt. • A bolt listázza az elérhető termékeket és árakat. • A játékos kijelöl egy terméket. • A játékos megadja a mennyiséget. • A játékos a „Vásárlás” gombra kattint, vagy kiadja a vásárolt 3 parancsot. • A rendszer ellenőrzi: ○ van-e elég pénz, ○ a termék megvásárolható-e. • Siker esetén a rendszer levonja az összeget. • A megvásárolt termék bekerül a játékos vagy a hókotró készletébe. • A Játékosinformációs panel frissíti a pénz- és készletadatokat. • A Naplópanel rögzíti a vásárlást.

Teszt-eset neve	5. Busz útvonalának automatikus kijelölése és teljesítése
Rövid leírás	Osztályok: Bus, Vehicle, Route, Terminal, RoadNetwork, Lane, BusdriverRole, Game, Skeleton Funkcionalitás: • A busz számára automatikusan generált útvonal kezelése, annak teljesítése és a jutalom kiszámítása. Aktor: Buszsofőr játékos, Rendszer

Teszt célja	Forgatókönyv: • A rendszer a kör elején automatikusan kiválaszt két terminált a Játéktéren. • A rendszer kiszámít egy útvonalat a két pont között. • A kiválasztott útvonal megjelenik a Játéktéren. • A Játékosinformációs panelen megjelenik az aktuális feladat, vagyis a kiindulási és célpont. • A játékos kiválasztja a buszt. • A játékos az Akciópanelen a „Mozgás” műveletet használja, vagy parancsot ad ki (mozgas bus_1 ...). • A rendszer ellenőrzi, hogy a mozgás megfelel-e az útvonalnak és az út állapotának. • A busz lépésenként halad az útvonal mentén. • Amikor a busz eléri a célterminált: ◦ a rendszer kiszámítja a megtett távolságot, ◦ ennek megfelelően növeli a játékos pénzét. • A Naplópanel rögzíti a sikeres teljesítést és a jutalmat. • A rendszer automatikusan generál egy új útvonalat, és a folyamat újraindul.
--------------------	---

Teszt-eset neve	6. Busz közlekedtetése forduló teljesítéséhez
Rövid leírás	Osztályok: Bus, Vehicle, Route, Lane, Road, BusdriverRole, Game, Skeleton Funkcionalitás: • A busz lépésenkénti mozgatása az útvonal mentén és a forduló teljesítésének ellenőrzése. Aktor: Buszsofőr játékos, Rendszer
Teszt célja	Forgatókönyv: • A játékos kiválasztja a buszt a Játéktér nézetben. • Az Akciópanelen a „Mozgás” opciót választja. • A játékos kiadja a mozgas bus_1 ut_5 parancsot, vagy a térképen kijelöli a következő szakaszt. • A rendszer ellenőrzi, hogy a lépés megfelel-e a busz útvonalának. • A busz a következő útszakaszra lép. • A rendszer frissíti a busz pozícióját a Játéktéren. • A folyamat addig ismétlődik, amíg a busz el nem éri a következő terminált. • A rendszer ellenőrzi, hogy a forduló teljesült-e.

	• Siker esetén a rendszer pontot ad. • A Naplópanel rögzíti a forduló befejezését.
--	---

Teszt-eset neve	7. Autók automatikus közlekedése lakhely és munkahely között
Rövid leírás	Osztályok: Car, Vehicle, Residence, Workplace, Route, RoadNetwork, Lane, Game, Skeleton Funkcionalitás: • Az autók automatikus mozgása a lakhely és munkahely között, valamint az útvonal követése. Aktor: Rendszer
Teszt célja	Forgatókönyv: • A rendszer létrehozza az autókat a játék inicializálásakor. • Minden autóhoz tartozik egy lakhely és egy munkahely csomópont a Játéktéren. • A rendszer kiszámít egy útvonalat az autó számára. • Az autók automatikusan elindulnak. • Minden körben a rendszer lépteti az autókat: ◦ ellenőrzi a következő útszakasz állapotát, ◦ ha járható, az autó továbblép. • Az autók pozíciója folyamatosan frissül a Játéktéren. • Ha az autó eléri a munkahelyet, a rendszer rögzíti a célba érést. • Az autó új útvonalat kaphat, például vissza a lakhelyre.

Teszt-eset neve	8. Útvonal újratervezése akadály esetén
Rövid leírás	Osztályok: Car, Vehicle, Route, RoadNetwork, Lane, LaneState, Game, Skeleton Funkcionalitás: • Az autó útvonalának automatikus újratervezése, amikor egy útszakasz járhatatlanná válik, valamint új legrövidebb útvonal keresése és követése. Aktor: Rendszer

Teszt célja	Forgatókönyv: ● Az autó haladása során egy útszakasz állapota megváltozik. ● A rendszer érzékeli, hogy a következő szakasz nem járható. ● A Naplópanel figyelmeztetést jelenít meg. ● A rendszer automatikusan elindítja az útvonal újratervezését. ● A rendszer új legrövidebb járható útvonalat keres. ● Ha talál megfelelőt, a Játéktér frissíti a kijelölt útvonalat. ● A jármű az új útvonalon halad tovább.
--------------------	--

Teszt-eset neve	9. Ütközések kezelése automatikus és játékos által vezérelt járművek között
Rövid leírás	Osztályok: Vehicle, Car, Bus, Lane, Road, Game, Skeleton Funkcionalitás: ● Ütközések detektálása és kezelése különböző járművek között, valamint a következmények alkalmazása. Aktor: Rendszer, Buszsofőr játékos
Teszt célja	Forgatókönyv: ● Egy jármű mozgása során a rendszer meghatározza a következő pozíciót. ● A rendszer ellenőrzi, hogy a célpozíció foglalt-e. A) Automatikus ütközés (autó-autó): ● Két autó ugyanarra az útszakaszra lépne. ● A rendszer ütközést detektál. ● A Játéktér vizuálisan jelzi az ütközést. ● A Naplópanel kiírja az eseményt. ● A rendszer alkalmazza a következményeket, például megállást vagy blokkolást. B) Játékos által okozott ütközés (busz): ● A játékos mozgási parancsot ad ki a buszra. ● A busz egy olyan útszakaszra lépne, ahol más jármű tartózkodik. ● A rendszer ütközést detektál. ● A Naplópanel jelzi, hogy a balesetet a játékos okozta. ● A rendszer alkalmazza a következményeket, például büntetést vagy megállást.

Teszt-eset neve	10. Autó és busz elakadása mély hóban
Rövid leírás	Osztályok: Vehicle, Car, Bus, Lane, LaneState, RoadNetwork, Road, Weather, Skeleton Funkcionalitás: Túl nagy hóvastagság felismerése, az autó és a busz alkalmazkodása (elakadnak). Aktor: Rendszer, Autó, Buszsofőr
Teszt célja	- Az autó, busz a következő útszakaszra lépne. - A rendszer ellenőrzi a hóvastagságot. - Ha mély hó, a jármű elakad. - A Naplópanel rögzíti az eseményt. - Az út megtisztítása után az elakadás megszűnik.

Teszt-eset neve	11. Fej hatástalanná válása készlet hiányában
Rövid leírás	Osztályok: CleanerRole, Snowplow, GravelSpreaderHead, SaltSpreaderHead, DragonHead, Skeleton Funkcionalitás: Zuzalékot szóró, sószóró és sárkány fej kezeli a hozzájuk tartozó nyersanyag (zuzalék, só, biokerozin) hiányát. Aktor: Rendszer, Takarító játékos
Teszt célja	- A játékos takarítást indít vagy folytatja a megkezdett takarítást a zuzalékot szóró, sószóró vagy sárkány fejjel. - A rendszer ellenőrzi a készletet. - Ha a készlet 0, a fej nem fejt ki hatást.

	4. A Naplópanel figyelmeztetést ír ki. 5. A játékosnak a továbbiakban 2 lehetősége van a sikeres takarításra: fejcserét kezdeményez vagy vásárol a Boltból, ezzel feltölti a készletet.
--	---

Teszt-eset neve	12. Játék kiértékelése
Rövid leírás	Osztályok: Game, Player, Role, BusDriverRole, CleanerRole Funkcionalitás: A játék végi állapot detektálásának ellenőrzése (a körlimit elérésekor). A pontszámítási logika helyességének tesztelése mind a buszsofőr, mind a takarító szerepkörök esetén. A végeredmény megfelelő formátumban történő, konzolos megjelenítésének ellenőrzése. Aktor: Rendszer, Játékos
Teszt célja	- A rendszer a Game.tick() hívások során érzékeli, hogy a currentRound >= maxRound feltétel teljesült. - A játék állapota befejezettre vált (az isOver() igazat ad vissza). - A rendszer végigiterál a Player listán, és lekéri a pontszámokat a getSumPoints(), illetve a szerepkörökhöz tartozó getScore() metódusok meghívásával. - A rendszer összegzi az eredményeket, majd a Konzolra írja: "Játék vége! Pontszámok: Játékos1: 120 pont, Játékos2: 85 pont." - A rendszer ezután megszakítja a játékmenetet, és parancsot vár a kilépéshez vagy a főmenübe való visszatéréshez.

Teszt-eset neve	13. Sáv állapotának lekérdezése
Rövid leírás	Osztályok: RoadNetwork, Road, Lane, LaneState (IceSheet, Gravel állapotok)

	Funkcionalitás: A parancsértelmező tesztelése az inspect parancs és a sáv azonosító (lane_id) felismerésére. A kiválasztott Lane objektum attribútumainak pontos lekérdezése. A sáv áthaladhatóságának (isPassable()) és dinamikus súlyának (getDynamicWeight()) helyes kiszámítása az aktuális állapot alapján. Aktor: Játékos, Rendszer
Teszt célja	1. A Játékos a konzolon kiadja az inspect <lane_id> parancsot. 2. A Rendszer a RoadNetwork-on keresztül megkeresi a kért azonosítójú sávot. 3. A Rendszer lekéri a Lane objektum fizikai attribútumait (snowThickness, iceThickness). 4. A Rendszer meghívja a LaneState interfészen keresztül a sáv aktuális állapotának isPassable() és getDynamicWeight() metódusait. 5. A Konzol strukturáltan megjeleníti a kinyert adatokat (pl.: "Sáv: L5", vastagságok, dinamikus súly).

Teszt-eset neve	14. Jeges út érdesítése zúzalékkal
Rövid leírás	Osztályok: CleanerRole, Snowplow, GravelSpreaderHead, Head, Lane, LaneState, IceSheet, Gravel Funkcionalitás: A hókotró eszközcseréjének (changeHead) tesztelése a zúzalékszóró fejre. A tisztítási művelet (clean) és a készletellenőrzés (gravelStock) összekapcsolásának ellenőrzése. A polimorfizmus tesztelése: a sáv állapotának (LaneState) sikeres átmenete IceSheet-ből Gravel típusúba. Aktor: Takarító játékos, Rendszer
Teszt célja	1. A Takarító játékos kiadja a fej_csere gravelspreader parancsot. A rendszer beállít egy GravelSpreaderHead objektumot a Snowplow currentHead attribútumaként. 2. A Játékos egy olyan Lane-re navigálja a hókotró, amelynek currentState-je egy IceSheet példány, majd kiadja a clean parancsot. 3. A Rendszer a Snowplow objektumban ellenőrzi, hogy a gravelStock > 0 feltétel igaz-e. 4. A GravelSpreaderHead clean() metódusa lefut, a gravelStock csökken, és a Lane currentState-jét egy Gravel objektumra cseréli.

	5. A Konzol kiírja a visszajelzést: "A sáv felszórva zúzalékkal, a csúszásveszély megszűnt."
--	--

Teszt-eset neve	15. Vékony hóréteg eltakarítása SweeperHead-del
Rövid leírás	• Osztályok: CleanerRole, Snowplow, SweeperHead, Head, Lane, LaneState, ThinSnow, Clear • Funkcionalitás: A hókotró eszközcserejének (changeHead) tesztelése a seprű fejre. A tisztítási művelet (clean) végrehajtásának ellenőrzése. A polimorfizmus tesztelése: a sáv állapotának (LaneState) sikeres átmenete ThinSnow-ból Clear típusúba. • Aktor: Takarító játékos, Rendszer
Teszt célja	1. A Takarító játékos kiadja a fej_csere sweeper parancsot. A rendszer beállít egy SweeperHead objektumot a Snowplow currentHead attribútumaként. 2. A Játékos egy olyan Lane-re navigálja a hókotró, amelynek állapota (currentLaneState) egy ThinSnow példány, majd kiadja a clean parancsot. 3. A SweeperHead clean() metódusa lefut, és a Lane currentLaneState-jét egy Clear objektumra cseréli. 4. A Konzol kiírja a visszajelzést: "A vékony hóréteg lesöpörve, az út tiszta."

Teszt-eset neve	16. Vastag hóréteg eltakarítása ThrowerHead-del
Rövid leírás	• Osztályok: CleanerRole, Snowplow, ThrowerHead, Head, Lane, LaneState, DeepSnow, Clear • Funkcionalitás: A hókotró eszközcserejének (changeHead) tesztelése a hómaró fejre. A tisztítási művelet (clean) végrehajtásának ellenőrzése. A

	polimorfizmus tesztelése: a sáv állapotának (LaneState) sikeres átmenete DeepSnow-ból Clear típusúba. • Aktor: Takarító játékos, Rendszer
Teszt célja	1. A Takarító játékos kiadja a fej_csere thrower parancsot. A rendszer beállít egy ThrowerHead objektumot a Snowplow currentHead attribútumaként. 2. A Játékos egy olyan Lane-re navigálja a hókotrót, amelynek állapota (currentLaneState) egy DeepSnow példány, majd kiadja a clean parancsot. 3. A ThrowerHead clean() metódusa lefut, és a Lane currentLaneState-jét egy Clear objektumra cseréli. 4. A Konzol kiírja a visszajelzést: "A vastag hóréteg eltisztítva, az út megtisztítva."

Teszt-eset neve	17. Jégpáncél feltörése jégtörővel
Rövid leírás	• Osztályok: CleanerRole, Snowplow, IcebreakerHead, Head, Lane, LaneState, IceSheet, BrokenIce • Funkcionalitás: A hókotró eszközcseréjének (changeHead) tesztelése a jégtörő fejre. A tisztítási művelet (clean) végrehajtásának ellenőrzése. A polimorfizmus tesztelése: a sáv állapotának (LaneState) sikeres átmenete IceSheet-ből BrokenIce típusúba. • Aktor: Takarító játékos, Rendszer
Teszt célja	1. A Takarító játékos kiadja a fej_csere icebreaker parancsot. A rendszer beállít egy IcebreakerHead objektumot a Snowplow currentHead attribútumaként. 2. A Játékos egy olyan Lane-re navigálja a hókotrót, amelynek állapota (currentLaneState) egy IceSheet példány, majd kiadja a clean parancsot. 3. Az IcebreakerHead clean() metódusa lefut, és a Lane currentLaneState-jét egy BrokenIce objektumra cseréli.

	4. A Konzol kiírja a visszajelzést: "A jégpáncél feltörve, az út tapadása javult."
--	--

Teszt-eset neve	18. Jég felolvasztása sószórással
Rövid leírás	• Osztályok: CleanerRole, Snowplow, SaltSpreaderHead, Head, Lane, LaneState, IceSheet, Clear • Funkcionalitás: A hókotró eszközcseréjének (changeHead) tesztelése a sószóró fejre. A tisztítási művelet (clean) és a készletellenőrzés (saltStock) összekapcsolásának ellenőrzése. A polimorfizmus tesztelése: a sáv állapotának (LaneState) sikeres átmenete IceSheet-ből Clear típusúba. • Aktor: Takarító játékos, Rendszer
Teszt célja	1. A Takarító játékos kiadja a fej_csere saltspreader parancsot. A rendszer beállít egy SaltSpreaderHead objektumot a Snowplow currentHead attribútumaként. 2. A Játékos egy olyan Lane-re navigálja a hókotrót, amelynek állapota (currentState) egy IceSheet példány, majd kiadja a clean parancsot. 3. A Rendszer a Snowplow objektumban ellenőrzi, hogy a saltStock > 0 feltétel igaz-e. 4. A SaltSpreaderHead clean() metódusa lefut, a saltStock csökken, és a Lane currentState-jét egy Clear objektumra cseréli. 5. A Konzol kiírja a visszajelzést: "A sáv felszózva, a jég elolvadt, az út tiszta."

Teszt-eset neve	19. Hó és jég leolvasztása sárkányfejjel
------------------------	--

Rövid leírás	• Osztályok: CleanerRole, Snowplow, DragonHead, Head, Lane, LaneState, IceSheet, DeepSnow, Clear • Funkcionalitás: A hókotró eszközcserejének (changeHead) tesztelése a sárkányfejre. A tisztítási művelet (clean) és a készletellenőrzés (biokeroseneStock) összekapcsolásának ellenőrzése. A polimorfizmus tesztelése: a sáv állapotának (LaneState) sikeres átmenete hóval vagy jéggel borított állapotból (pl. IceSheet vagy DeepSnow) Clear típusúba. • Aktor: Takarító játékos, Rendszer
Teszt célja	1. A Takarító játékos kiadja a fej_csere dragonhead parancsot. A rendszer beállít egy DragonHead objektumot a Snowplow currentHead attribútumaként. 2. A Játékos egy olyan Lane-re navigálja a hókotró, amelynek állapota (currentLaneState) egy jéggel vagy hóval borított példány (pl. IceSheet), majd kiadja a clean parancsot. 3. A Rendszer a Snowplow objektumban ellenőrzi, hogy a biokeroseneStock > 0 feltétel igaz-e. 4. A DragonHead clean() metódusa lefut, a biokeroseneStock csökken, és a Lane currentLaneState-jét egy Clear objektumra cseréli. 5. A Konzol kiírja a visszajelzést: "A sáv lángszóróval megtisztítva, az út teljesen tiszta."

7.4 Tesztelést támogató segéd- és fordítóprogramok specifikálása

A tesztek elvárt és tényleges kimeneteit külön fájlokba tároljuk el.

A teszt lefutása után ezt a két fájlt összehasonlítjuk windows parancssor fc parancsával.

fc assert.txt output.txt melynek két eredménye lehet: egyezés illetve eltérés, ha eltér kiírjuk a két fájl tartalmát, egyébként pedig a sikerességet.

Az eredményt a parancs a windows parancssori konzolra írja ki.

Napló

Kezdet	Időtartam	Résztevők	Leírás
2026.04.03. 18:00	2 óra	Czap Bocskai Tóth Dénes	Megbeszélés, feladatok kiosztása heti feladat átbeszélése
2026.04.07.	2,5 óra	Tóth	Részletes use-case-ek megírása 1-9-ig
2026.04.08.	2 óra	Jandó	Use-case-ek megírása 15-19-ig
2026.04.08.	3 óra	Czap	Részletes use-case-ek megírása 12-14-ig. Uml diagram változtatása a módosításnak megfelelően.
2026.04.08.	4 óra	Bocskai	Use-case-ek megírása 10, 11 Új osztályok, változások leírása Szekvencia diagramok elkészítése
2026.04.08. 20:00	1,5 óra	Czap Bocskai Tóth Jandó	Heti feladat follow-up, további feladattal való egyeztetés
2026.04.09.	2 óra	Tóth	Tesztelési terv megírása 1-9-ig
2026.04.09.	2 óra	Czap	Tesztelési terv megírása 12-14. Tesztelést támogató segédprogramok specifikálása.
2026.04.10	0,5 óra	Bocskai	Tesztelési terv megírása 10, 11
2026.04.10	4 óra	Dénes	Prototípus interface
2026.04.12	2 óra	Jandó	Tesztelési terv megírása 15-19-ig
2026.04.12	1 óra	Tóth	Apró javítások a dokumentumban.

8. Részletes tervek

23 – githappens

Konzulens:

Haragos Gergő Viktor

Csapattagok

Czap László	NS7V2T	czap.laszlo@edu.bme.hu
Bocskai Zsófia	UGSTW9	bocskaizsofi@gmail.com
Tóth Márton	K01YXA	toth.marton21@gmail.com
Dénes Péter István	PO6MVA	denespeter03tab@gmail.com
Jandó Barnabás Tamás	AWQ77G	jandobarnabas@gmail.com

2026.04.20.

8. Részletes tervek

8.1 Osztályok és metódusok tervei.

8.1.1

Lane

- **Felelősség**

A Lane osztály az úthálózat gráfjának egy irányított élét reprezentálja, amelyen a járművek közlekednek. Ez az osztály a "gazdája" az útszakasz fizikai állapotának: nyilvántartja a lehullott hó mennyiségét, a kialakult jégpáncél vastagságát és az esetleges balesetek tényét. Ezen adatok alapján felelős annak kiszámításáért, hogy a sáv az adott pillanatban járható-e, valamint meghatározza a sáv "dinamikus súlyát", amely jelzi az áthaladás nehézségét az útvonaltervező algoritmus számára.

- **Össztályok**

Nincs.

- **Interfészek**

Nincs.

- **Asszociációk**

- **source: Node** — A sáv kiinduló csomópontja.
- **destination: Node** — A sáv célállomása.
- **parentRoad: Road** — Az az út objektum, amelyhez ez a sáv tartozik.
- **currentState: LaneState** — A sáv állapotát reprezentáló objektum.

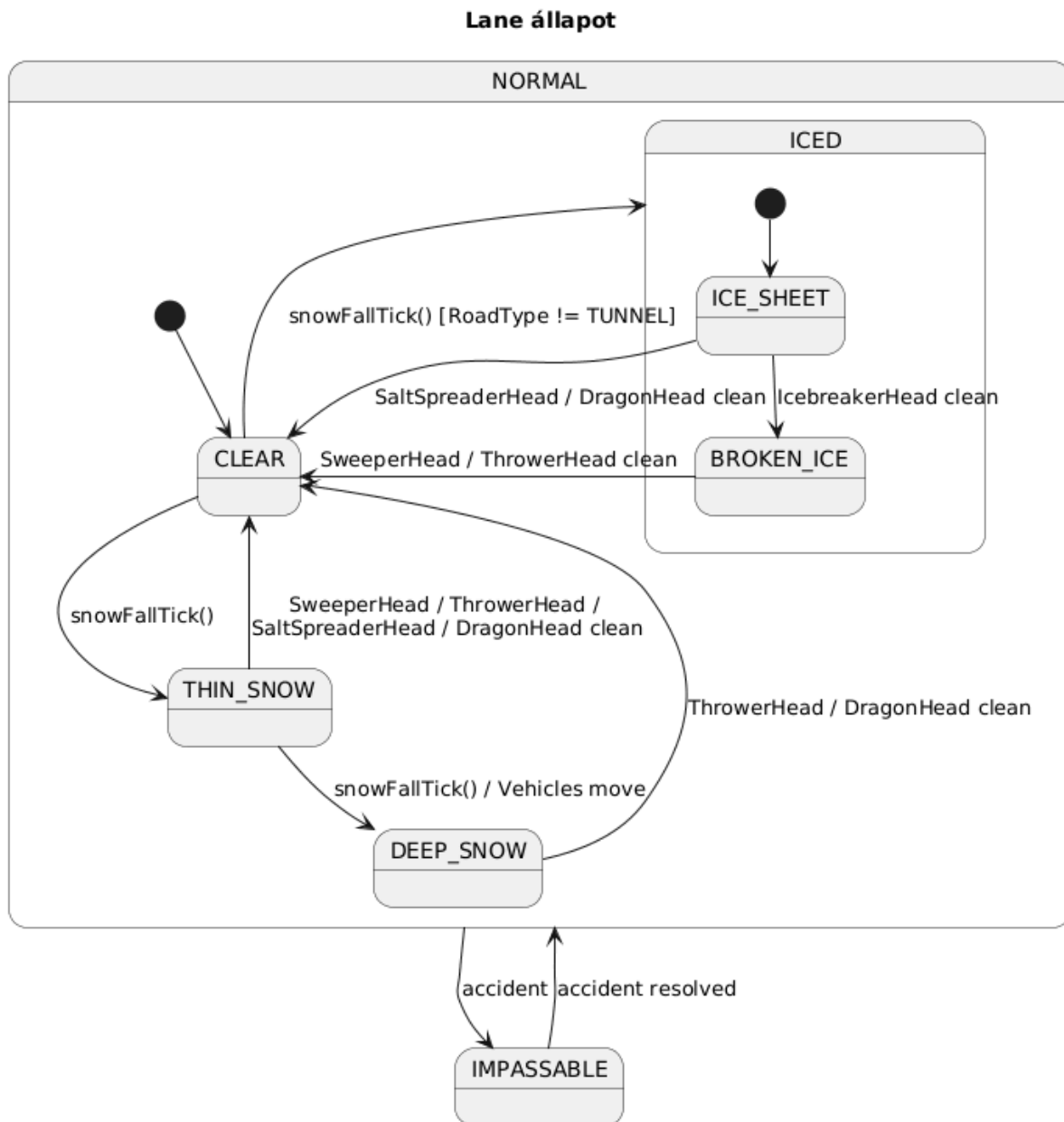
- **Attribútumok**

- **– snowValue: int** — a sávon felhalmozódott hó mennyiségi mutatója.
- **– snowThickness: double** — a hóréteg aktuális fizikai vastagsága.
- **– iceThickness: double** — az úton lévő jégpáncél vastagsága.
- **– gravelThickness: double** — a sávon lévő zúzalék fizikai vastagsága (ÚJ, a GravelSpreaderHead fejjel szórt zúzalék mennyisége).
- **– hasAccident: boolean** — jelzi, ha a sávon baleset történt, ami akadályozza a haladást.
- **– waitTimer: int** — baleset vagy elakadás esetén a kényszerű várakozási időt mérő számláló.
- **– currentState: LaneState** — a sáv aktuális állapotát reprezentáló objektum (State tervezési minta).

- **Metódusok**

- **+ setState(newState: LaneState): void** - Megváltoztatja a sáv jelenlegi állapotát a paraméterben kapott új állapotra
- **+ isPassable(): boolean** — kiértékeli a sáv aktuális állapotát (hó, jég, baleset) és visszaadja, hogy a járművek áthajthatnak-e rajta. Delegálja a kérdést a currentState objektumnak.
return currentState.isPassable()
- **+ getDynamicWeight(): double** — kiszámítja a sáv súlyozását az útvonalkereséshez; a mélyebb hó vagy jegesedés növeli az értéket, így a járművek preferálják a tisztább szakaszokat. Delegálja a számítást a currentState objektumnak.
return currentState.getDynamicWeight()

- + **getLaneState(): LaneState** — visszaadja a sáv leírására szolgáló állapot-objektumot.
return currentState
- + **applyWeather(snowamount: int): void** - Módosítja a sáv állapotát az időjárási viszonyok hatásának megfelelően. (havazás mennyisége)
- + **hasAccident(): boolean** - Visszaadja, hogy a sávon van-e baleset.
- + **change(amount: int): void** módosítja a hó, jég vagy zúzalék mennyiségét a sávon havazás vagy hókotrás hatására. A takarítás során a kotrófej hívja meg, az amount értékével csökkenti a megfelelő vastagság-attribútumot, és ha a vastagság eléri a nullát, módosítja az állapotot.
if currentState is DeepSnow or ThinSnow:
snowThickness = max(0, snowThickness - amount)
snowValue = max(0, snowValue - amount)
if snowThickness == 0: changeState(Clear)
else if currentState is IceSheet or BrokenIce:
iceThickness = max(0, iceThickness - amount)
if iceThickness == 0: changeState(Clear)
else if currentState is Gravel:
gravelThickness = max(0, gravelThickness - amount)
if gravelThickness == 0: changeState(IceSheet)



8.1.2

Road (absztrakt)

- **Felelősség**

A Road absztrakt osztály két csomópontot összekötő fizikai útszakaszt reprezentál a játékteren. Feladata az egy irányba haladó sávok összefogása és koordinálása. Emellett támogatja a járművek navigációját azáltal, hogy segít meghatározni a sávok közötti szomszédsági viszonyokat.

- **Össztályok**

Nincs.

- **Interfészek**

Nincs.

- **Asszociációk**

- **lanes:** **List<Lane>** — Az adott útszakaszhoz tartozó sávok listája.
- **source:** **Node** — Az útszakasz kiinduló csomópontja.
- **destination:** **Node** — Az útszakasz célállomását jelentő csomópont.
- **Attribútumok**
 - – **lanes:** **List<Lane>** — az útszakaszt felépítő sávok gyűjteménye.
- **Metódusok**
 - + **getAdjacentLane(Lane lane, int index): Lane** — megkeresi és visszaadja a paraméterként kapott sáv melletti szomszédos sávot a megadott index (irány) alapján. Ez a metódus teszi lehetővé a járművek számára a sávváltást akadályok vagy balesetek esetén.
`pos = lanes.indexOf(lane)`
`newPos = pos + index`
`if newPos < 0 or newPos >= lanes.size():`
 `return null`
 `return lanes.get(newPos)`
 - + **abstract applyWeatherEffects(weather: Weather): void** — absztrakt metódus, amelyet a leszármazottak felüldefiniálnak. Az időjárás hatását alkalmazza az útszakasz sávjaira.
 - **addLane(lane: Lane): void** - A paraméterben átvett Lane objektumot hozzáadja a lanes listához.

Az osztályhierarchia konkrét tagjai:

- **NormalRoad:** hagyományos útszakasz. Az `applyWeatherEffects` közvetlenül érvényesíti az időjárási hatásokat a sávokon: a `Weather`-től kapott havazásmennyiséget minden sávra alkalmazza.
`snowAmount = weather.getSnowAmount()`
 for each lane in lanes:
 `lane.change(snowAmount)`
- **Bridge:** híd. Az `applyWeatherEffects` a `Weather.snowfallTick(this)` visszahívást használja, mivel a hidakon fokozott jegesedés lép fel; a `Weather` a `Road` típusa alapján határozza meg az intenzitást.
 `weather.snowfallTick(this)`
- **Tunnel:** alagút. Az `applyWeatherEffects` üres implementációt ad, mivel az alagutakban az időjárás hatása korlátozott: a hó vagy jég általában nem befolyásolja az útsávok állapotát. (Az alagutat védi a fedél.)

8.1.3

RoadNetwork

- **Felelősség**

A `RoadNetwork` osztály felelős Zúzmaraváros teljes úthálózatának reprezentálásáért és kezeléséért. Ez az osztály működik a játék útvonalkereső motorjaként: nyilvántartja a város összes csomópontját és útszakaszát, valamint kiszámítja a járművek számára a céljukhoz vezető legrövidebb, aktuálisan járható utat. Figyelembe veszi az utak típusát (híd, alagút, felszíni út) és azok aktuális állapotát a tervezés során.

- **Ősosztályok**

Nincs.

- **Interfészek**

Nincs.

- **Asszociációk**
 - **nodes: List<Node>** — Az úthálózatot felépítő összes csomópont listája.
 - **roads: List<Road>** — Az úthálózatot felépítő összes útszakasz (élek) listája.
- **Attribútumok**
 - **– nodes: List<Node>** — a városban található összes kereszteződés, lakóhely és munkahely gyűjteménye.
 - **– roads: List<Road>** — a csomópontokat összekötő útszakaszok, hidak és alagutak listája.
- **Metódusok**
 - **+ getShortestPath(Node from, Node to): Route** meghatározza a két megadott csomópont közötti legrövidebb, akadálymentes útvonalat. A számítás során a lane.getDynamicWeight() értéket használja súlyként.
Súlyozási sorrend: Clear (legkisebb súly) < Gravel < ThinSnow < BrokenIce. A DeepSnow, IceSheet és Impassable állapotok esetén a súly végtelen, mivel az isPassable() hamisat ad vissza, így az algoritmus ezeket az éleket kihagyja. Dijkstra-algoritmust alkalmaz a dinamikus súlyok felhasználásával
distance = map; előd = map; Q = min-priority queue
for each node n in nodes:
distance[n] = $+\infty$
distance[from] = 0
Q.insert(from, 0)
while Q not empty:
current = Q.extractMin()
if current == to: break
for each lane l kilépve current-ból:
if not l.isPassable(): continue
w = l.getDynamicWeight()
neighbor = lane célcsomópontja
if distance[current] + w < distance[neighbor]:
distance[neighbor] = distance[current] + w
előd[neighbor] = l
Q.insert(neighbor, distance[neighbor])
return buildRoute(előd, to)
 - **+ getLane(Lane coord): Lane** — visszaadja a paraméterként megadott koordinátákhoz vagy azonosítóhoz tartozó konkrét sáv objektumot.
for each road r in roads:
for each lane l in r.lanes:
if l matches coord: return l
return null

8.1.4

Route

- **Felelősség**

A Route osztály egy konkrét útvonaltervet reprezentál, amelyet az úthálózat útvonalkereső algoritmus állít elő a járművek számára. Feladata a célba jutáshoz szükséges sávok rendezett listájának tárolása. A szimuláció során ez az osztály felelős azért, hogy a jármű haladása közben kiszolgálja ("adagolja") a következő sávot, amelyre a járműnek rá kell hajtania.

- **Össztályok**

Nincs.

- **Interfészek**

Nincs.

- **Asszociációk**

- **lanes: List<Lane>** — Az útvonalat felépítő sávok rendezett sorozata.

- **Attribútumok**

- **– lanes: List<Lane>** — a jármű által bejárando sávok listája.

- **Metódusok**

- **+ getNextLane(Lane curr): Lane** — megkeresi a paraméterként kapott aktuális sávot az útvonalban, és visszaadja a soron következő sáv objektumot. Ezzel irányítja a járművet az útvonal mentén.
`idx = lanes.indexOf(curr)`
`if idx == -1 or idx == lanes.size() - 1:`
`return null`
`return lanes.get(idx + 1)`
- **+ getLength(): double** — visszaadja az útvonal teljes hosszát, amely az útvonalat alkotó sávok hosszának összege.
`sum = 0`
`for each lane in lanes:`
`sum += lane.length`
`return sum`
- **+ getLanes(): List<Lane>** - Visszaadja az útvonalon szereplő sávokat

8.1.5

Node

- **Felelősség**

Az absztrakt Node osztály az úthálózatot alkotó gráf csúcspontjait reprezentálja, ahol a város különböző útszakaszai találkoznak. Feladata a földrajzi vagy logikai pont azonosítása. A leszármazott osztályok (Intersection, Terminal, Residence, Workplace) speciális viselkedéssel egészítik ki a jármű-érkezési eseménykezelést.

- **Össztályok**

Nincs.

- **Interfészek**

Nincs.

- **Asszociációk**

- **A RoadNetwork osztály felhasználja.**

- **Attribútumok**

- **– id: String** — a csomópont egyedi azonosítója vagy neve.

- **Metódusok**

- **+ getId(): String** — visszaadja a csomópont egyedi azonosítóját.
- **+ onVehicleEnter(vehicle: Vehicle): void** — a jármű érkezésének kezelése.
 Az alaposztályban üres; a leszármazottak felüldefiniálják.

A Node hierarchia konkrét tagjai:

- **Intersection:** útkereszteződés, ahol több útszakasz találkozik. Felüldefiniálja az onVehicleEnter metódust: lehetővé teszi a járművek számára az egyik útról a másikra történő áthaladást.

```

nextLane = vehicle.currentRoute.getNextLane(vehicle.currentLane)
if nextLane != null:
    vehicle.changeLane(nextLane)

```

- **Terminal:** busz végállomás. Felüldefiniálja az onVehicleEnter(vehicle: Vehicle) metódust. Mivel a Terminal egy statikus pályaelem, nem ő osztja a pontokat. A metódus mindössze fogadja a járművet, vagy esetleg naplózza az érkezés tényét. A forduló befejezésének adminisztrálását a Bus és a BusDriverRole végzi a saját működése (pl. a mozgás utáni ellenőrzés) során.
- **Residence:** lakóhely csomópont. Az autók kiindulási pontja. Az onVehicleEnter a jármű érkezését kezeli.
- **Workplace:** munkahely csomópont. Az autók célállomása. Az onVehicleEnter a jármű érkezését kezeli.

8.1.6 Vehicle

- **Felelősség**

A Vehicle absztrakt osztály a játékban mozgó összes jármű közös ősosztálya. Feladata a járművek alapvető állapotának és viselkedésének összefogása: nyilvántartja az aktuális sávot, a sávon belüli pozíciót és a haladási sebességet. A leszármazott osztályok (Bus, Car, Snowplow) erre az alapra építve valósítják meg a saját speciális működésüket. A Vehicle biztosítja az egységes mozgási modellt a szimulációban, így minden jármű ugyanazon általános szabályok szerint halad az úthálózaton.

- **Ősosztályok**

Nincs.

- **Interfészek**

Nincs.

- **Asszociációk**

- **currentLane: Lane** — az a sáv, amelyen a jármű éppen halad.

- **Attribútumok**

- - **id: String** — a jármű egyedi azonosítója.
- - **currentLane: Lane** — az aktuális sáv, amelyen a jármű tartózkodik.
- - **positionOnLane: double** — a jármű pozíciója az aktuális sáv mentén.
- - **speed: double** — a jármű aktuális sebessége.

- **Metódusok**

- + **move(): void** — a járművet előre mozgatja az aktuális sávon a sebessége alapján. Ha a jármű eléri a sáv végét, a megfelelő csomópont onVehicleEnter() metódusa kezelheti a továbblépést.

```

if currentLane == null: return
positionOnLane += speed
if positionOnLane >= currentLane.length:
    target = currentLane.destination

```



```
positionOnLane = 0
target.onVehicleEnter(this)
```

- **+ tick(): void** — a szimuláció egy körének megfelelő frissítést hajt végre. Az alapsztályban az általános viselkedés a mozgás végrehajtása; a leszármazottak ezt felüldefiniálhatják vagy kiegészíthetik.
- **+ getCurrentLane(): Lane** — visszaadja azt a sávot, amelyen a jármű aktuálisan tartózkodik.
- **+ changeLane(targetLane: Lane): boolean** — megpróbálja a járművet a megadott cél sávra áthelyezni. Siker esetén frissíti az aktuális sávot és true értékkel tér vissza, ellenkező esetben false-szal.
- **+ updatePositionOn(lane: Lane): void** — frissíti a jármű pozícióját a megadott sávon, például mozgás vagy sávváltás után.
- **+ setCurrentLane(lane: Lane): void** — beállítja a jármű aktuális sávját a paraméterként kapott sávra.
- **+ getSpeed(): double** — visszaadja a jármű aktuális sebességét.

8.1.7 Bus

- **Felelősség**

A Bus osztály a városi buszjáratot reprezentálja. A busz két végállomás között közlekedik, és egy előre kijelölt útvonal mentén halad. Feladata a közlekedés lebonyolítása, a végállomások elérése, valamint a körök végrehajtása a szimulációban. A busz működése összekapcsolódik a BusDriverRole szereppel, amely az útvonal kijelöléséért és a teljesített fordulókat nyilvántartásáért felel.

- **Össztályok**

Vehicle

- **Interfészek**

Nincs.

- **Asszociációk**

- **terminalA: Terminal** — az egyik végállomás.
- **terminalB: Terminal** — a másik végállomás.
- **currentRoute: Route** — a busz által követett aktuális útvonal.

- **Attribútumok**

- **- terminalA: Terminal** — a busz egyik végállomása.
- **- terminalB: Terminal** — a busz másik végállomása.
- **- immobileTime: int** — azt méri, hogy a busz mennyi ideje nem tud haladni.
- **- currentRoute: Route** — a busz számára kijelölt aktuális útvonal.

- **Metódusok**

- **+ tick(): void** — a busz egy környi működését hajtja végre. Ha az aktuális sáv járható, a busz mozog; ha nem, várakozik vagy megpróbálhat később továbbhaladni. Ha végállomásra ér, a további kezelés a Route és a Terminal logikáján keresztül történik.

```

if currentLane == null or currentRoute == null: return
if currentLane.isPassable():
    immobileTime = 0
    move()
else:
    immobileTime++

```

- + **getImmobileTime(): int** - Visszaadja, hogy a busz mennyi ideje nem tud haladni.

8.1.8 Car

- **Felelősség**

A Car osztály a városban lakóhely és munkahely között közlekedő személyautót reprezentálja. Az autó célja, hogy mindig a legrövidebb járható útvonalon jusson el a célállomásra. Az útvonaltervezést a RoadNetwork végzi, míg a Car saját feladata a kijelölt útvonal követése és az aktuális közlekedési helyzethez való alkalmazkodás. Az autó reagálhat az utak járhatatlanná válására, és szükség esetén újratervezés kezdeményezhető.

- **Ősosztályok**

Vehicle

- **Interfészek**

Nincs.

- **Asszociációk**

- **residence: Residence** — az autó kiindulási pontja.
- **workplace: Workplace** — az autó célállomása.
- **currentRoad: Road** — az az útszakasz, amelyhez az autó aktuálisan kapcsolódik.

- **Attribútumok**

- - **residence: Residence** — a járműhöz tartozó lakóhely.
- - **workplace: Workplace** — a járműhöz tartozó munkahely.
- - **currentRoad: Road** — az aktuális útszakasz.

- **Metódusok**

- + **changeLane(targetLane: Lane): boolean** — megkísérli a járművet a megadott cél sávra áthelyezni. Siker esetén frissíti az aktuális sávot és true értékkel tér vissza, ellenkező esetben false-szal.
- + **stopAndWait(): void** — megállítja a járművet, és várakozó állapotba helyezi, például ha a következő sáv nem járható vagy akadály található.
- + **setCurrentRoute(route: Route): void** — beállítja a jármű aktuálisan követett útvonalát a megadott Route objektumra.

8.1.9 Snowplow

- **Felelősség**

A Snowplow osztály a hókotró járművet reprezentálja. Feladata az útszakaszok tisztítása különböző cserélhető fejek segítségével. A hókotró működése szorosan kapcsolódik a CleanerRole szerephez, amely irányítja és felszereli a járművet. A Snowplow felel a pillanatnyilag felszerelt fej nyilvántartásáért, az anyagkészletek (só, bio-kerozin, zúzott kő) kezeléséért, valamint a takarítási művelet végrehajtásáért.

- **Ősosztályok**

Vehicle

- **Interfészek**

Nincs.

- **Asszociációk**

- **currentHead: Head** — az éppen felszerelt tisztítófej.

- **Attribútumok**

- - **currentHead: Head** — az aktuálisan használt tisztítófej.
- - **saltStock: int** — a rendelkezésre álló só mennyisége.
- - **biokeroseneStock: int** — a rendelkezésre álló bio-kerozin mennyisége.
- - **gravelStock: int** — a hókotróban tárolt zúzott kő mennyisége.

- **Metódusok**

- + **changeHead(Head newHead): void** — lecseréli a hókotró aktuális fejét az új fejre.

```
currentHead = newHead
```

- + **clean(Lane lane): void** — a megadott sáv takarítását végzi el az aktuális fej segítségével. A konkrét tisztítási logika a Head leszármazottjaiban van megvalósítva

```
if currentHead == null or lane == null: return
currentHead.clean(lane, this)
```

- + **addBiokerosene(amount: int): void** - a rendelkezésre álló bio-kerozin mennyiséghez hozzáadja a paraméterbenkapott mennyiséget

- + **getBiokeroseneStock(): int** - Visszaadja a bio-kerozin mennyiséget

8.1.10 Head

- **Felelősség**

A Head absztrakt osztály a hókotróra szerelhető tisztítófejek közös ősosztálya. Feladata egységes interfészt biztosítani a különféle tisztítási módok számára. A leszármazottak eltérő takarítási stratégiákat valósítanak meg: van, amelyik a havat tolja, van, amelyik jeget tör, van, amelyik sót szór, van, amelyik lánggal olvaszt, és van, amelyik zúzott követ terít az útra. Ez a hierarchia elkülöníti a hókotró járművet a konkrét takarítási algoritmusoktól.

- **Ősosztályok**

Nincs.

- **Interfészek**

Buyable

- **Asszociációk**

A snowplow használja.

- **Attribútumok**

Nincs.

- **Metódusok**

- **+ clean(Lane lane, Snowplow snowplow): void** — absztrakt művelet, amely a megadott sáv tisztítását végzi el a hókotró erőforrásainak és a fej saját működésének megfelelően. A leszármazottak felüldefiniálják.
- **+ getPrice(): int** — visszaadja a fej árát, hogy a boltban megvásárolható legyen. A konkrét értéket a leszármazottak adják meg.

8.1.11 DragonHead

- **Felelősség**

A DragonHead egy speciális tisztítófej, amely bio-kerozin felhasználásával magas hőmérséklettel olvasztja el a havat vagy jeget. Elsősorban azokban a helyzetekben hasznos, amikor a mechanikus tisztítás nem elegendő. Működése erőforrás-függő: ha a hókotró bio-kerozin készlete elfogy, a fej nem tud hatékonyan dolgozni.

- **Ősosztályok**

Head

- **Interfészek**

Buyable

- **Attribútumok**

Nincs.

- **Metódusok**

- **+ clean(Lane lane, Snowplow snowplow): void** — bio-kerozint használva megtisztítja a sávot. Ha van elegendő készlet, a hó- vagy jégréteget megszünteti vagy jelentősen csökkenti.

```
if snowplow.biokeroseneStock <= 0: return
if lane.getLaneState() is Gravel: return
snowplow.biokeroseneStock--
```

lane.change(largeAmount)

- **+ getPrice(): int** — visszaadja a DragonHead árát.

8.1.12 SweeperHead

- **Felelősség**

A SweeperHead olyan mechanikus tisztítófej, amely a havat vagy a fellazított jeget eltakarítja. A zúzalékot ugyanúgy eltakarítja, mint a havat. Vastag, de még nem teljesen lefagyott hó eltávolítására alkalmas, valamint képes a korábban kiszórt zúzott kő eltávolítására is, ha arra már nincs tovább szükség az adott sávon.

- **Ősosztályok**

Head

- **Interfészek**

Buyable

- **Attribútumok**

Nincs.

- **Metódusok**

- **+ clean(Lane lane, Snowplow snowplow): void** — a sávon lévő vékonyabb hó vagy fellazult réteg eltávolítását végzi.

lane.change(mediumAmount)

- **+ getPrice(): int** — visszaadja a SweeperHead árát.

8.1.13 ThrowerHead

- **Felelősség**

A ThrowerHead olyan mechanikus tisztítófej, amely a havat, zúzottkövet vagy a fellazított jeget oldalra dobja. Feladata a sáv gyors felszabadítása olyan helyzetekben, amikor a felületen még nem alakult ki kemény jégpáncél. Ez a fej a vastagabb, de még nem teljesen lefagyott hó eltávolítására alkalmas.

- **Ősosztályok**

Head

- **Interfészek**

Buyable

- **Attribútumok**

Nincs.

- **Metódusok**

- **+ clean(Lane lane, Snowplow snowplow): void** — oldalra szórja a havat vagy a fellazított jeget, ezzel csökkenti a sáv terheltségét.

```
lane.change(mediumOrLargeAmount)
```

- **+ getPrice(): int** — visszaadja a ThrowerHead árát.

8.1.14 IcebreakerHead

- **Felelősség**

Az IcebreakerHead feladata a kemény, letapadt jégréteg mechanikus feltörése. Ez a fej elsősorban nem a teljes tisztítást végzi el, hanem a jég állapotát alakítja át kezelhetőbbé, például jégpáncélból feltört jéggé. Ezzel előkészíti a felületet más fejek, például a SweeperHead vagy ThrowerHead számára.

- **Ősosztályok**

Head

- **Interfészek**

Buyable

- **Attribútumok**

Nincs.

- **Metódusok**

- **+ clean(Lane lane, Snowplow snowplow): void** — feltöri a jégpáncélt, hogy az út később teljesen megtisztítható legyen.

```
if lane.getLaneState() is Gravel: return
```

```
if lane.getLaneState() is IceSheet:
```

```
    lane.change(smallAmount)
```

```
else:
```

```
    lane.change(smallAmount)
```

- **+ getPrice(): int** — visszaadja a IcebreakerHead árát.

8.1.15 SaltSpreaderHead

- **Felelősség**

A SaltSpreaderHead a hókotró sókészletét használva csökkenti az utak jegesedését és javítja a tapadást. Ez a fej nem feltétlenül távolítja el teljesen a havat vagy a jeget, hanem kémiai úton mérsékli a csúszósságot és segít megelőzni az újrafagyást. Működéséhez szükség van a hókotró sókészletére.

- **Össztályok**

Head

- **Interfészek**

Buyable

- **Attribútumok**

Nincs.

- **Metódusok**

- **+ clean(Lane lane, Snowplow snowplow): void** — sót szór a sávra. Ha van elegendő sókészlet, csökkenti a jegesedés hatását vagy javítja az út járhatóságát.

```
if snowplow.saltStock <= 0: return
if lane.getLaneState() is Gravel: return
snowplow.saltStock--
lane.change(smallAmount)
```

- **+ getPrice(): int** — visszaadja a SaltSpreaderHead árát.

8.1.16 GravelSpreaderHead

- **Felelősség**

A GravelSpreaderHead speciális kotrófej, amely zúzott követ, azaz zúzalékot szór a jármű előtt az útfelületre. Elsődleges célja nem a hó fizikai eltávolítása, hanem a csúszós, jeges útfelület biztonságosabbá tétele. Működéséhez zúzott kő szükséges, amelyből csak véges mennyiség tárolható a hókotrón. A készlet a Store objektumban vásárolható meg. A fej használata új állapotot hozhat létre a sávon: a Gravel állapotot.

- **Össztályok**

Head

- **Interfészek**

Buyable

- **Attribútumok**

Nincs.

- **Metódusok**

- **+ clean(Lane lane, Snowplow snowplow): void** — ellenőrzi a hókotró zúzottkő-készletét, majd annak felhasználásával a sávot kezeli. Ha van elegendő zúzalék, a sáv állapotát Gravel állapotba helyezi vagy növeli a zúzalék borítottságát.

```
if lane == null or snowplow == null: return
```

```
if snowplow.gravelStock <= 0: return
```

```
snowplow.gravelStock--
```

```
if lane.getLaneState() is IceSheet or lane.getLaneState() is BrokenIce or
```

```
lane.getLaneState() is Clear:
```

```
    lane.changeState(new Gravel())
```

- **+ getPrice(): int** — visszaadja a GravelSpreaderHead árát.

8.1.17 Store

- **Felelősség**

A Store osztály a bolt reprezentálására szolgál a szimulációban, ahol a takarító szerepkörű játékosok (CleanerRole) új felszereléseket (pl. különböző hókotró fejeket) vásárolhatnak. Felelős a megvásárolható tárgyak listájának nyilvántartásáért és a tranzakciók lebonyolításáért.

- **Ősosztályok**
Nincs
- **Interfészek**
Nincs
- **Asszociációk**
inventory: List<Buyable> — A boltban elérhető, megvásárolható elemek gyűjteménye.
- **Attribútumok**
– inventory: List<Buyable> — a bolt kínálatát alkotó elemek listája.
- **Metódusok**
+ buy(CleanerRole cleanerRole, Buyable item): boolean — Lebonyolítja a vásárlást.
Ellenőrzi, hogy a játékosnak van-e elég pénze a tárgy megvételéhez. Ha igen, megtörténik a pénzlevonás és a tárgy átadása, majd igaz értékkel tér vissza.
price = item.getPrice().

8.1.18 Buyable (interface)

- **Felelősség**
A Buyable egy interfész, amely azokat az elemeket (például a zöld színnel jelölt hókotró fejeket: Head) fogja össze, amelyek a boltban (Store) megvásárolhatók. Biztosítja, hogy minden ilyen objektum lekérdezhető árral rendelkezzen.
- **Ősosztályok**
Nincs
- **Interfészek**
Nincs
- **Attribútumok**
Nincs
- **Metódusok**
+ getPrice(): int — Visszaadja az adott elem vételárát.

8.1.19 LaneState (interface)

- **Felelősség**
A LaneState interfész a sávok (Lane) időjárási és felületi állapotait határozza meg (Állapot / State tervezési minta). Célja, hogy egységesítse az egyes állapotok viselkedését, meghatározva, hogy az adott sáv járható-e, mennyi a rajta való áthaladás "költsége" (súlya), és hogyan reagál a további hóesésre.

- **Össosztályok**
Nincs
- **Interfészek**
Nincs
- **Attribútumok**
Nincs
- **Metódusok**
 - + **isPassable(): boolean** — Visszaadja, hogy az adott állapotban a sáv járható-e a járművek számára.
 - + **getDynamicWeight(): double** — Visszaadja a sáv dinamikus súlyát (pl. az útvonalkereső algoritmus számára), amely az útviszonyoktól függően változik.
 - + **handleWeatherChange(snowAmount: int): LaneState** — A kapott hó mennyiség (vagy letakarítás) függvényében kezeli az állapotváltozást, és visszatér az új (vagy a jelenlegi) állapottal.

8.1.20 BrokenIce

- **Felelősség**

A feltört jég állapotot reprezentálja egy sávon. Olyan esetekben jön létre, amikor a jégréteget már felszakították (pl. IcebreakerHead segítségével), de a sávot még nem takarították le teljesen.

- **Ősosztályok**

Nincs

- **Interfészek**

LaneState

- **Attribútumok**

Nincs

- **Metódusok**

+ **isPassable(): boolean** — Megadja, hogy a feltört jég akadályozza-e az áthaladást.

+ **getDynamicWeight(): double** — Visszaadja a feltört jégre jellemző dinamikus súlyt.

+ **handleWeatherChange(snowAmount: int): LaneState** — Kezeli az időjárás változását, szükség esetén új állapotba (pl. újra összefüggő jégbe vagy havas állapotba) lépteti a sávot.

8.1.21 Clear

- **Felelősség**

A tiszta (hó- és jégmentes) útállapotot reprezentálja. Ez az alapértelmezett, akadálymentes állapot, amikor a járművek a leggyorsabban tudnak haladni.

- **Ősosztályok**

Nincs

- **Interfészek**

LaneState

- **Attribútumok**

Nincs

- **Metódusok**

+ **isPassable(): boolean** — Tiszta út esetén mindig true értékkel tér vissza.

+ **getDynamicWeight(): double** — Visszaadja az alapértelmezett, legalacsonyabb dinamikus súlyt.

+ **handleWeatherChange(snowAmount: int): LaneState** — Ha a snowAmount eléri egy bizonyos küszöböt, a metódus visszatérhet például egy új ThinSnow objektummal.

8.1.22 ThinSnow

- **Felelősség**

A vékony hóréteggel borított állapotot reprezentálja. A járművek még képesek lehetnek haladni rajta, de a sáv súlya már megnövekedett a tiszta állapothoz képest.

- **Össztályok**
Nincs
- **Interfészek**
LaneState
- **Attribútumok**
Nincs
- **Metódusok**
 - + **isPassable(): boolean** — Megadja a járhatóságot vékony hó esetén.
 - + **getDynamicWeight(): double** — Visszaadja a vékony hóra jellemző dinamikus súlyt.
 - + **handleWeatherChange(snowAmount: int): LaneState** — További hóesés esetén átválthat DeepSnow állapotba.

8.1.23 IceSheet

- **Felelősség**

Az összefüggő jégréteg állapotot reprezentálja egy sávon. Jellemzően csúszós és rendkívül nehezen járható, a megfelelő takarító jármű (jégtörő) beavatkozását igényli a járhatóság visszaállításához.
- **Össztályok**
Nincs
- **Interfészek**
LaneState
- **Attribútumok**
Nincs
- **Metódusok**
 - + **isPassable(): boolean** — Megadja a járhatóságot a jégpályán.
 - + **getDynamicWeight(): double** — Visszaadja a jégréteghez tartozó extrém magas dinamikus súlyt.
 - + **handleWeatherChange(snowAmount: int): LaneState** — Frissíti a jégréteg állapotát az időjárás vagy a külső beavatkozások függvényében.

8.1.24 DeepSnow

- **Felelősség**

A mély hó állapotot reprezentálja egy sávon. Ebben az állapotban az út jellemzően járhatatlanná válik a legtöbb jármű számára, amíg egy hókotró (Snowplow) meg nem tisztítja.

- **Ősosztályok**
Nincs
- **Interfészek**
LaneState
- **Attribútumok**
Nincs
- **Metódusok**
 - + **isPassable(): boolean** — Visszaadja a járhatóságot (valószínűleg false).
 - + **getDynamicWeight(): double** — Visszaadja a mély hóhoz tartozó maximális dinamikus súlyt.
 - + **handleWeatherChange(snowAmount: int): LaneState** — Kezeli a hőmennyiség változását (pl. hóolvadás esetén visszaválthat ThinSnow vagy Clear állapotba).

8.1.25 Bridge

- **Felelősség**
A Bridge osztály egy hidat reprezentál, ami egy speciális útként szolgál.
- **Ősosztályok**
Road
- **Interfészek**
Nincs
- **Attribútumok**
Nincs saját attribútuma.
- **Metódusok**
 - + **applyWeatherEffects(weather: Weather): void**- Az időjárás hatását alkalmazza az útszakasz sávjaira.

8.1.26 Tunnel

- **Felelősség**
A Tunnel osztály egy alagutat modellez. Az alagutakban az időjárás hatása korlátozott, ezért a hó vagy jég általában nem befolyásolja az útsávok állapotát.

- **Össztályok**
Road
- **Interfészek**
Nincs
- **Attribútumok**
Nincs saját attribútuma.
- **Metódusok**
+ **applyWeatherEffects(weather: Weather): void** - Az időjárás hatását alkalmazza az útszakasz sávjaira.

8.1.27 NormalRoad

- **Felelősség**
A NormalRoad egy hagyományos útszakaszt modellez. Az időjárási hatások közvetlenül érvényesülnek rajta, így a sávok állapota a hó vagy jég mennyiségének megfelelően változhat.
- **Össztályok**
Road
- **Interfészek**
Nincs
- **Attribútumok**
Nincs saját attribútuma.
- **Metódusok**
+ **applyWeatherEffects(weather: Weather): void** - Az időjárás hatását alkalmazza az útszakasz sávjaira.

8.1.28 Player

- **Felelősség**
A Player egy valódi emberi résztvevőt reprezentál a rendszerben. A játékos a játék emberi irányítója, de önmagában kevés közvetlen műveletet végez, a tényleges interakciók a hozzá tartozó Role segítségével valósulnak meg. Egy játékos egyszerre több szerepkört is betölthet, és ezek a szerepkörök határozzák meg, hogy milyen járműveket irányíthat, milyen döntéseket hozhat, és hogyan szerez pontokat a játék során. A játékos stratégiai döntéseket hoz, amik hatással vannak a Zúzmaraváros állapotára, illetve a saját pontszámára.

- **Ősosztályok**
Nincs őosztály.
- **Interfészek**
Nincs
- **Asszociációk**
 - **Player - Role:** A kompozíció túloldalán a Role osztály áll. A kapcsolat célja, hogy a játékos a szerepkörain keresztül tudjon cselekedni a játékban. Egy játékos több szerepkört is tartalmazhat, és ezek határozzák meg, milyen járműveket irányíthat és milyen döntéseket hozhat.
 - **Player - Game:** Az kompozíció túloldalán a Game osztály áll. A kapcsolat célja, hogy a Game nyilvántartsa az összes játékost. A Player résztvevője a Game-nek.
- **Attribútumok**
 - id: int - A játékos egyedi azonosítója.
 - name: String - A játékos neve.
- **Metódusok**
 - + **getName(): String** - A játékos nevét adja meg
 - + **getSumPoints(): int** - Összegzi a szerepkörök által szerzett pontokat.

8.1.29 Role

- **Felelősség**
A Role (Szerepkör) absztrakt osztály felel a járművek irányításáért. A játékosok különböző szerepkörökön keresztül irányíthatják a hozzájuk tartozó járműveket, végezhetnek további járművekhez kapcsolódó műveleteket. Egy játékos több szerepkört is felvehet egyszerre.
- **Ősosztályok**
Nincs őosztály.
- **Interfészek**
Nincs
- **Asszociációk**
 - **Roles - Player (Játékos):** A kompozíció túloldalán a Player (Játékos) osztály áll. A kapcsolat célja, hogy a szerepkörökön keresztül a játékos tudjon cselekedni a játékban. Egy játékos több szerepkört is tartalmazhat, és ezek határozzák meg, milyen járműveket irányíthat és milyen döntéseket hozhat.
- **Attribútumok**
Nincs attribútum.
- **Metódusok**
 - + **getScore(): int**- Megadja az adott szerepkör által szerzett pontszámot

8.1.30 BusDriverRole

- **Felelősség**

A BuszvezetőSzerep a Role leszármazottja. A buszvezető felelős a buszok mozgatásáért, útvonalak megtervezéséért és a fordulók teljesítéséért két végállomás között. A szerepkör pontszáma a sikeresen teljesített fordulókból adódik.

- **Őszosztályok**

Role

- **Interfészek**

Nincs

- **Asszociációk**

- **BusDriverRole - Bus:** Az asszociáció túloldalán a Bus osztály áll. A kapcsolat célja, hogy a buszvezető irányíthassa a buszokat és teljesíthesse vele a fordulókat.

- **Attribútumok**

- completedRounds: int - A buszvezető által teljesített fordulók száma.
- bus: Bus - A buszvezető által irányított busz.

- **Metódusok**

- + **assignRoute(bus: Bus, destination: Node): int** - A busz aktuális állapotából a cél csomópontba megtalálja a legrövidebb útvonalat. A metódus a RoadNetwork osztály getShortestPath() függvényét használja, amely a Dijkstra-algoritmusra épül, és a sávok dinamikus súlyát (lane.getDynamicWeight()) veszi figyelembe. Az útvonal kiszámítása után a metódus beállítja a busz currentRoute attribútumát, hogy a jármű a következő szimulációs ciklusban ezen az útvonalon haladhasson tovább. A metódus visszatérési értéke a megtalált útvonal teljes hossza (összegzett súly), vagy 0, ha nem található járható útvonal.
- + **incrementCompletedRounds(): void** - Növeli a teljesített fordulók számát.
- + **getScore():int** - A Role metódusának felüldefiniált metódusa, megadja a buszvezető szerepkör által szerzett pontszámot.

8.1.31 CleanerRole

- **Felelősség**

A CleanerRole (TakarítóSzerepkör) a Role leszármazottja. A takarító felelős a hó eltávolításáért, a hókotrók irányításáért és a hókotrókat érintő gazdasági döntésekért. A szerepkör kezeli a játékos pénzét, vásárolhat a Boltban, és működteti a hókotrók fejeit.

- **Össz osztályok**
Role
- **Interfészek**
Nincs
- **Asszociációk**
 - **CleanerRole - Snowplow (Hókotró):** Az asszociáció túloldalán a Hókotró osztály áll. A kapcsolat célja, hogy a takarító irányíthassa a hókotrókat és takarítja vele az utakat.
 - **CleanerRole – Store (Bolt):** A kapcsolat célja, hogy a takarító vásárolhasson nyersanyagot, új fejeket vagy hókotrókat.
- **Attribútumok**
 - **money: int** - A takarító jelenlegi vagyona.
 - **currentHead: Head** - A takarító által jelenleg használt kotrófej
 - **snowplow: Snowplow** - A takarító hókotrója
- **Metódusok**
 - + **buy(role: Role, item: Buyable): boolean** - A takarító kezdeményezi a vásárlást.
 - + **buy(store: Store, item: Buyable): boolean** - A boltból a kotrófejek működéséhez szükséges üzemanyagokat, új kotrófejeket és hókotrókat vásárolhat.
 - + **getMoney(): int** - A takarító aktuális pénzmennyiségét adja vissza
 - + **changeMoney(amount: int): void** - A takarító aktuális pénzmennyiségéhez hozzáadja a paraméterben megadott mennyiséget.
 - + **decreaseMoney(price: int): void** - A takarító pénzét a paraméterben megadott mennyiséggel csökkenti (vásárláskor).
 - + **addSalt(amount: int): void** - A takarító hókotrójához hozzáadja a paraméterben megadott só mennyiséget
 - + **addGravel(amount: int): void** - A takarító hókotrójához hozzáadja a paraméterben megadott zúzalék mennyiséget
 - + **addBiokerosene(amount: int): void** - A takarító hókotrójához hozzáadja a paraméterben megadott biokerozin mennyiséget
 - + **getSnowplow(): Snowplow** - Megadja a takarító hókotróját
 - + **addHead(head: newHead): void** - Új fejet ad a takarítónak.
 - + **controlSnowplow(sp: Snowplow): void** - A takarító irányítja a hókotrókat. Ezt úgy valósítja meg, hogy a meghívja a takarító hókotrójának clean() metódusát, amely a fej típusának megfelelően módosítja a sáv állapotát. A metódus nem tér vissza értékkel, de mellékhatásként módosítja az utak állapotát.
 - + **getScore(): int** - A Role metódusának felüldefiniált metódusa, megadja a takarító által szerzett pontszámot.

8.1.32 Weather

- **Felelősség**

A Weather osztály felel a szimuláció során az időjárási viszonyok (havazás, esetleg fagyás) modellezéséért. A Game osztály a játékciklus során (minden körben) megszólítja. Amikor az időjárás sorra kerül, végigmegy az úthálózat összes érintett felszíni sávján (Lane), és azokon növeli a hó vastagságát, ami hatással van a sávok járhatóságára és a járművek haladására.

- **Össosztályok**

Nincs össosztálya.

- **Interfészek**

Nincs megvalósított interfész.

- **Attribútumok**

- **-roadNetwork:** ezen keresztül éri el az időjárás az úthálózatot, hogy lekérdezhesse a csomópontokat és az utakat/sávokat, amelyekre a hó hullik. Láthatósága: private (-), Típusa: RoadNetwork
- **-snowIntensity:** Eltárolja a havazás mértékét vagy valószínűségét. Ennek alapján dől el, hogy egy körben mennyi hó esik a sávokra. Láthatósága: private (-), Típusa: int vagy double.

- **Metódusok**

- **+ Weather(network: RoadNetwork):** Konstruktor, amely példányosítja az időjárás objektumot. Paraméterként kapja meg a szimulált úthálózat referenciáját, amit eltárol saját attribútumába
- **+ void snowfallTick(RoadNetwork roadNetwork):** Kiszámítja és alkalmazza a hóesés mértékét az úthálózat sávjaira az aktuális körben.

8.1.33 Game

- **Felelősség**

A Game osztály a szimuláció központi vezérlője, amely koordinálja a teljes játékmenetet, kezeli a körök számát, valamint nyilvántartja a résztvevő játékosokat és járműveket. Felelős a játéklógika léptetéséért (minden körben megszólítja a megfelelő objektumokat) és a befejezési feltételek ellenőrzéséért.

- **Össosztályok**

Nincs össosztálya.

- **Interfészek**

Nincs megvalósított interfész.

- **Attribútumok**

- **- currentRound:** Az aktuális szimulációs kör sorszámát tárolja. Láthatósága: private (-), Típusa: int.
- **- maxRound:** A játék maximális időtartamát határozza meg körökben. Láthatósága: private (-), Típusa: int.
- **- vehicles:** A rendszerben lévő, szimulációban részt vevő járművek listája. Láthatósága: private (-), Típusa: List<Vehicle>.
- **- players:** A játékban regisztrált résztvevő játékosok gyűjteménye. Láthatósága: private (-), Típusa: List<Player>.

- **Metódusok**

- **+ void tick():** Egy egységgel előre lépteti a játék állapotát és frissíti a belső logikát.

- **+ boolean isOver():** Megvizsgálja, hogy a szimuláció elérte-e a maximális körszámot vagy véget ért-e.

8.1.34 Intersection

- **Felelősség**

Az Intersection osztály felel az úthálózat (RoadNetwork) csomópontjainak reprezentálásáért. Ez az osztály köti össze az egyes útszakaszokat (Road vagy Lane), és biztosítja a járművek (Vehicle) számára az áthaladást és a kanyarodást az egyik útszakaszból a másikra. Szerepet játszik a járművek útvonalkeresésében (routing) is, hiszen innen kérdezhető le, hogy egy adott pontból mely további utak érhetőek el.

- **Ősosztályok**

Node

- **Interfészek**

Nincs megvalósított interface.

- **Attribútumok**

- **- id: int** — A kereszteződés egyedi azonosítója (hasznos gráfbejárásnál és útvonaltervezésnél).
- **- connectedRoads: List<Road>** — A kereszteződésbe befutó, illetve onnan kiinduló útszakaszok gyűjteménye. Ebből tudja a jármű, hogy merre mehet tovább.
- **- vehiclesInside: List<Vehicle>** — A kereszteződésben éppen aktuálisan tartózkodó járművek listája (amennyiben a szimulációban az áthaladás nem instant történik, hanem időt vagy helyet foglal)

- **Metódusok**

- **+ Intersection(id: int): void** — Konstruktor, amely inicializálja a kereszteződést az egyedi azonosítóval, és létrehozza a kapcsolatokat tároló üres listákat.
- **+ addConnection(road: Road): void** — Beköt egy új útszakaszt a kereszteződésbe az inicializálás vagy a pályaépítés során. Hozzáadja az átadott Road objektumot a connectedRoads listához.
- **+ getConnectedRoads():void** — Visszaadja a csomópontához csatlakozó utak listáját. Ezt a metódust hívják meg a járművek vagy a navigációs algoritmusok a következő lépés megtervezésekor.
- **+ requestEntry(vehicle: Vehicle, from: Road, to: Road): void** — Egy jármű ezen a metóduson keresztül jelzi az áthaladási szándékát. A metódus ellenőrzi, hogy a kereszteződés járható-e (nincs-e dugó, baleset, vagy ha van lámparendszer, akkor zöld-e), és visszaad egy logikai értéket. Ha igaz, a jármű beléphet a kereszteződésbe és áttérhet a cél útszakaszra (to).
- **+ onVehicleEnter(vehicle: Vehicle): void** — felüldefiniált metódus; hozzáadja a járművet a vehiclesInside listához, és kezeli az áthaladást a következő útszakaszra.

8.1.35 Residence

- **Felelősség**

A Residence osztály felel egy olyan hálózati csomópont reprezentálásáért, amely a személyautók (Car) kiindulási pontjaként és végső célállomásaként szolgál. Generálja a civil forgalmat: innen indulnak a járművek a munkahelyükre (Workplace), és ide térnek vissza. Felelős az indulásra váró vagy parkoló autók nyilvántartásáért.

- **Ősosztályok**

Node

- **Interfészek**

Nincs megvalósított interface.

- **Attribútumok**

- - **id: int** — A kereszteződés egyedi azonosítója (hasznos gráfbejárásnál és útvonaltervezésnél).
- - **connectedRoads: List<Road>** — A kereszteződésbe befutó, illetve onnan kiinduló útszakaszok gyűjteménye. Ebből tudja a jármű, hogy merre mehet tovább.
- - **parkedCars: List<Car>** — A lakóhelynél éppen parkoló (otthon tartózkodó) autók listája.

- **Metódusok**

- + **Residence(id: int): void** — Konstruktor, amely létrehozza a lakóhely csomópontot a megadott azonosítóval.
- + **spawnCar(destination: Workplace): void** — Elindít egy autót a parkedCars listából a megadott munkahely felé. Kikéri az útvonaltervezőtől a megfelelő útvonalat, és a jármű állapotát MOVING-ra állítja.
- + **acceptCar(car: Car): void** — Fogadja az útról megérkező autót. Eltárolja a parkedCars listában, és a jármű állapotát frissíti (pl. IDLE).
- + **onVehicleEnter(vehicle: Vehicle): void** — felüldefiniált metódus; ha a belépő jármű egy Car, akkor meghívja az acceptCar() metódust.

8.1.36 Workplace

- **Felelősség**

A Workplace csomópont a személyautók (Car) napközbeni célállomása. Képes fogadni az érkező civil forgalmat, eltárolja a járműveket egy bizonyos ideig (a munkaidő szimulálása), majd a megfelelő időben vagy parancsra visszaindítja őket a lakóhelyükre (Residence).

- **Ősosztályok**

Node

- **Interfészek**

Nincs megvalósított interface.

- **Attribútumok**

- - **id: int** — A kereszteződés egyedi azonosítója (hasznos gráfbejárásnál és útvonaltervezésnél).
- - **connectedRoads: List<Road>** — A kereszteződésbe befutó, illetve onnan kiinduló útszakaszok gyűjteménye. Ebből tudja a jármű, hogy merre mehet tovább.
- - **workingCars: List<Car>** — Az éppen a munkahelyen parkoló autók listája.

- **Metódusok**

- + **Workplace(id: int): void** — Konstruktor, amely létrehozza a lakóhely csomópontot a megadott azonosítóval.
- + **acceptCar(car: Car): void** — Fogadja a lakóhelyről megérkező autót. Beteszi a workingCars listába, és módosítja az autó állapotát.
- + **releaseCar(destination: Residence): void** — A munkaidő lejártával (vagy a Game objektum utasítására) elindít egy autót vissza a lakóhelyére. Eltávolítja a workingCarslistából és beállítja az útvonalát.
- + **onVehicleEnter(vehicle: Vehicle): void** — felüldefiniált metódus; ha a belépő jármű egy Car, akkor meghívja az acceptCar() metódust.

8.1.37 Terminal

- **Felelősség**

A Terminal a tömegközlekedési és karbantartó járművek (pl. Bus, Snowplow) központi bázisa. Itt várakoznak a járművek (AT_TERMINAL állapotban), itt kapják meg a feladataikat/útvonalukat (assignRoute), és ide térnek vissza a körzetük bejárása után. A hókotrók esetében itt történhet az erőforrások (pl. üzemanyag, só) feltöltése is.

- **Ősosztályok**

Node

- **Interfészek**

Nincs megvalósított interface.

- **Attribútumok**

- - **id: int** — A kereszteződés egyedi azonosítója (hasznos gráfbejárásnál és útvonaltervezésnél).
- - **connectedRoads: List<Road>** — A kereszteződésbe befutó, illetve onnan kiinduló útszakaszok gyűjteménye. Ebből tudja a jármű, hogy merre mehet tovább.
- - **stationedBuses: List<Bus>** — A végállomáson várakozó buszok.
- - **stationedPlows: List<Snowplow>** — A bázison lévő hókotrók listája.

- **Metódusok**
- + **Terminal(id: int): void** — Konstruktor, amely létrehozza a lakóhely csomópontot a megadott azonosítóval.
- + **assignRouteToBus(bus: Bus, route: Route): void** — Kiválaszt egy buszt a stationedBuses listából, kijelöli számára az útvonalat, és elindítja a hálózaton (MOVING állapotba lépteti). Az analízis modell állapotgépe alapján ez egy kritikus metódus.
- + **dispatchSnowplow(plow: Snowplow, targetArea: List<Road>): void** — Szükség (havazás) esetén elindít egy hókotrókat a megadott útszakaszok takarítására (CLEANING állapotba lépteti).
- + **acceptVehicle(vehicle: Vehicle): void** — Fogadja a műszakjukat vagy útvonalukat befejező buszokat és hókotrókat. Visszahelyezi őket a megfelelő tároló listába, és állapotukat alaphelyzetbe (AT_TERMINAL vagy FINISHED) állítja.
- + **onVehicleEnter(vehicle: Vehicle): void** — felüldefiniált metódus a Node osztályból; meghívja az acceptVehicle() metódust, amikor egy jármű belép a terminálba.

8.2 A tesztek részletes tervei, leírásuk a teszt nyelvén

8.2.1

Teszteset 1: Játék indítása

- **Leírás**
Ellenőrizni, hogy a rendszer képes-e egy konfigurációs fájl alapján megfelelően felépíteni a játékteret és inicializálni a résztvevőket.
- **Ellenőrzött funkcionalitás**
 - Konfigurációs fájlok betöltése
 - Térkép és hálózat felépítése
 - Járművek útvonaltervezése
 - Járművek állapotgépes léptetése
 - Időjárás (havazás) szimulálása
 - Hókotrók takarítási logikája
 - Körökalapú játékmenet (Game loop)
- **Várható hibahelyek**
 - Szintaktikai hibák betöltéskor
 - Hiányzó gráfélek (zsákutcák)
 - Végtelen hurok útvonaltervezéskor
 - Érvénytelen jármű-állapotátmenetek
 - Null referenciák léptetéskor
 - Negatív hóvastagság vagy erőforrás
 - Hibás körszámlálás vagy végfeltétel

- **Bemenet:**

```
load basic_map.txt
state
exit
```

- **Elvárt kimenet:**

[RoadNetwork] [Status]: Initialized

[Nodes] [Count]: 10

[Lanes] [Count]: 15

[Vehicles] [Count]: 4

[Players] [Count]: 2

[Car_1] [Position]: Residence_1

[Bus_1] [Position]: Terminal_A

[Snowplow_1] [Position]: Terminal_A

[Weather] [CurrentSnowIntensity]: 0.0

[Game] [CurrentRound]: 1

8.2.2

Teszteset 2: Hókotró mozgatása és út tisztítása

- **Leírás** A teszt célja ellenőrizni a hókotró mozgási logikáját (szomszédos szakaszra lépés), valamint a takarítási funkció sikerességét, amely során a sáv állapota és járhatósága a kotrófej hatására megváltozik.
- **Ellenőrzött funkcionális**
 - Hókotró pozíciójának és állapotának frissítése
 - Sávok közötti szomszédossági kapcsolatok ellenőrzése
 - Takarítási algoritmus (hóvastagság csökkentése)
 - Sávállapot-átmenetek kezelése (pl. Snowy -> Clear)
 - Járhatósági flag (isPassable) automatikus kalkulációja
 - Üzemanyag-fogyasztás és pontszámítás
- **Várható hibahelyek**
 - Mozgás nem szomszédos vagy érvénytelen útszakaszra
 - Takarítás kísérlete mozgás közben vagy nem megfelelő állapotban
 - Hibás kotrófej-típus miatti eredménytelen takarítás
 - A sáv állapota nem frissül a hóvastagság nullázódása után
 - Erőforrás (üzemanyag) hiánya esetén is végrehajtott művelet
 - Negatív értékbe forduló hóvastagság vagy pontszám
- **Bemenet:**

load test_map.txt

mozgas snowplow_1 lane_12

takarit snowplow_1

state

exit

- **Elvárt kimenet:**

[snowplow_1] [currentLane]: Terminal_A -> lane_12

[snowplow_1] [state]: AT_TERMINAL -> READY_TO_CLEAN

[snowplow_1] [fuel]: 100 -> 95

[lane_12] [snowThickness]: 5.0 -> 0.0

[lane_12] [currentState]: Snowy -> Clear

[lane_12] [isPassable]: false -> true

[player_cleaner] [points]: 100 -> 150

8.2.3

Teszteset 3: Kotrófej cseréje

- **Leírás** A takarító játékos lecseréli a hókotró aktuális fejét egy másik típusra, hogy más útállapotot is kezelni tudjon.
- **Ellenőrzött funkcionalitás**
 - Szöveges parancs (fej_csere) és argumentumainak sikeres értelmezése
 - A megfelelő járműpéldány kikeresése az azonosító (snowplow_1) alapján
 - A régi kotrófej objektum lecserélése (vagy típusának módosítása) az új fejtípusra (IcebreakerHead)
 - Visszajelző üzenet generálása a konzolra
- **Várható hibahelyek**
 - Érvénytelen vagy elgévelt kotrófej típus megadása
 - Nem létező vagy nem megfelelő típusú járműre (pl. Buszra) kiadott parancs
 - Fejcsere kísérlete nem megfelelő jármű-állapotban (pl. a kotró éppen CLEANING vagy MOVING állapotban van, és a szabályok szerint csak álló helyzetben szerelhető)
- **Bemenet:**

```
load test_map.txt
fej_csere snowplow_1 icebreaker
state
exit
```

- **Elvárt kimenet:**

```
[snowplow_1] [currentHead]: SweeperHead -> IcebreakerHead
```

8.2.4

Teszteset 4: Vásárlás a boltban

- **Leírás**

A tesztet azt ellenőrzi, hogy a takarító játékos képes-e a boltban megvásárolni egy kiválasztott terméket, ha rendelkezik elegendő pénzzel. A vásárlás során a rendszernek helyesen kell levonnia az árat, a megvásárolt elemet pedig a megfelelő készletbe kell helyeznie. A teszt a bolt működését, a Buyable interfészen keresztüli árlekérdezést és a CleanerRole pénzkezelését ellenőrzi.

- **Ellenőrzött funkcionalitás, várható hibahelyek**
 - Érintett osztályok: Store, Buyable, CleanerRole, Player, Snowplow.
 - A bolt termékkínálatának megjelenítése és kiválasztása.
 - A vásárlás során a Buyable.getPrice() használata.
 - A Store.buy(cleanerRole, item) működésének helyessége.
 - A pénz levonása a takarító játékostól.
 - A megvásárolt készlet hozzáadása a hókotró megfelelő erőforrásához.
 - Várható hibahelyek: a rendszer nem ellenőrzi helyesen a pénz mennyiségét; a vásárlás sikeres, de a pénz nem változik; a készlet nem frissül; hibás termékazonosító vagy mennyiség esetén a rendszer mégis végrehajtja a vásárlást.

- **Bemenet:**

```
load test4_arrange.txt
bolt_nyit
vasarol salt 3
save test4_out.txt
exit
```

Elvárt kimenet:

```
[store] [open]: false -> true
[store] [selectedItem]: null -> salt
[cleaner_1] [money]: 100 -> 70
[plow_1] [saltStock]: 0 -> 3
[log] Vásárlás sikeres: salt x3
```

8.2.5

Teszteset 5: Busz útvonalának automatikus kijelölése és teljesítése

- **Leírás**

A teszteset azt vizsgálja, hogy a rendszer képes-e automatikusan kijelölni egy útvonalat a busz számára két terminál között, majd a busz a kijelölt útvonalon végighaladva teljesíti-e a fordulót. A teszt során a jutalom kiszámítása és jóváírása is ellenőrzésre kerül.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

- Érintett osztályok: Bus, Vehicle, Route, Terminal, RoadNetwork, Lane, BusDriverRole, Game.
- Két terminál automatikus kiválasztása a rendszer által.
- A RoadNetwork.getShortestPath() helyes működése.
- Az útvonal hozzárendelése a buszhoz.
- A busz mozgása a kijelölt útvonal mentén.
- A terminál elérésének felismerése.
- A jutalom kiszámítása a megtett távolság alapján.
- Új útvonal automatikus generálása a teljesítés után.
- Várható hibahelyek: a rendszer nem tud útvonalat generálni két terminál között; a busz olyan sávra lép, amely nem része az útvonalnak; a terminál elérése után nem történik jutalomadás; az új forduló nem indul el automatikusan.

- **Bemenet:**

```
load test5_arrange.txt
auto_utvonal bus_1
mozgas bus_1 lane_t1_1
mozgas bus_1 lane_t1_2
mozgas bus_1 lane_t2
save test5_out.txt
exit
```

Elvárt kimenet:

```
[system] [selectedStart]: term_a
[system] [selectedDestination]: term_b
[bus_1] [currentRoute]: null -> route_1
[bus_1] [currentLane]: lane_start -> lane_t1_1
```

```
[bus_1] [currentLane]: lane_t1_1 -> lane_t1_2
[bus_1] [currentLane]: lane_t1_2 -> lane_t2
[bus_1] [location]: úton -> term_b
[busdriver_1] [money]: 100 -> 160
[log] Forduló teljesítve: bus_1, jutalom: 60
[bus_1] [currentRoute]: route_1 -> route_2
```

8.2.6

Teszteset 6: Busz közlekedtetése forduló teljesítéséhez

- **Leírás**

A teszteset azt ellenőrzi, hogy a játékos lépésenként képes-e a buszt a kijelölt útvonal mentén mozgatni, és a rendszer helyesen érzékeli-e a forduló befejezését. A teszt célja annak vizsgálata, hogy a busz csak az útvonalnak megfelelő következő sávra léphet, és a teljesítés után a rendszer pontot vagy jutalmat ad.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

- Érintett osztályok: Bus, Vehicle, Route, Lane, Road, BusDriverRole, Game.
- A busz kiválasztása és mozgatása játékos parancs alapján.
- Az útvonal-követési logika helyes működése.
- Az egyes lépések után a pozíció frissítése.
- A forduló teljesítésének felismerése.
- Jutalom vagy pont jóváírása.
- Várható hibahelyek: a busz rossz sávra is továbbléphet; a rendszer nem ellenőrzi, hogy a lépés része-e az útvonalnak; a forduló végén nem történik állapotfrissítés; a naplózás elmarad.

- **Bemenet:**

```
load test6_arrange.txt
kijelol bus_1
mozgas bus_1 lane_21
mozgas bus_1 lane_22
mozgas bus_1 lane_term
save test6_out.txt
exit
```

Elvárt kimenet:

```
[selectedVehicle] null -> bus_1
[bus_1] [currentLane]: lane_start -> lane_21
[bus_1] [currentLane]: lane_21 -> lane_22
[bus_1] [currentLane]: lane_22 -> lane_term
[bus_1] [location]: úton -> terminal_2
[busdriver_1] [completedRounds]: 0 -> 1
[busdriver_1] [money]: 80 -> 130
[log] Forduló befejezve: bus_1
```

8.2.7

Teszteset 7: Autók automatikus közlekedése lakhely és munkahely között

- **Leírás**

A teszteset azt ellenőrzi, hogy az autók a rendszer által automatikusan számított útvonalon elindulnak-e a lakhelyükről a munkahelyükre, és minden körben helyesen haladnak-e tovább. A teszt során a rendszernek figyelembe kell vennie az utak járhatóságát, és el kell juttatnia az autót a célsomópontig.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

- Érintett osztályok: Car, Vehicle, Residence, Workplace, Route, RoadNetwork, Lane, Game.
- Az autó létrehozása és kiindulási/cél csomópontjának kezelése.
- Az automatikus útvonaltervezés helyes működése.
- Az autó automatikus léptetése körönként.
- A következő sáv járhatóságának ellenőrzése.
- A célba érkezés rögzítése.
- Várható hibahelyek: az autó nem kap útvonalat; nem indul el automatikusan; nem ellenőrzi a következő sáv állapotát; célba éréskor nem frissül az állapot; a rendszer rossz célponthoz rendeli az autót.

- **Bemenet:**

```
load test7_arrange.txt
tick
tick
tick
save test7_out.txt
exit
```

Elvárt kimenet:

```
[car_1] [residence]: home_1
[car_1] [workplace]: work_1
[car_1] [currentRoute]: null -> route_car_1
[car_1] [currentLane]: lane_home -> lane_mid
[car_1] [currentLane]: lane_mid -> lane_work
[car_1] [location]: úton -> work_1
[log] car_1 megérkezett a munkahely
```

8.2.8

Teszteteset 8: Útvonal újratervezése akadály esetén

- **Leírás**

A szimuláció során egy autó (Car) a kijelölt útvonalán (Route) halad. A soron következő sáv (Lane) időjárási okokból járhatatlanná válik (pl. DeepSnow állapotba lép). Az autó a továbblépés előtt ellenőrzi a sáv járhatóságát. Mivel az akadályozott, a rendszer újratervezi az útvonalat a hálózat (RoadNetwork) aktuális állapotát figyelembe véve, majd az autó a sikeres újratervezés után az új, alternatív sávra lép.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

- **Érintett osztályok:** Car, Route, RoadNetwork, Lane, LaneState, DeepSnow, Game.
- **Útvonalkereső algoritmus tesztelése:** A RoadNetwork útvonalkereső metódusának (pl. getShortestPath()) ellenőrzése, hogy az érvénytelen (isPassable() == false) sávokat helyesen kizárja-e a gráfból, és megtalálja-e az alternatív utat.
- **Állapotfrissítés tesztelése:** A Car objektum currentRoute attribútumának cseréje az új útvonalra.
- **Várható hibahelyek:** Az útvonalkereső nem veszi figyelembe a sávok dinamikus állapotát (nem hívja meg az isPassable()-t), így visszatervezi a járművet az akadályba.
A jármű beragad az aktuális sávra, mert a kivételkezelés vagy az állapotgép megszakítja a mozgás (move()) logikáját.
Az új útvonal megkapása után a jármű nem a megfelelő (első alternatív) sávra lép át.

- **Bemenet**

```
load test8_arrange.txt
allapot_allit lane_next deepsnow
mozgas car_1
mozgas car_1
save test8_out.txt
exit
```

- **Elvárt kimenet**

```
[lane_next] [currentState]: Clear -> DeepSnow
[Console]: "Figyelmeztetés: A következő útszakasz járhatatlan. Útvonal
újratervezése..."
[car_1] [currentRoute]: route_original -> route_recalculated
[Console]: "Új útvonal sikeresen kijelölve."
[car_1] [currentLane]: lane_current -> lane_bypass_1
```

8.2.9

Teszteteset 9: Ütközések kezelése automatikus és játékos által vezérelt járművek között

- **Leírás**

A szimuláció során a rendszer folyamatosan ellenőrzi a járművek pozícióját és az általuk elfoglalt sávokat (Lane). Ez a tesztet két eltérő baleseti szituációt vizsgál:

- **A) Automatikus ütközés:** Két rendszer által vezérelt autó (Car) próbál ugyanarra a sávra lépni. A rendszer ezt detektálja, a sávot balesetesnek jelöli, és megállítja az érintett járműveket.
- **B) Játékos által okozott ütközés:** Egy játékos által irányított busz (Bus) hajt rá egy már foglalt sávra. A baleset detektálása mellett a rendszer egyedi büntetést is kiszab a felelős játékosra (BusDriverRole).
- **Ellenőrzött funkcionalitás, várható hibahelyek**
 - **Érintett osztályok:** Vehicle, Car, Bus, BusDriverRole, Lane, Road, Game.
 - **Ütközés detektálás tesztelése:** Annak ellenőrzése, hogy a rendszer (vagy maga a Lane objektum) helyesen felismeri-e, ha több jármű tartózkodik ugyanazon az útszakaszon, és megfelelően beállítja-e az ehhez tartozó állapotot (pl. a hasAccident attribútumot).
 - **Következmények érvényesítése:** A járművek mozgásának blokkolása (sebesség lenullázása), valamint a játékos pontszámának (score) csökkentése baleset okozása esetén.
 - **Várható hibahelyek:** A rendszer nem érzékeli az egybeeső pozíciókat, és a járművek "áthajtanak" egymáson (nincs ütközésvizsgálat a move() metódusban vagy a Lane elfoglalásakor). Az ütközés megtörténik, de a járművek tovább haladnak. A buszsofőr játékos profiljában nem hajtódik végre a pontlevonás a baleset okozása után.
- **Bemenet**

```
load test9_arrange.txt
// A) eset kiváltása: két autó ugyanarra a sávra lép
mozgas car_1 lane_conflict_1
mozgas car_2 lane_conflict_1
// B) eset kiváltása: a busz ráfut egy foglalt sávra
mozgas bus_1 lane_conflict_2
save test9_out.txt
exit
```
- **Elvárt kimenet**

```
[lane_conflict_1] [hasAccident]: false -> true
[Console]: "Ütközés történt két automatikus jármű között (car_1, car_2). Az útszakasz blokkolva."
[car_1] [speed]: 50.0 -> 0.0
[car_2] [speed]: 50.0 -> 0.0
[lane_conflict_2] [hasAccident]: false -> true
[Console]: "Baleset: A játékos által vezetett bus_1 ütközött. Büntetés kiszabva."
[bus_1] [speed]: 40.0 -> 0.0
[busdriver_1] [score]: 100 -> 50
```

8.2.10

Teszt eset 10: Autó és busz elakadása mély hóban

- **Leírás**
A szimuláció során a járművek (egy rendszer által vezérelt autó és egy játékos által vezetett busz) haladnak az útvonalukon. A következő útszakaszt azonban erős havazás éri, így annak állapota mély hóra (DeepSnow) változik. Amikor a járművek erre a sávra próbálnak lépni (vagy már rajta vannak), a rendszer ellenőrzi a járhatóságot. Mivel a mély hó akadályozza a haladást, a járművek elakadnak, sebességük nullára csökken. Az eseményt a rendszer naplózza. Miután egy hókotró (vagy az időjárás enyhülése) megtisztítja az útszakaszt (visszaállítva azt Clear állapotba), az elakadás megszűnik, és a járművek folytatják útjukat.
- **Ellenőrzött funkcionalitás, várható hibahelyek**
 - **Érintett osztályok:** Vehicle, Car, Bus, Lane, LaneState, DeepSnow, Clear, RoadNetwork, Weather, Game.
 - **Járhatatlanság felismerése:** Az isPassable() metódus helyes működése DeepSnow állapotban (elvárás: false), és ennek érvényesítése a járművek move() logikájában.
 - **Elakadás feloldása:** Az út állapotának megváltozásakor (pl. handleWeatherChange() vagy clean() hatására) a járművek ismételt elindulásának tesztelése.
 - **Várható hibahelyek:** * A járművek figyelmen kívül hagyják a sáv állapotát és zökkenőmentesen áthajtanak a mély havon.
Az elakadás megtörténik, de a járművek a hó eltakarítása után is "beragadva" maradnak, és nem kapják vissza az eredeti sebességüket/mozgási képességüket.
A Naplópanel nem kap értesítést az elakadás tényéről.
- **Bemenet**
load test10_arrange.txt
// A sávok állapota havazás miatt megváltozik
allapot_allit lane_snow_car deepsnow
allapot_allit lane_snow_bus deepsnow
// A járművek megpróbálnak mozogni
mozgas car_1 lane_snow_car
mozgas bus_1 lane_snow_bus
// Egy hókotró megtisztítja az elzárt utakat
takarit plow_1 lane_snow_car
takarit plow_1 lane_snow_bus
// A járművek újra próbálkoznak
mozgas car_1
mozgas bus_1
save test10_out.txt
exit
- **Elvárt kimenet**
[lane_snow_car] [currentState]: Clear -> DeepSnow
[lane_snow_bus] [currentState]: Clear -> DeepSnow

```
[Console]: "Az autó (car_1) elakadt a mély hóban."
[car_1] [speed]: 50.0 -> 0.0
[Console]: "A busz (bus_1) elakadt a mély hóban."
[bus_1] [speed]: 40.0 -> 0.0
[lane_snow_car] [currentState]: DeepSnow -> Clear
[lane_snow_bus] [currentState]: DeepSnow -> Clear
[Console]: "Az út letakarítva. Az autó (car_1) kiszabadult az elakadásból."
[car_1] [speed]: 0.0 -> 50.0
[car_1] [currentLane]: lane_snow_car -> lane_next_car
[Console]: "Az út letakarítva. A busz (bus_1) kiszabadult az elakadásból."
[bus_1] [speed]: 0.0 -> 40.0
[bus_1] [currentLane]: lane_snow_bus -> lane_next_bus
```

8.2.11

Teszt eset 11: Fej hatástalanná válása készlet hiányában

- **Leírás**

A szimuláció során a takarító játékos (CleanerRole) egy nyersanyagot igénylő hókotró fejjel (ebben a konkrét esetben SaltSpreaderHead) próbál megtisztítani egy jéggel borított útszakaszt (IceSheet). Mivel a hókotróhoz tartozó sókészlet (saltStock) nullán áll, a takarítási kísérlet hatástalan marad: a sáv állapota nem változik, és a rendszer figyelmeztetést küld a Naplópanelre. A probléma megoldására a játékos a Boltban (Store) vásárol újabb adag sót. A sikeres tranzakció után a készlet feltöltődik, és a megismételt takarítási parancs immár elvégzi a jég felolvasztását.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

- **Érintett osztályok:** CleanerRole, Snowplow, SaltSpreaderHead, Lane, LaneState, IceSheet, Clear, Store.
- **Készletellenőrzés tesztelése:** Annak ellenőrzése, hogy a SaltSpreaderHead (vagy a DragonHead, GravelSpreaderHead) clean() metódusa megszakítja-e a folyamatot vagy hatástalan marad-e, ha a releváns készlet (saltStock, biokeroseneStock stb.) értéke 0.
- **Kereskedelem és készletfrissítés tesztelése:** A Store és a Buyable interfészen keresztüli vásárlás (buy()) működése. Levonódik-e a pénz (money), és frissül-e a hókotró készlet-attribútuma.
- **Várható hibahelyek:**
 - A fej a 0 készlet ellenére is megváltoztatja a sáv állapotát (nincs feltételhez kötve az állapotváltás).
 - A készlet attribútum az első takarítás után negatív értékbe fordul (-1).
 - A bolti vásárlás során a pénz levonásra kerül, de a jármű készlete nem frissül, így a második takarítás is megghiúsul.

- **Bemenet**

```
load test11_arrange.txt
// 1. Kísérlet takarításra üres készlettel
takarit plow_1 lane_ice_1
// 2. Készletfeltöltés a boltból (játékos döntése a vásárlás mellett)
vasarol cleaner_1 salt
// 3. Újabb kísérlet a feltöltött készlettel
```

```
takarit plow_1 lane_ice_1
save test11_out.txt
exit
stat
```

- **Elvárt kimenet**

```
[Console]: "Figyelmeztetés: A takarítás sikertelen. A sósóró fej készlete kimerült!"
[lane_ice_1] [currentState]: IceSheet -> IceSheet
[cleaner_1] [money]: 200 -> 150
[plow_1] [saltStock]: 0 -> 10
[plow_1] [saltStock]: 10 -> 9
[lane_ice_1] [currentState]: IceSheet -> Clear
[Console]: "A sáv felsózva, a jég elolvadt, az út tiszta."
```

8.2.12

Tesztet 12: Játék kiértékelése

- **Leírás**

A játék végi állapot detektálásának ellenőrzése (a körlimit elérésekor). A pontszámítási logika helyességének tesztelése mind a buszsofőr, mind a takarító szerepkörök esetén. A végeredmény megfelelő formátumban történő, konzolos megjelenítésének ellenőrzése.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

- Érintett osztályok: Game, Player, Role, BusDriverRole, CleanerRole
- A rendszer felismeri a játék végét (currentRound >= maxRound feltétel teljesült) és a játék állapota befejezettre vált (isOver() igazat ad vissza).
- A rendszer minden játékosra lekéri a pontszámokat a getSumPoints() és a szerepkörökhöz tartozó getScore() metódusok segítségével.
- A rendszer összegzi az eredményeket, majd a konzolra írja őket.
- Várható hibahelyek: A rendszer nem nem összegzi a játékos különböző szerepköreiből szerzett pontszámát; helytelenül számol szerepkörhöz tartozó pontszámot; az eredmények felsorolásakor nem veszi figyelembe az összes játékost.

- **Bemenet**

```
load test12_arrange.txt
```

```
//várunk amíg a tick() növeli a kör számát
//enter leütésével gyorsítható
save test12_out.txt
exit
```

- **Elvárt kimenet**

```
[game] [currentRound]: 9 -> 10
[Console]: "Játék vége! Pontszámok: Játékos1: 120 pont, Játékos2: 85 pont."
```


8.2.13

Teszteset 13: Sáv állapotának lekérdezése

- **Leírás**
A szimuláció célja a kiválasztott Lane objektum attribútumainak pontos lekérdezése. A sáv áthaladhatóságának (isPassable()) és dinamikus súlyának (getDynamicWeight()) helyes kiszámítása az aktuális állapot alapján.
- **Ellenőrzött funkcionalitás, várható hibahelyek**
 - Érintett osztályok: RoadNetwork, Road, Lane, LaneState, IceSheet, Gravel, DeepSnow, ThinSnow, BrokenIce, Clear
 - Ellenőrzi, hogy a rendszer a RoadNetwork-ön keresztül megtalálja-e a kért azonosítójú sávot.
 - A rendszer helyesen lekéri a sáv fizikai attribútumait, a sáv aktuális állapotát és kiszámolja a hozzá tartozó dinamikus súlyt.
 - Várható hibahelyek: A rendszer nem találja meg a sávot; az adatok lekérdezése és/vagy megjelenítése hibás; a kiszámolt dinamikus súly nem felel meg az elvártnak.
- **Bemenet**
load test13_arrange.txt
inspect lane_1
save test13_out.txt
exit
- **Elvárt kimenet**
[Console]: "Sáv: L5; snowThickness=3; iceThickness=0; gravelThickness=0; isPassable=true; DynamicWeight=4"

8.2.14

Teszteset 14: Jeges út érdesítése zúzalékkal

- **Leírás**
A hókotró a zúzalékot szóró (GravelSpreaderHead) fejjel zúzalékot juttat a jégpáncélra, aminek hatására az úton járni lehet és a sáv Gravel állapotba kerül.
- **Ellenőrzött funkcionalitás, várható hibahelyek**
 - Érintett osztályok: CleanerRole, Snowplow, GravelSpreaderHead, Head, Vehicle, Lane, LaneState, IceSheet, Gravel.
 - A hókotró fejének cseréje GravelSpreaderHead típusra.
 - A CleanerRole.controlSnowplow → Snowplow.clean → GravelSpreaderHead.clean hívási lánc helyessége a zúzalék készlet csökkentésével.
 - Várható hibahelyek: az állapot nem Gravel-re változik; a change() nem kerül meghívásra; a készlet nem csökken; a fej a 0 készlet ellenére is megváltoztatja a sáv állapotát.
- **Bemenet**
load test14_arrange.txt
fej_csere plow_1 gravelspreader
mozgas plow_1 lane_ice2
takarit plow_1

```
save test14_out.txt
exit
```

- **Elvárt kimenet**

```
[plow_1] [currentHead]: SweeperHead -> GravelSpreaderHead
[plow_1] [currentLane]: lane_start -> lane_ice2
[plow_1] [gravelStock]: 10 -> 9
[lane_ice2] [currentState]: IceSheet -> Gravel
[lane_ice2] [gravelThickness]: 0.0 -> 1.0
[cleaner_1] [money]: 100 -> 150
```

8.2.15

Teszteset 15: Vékony hóréteg eltakarítása SweeperHead-del

- **Leírás**

A hókotró a söprő (SweeperHead) fejjel eltávolítja a vékony hóréteget (ThinSnow) a sávról. A takarítás eredményeként a sáv állapota Clear típusúra változik, az út ismét járható lesz.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

- Érintett osztályok: CleanerRole, Snowplow, SweeperHead, Head, Vehicle, Lane, LaneState, ThinSnow, Clear.
- A hókotró fejének dinamikus cseréje (changeHead) SweeperHead típusra.
- A polimorfizmus helyes működése: az állapot-átmenet ThinSnow → Clear.
- A CleanerRole.controlSnowplow → Snowplow.clean → ThrowerHead.clean hívási lánc helyessége.
- Várható hibahelyek: a fej cseréje után a régi fej maradna a Snowplow-ban; a Lane állapota nem változik.

- **Bemenet:**

```
load test15_arrange.txt
fej_csere plow_1 sweeper
mozgas plow_1 lane_5
takarit plow_1
save kimenet_15.txt
exit
```

Elvárt kimenet:

```
[plow_1] [currentHead]: ThrowerHead -> SweeperHead
[plow_1] [currentLane]: lane_1 -> lane_5
[lane_5] [currentState]: ThinSnow -> Clear
[lane_5] [snowThickness]: 3.0 -> 0.0
[cleaner_1] [money]: 100 -> 150
```

8.2.16

Teszteset 16: Vastag hóréteg eltakarítása ThrowerHead-del

- **Leírás**

A hókotró a hányó (ThrowerHead) fejjel eltávolítja a vastag, mély havat (DeepSnow) a sávról. A takarítás eredményeként a sáv állapota Clear típusúra változik, az út ismét járható lesz.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

- Érintett osztályok: CleanerRole, Snowplow, ThrowerHead, Head, Vehicle, Lane, LaneState, DeepSnow, Clear.
- A hókotró fejének dinamikus cseréje (changeHead) ThrowerHead típusra.
- A polimorfizmus helyes működése: az állapot-átmenet DeepSnow → Clear.
- A CleanerRole.controlSnowplow → Snowplow.clean → ThrowerHead.clean hívási lánc helyessége.
- Várható hibahelyek: a fej cseréje után a régi fej maradna a Snowplow-ban; a Lane állapota nem változik; az amount paraméter nem kerül átadásra a change() metódusnak.

- **Bemenet:**

```
load paja_config.txt
fej_csere plow_1 thrower
mozgas plow_1 lane_5
takarit plow_1
save kimenet_16.txt
exit
```

Elvárt kimenet:

```
[plow_1] [currentHead]: SweeperHead -> ThrowerHead
[plow_1] [currentLane]: lane_1 -> lane_5
[lane_5] [currentState]: DeepSnow -> Clear
[lane_5] [snowThickness]: 5.0 -> 0.0
[cleaner_1] [money]: 100 -> 150
```

8.2.17

Teszteset 17: Jégpáncél feltörése jégtörővel

- **Leírás**

A hókotró a jégtörő (IcebreakerHead) fejjel feltöri az összefüggő jégpáncélt (IceSheet) a sávon, így a jég tört jég (BrokenIce) állapotba kerül, ami javítja a tapadást.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

- Érintett osztályok: CleanerRole, Snowplow, IcebreakerHead, Head, Vehicle, Lane, LaneState, IceSheet, BrokenIce.
- A hókotró fejének cseréje IcebreakerHead típusra.
- A polimorfizmus tesztelése: az állapot-átmenet IceSheet → BrokenIce.
- Az IcebreakerHead nem hívja a Lane.change() metódust (csak az állapotot változtatja), eltérően a hó-eltávolító fejektől.
- Várható hibahelyek: rossz állapotra cserélné (pl. Clear-re BrokenIce helyett); a fej cseréje nem érvényesül.

- **Bemenet**

```
load test17_arrange.txt
fej_csere plow_1 icebreaker
mozgas plow_1 lane_ice
takarit plow_1
save test17_out.txt
exit
```

- **Elvárt kimenet**

```
[plow_1] [currentHead]: SweeperHead -> IcebreakerHead
[plow_1] [currentLane]: lane_start -> lane_ice
[lane_ice] [currentState]: IceSheet -> BrokenIce
```

8.2.18

Teszteset 18: Jég felolvasztása sószórással

- **Leírás**

A hókotró a sószóró (SaltSpreaderHead) fejjel sót juttat a jégpáncélra, aminek hatására a jég felolvad és a sáv Clear állapotba kerül.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

- Érintett osztályok: CleanerRole, Snowplow, SaltSpreaderHead, Head, Vehicle, Lane, LaneState, IceSheet, Clear.
- A hókotró fejének cseréje SaltSpreaderHead típusra.
- A polimorfizmus tesztelése: IceSheet → Clear átmenet, a Lane.change(amount) meghívása.
- Várható hibahelyek: az állapot nem Clear-re változik; a change() nem kerül meghívásra, így a jégvastagság nem csökken.

- **Bemenet**

```
load test18_arrange.txt
fej_csere plow_1 saltspreader
mozgas plow_1 lane_ice2
takarit plow_1
save test18_out.txt
exit
```

- **Elvárt kimenet**

```
[plow_1] [currentHead]: SweeperHead -> SaltSpreaderHead
[plow_1] [currentLane]: lane_start -> lane_ice2
[plow_1] [saltStock]: 10 -> 9
[lane_ice2] [currentState]: IceSheet -> Clear
[lane_ice2] [iceThickness]: 5.0 -> 0.0
[cleaner_1] [money]: 100 -> 150
```

8.2.19

Teszteset 19: Hó és jég leolvasztása sárkányfejjel

- **Leírás**

A hókotró a sárkány (DragonHead) fejjel intenzív hővel azonnal leolvasztja a havat és a jeget a sávról. A művelet biokerozint fogyaszt.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

- Érintett osztályok: CleanerRole, Snowplow, DragonHead, Head, Vehicle, Lane, LaneState, IceSheet / DeepSnow, Clear.
- A hókotró fejének cseréje DragonHead típusra.
- A polimorfizmus tesztelése: a sáv állapota akár DeepSnow, akár IceSheet volt, Clear állapotra változik.
- Várható hibahelyek: a DragonHead.clean() nem helyesen állítja be az állapotot; a Lane.change() hívása elmarad.

- **Bemenet**

```
load test19_arrange.txt
fej_csere plow_1 dragonhead
mozgas plow_1 lane_mixed
takarit plow_1
save test19_out.txt
```

exit

- **Elvárt kimenet**

[plow_1] [currentHead]: SweeperHead -> DragonHead

[plow_1] [currentLane]: lane_start -> lane_mixed

[plow_1] [biokeroseneStock]: 5 -> 4

[lane_mixed] [currentState]: DeepSnow -> Clear

[lane_mixed] [snowThickness]: 8.0 -> 0.0

[cleaner_1] [money]: 150 -> 200

8.3 A tesztelést támogató programok tervei

A prototípus tesztelése során az elvárt (assertált) és a tényleges (generált) kimenetek összehasonlítására nem készítünk külön, egyedi fejlesztésű szoftvert, hanem a beépített Windows parancssori eszközöket, egész pontosan az fc (File Compare) parancsot használjuk. Ennek segítségével a tesztek futtatása és kiértékelése egyszerűen és hatékonyan automatizálható egy parancsfájl (batch script) segítségével.

A tesztkiértékelés logikája és folyamata:

1. Kimenetek fájlba mentése: Minden egyes tesztesethez tartozik egy előre megírt, a helyes működést specifikáló szöveges fájl (pl. assert.txt). A teszt futtatása során a rendszer által generált konzolos kimenetet egy külön fájlba (pl. output.txt) irányítjuk át.
2. Összehasonlítás parancssorból: A teszt lefutása után a tesztelő script meghívja a fájl-összehasonlító parancsot a következő szintaxissal: fc assert.txt output.txt
3. Eredmény kiértékelése és konzolos megjelenítése: Az összehasonlításnak két lehetséges kimenetele van, amelyet a parancssori konzolra írunk ki:
 - Egyezés (**Sikeres**): Amennyiben a két fájl tartalma maradéktalanul megegyezik, a program a teszt sikeres lefutását jelző üzenetet ír ki a konzolra.
 - Eltérés (**Hiba**): Ha az fc parancs különbséget észlel, a script jelzi a hibát, majd a könnyebb hibakeresés érdekében a képernyőre (konzolra) íratja mind az elvárt, mind a tényleges fájl tartalmát.

8.4 Napló

Kezdet	Időtartam	Résztevők	Leírás
2026.04.15. 20:00	1,5 óra	Bocskai Jandó Dénes Tóth	Meeting: Heti feladat átbeszélése, és feladatok kiosztása
2026.04.17.	6 óra	Czap	Lane, Route, Road, RoadNetwork, Node osztályok és metódusainak tervei. Lane állapotgép. 16-19-es tesztesetek.
2026.04.17	5 óra	Tóth	Vehicle, Bus, Car, Snowplow, Head, DragonHead, SweeperHead, ThrowerHead, IcebreakerHead, SaltSpreaderHead, GravelSpreaderHead osztályok és metódusainak tervei. 4-7-es tesztesetek.
2026. 04. 17.	6 óra	Dénes	Osztályok és tesztelést támogató programok terveinek írása
2026. 04. 17.	5 óra	Bocskai	Player, Role, CleanerRole, BusDriverRole osztályok leírása és metódusainak tervei. 12-15 tesztesetek.
2026.04.18	5 óra	Jandó	Buyable, Store, LaneState, Brokenice, Clear, ThinSnow, IceSheet, DeepSnow osztályok és metódusainak tervei. 8-11-es tesztesetek.
2026. 04. 19	2 óra	Dénes	Teszt esetek írása
2026. 04. 19	2 óra	Bocskai Tóth	Osztályleírások összehasonlítása az UML diagramokkal és a Skeletonnal, ezek alapján javítások és javaslatok leírása

10. Prototípus beadása

23 – githappens

Konzulens:

Haragos Gergő Viktor

Csapattagok

Czap László	NS7V2T	czap.laszlo@edu.bme.hu
Bocskai Zsófia	UGSTW9	bocskaizsofi@gmail.com
Tóth Márton	K01YXA	toth.marton21@gmail.com
Dénes Péter István	PO6MVA	denespeter03tab@gmail.com
Jandó Barnabás Tamás	AWQ77G	jandobarnabas@gmail.com

2026.05.04.

10. Prototípus beadása

10.0 Javítások

10.0.1 Teszteset 2: Hókotró mozgása és út tisztítása

- **A javítás oka:**

A teszt kimenete nem illeszkedett a többi tesztéhez, így a konzisztencia és a helyes működés tükrözésének érdekében javítottuk..

- **Leírás** A teszt célja ellenőrizni a hókotró mozgási logikáját (szomszédos szakaszra lépés), valamint a takarítási funkció sikerességét, amely során a sáv állapota és járhatósága a kotrófej hatására megváltozik.
- **Ellenőrzött funkcionalitás**
 - Hókotró pozíciójának és állapotának frissítése
 - Sávok közötti szomszédossági kapcsolatok ellenőrzése
 - Takarítási algoritmus (hóvastagság csökkentése)
 - Sávállapot-átmenetek kezelése (pl. DeepSnow-> Clear)
 - Járhatósági flag (isPassable) automatikus kalkulációja
 - Üzemanyag-fogyasztás és pontszámítás
- **Várható hibahelyek**
 - Mozgás nem szomszédos vagy érvénytelen útszakaszra
 - Takarítás kísérlete mozgás közben vagy nem megfelelő állapotban
 - Hibás kotrófej-típus miatti eredménytelen takarítás
 - A sáv állapota nem frissül a hóvastagság nullázódása után
 - Erőforrás (üzemanyag) hiánya esetén is végrehajtott művelet
 - Negatív értékbe forduló hóvastagság vagy pontszám

- **Bemenet:**

```
load test_map.txt
mozgas snowplow_1 lane_12
takarit snowplow_1
exit
```

- **Elvárt kimenet:**

```
[snowplow_1] [currentLane]: Terminal_A -> lane_12
[lane_12] [currentState]: DeepSnow -> Clear
[lane_5] [snowThickness]: 5.0 -> 0.0
[player_cleaner] [money]: 100 -> 150
```

10.0.2 Teszteset 3: Kotrófej cseréje

- **A javítás oka:**

A felesleges state parancsot töröltük a bemenetből.

- **Leírás** A takarító játékos lecseréli a hókotró aktuális fejét egy másik típusra, hogy más útállapotot is kezelni tudjon.
- **Ellenőrzött funkcionalitás**

- Szöveges parancs (fej_csere) és argumentumainak sikeres értelmezése
- A megfelelő járműpéldány kikeresése az azonosító (snowplow_1) alapján
- A régi kotrófej objektum lecserélése (vagy típusának módosítása) az új fejtípusra (IcebreakerHead)
- Visszajelző üzenet generálása a konzolra
- **Várható hibahelyek**
 - Érvénytelen vagy elépelt kotrófej típus megadása
 - Nem létező vagy nem megfelelő típusú járműre (pl. Buszra) kiadott parancs
 - Fejcsere kísérlete nem megfelelő jármű-állapotban (pl. a kotró éppen CLEANING vagy MOVING állapotban van, és a szabályok szerint csak álló helyzetben szerelhető)

- **Bemenet:**

```
load test_map.txt
fej_csere snowplow_1 icebreaker
exit
```

- **Elvárt kimenet:**

```
[snowplow_1] [currentHead]: SweeperHead -> IcebreakerHead
```

10.0.3 Teszteset 13: Sáv állapotának lekérdezése teszt javítása

- **A javítás oka:**

A DynamicWeight meghatározása az osztályok implementálása közben változott.

- **Leírás**

A szimuláció célja a kiválasztott Lane objektum attribútumainak pontos lekérdezése. A sáv áthaladhatóságának (isPassable()) és dinamikus súlyának (getDynamicWeight()) helyes kiszámítása az aktuális állapot alapján.

- **Ellenőrzött funkcionalitás, várható hibahelyek**

- Érintett osztályok: RoadNetwork, Road, Lane, LaneState, IceSheet, Gravel, DeepSnow, ThinSnow, BrokenIce, Clear
- Ellenőrzi, hogy a rendszer a RoadNetwork-on keresztül megtalálja-e a kért azonosítójú sávot.
- A rendszer helyesen lekéri a sáv fizikai attribútumait, a sáv aktuális állapotát és kiszámolja a hozzá tartozó dinamikus súlyt.
- Várható hibahelyek: A rendszer nem találja meg a sávot; az adatok lekérdezése és/vagy megjelenítése hibás; a kiszámolt dinamikus súly nem felel meg az elvártaknak.

- **Bemenet**

```
load test13_arrange.txt
inspect lane_1
save test13_out.txt
exit
```

- **Elvárt kimenet**

```
[Console]: "Sáv: lane_1; snowThickness=3; iceThickness=0; gravelThickness=0;
isPassable=true; DynamicWeight=1.5"
```

10.1 Fordítási és futtatási útmutató

10.1.1 Fájllista

Fájl neve	Méret	Keletkezés ideje	Tartalom
Forrásfájlok (src/ mappa)			
Vehicle.java	2833	2026.05.01 00:15	Absztrakt jármű ösosztály (közös attribútumok és viselkedés)
Snowplow.java	2674	2026.05.02 22:41	Hókotró osztály — só/biokerozin/zúzottkő készlet, kotrófej kezelése
Bus.java	2470	2026.05.02 22:29	Busz osztály — végállomások közti közlekedés
Car.java	3247	2026.05.02 22:29	Autó osztály — lakó- és munkahely közti közlekedés
Head.java	388	2026.05.02 22:29	Absztrakt kotrófej ösosztály (Buyable interfészt implementálja)
SweeperHead.java	1031	2026.05.02 22:41	Söprőfej — vékony hó takarítása
GravelSpreaderHead.java	1255	2026.05.02 22:41	Zúzottkő szóró fej — letört jég kezelése
SaltSpreaderHead.java	1002	2026.05.02 22:41	Sósórozó fej — jégolvasztás
IcebreakerHead.java	1171	2026.05.02 22:41	Jégtörő fej — jégtábla letörése
DragonHead.java	1053	2026.05.02 22:41	Sárkányfej — vastag hó kezelése biokerozinnal
ThrowerHead.java	789	2026.05.02 22:41	Hódobó fej — havat máshová helyez át
Road.java	3541	2026.05.02 22:41	Absztrakt útszakasz ösosztály
NormalRoad.java	724	2026.05.02 22:41	Hagyományos útszakasz
Bridge.java	356	2026.05.02 22:41	Híd típusú útszakasz
Tunnel.java	357	2026.05.02 22:41	Alagút típusú útszakasz
Lane.java	7929	2026.05.02 22:41	Sáv osztály — hó/jég/zúzottkő vastagság, állapot, isPassable, getDynamicWeight
Intersection.java	1310	2026.05.02 22:29	Útkereszteződés csomópont
Route.java	2683	2026.05.02 22:29	Útvonal — sávok rendezett listája
RoadNetwork.java	3652	2026.05.02 22:41	Úthálózat — útszakaszok, sávok, csomópontok kezelése és lekérdezése
Node.java	900	2026.05.02 22:41	Absztrakt csomópont ösosztály
Residence.java	812	2026.05.02 22:29	Lakóhely csomópont

Workplace.java	841	2026.05.02 22:41	Munkahely csomópont
Terminal.java	808	2026.05.02 22:29	Buszvégállomás csomópont
LaneState.java	789	2026.04.30 19:17	Sáv-állapot interfész (State pattern)
Clear.java	1422	2026.04.30 19:17	Tiszta sáv állapot
ThinSnow.java	1178	2026.04.30 19:17	Vékony hó állapot
DeepSnow.java	1395	2026.04.30 19:17	Vastag hó állapot
IceSheet.java	1135	2026.04.30 19:17	Jégtábla állapot
BrokenIce.java	1287	2026.05.02 22:29	Letört jég állapot
Gravel.java	1773	2026.05.02 22:29	Felszórt zúzottkő állapot
Role.java	485	2026.04.30 19:17	Absztrakt szerepkör ösztály
BusdriverRole.java	5118	2026.05.02 22:29	Buszsofőr szerepkör — pénz, pontszám, körök
CleanerRole.java	4142	2026.05.02 22:41	Takarító szerepkör — hókotró vezérlése, pénz
Game.java	2929	2026.05.02 22:41	Játék-vezérlő — körök, járművek, játékosok
Player.java	1723	2026.04.30 19:17	Játékos osztály — id, név, szerepkörök
Weather.java	1592	2026.05.02 22:41	Időjárás — hóesés és olvadás logika
Store.java	2210	2026.05.02 22:41	Bolt — vásárolható kotrófejek és készletek
Buyable.java	347	2026.04.30 19:17	Vásárolható elemek interfésze (ár)
Tesztfájlok (tests/ mappa)			
TestCase.java	277	2026.05.02 22:41	Tesztosztályok közös interfésze (run() metódus)
MainRunner.java	1488	2026.05.02 22:41	Batch tesztfutató — reflexióval példányosítja a megadott testN-t
InteractiveRunner.java	4951	2026.05.02 22:41	Interaktív konzolos tesztfutató — load testN arrange.txt parancs
ArrangeContext.java	18843	2026.05.02 22:41	Arrange-nyelv értelmezője, kezdeti szcenárió felépítése
TestSupport.java	30518	2026.05.02 22:41	Közös segédosztály a parancs-dispatch-hez (fej_csere, mozgás, takarít, inspect, vasarol stb.)
TestContext.java	4211	2026.05.02 22:41	Megosztott teszt-állapot tároló (lanes, járművek, szerepkörök, útvonalak)
test1.java	2849	2026.05.03 20:24	Játék indítása — konfiguráció alapján a játéktér felépítése és a résztvevők inicializálása.
test2.java	1867	2026.05.03 20:24	Jármű sáv váltása és a sáv járhatóságának ellenőrzése (Clear vs. DeepSnow).

test3.java	891	2026.05.03 20:24	SweeperHead-del vékony hó takarítása, a sáv visszaállítása Clear állapotba.
test4.java	1307	2026.05.02 22:41	Bolt-vásárlás — bolt_nyit, item-választás, só vásárlása, pénz- és készlet-módosítás.
test5.java	2215	2026.05.02 22:41	Busz automatikus útvonal-kijelölése és teljesítése (terminál-érzékelés, money).
test6.java	1820	2026.05.02 22:41	Busz forduló teljesítése — kijelölés, mozgás, completedRounds++, money.
test7.java	1764	2026.05.02 22:41	Autó automatikus útvonal-haladása (tick alapján), munkahely-megérkezés.
test8.java	1751	2026.05.02 22:41	Útvonal újratervezése, ha akadály miatt a következő szakasz járhatatlan.
test9.java	2335	2026.05.02 22:41	Ütközések kezelése automatikus és játékos által vezérelt járművek között.
test10.java	2576	2026.05.02 22:41	Autó és busz mély hóban való elakadása, takarítás után kiszabadítás.
test11.java	1547	2026.05.02 22:41	Fej hatástalansága készlet hiányában — só nélkül takarít, vásárlás, újra takarít.
test12.java	2188	2026.05.02 22:41	Játék kiértékelése — körlimit elérése, pontszámítás (cleaner + busdriver).
test13.java	1420	2026.05.02 22:41	Sáv attribútumainak lekérdezése (isPassable, getDynamicWeight).
test14.java	1443	2026.05.02 22:41	Jeges út érdesítése GravelSpreaderHead-del (IceSheet → Gravel).
test15.java	1434	2026.05.02 22:41	SweeperHead-del vékony hó eltávolítása (ThinSnow → Clear).
test16.java	1468	2026.05.02 22:41	ThrowerHead-del vastag hóréteg eltávolítása (DeepSnow → Clear).
test17.java	1340	2026.05.02 21:45	IcebreakerHead-del jégpáncél feltörése (IceSheet → BrokenIce, nincs fizetés).
test18.java	1363	2026.05.02 21:45	SaltSpreaderHead-del jég felolvasztása (IceSheet → Clear, 1 só fogyasztása).
test19.java	1390	2026.05.02 21:45	DragonHead-del hó és jég leolvasztása (DeepSnow → Clear, 1 biokerozin fogyasztása).
Futtató szkriptek és tesztadatok			
run_tests.bat	1867	2026.05.01 00:15	Windows batch szkript — fordítás + összes teszt futtatása + assert összehasonlítás
run_tests.sh	2197	2026.05.02 22:41	Linux/Mac shell szkript — ugyanaz, mint a .bat (CRLF-toleráns diff-fel)
Tesztadatok — assert (test data/assert/)			

test1_assert.txt	291	2026.05.03 20:18	Játéktér inicializálási állapot ellenőrzése (RoadNetwork, Nodes, Lanes, Vehicles, Players, Weather, Game).
test2_assert.txt	169	2026.05.03 20:21	Sávváltás után snowplow állapotai, lane snowThickness/state/isPassable, player points.
test3_assert.txt	58	2026.05.03 20:21	Hókotró fej-csere — SweeperHead → IcebreakerHead.
test4_assert.txt	165	2026.04.30 19:03	Bolt nyitás, salt kiválasztása, money 100→70, saltStock 0→3, vásárlási üzenet.
test5_assert.txt	417	2026.04.30 19:03	Busz útvonal-haladás lépésről lépésre, terminálba érkezés, busdriver pénzjutalom.
test6_assert.txt	314	2026.04.30 19:03	Busz forduló — sávok közti haladás, terminál_2-be érkezés, completedRounds 0→1.
test7_assert.txt	268	2026.04.30 19:03	Autó residence/workplace/route, sávok közti haladás, work_1-be érkezés.
test8_assert.txt	304	2026.04.30 19:03	Sáv járhatatlanná válása, figyelmeztető Console üzenet, route újratervezése, bypass-sávra váltás.
test9_assert.txt	419	2026.04.30 19:03	Ütközés-érzékelés (hasAccident), sebesség 0-ra állása, balesetek logolása.
test10_assert.txt	694	2026.04.30 19:03	Autó és busz elakadása mély hóban, takarítás, kiszabadítás (DeepSnow → Clear).
test11_assert.txt	347	2026.04.30 19:03	Sószó üres készlettel — figyelmeztetés, vásárlás, sikeres takarítás (IceSheet → Clear).
test12_assert.txt	115	2026.05.02 11:46	Játék vége detektálás (currentRound 9→10), pontszámítás, Console üzenet.
test13_assert.txt	113	2026.05.02 11:46	Sáv lekérdezés Console kimenet (snowThickness, iceThickness, gravelThickness, isPassable, DynamicWeight).
test14_assert.txt	265	2026.05.02 11:46	GravelSpreaderHead — fej-csere, sávmozgás, gravelStock 10→9, IceSheet→Gravel, money +50.
test15_assert.txt	207	2026.04.30 19:03	SweeperHead — fej-csere ThrowerHead-ről, ThinSnow→Clear, snowThickness 3.0→0.0, money +50.
test16_assert.txt	203	2026.04.27 23:20	ThrowerHead — fej-csere, sávmozgás, DeepSnow→Clear, snowThickness 5.0→0.0, money +50.
test17_assert.txt	151	2026.04.28 18:36	IcebreakerHead — fej-csere, sávmozgás, IceSheet→BrokenIce (nincs pénzjutalom).
test18_assert.txt	255	2026.04.28 18:38	SaltSpreaderHead — fej-csere, saltStock 10→9, IceSheet→Clear, iceThickness 5.0→0.0, money +50.

test19_assert.txt	259	2026.04.28 18:39	DragonHead — fej-csere, biokeroseneStock 5→4, DeepSnow→Clear, snowThickness 8.0→0.0, money +50.
Tesztadatok — arrange szcenáriók (test data/arrange/)			
test1_arrange.txt	1223	2026.05.03 20:21	Test1 szcenárió — Game indítása
test2_arrange.txt	253	2026.05.03 20:21	Test2 szcenárió — Snowplow, CleanerRole, Lane felépítése.
test3_arrange.txt	188	2026.05.03 20:21	Test3 szcenárió — Snowplow, CleanerRole felépítése.
test4_arrange.txt	179	2026.05.02 22:41	Test4 szcenárió — Snowplow, CleanerRole, Lane, Store felépítése.
test5_arrange.txt	601	2026.05.02 22:41	Test5 szcenárió — Bus, BusdriverRole, terminálok, route_1 sávjainak felépítése.
test6_arrange.txt	382	2026.05.02 22:41	Test6 szcenárió — Bus, route, terminálok, completedRounds-számláló indítás.
test7_arrange.txt	495	2026.05.02 22:41	Test7 szcenárió — Car, Residence, Workplace, automatikus route felépítése.
test8_arrange.txt	452	2026.05.02 22:41	Test8 szcenárió — Car, route_original, Lane-ek (köztük az újratervezéshez bypass).
test9_arrange.txt	594	2026.05.02 22:41	Test9 szcenárió — több jármű (autók + játékos busz) ütközéshez beállítva.
test10_arrange.txt	740	2026.05.02 22:41	Test10 szcenárió — Car és Bus mély havas sávokon, CleanerRole + plow.
test11_arrange.txt	216	2026.05.02 22:41	Test11 szcenárió — Plow üres saltStock-kal, Store, IceSheet sáv.
test12_arrange.txt	594	2026.05.02 22:41	Test12 szcenárió — Game (currentRound=9, max=10), Player-ek, szerepkörök.
test13_arrange.txt	185	2026.05.02 22:41	Test13 szcenárió — egyetlen Lane (lane_1) attribútum-lekérdezéshez.
test14_arrange.txt	239	2026.05.02 22:41	Test14 szcenárió — Plow gravelStock=10, IceSheet sáv (lane_ice2), CleanerRole.
test15_arrange.txt	216	2026.05.02 22:41	Test15 szcenárió — Plow ThrowerHead-del, ThinSnow sáv (lane_5), CleanerRole.
test16_arrange.txt	215	2026.05.02 22:41	Test16 szcenárió — Plow SweeperHead-del, DeepSnow sáv (lane_5), CleanerRole.
test17_arrange.txt	236	2026.05.02 22:41	Test17 szcenárió — Plow SweeperHead-del, IceSheet sáv (lane_ice), CleanerRole.
test18_arrange.txt	242	2026.05.02 22:41	Test18 szcenárió — Plow saltStock=10, IceSheet sáv (lane_ice2), CleanerRole.
test19_arrange.txt	236	2026.05.02 22:41	Test19 szcenárió — Plow biokeroseneStock=5, DeepSnow sáv (lane_mixed), CleanerRole.
Tesztadatok — bemeneti parancsfájlok (test data/input/)			
test1_in.txt	37	2026.05.03 20:27	Test1 parancsai — állapot lekérése
test2_in.txt	77	2026.05.03 13:23	Test2 parancsai — hókotró sávra irányítása, takarítás
test3_in.txt	63	2026.05.03 13:23	Test3 parancsai — fejcsere.

test4_in.txt	75	2026.04.30 19:03	Test4 parancsai — bolt_nyit, salt kiválasztása, vasarol salt 3.
test5_in.txt	138	2026.04.30 19:03	Test5 parancsai — start/dest kiválasztása, set_vehicle_route, mozgás-sorozat.
test6_in.txt	131	2026.04.30 19:03	Test6 parancsai — vehicle kiválasztása, mozgás-sorozat a teljes körhöz.
test7_in.txt	66	2026.04.30 19:03	Test7 parancsai — car_1 tick-elése (több lépésben).
test8_in.txt	110	2026.04.30 19:03	Test8 parancsai — állapot_allit (akadály), majd mozgás (újratervezés).
test9_in.txt	138	2026.04.30 19:03	Test9 parancsai — mozgás-sorozat ütközés-előidézéshöz.
test10_in.txt	270	2026.04.30 19:03	Test10 parancsai — állapot_allit, mozgás (elakadás), takarit, mozgás (kiszabadítás).
test11_in.txt	134	2026.04.30 19:03	Test11 parancsai — takarit (üres készlettel), bolt_nyit, vasarol, takarit (sikeres).
test12_in.txt	52	2026.05.01 13:52	Test12 parancsai — utolsó kör befejezése, eredmény-kiértékelés.
test13_in.txt	66	2026.04.30 19:03	Test13 parancsai — inspect lane_1, save, exit.
test14_in.txt	124	2026.04.30 19:03	Test14 parancsai — fej_csere GravelSpreaderHead, mozgás, takarit.
test15_in.txt	114	2026.04.30 19:03	Test15 parancsai — fej_csere SweeperHead, mozgás, takarit.
test16_in.txt	106	2026.04.27 23:13	Test16 parancsai — fej_csere ThrowerHead, mozgás, takarit.
test17_in.txt	119	2026.04.28 18:37	Test17 parancsai — fej_csere IcebreakerHead, mozgás, takarit.
test18_in.txt	122	2026.04.28 18:38	Test18 parancsai — fej_csere SaltSpreaderHead, mozgás, takarit.
test19_in.txt	121	2026.04.28 18:39	Test19 parancsai — fej_csere DragonHead, mozgás, takarit.

10.1.2 Fordítás

A program JDK 17 (vagy újabb) környezetet igényel; a javac és java parancsoknak elérhetőeknek kell lenniük a PATH-on. További külső függőség, build-eszközre nincs szükség.

A bináris kód előállításához a prototype/ munkamappából a következő egyetlen javac parancs futtatandó:

```
javac -encoding UTF-8 -d bin src/*.java tests/*.java
```

Ez a src/ és tests/ mappákban található összes .java forrásfájlt UTF-8 kódolással lefordítja, a kimeneti .class fájlokat pedig a bin/ mappába helyezi. A bin/ mappa hiányában az automatikusan létrejön.

A fordítás kényelmesebb módon a mellékelt run_tests.bat (Windows) vagy run_tests.sh (Linux/Mac) szkript indításával is elvégezhető, ezek a fordítást és az összes teszt futtatását is egy lépésben elvégzik.

10.1.3 Futtatás

A lefordított program kétféle módon futtatható, mindkettő a prototype/ munkamappából, a bin/ osztályútvonal megadásával. UTF-8 kimenethez a -Dfile.encoding=UTF-8 -Dstdout.encoding=UTF-8 JVM-kapcsolók megadása ajánlott.

a) Batch (kötegelt) tesztfuttatás, összes teszt egyszerre:

A mellékelt szkript fordítást és az összes tesztet egy menetben lefuttatja, majd a kimenetet összehasonlítja a test_data/assert/ mappában tárolt elvárt eredményekkel:

Windows: run_tests.bat

Linux / Mac: ./run_tests.sh

Egyetlen konkrét tesztet egyenkénti, batch módú futtatása a MainRunner indító osztályon keresztül lehetséges (példa a 13. tesztre):

```
java -Dfile.encoding=UTF-8 -Dstdout.encoding=UTF-8 -cp bin tests.MainRunner test13
```

A MainRunner futás kimenete közvetlenül az assert fájlok formátumában jelenik meg a konzolon. Fájlbá átirányítva a Windows fc parancsával egy az egyben összehasonlítható az elvárt eredménnyel:

```
java -Dfile.encoding=UTF-8 -Dstdout.encoding=UTF-8 -cp bin tests.MainRunner test13 > test_data/output/test13_out.txt
```

```
fc test_data/output/test13_out.txt test_data/assert/test13_assert.txt
```

b) Interaktív konzolos futtatás — kézi parancsbevitel

Az InteractiveRunner indító osztály a 7. heti dokumentumban definiált konzolos parancsnyelvet implementálja. Indítás:

```
java -Dfile.encoding=UTF-8 -Dstdout.encoding=UTF-8 -cp bin tests.InteractiveRunner
```

Indítás után a programot soronként vezérelhetjük. A beírt parancsok megegyeznek a test_data/input/testN_in.txt fájlokban szereplőkkel. A munkamenet kötelezően egy load paranccsal kezdődik, amely a test_data/arrange/ mappából beolvassa a kívánt szcenárió-fájlt, és felépíti a kezdeti állapotot. Példa:

```
> load test13_arrange.txt
[load] test13_arrange.txt betöltve.
> inspect lane_1
> exit
```

Innentől a futás-idejű parancsok: fej_csere, mozgás, takarít, állapot_allit, inspect, bolt_nyit, vasarol az imént betöltött szcenárión hajtódnak végre. A munkamenet az exit paranccsal vagy Ctrl+Z / Ctrl+D billentyűkombinációval zárható le.

Kimenet összehasonlítása az elvárt eredménnyel

Kétféle módon történhet:

(a) Kézzel begépelve a parancsokat a kimenet a konzolon jelenik meg, ahol szemmel vethető össze a test_data/assert/testN_assert.txt tartalmával.

(b) A test_data/input/testN_in.txt bemenet és a kimenet operációs rendszer szintű átirányításával a kimenet fájlba menthető, majd a Windows beépített fc paranccsal vethető össze az assert fájljal:

```
java -Dfile.encoding=UTF-8 -Dstdout.encoding=UTF-8 -cp bin tests.InteractiveRunner < test_data/input/test13_in.txt > test_data/output/test13_out.txt
```

```
fc test_data/output/test13_out.txt test_data/assert/test13_assert.txt
```

Az interaktív futtatás kimenete tartalmaz pár plusz sort, amelyek az assert-ben nincsenek (indító fejléc, a > promptok, a [load] ... betöltve. üzenet és a Kilépés. sor) ezeket az fc különbséggként mutatja, szemmel kell ignorálni.

10.2 Tesztek jegyzőkönyvei

10.2.1 Teszteset1

Tesztelő neve	Dénes Péter István
Teszt időpontja	2026.05.03. 20:10

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.2 Teszteset2

Tesztelő neve	Dénes Péter István
Teszt időpontja	2026.05.03. 20:17

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.3 Teszteset3

Tesztelő neve	Dénes Péter István
Teszt időpontja	2026.05.03. 20:19

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.4 Teszteset4

Tesztelő neve	Tóth Márton
Teszt időpontja	2026.05.03. 11:20

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.5 Teszteset5

Tesztelő neve	Tóth Márton
Teszt időpontja	2026.05.03. 11:30

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.6 Teszteset6

Tesztelő neve	Tóth Márton
Teszt időpontja	2026.05.03. 11:40

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.7 Teszteset7

Tesztelő neve	Tóth Márton
Teszt időpontja	2026.05.03. 11:50

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.8 Teszteset8

Tesztelő neve	Jandó Barnabás
Teszt időpontja	2026.05.03 13:20

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.9 Teszteset9

Tesztelő neve	Jandó Barnabás
Teszt időpontja	2026.05.03 13:24

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-

Lehetséges hibaok	-
Változtatások	-

10.2.10 Teszteset10

Tesztelő neve	Jandó Barnabás
Teszt időpontja	2026.05.03 13:27

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.11 Teszteset11

Tesztelő neve	Jandó Barnabás
Teszt időpontja	2026.05.03 13:31

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.12 Teszteset12

Tesztelő neve	Bocskai Zsófia
Teszt időpontja	2026.05.01. 9:20

Tesztelő neve	Bocskai Zsófia
Teszt időpontja	2026.05.01. 9:05
Teszt eredménye	A játékos összeadott pont értékei nem egyeztek.
Lehetséges hibaok	A CleanerRole szerepkör pontszáma rosszul volt számolva.
Változtatások	A CleanerRole szerepkör pontszám számolásának javítása.

10.2.13 Teszteset13

Tesztelő neve	Bocskai Zsófia
Teszt időpontja	2026.05.01. 10:03

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.14 Teszteset14

Tesztelő neve	Bocskai Zsófia
Teszt időpontja	2026.05.01. 11:18

Tesztelő neve	Bocskai Zsófia
Teszt időpontja	2026.05.01. 11:09
Teszt eredménye	A takarítás során nem csökkent a zúzottkő készlet. kimenet: [plow_1] [currentHead]: SweeperHead -> GravelSpreaderHead [plow_1] [currentLane]: lane_start -> lane_ice2 [plow_1] [gravelStock]: 10 -> 9 [lane_ice2] [currentState]: IceSheet -> Gravel [cleaner_1] [money]: 100 -> 150
Lehetséges hibaok	A GravelSpreaderHead osztály clean metódusában nem csökkenti a zúzottkő számot.
Változtatások	A GravelSpreaderHead osztály clean metódusába a zúzottkő szám csökkentésének hozzáadása.

10.2.15 Teszteset15

Tesztelő neve	Bocskai Zsófia
Teszt időpontja	2026.05.01. 12:10

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.16 Teszteset16

Tesztelő neve	Czap László
Teszt időpontja	2026.05.02. 19:42

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.17 Teszteset17

Tesztelő neve	Czap László
Teszt időpontja	2026.05.02. 20:13

Tesztelő neve	Czap László
Teszt időpontja	2026.05.02. 20:05
Teszt eredménye	A jégfelület feltörése után a takarító indokolatlanul pénzjutalmat kapott. kimenet: [plow_1] [currentHead]: SweeperHead -> IcebreakerHead [plow_1] [currentLane]: lane_start -> lane_ice [lane_ice] [currentState]: IceSheet -> BrokenIce [cleaner_1] [money]: 100 -> 150

Lehetséges hibaok	A takarítás dispatch (doCleanWithMoney) feltétel nélkül adott pénzjutalmat, függetlenül attól, hogy a takarítás teljes volt-e. Az IcebreakerHead csak feltöri a jeget (IceSheet → BrokenIce), így ezért nem jár pénz.
Változtatások	A pénzjutalom kifizetése feltételhez kötve a CleanerRole.controlSnowplow metódusban: csak akkor fizet, ha az új sávállapot Clear.

10.2.18 Teszteset18

Tesztelő neve	Czap László
Teszt időpontja	2026.05.02. 20:38

Tesztelő neve	-
Teszt időpontja	-
Teszt eredménye	-
Lehetséges hibaok	-
Változtatások	-

10.2.19 Teszteset19

Tesztelő neve	Czap László
Teszt időpontja	2026.05.02. 21:07

Tesztelő neve	Czap László
Teszt időpontja	2026.05.02. 20:55
Teszt eredménye	A sárkányfej használata után a hóvastagság nem nullázódott teljesen, csak 1 egységgel csökkent. kimenet: [plow_1] [currentHead]: SweeperHead -> DragonHead [plow_1] [currentLane]: lane_start -> lane_mixed [plow_1] [biokeroseneStock]: 5 -> 4 [lane_mixed] [currentState]: DeepSnow -> Clear [lane_mixed] [snowThickness]: 8.0 -> 7.0 [cleaner_1] [money]: 150 -> 200
Lehetséges hibaok	A DragonHead.clean metódus a Lane.change(1) hívással csak 1 egységgel csökkentette a hóvastagságot, nem nullázta le teljesen.
Változtatások	A DragonHead.clean metódusban a hóvastagság teljes nullázásának kikényszerítése (a meglévő snowThickness értékének továbbadása a change hívásnak).

10.3Értékelés

Tag neve	Tag neptun	Munka százalékban
Czap László	NS7V2T	20%
Bocskai Zsófia	UGSTW9	20%
Tóth Márton	K01YXA	20%
Dénes Péter István	PO6MVA	20%
Jandó Barnabás Tamás	AWQ77G	20%

10.4 Napló

Kezdet	Időtartam	Résztevők	Leírás
2026.04.20. 21:00	2 óra	Bocskai Czap Jandó Tóth	Heti feladat felosztása, feladat átbeszélése
2026.04.23.	3 óra	Tóth	Járművek és hókotró fejek osztályainak megírása. Teszt 4-7 elkészítése
2026.04.23.	2 óra	Czap	Road, RoadNetwork, Node, Route, Lane osztályok megírása, Javadoc kommentek írása.
2026.04.24	3 óra	Dénes	Game, Weather osztály írása
2026.04.24.	2,5 óra	Jandó	LaneState, Clear, ThinSnow, DeepSnow, IceSheet, Brokenice, Buyable, Store osztályok megírása
2026.04.24.	3 óra	Czap	Tesztelést támogató struktúrák megírása.
2026.04.25.	2 óra	Bocskai	Player, Role, BusdriverRole, CleanerRole osztályok megírása, 12-15 tesztek megírása
2026.04.25.	2 óra	Czap	16-19 tesztek megírása.
2026.04.25.	2	Jandó	8-11 tesztek megírása
2026.04.27	2 óra	Dénes	tesztek írása, osztályok alakítása
2026.04.28.	2 óra	Tóth	Hiányos osztályok megírása, osztályokba a hibák kijavítása
2026.04.28.	1 óra	Tóth	Saját tesztek kijavítása, és formátum kijavítása
2026.04.29.	1 óra	Jandó	Saját tesztek javítása

2026.04.30	2 óra	Dénes	Tesztek javítása, getterek setterek
2026.05.01.	3 óra	Bocskai	12-15 tesztek javítása
2026.05.02.	2 óra	Bocskai	Osztályok kiegészítése kommentekkel
2026.05.03.	1 óra	Czap	Fájl lista kigyűjtése.

11. Grafikus változat tervei

23 – githappens

Konzulens:

Haragos Gergő Viktor

Csapattagok

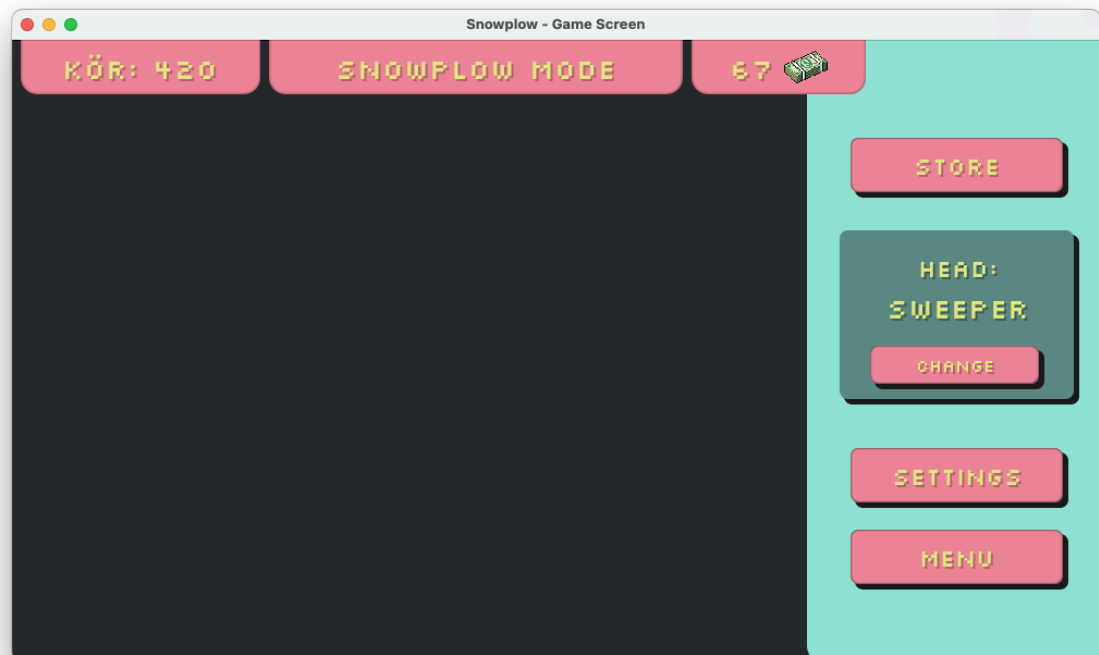
Czap László	NS7V2T	cz.laci04@gmail.com
Bocskai Zsófia	UGSTW9	bocskaizsofi@gmail.com
Tóth Márton	K01YXA	toth.marton21@gmail.com
Dénes Péter István	PO6MVA	denespeter03tab@gmail.com
Jandó Barnabás Tamás	AWQ77G	jandobarnabas@gmail.com

2026.05.11

Grafikus felület specifikációja

11.1 A grafikus interfész





11.2 A grafikus rendszer architektúrája

11.2.1A felület működési elve

A grafikus rendszer a Model–View–Controller (MVC) architektúrát követi. A modell (Model) réteget a játék logikai osztályai alkotják, például a Game, Player, Vehicle, RoadNetwork és Store. Ezek tárolják a játék aktuális állapotát és végzik a szimulációs logikát. A modell nem tartalmaz közvetlen grafikus függőségeket

A nézet (View) réteghez tartoznak a grafikus komponensek, például a GameScreen, SnowplowMenu, TopPill és StyledButton. Ezek feladata kizárólag a játék állapotának megjelenítése.

A vezérlő (Controller) réteg kezeli a felhasználói bemeneteket és összekapcsolja a modellt a nézettel. Ide tartozik például a GameController, KeyboardController, VehicleController és ScreenController. A kontrollerek dolgozzák fel a billentyűzet és egér eseményeit, majd meghívják a modell megfelelő metódusait.

A játék működését a GameLoop osztály vezérli, amely Swing Timer segítségével időszakosan meghívja a Game.tick()metódust. A modell frissítése után a GameController lekérdezi az aktuális állapotot, majd frissíti a grafikus felület elemeit.

A rendszer observer alapú frissítést is használ. A ModelObserver mechanizmus segítségével a grafikus komponensek automatikusan értesülnek a modell állapotváltozásairól, így a felület mindig a játék aktuális állapotát jeleníti meg.

A játék indításakor a Main osztály inicializálja a modellt, a grafikus felületet és a kontrollereket. A ScreenController kezeli a különböző képernyők – főmenü, játék, bolt és beállítások – közötti váltást.

11.2.2A felület osztály-struktúrája



A grafikus objektumok felsorolása

Megváltozott osztályok:

11.3.1 Game

- **Felelősség**

A szimuláció központi vezérlője (Modell). A MVC architektúra bevezetésével a felelőssége kibővült: folyamatosan nyilvántartja a valós idejű (tick-alapú) idő múlását, menedzseli a másodperceket és a 10 perces köröket. Ezen felül a megfelelő getter és segédmetódusokon keresztül transzparensen publikálja a belső állapotát (járművek helyzete, hálózat állapota, játékos adatai, idő) a GUI (GamePanel, InfoPanel) és a Controller számára, anélkül, hogy közvetlenül importálná a Swing elemeket.

- **Ősosztályok**

Nincs

- **Interfészek**

Nincs

- **Attribútumok**

- **ticksInCurrentRound**: Az aktuális körben eltelt tick-ek (szimulációs másodpercek) számát tárolja. Láthatósága: - (private), típusa: int
- **TICKS_PER_ROUND**: Konstans, amely megadja egy szimulációs kör hosszát tick-ekben (alapértelmezetten 600, azaz 10 perc). Láthatósága: - (private static final), típusa: int

- **Metódusok**

- **Bus getBus()**: Visszaadja a játékban szereplő busz referenciáját. Típusa: Bus
- **Snowplow getSnowplow()**: Visszaadja a hókotró referenciáját. Típusa: Snowplow
- **List<Car> getCars()**: Visszaadja a rendszerben lévő automatikus autók listáját. Típusa: List<Car>
- **RoadNetwork getRoadNetwork()**: Visszaadja a pályahálózat (csomópontok, utak) objektumát. Típusa: RoadNetwork
- **Player getPlayer()**: Visszaadja az emberi játékost reprezentáló objektumot. Típusa: Player
- **Store getStore()**: Visszaadja a bolt objektumát a StoreDialog kezeléséhez. Típusa: Store
- **String getFormattedTime()**: Segédmetódus az InfoPanel számára. Az aktuális körből eltelt tickeket (másodperceket) konvertálja és formázza egy "MM:SS" (perc:másodperc) alakú szöveggé. Típusa: String
- **String getCurrentControlledVehicleName()**: Lekérdezi a játékos aktuális szerepkörét, és visszaadja az általa épp vezetett jármű nevét, amit a GUI megjelenít. Típusa: String
- **void tick()**: A szimuláció egy lépését (1 másodperc) végrehajtó algoritmus. Meghívja a Weather hóesés metódusát, végigiterál és meghívja a tick()metódust az összes Vehicle objektumon, majd megnöveli a ticksInCurrentRound értékét. Ha az eléri a maximumot, lépteti a kört. Típusa: void

- **void checkRoleSwitch()**: Belső metódus, amely egy kör (10 perc / 600 tick) letelte után meghívódik, és kezdeményezi a játékos szerepkörének (Role) megváltoztatását az üzleti logika szerint. Típusa: void

11.3.2 Player

- **Felelősség**

A Player osztály a játékost reprezentálja a modellben. Feladata a játékos aktuális szerepkörének, pontszámának és pénzének nyilvántartása. A grafikus felület számára lekérdezhetővé teszi a játékos állapotát, például az aktuális szerepkört és az elért pontszámot.

- **Össztályok**

Nincs

- **Interfészek**

Nincs

- **Attribútumok**

- **currentRole**: A játékos éppen aktuális, aktív szerepköre (pl. CleanerRole vagy BusDriverRole). Láthatósága: - (private), típusa: Role
- **score**: A játékos általános pontszáma vagy vagyona. Láthatósága: - (private), típusa: int

- **Metódusok**

Role getCurrentRole(): Visszaadja a játékos jelenlegi aktív szerepkörét. Ezt használja az InfoPanel a "Szerep:" sor kitöltéséhez, illetve a HeadChangeDialog, hogy ellenőrizze a jogosultságot. Típusa: Role

void setCurrentRole(Role role): Beállítja a játékos új aktív szerepkörét (jellemzően a Game hívja meg körváltáskor). Típusa: void

int getScore(): Lekérdezi a játékos (vagy delegálva az aktív szerepkör) aktuális pontszámát/pénzét a GUI számára. Típusa: int

void addScore(int points): Növeli a játékos pontszámát a megadott értékkel (pl. sikeres forduló vagy takarítás után). Típusa: void

11.3.3 Vehicle

- **Felelősség**

A járművek eddig is tartalmaztak egy tick() metódust az UML alapján, de a MVC és a folyamatos, időzített Swing Timer bevezetésével ennek a metódusnak a felelőssége pontosításra került. A járműveknek saját magukat kell folyamatosan előre mozgatniuk a sávjukon az idő múlásának (tickeknek) megfelelően.

- **Össztályok**

Legősebb osztály: Vehicle -> Leszármazottai: Car, Bus, Snowplow

- **Interfészek**

Nincs

- **Attribútumok**

- Nincs új attribútum, a meglévő speed és positionOnLane kerül aktív felhasználásra a módosított algoritmusban.

- **Metódusok**

- **void tick():** A jármű folyamatos mozgásáért felelős algoritmus, amelyet a Game.tick() hív meg másodpercenként. A metóduson belül a jármű ellenőrzi a sáv (Lane) járhatóságát (isPassable()). Ha járható, a positionOnLane értékét megnöveli a speed értékével. Ha a positionOnLane eléri a sáv hosszát, a jármű meghívja a következő csomópont (Node) onVehicleEnter() metódusát. Típusa: void

Új osztályok:

11.3.4 GameScreen

- **Felelősség**

A játék grafikus felületének megjelenítése és kezelése. Az osztály a játék főablakát (JFrame) valósítja meg, amely tartalmazza a háttérpanelt, a felső információs sávot és a jobb oldali menüt. MVC környezetben a nézet (View) szerepét tölti be, biztosítva, hogy a játék aktuális állapota vizuálisan megjelenjen a felhasználó számára.

- **Ősosztályok**

JFrame

- **Interfészek**

Nincs

- **Attribútumok**

- **silkscreenTitle: Font** — A címekhez használt egyedi betűtípus.
- **silkscreenNormal: Font** — A normál szövegekhez használt betűtípus.
- **silkscreenSmall: Font** — A kisebb szövegekhez használt betűtípus.
- **TEXT_COLOR: Color** — a sárgás-zöld pixel szövegszín.
- **PINK_COLOR: Color** — a rózsaszín dizájnelem színe.
- **DARK_SHADOW: Color** — a sötét árnyék színe a 3D hatásokhoz.
- **roundTopPill: TopPill** — a körszámot megjelenítő TopPill.
- **modeTopPill: TopPill** — az aktuális játékmódot megjelenítő TopPill
- **moneyTopPill: TopPill** — a pénzüsszeget megjelenítő TopPill
- **infoBox: GrayInfoBox** — a jobb oldali információs doboz, amely az aktuális fejet mutatja.

- **Metódusok**

- **loadCustomFont(): void** — betölti a Silkscreen betűtípust a fájlrendszerből, vagy hiba esetén alapértelmezett SansSerif betűtípust állít be. Láthatósága: private.

- + **roundChanged(int round): void** — frissíti a megjelenített körszámot a paraméterben megkapott körszámmra. Láthatósága: public.
- + **roleChanged(): void** — frissíti a szerepkört és a megjelenített játékmódot. Láthatósága: public.
- + **moneyChanged(): void** — frissíti a megjelenített pénzösszeget. Láthatósága: public.
- + **headChanged(String head): void** — frissíti az aktuális fejet az információs dobozban. Láthatósága: public.

11.3.5 MainBgPanel

- **Felelősség**

A játék fő háttérének kirajzolása, amely két színű: sötétkék bal oldal (játéktér) és menta zöld jobb oldal (menüpanel).

- **Ősosztályok**

JPanel

- **Interfészek**

Nincs

- **Attribútumok**

Nincs

- **Metódusok**

- + **paintComponent(Graphics g): void** — kirajzolja a háttér két színű felületét, lekerekített jobb oldali panellel. Láthatósága: protected

11.3.6 TopPill

- **Felelősség**

A felső sáv rózsaszín kapszuláinak megjelenítése, amelyek szöveget és opcionális ikont tartalmaznak (körszám, mód, pénz).

- **Ősosztályok**

JPanel

- **Interfészek**

Nincs

- **Attribútumok**

- **text: String** — a megjelenítendő szöveg.
- **icon: Image** — opcionális ikon.

- **Metódusok**

- + **paintComponent(Graphics g): void** — kirajzolja a kapszula háttérét, árnyékát, szövegét és ikonját. Láthatósága: protected.

- + **setText(String newText): void** — frissíti a kapszula szövegét és újrarajzolja a komponenst. Láthatósága: public.

11.3.7 GrayInfoBox

- **Felelősség**

A jobb oldali menü információs doboza, amely megjeleníti az aktuális fejet és a fejcserét kezdeményező gombot.

- **Ősosztályok**

JPanel

- **Interfészek**

Nincs

- **Attribútumok**

- **shadowSize: int** — az árnyék mérete.
- **currentHeadLabel: JLabel** — az aktuális hókotró fejet megjelenítő címke.

- **Metódusok**

- + **paintComponent(Graphics g): void** — kirajzolja a doboz háttérét és árnyékát. Láthatósága: protected.
- + **setCurrentHeadLabel(String head): void** — frissíti az aktuális hókotró fejet a címkén. Láthatósága: public.
- **createShadowedLabel(String text, Font font): JLabel** — árnyékos szövegű címke létrehozása a megadott betűtípussal. Láthatósága: private.

11.3.8 StyledButton

- **Felelősség**

Egyedi, 3D árnyékos gomb, amelyet a menüben használnak (STORE, SETTINGS, MENU, CHANGE).

- **Ősosztályok**

JPanel

- **Interfészek**

Nincs

- **Attribútumok**

- **shadowSize: int** — az árnyék mérete.

- **Metódusok**

- + **paintComponent(Graphics g): void** — kirajzolja a gomb árnyékát, háttérét és szövegét, valamint kezeli a lenyomott állapot színváltozását.

11.3.9 SnowplowMenu

- **Felelősség**

A SnowplowMenu osztály a játék főmenüjét valósítja meg. Feladata a játék indításához, betöltéséhez, beállításaihoz és kilépéséhez szükséges kezelőfelület megjelenítése. A háttérben animált hóesést és pixel-art stílusú dizájnt alkalmaz, amely a játék hangulatát alapozza meg. MVC környezetben a nézet (View) szerepét tölti be, és a felhasználói interakciókat (gombnyomásokat) kezeli.

- **Ősosztályok**

JFrame

- **Interfészek**

Nincs

- **Attribútumok**

- **silkscreenFont: Font** — az egyedi Silkscreen betűtípus, amelyet a menü gombjai használnak.

- **Metódusok**

- **loadCustomFont(): void** — betölti a Silkscreen betűtípust a fájlrendszerből, vagy hiba esetén alapértelmezett SansSerif betűtípust állít be. Láthatósága: private.

11.3.10 BackgroundPanel

- **Felelősség**

A háttérkép és az animált hóesés megjelenítése. A panel folyamatosan frissíti a hópelyhek pozícióját, így dinamikus, pixel-art stílusú mozgást hoz létre.

- **Ősosztályok**

JPanel

- **Interfészek**

Nincs

- **Attribútumok**

- **backgroundImage: Image** — a háttérkép, amelyet a panel megjeleníti.
- **snowflakes: List<Snowflake>** — a hópelyhek listája, amelyek az animációt alkotják.
- **animationTimer: Timer** — az animáció időzítője, amely a hópelyhek mozgását frissíti.

- **Metódusok**

- + **paintComponent(Graphics g): void** — kirajzolja a háttérképet és a hópelyheket. Láthatósága: protected.

11.3.11 Snowflake

- **Felelősség**

Egyetlen hópehely pozíciójának, sebességének és mozgásának kezelése.

- **Ősosztályok**

Nincs

- **Interfészek**

Nincs

- **Attribútumok**

- **x, y: int** — a hópehely aktuális pozíciója.
- **size: int** — a hópehely mérete (2–4 pixel).
- **speed: int** — az esési sebesség.
- **swayOffset: int** — az oldalirányú kilengés mértéke.
- **swayCounter: int** — a szinuszgörbe-alapú mozgás számlálója.

- **Metódusok**

- + **update(int screenWidth, int screenHeight): void** — frissíti a hópehely pozícióját, és ha eléri a képernyő alját, újraindítja felülről. Láthatósága: public.

11.3.12 MenuOverlayPanel

- **Felelősség**

A középső, félig áttetsző, lekerekített sarkú panel, amely a menü gombjait tartalmazza.

- **Ősosztályok**

Nincs

- **Interfészek**

Nincs

- **Attribútumok**

Nincs

- **Metódusok**

- + **paintComponent(Graphics g): void** — kirajzolja a panel áttetsző, lekerekített hátterét (#748CAB színnel). Láthatósága: protected.

11.3.13 StorePanel

- **Felelősség**

A StorePanel osztály a játék bolt-felületét valósítja meg. Feladata a játékos számára elérhető vásárlási lehetőségek (anyagok, járművek, fejek) megjelenítése és kezelése. MVC környezetben a nézet (View) szerepét tölti be, amely a játék logikai komponenseitől

függetlenül vizuálisan reprezentálja a bolt működését. A panel frissíti a játékos pénzüsszegét és kezeli a vásárlási eseményeket.

- **Ősosztályok**

JPanel

- **Interfészek**

Nincs

- **Attribútumok**

- **silkscreenTitle: Font** — a címekhez használt egyedi betűtípus.
- **silkscreenHeader: Font** — a fejléc szövegekhez használt betűtípus.
- **silkscreenNormal: Font** — a normál szövegekhez használt betűtípus.
- **silkscreenSmall: Font** — a kisebb szövegekhez használt betűtípus.
- **moneyTopPill: TopPill** — a játékos pénzüsszegét megjelenítő felső kapszula.

- **Metódusok**

- + **loadCustomFont(): void** — betölti a Silkscreen betűtípust, vagy hiba esetén alapértelmezett SansSerif betűtípust állít be. Láthatósága: private
- + **paintComponent(Graphics g): void** — kirajzolja a panel áttetsző, lekerekített hátterét (#748CAB színnel). Láthatósága: protected.

11.3.14 StoreColumnPanel

- **Felelősség**

Egy bolt-oszlop megjelenítése (pl. MATERIAL, VEHICLE, HEAD), amely tartalmazza az elemek listáját és a vásárlás gombot.

- **Ősosztályok**

JPanel

- **Interfészek**

Nincs

- **Attribútumok**

- **itemsContainer: JPanel** — az oszlopban található elemek tárolója.
- **buyButton: StyledButton** — a vásárlás gomb.

- **Metódusok**

- + **addItemRow(String itemName): void** — hozzáad egy új elemet az oszlophoz. Láthatósága: public
- + **getBuyButton(): StyledButton** — visszaadja a vásárlás gombot. Láthatósága: public
- + **getItemsContainer(): JPanel** — visszaadja az elemek tárolópaneljét. Láthatósága: public.
- + **paintComponent(Graphics g): void** — kirajzolja az oszlop áttetsző, lekerekített hátterét (#748CAB színnel). Láthatósága: protected.

11.3.15 ItemRow

- **Felelősség**

Egy boltban megjelenő sor, amely tartalmazza az elem nevét és a mennyiségválasztó léptetőt (JSpinner).

- **Ősosztályok**

JPanel

- **Interfészek**

Nincs

- **Attribútumok**

- **name: String** — az elem neve.
- **spinner: JSpinner** — a mennyiségválasztó komponens.

- **Metódusok**

- + **getAmount(): int** — visszaadja a kiválasztott mennyiséget.
- + **getItemName(): String** — visszaadja az elem nevét.

11.3.16 GameController

- **Felelősség**

A GameController osztály a grafikus játék vezérlőrétegének központi eleme. Feladata a modell (Game) és a grafikus nézet (GameScreen) összekapcsolása, a játék indítása, szüneteltetése, folytatása, valamint a felhasználói műveletek továbbítása a megfelelő alrendszerek felé. MVC környezetben Controller szerepet tölt be: a View-ből érkező eseményeket feldolgozza, meghívja a modell megfelelő metódusait, majd gondoskodik a felület frissítéséről.

- **Ősosztályok**

Nincs

- **Interfészek**

Nincs

- **Attribútumok**

- **game: Game** — A játék modellobjektuma, amely a szimuláció aktuális állapotát tárolja.
- **gameScreen: GameScreen** — A játék fő grafikus ablaka, amelyen a változások megjelennek.
- **gameLoop: GameLoop** — Az időzített játékfrissítést végző objektum.
- **keyboardController: KeyboardController** — A billentyűzetes irányítást kezelő objektum.
- **running: boolean** — Jelzi, hogy a játék jelenleg fut-e.

- **Metódusok**

- + **GameController(Game game, GameScreen gameScreen):** Konstruktor, amely eltárolja a modell és a nézet referenciáját, létrehozza a játékciklust és a billentyűzetkezelőt.
- + **startGame(): void** — Elindítja a játékot, aktiválja a játékciklust, és futó állapotba állítja a kontrollert.
- + **pauseGame(): void** — Megállítja a játékciklust, de a játék aktuális állapotát megtartja.
- + **resumeGame(): void** — Folytatja a korábban szüneteltetett játékot.
- + **stopGame(): void** — Leállítja a játékot, megszünteti a ciklus futását, és visszaállítja a futási állapotot.
- + **tick(): void** — Meghívja a Game.tick() metódust, majd frissíti a grafikus felületet.
- **refreshView(): void** — A modell aktuális állapota alapján frissíti a GameScreen megjelenített adatait, például a kört, játékmódot, pénzt és aktuális fejet.

11.3.17 GameLoop

- **Felelősség**

A GameLoop osztály a játék időzített futtatásáért felelős. Swing környezetben Timer segítségével rendszeres időközönként meghívja a GameController.tick() metódust. Így biztosítja, hogy a játék állapota folyamatosan frissüljön, a járművek mozogjanak, az idő haladjon, és a grafikus felület a modell aktuális állapotát jelenítse meg.

- **Össztályok**

Nincs

- **Interfészek**

ActionListener

- **Attribútumok**

- **timer: Timer** — A Swing időzítő, amely meghatározott időközönként eseményt generál.
- **controller: GameController** — A vezérlő objektum, amelynek a tick() metódusát a ciklus meghívja.
- **delay: int** — Az időzítő késleltetése milliszekundumban.

- **Metódusok**

- + **GameLoop(GameController controller, int delay):** Konstruktor, amely beállítja a vezérlőt és az időzítés gyakoriságát.
- + **start(): void** — Elindítja a Swing időzítőt.
- + **stop(): void** — Leállítja a Swing időzítőt.
- + **isRunning(): boolean** — Visszaadja, hogy az időzítő jelenleg aktív-e.
- + **actionPerformed(ActionEvent e): void** — Az időzítő eseményének kezelése; meghívja a controller.tick() metódust.

11.3.18 KeyboardController

- **Felelősség**

A KeyboardController osztály a játék billentyűzetes irányítását kezeli. Feladata a lenyomott billentyűk érzékelése, azok értelmezése, majd a megfelelő játékbeli művelet továbbítása a GameController vagy a modell felé. A játékos ezzel az osztállyal tudja irányítani a buszt vagy a hókotrót, illetve billentyűparancsokkal megnyitni a boltot, a beállításokat vagy a menüt.

- **Össztályok**

Nincs

- **Interfészek**

KeyListener

- **Attribútumok**

- **controller: GameController** — A központi játékvezérlő, amely felé a billentyűzetes műveletek továbbításra kerülnek.
- **game: Game** — A játékmodell, amelyből lekérdezhető az aktuális játékos, szerepkör és jármű.
- **pressedKeys: Set<Integer>** — A jelenleg lenyomott billentyűk kódjait tárolja.
- **enabled: boolean** — Jelzi, hogy a billentyűzetes irányítás aktív-e.

- **Metódusok**

- + **KeyboardController(GameController controller, Game game):** Konstruktor, amely beállítja a vezérlőt és a modellt.
- + **keyPressed(KeyEvent e): void** — Lementi a lenyomott billentyű kódját, majd meghívja a billentyűhöz tartozó műveletet.
- + **keyReleased(KeyEvent e): void** — Eltávolítja a billentyű kódját a lenyomott billentyűk halmazából.
- + **keyTyped(KeyEvent e): void** — Karakteralapú billentyűesemények kezelése; jelen projektben nem végez külön műveletet.
- + **isKeyPressed(int keyCode): boolean** — Megadja, hogy az adott billentyű jelenleg le van-e nyomva.
- + **setEnabled(boolean enabled): void** — Engedélyezi vagy letiltja a billentyűzetes vezérlést.
- **handleMovementKey(int keyCode): void** — A mozgáshoz tartozó billentyűket dolgozza fel.
- **handleActionKey(int keyCode): void** — Az egyéb funkcióbillentyűket kezeli, például bolt, menü vagy szünet.

11.3.19 VehicleController

- **Felelősség**

A VehicleController osztály a járművek irányításáért felelős. Feladata eldönteni, hogy az aktuális játékos milyen szerepkörben van, melyik járművet vezérli, és a billentyűzetes input alapján milyen mozgási vagy műveleti parancsot kell végrehajtani. A Controller nem tárolja önállóan a járművek állapotát, hanem a modell megfelelő objektumain hív meg műveleteket.

- **Össztályok**

Nincs

- **Interfészek**

Nincs

- **Attribútumok**

- **game: Game** — A játékmodell referenciája.
- **controlledVehicle: Vehicle** — Az aktuálisan irányított jármű referenciája.
- **lastDirection: String** — Az utoljára kiadott irányítási parancs neve.

- **Metódusok**

- + **VehicleController(Game game):** Konstruktor, amely eltárolja a játékmodellt.
- + **updateControlledVehicle(): void** — Lekérdezi a játékostól az aktuális szerepkört, és beállítja az irányított járművet.
- + **moveUp(): void** — Felfelé vagy előre irányú mozgási parancsot ad az aktuális járműnek.
- + **moveDown(): void** — Lefelé vagy hátra irányú mozgási parancsot ad az aktuális járműnek.
- + **moveLeft(): void** — Balra irányú mozgási parancsot ad az aktuális járműnek.
- + **moveRight(): void** — Jobbra irányú mozgási parancsot ad az aktuális járműnek.
- + **stopMovement(): void** — Megállítja vagy semleges állapotba helyezi az aktuálisan vezérelt járművet.
- **hasControlledVehicle(): boolean** — Ellenőrzi, hogy van-e jelenleg irányítható jármű.

11.3.20 ScreenController

- **Felelősség**

A ScreenController osztály a különböző grafikus képernyők közötti váltást kezeli. Feladata a főmenü, a játékképernyő, a bolt és a beállítások megjelenítésének vezérlése. Ezzel elkülönül a képernyőváltási logika a konkrét grafikus komponensektől, így a View osztályok egyszerűbbek és átláthatóbbak maradnak.

- **Össztályok**

Nincs

- **Interfészek**

Nincs

- **Attribútumok**

- **menuScreen: SnowplowMenu** — A főmenü grafikus ablaka.
- **gameScreen: GameScreen** — A játék fő grafikus ablaka.
- **settingsScreen: SettingsScreen** — A játék beállítások grafikus ablaka.
- **gamePanel: GamePanel** — A játéktér kirajzolásáért felelős panel.
- **storeScreen: StoreScreen** — A játék boltot megjelenítő ablaka.
- **currentScreen: JFrame** — Az éppen aktív képernyő referenciája.

- **gameController: GameController** — A játék vezérlője, amely a képernyőváltások során indítható vagy leállítható.

- **Metódusok**

- + **ScreenController(SnowplowMenu menuScreen, GameScreen gameScreen, GameController gameController)**: Konstruktor, amely eltárolja a kezelendő képernyőket.
- + **showMenu(): void** — Megjeleníti a főmenüt, és szükség esetén elrejt a játékképernyőt.
- + **showGame(): void** — Megjeleníti a játéklapot, elrejt a menüt, és elindítja a játékot.
- + **showSettings(): void** — Megnyitja a beállítások felületét.
- + **showStore(): void** — Megnyitja a bolt felületét.
- + **exitApplication(): void** — Bezárja az alkalmazást.

11.3.21 InputAction

- **Felelősség**

Az InputAction felsorolás a játékban értelmezhető bemeneti parancsokat egységesíti. Segítségével a billentyűzetből érkező nyers billentyűkódok nem közvetlenül a modellhez jutnak el, hanem először játékbeli parancsokká alakulnak. Ez átláthatóbbá teszi az inputkezelést, és később lehetővé teszi más vezérlési módok bevezetését is.

- **Őszosztályok**

Enum

- **Interfészek**

Nincs

- **Attribútumok**

Nincs

- **Értékek**

- MOVE_UP** — Felfelé vagy előre mozgás.
- MOVE_DOWN** — Lefelé vagy hátra mozgás.
- MOVE_LEFT** — Balra mozgás.
- MOVE_RIGHT** — Jobbra mozgás.
- STOP** — Mozgás leállítása.
- OPEN_STORE** — Bolt megnyitása.
- OPEN_SETTINGS** — Beállítások megnyitása.
- OPEN_MENU** — Menü megnyitása.
- PAUSE** — Játék szüneteltetése.
- CONFIRM** — Kiválasztás vagy megerősítés.
- CANCEL** — Művelet megszakítása.

- **Metódusok**

Nincs külön metódusa

11.3.22 InputMapper

- **Felelősség**

Az InputMapper osztály a billentyűkódok és a játékbeli parancsok összerendelését végzi. Feladata, hogy a KeyEventobjektumokból érkező billentyűkódokat InputAction értékekké alakítsa. Ez azért hasznos, mert az irányítás kiosztása egy helyen módosítható, és a KeyboardController nem tartalmaz közvetlenül sok feltételes billentyűvizsgálatot.

- **Ősosztályok**

Nincs

- **Interfészek**

Nincs

- **Attribútumok**

- **keyBindings: Map<Integer, InputAction>** — A billentyűkódok és játékbeli műveletek összerendelését tárolja.

- **Metódusok**

+ **InputMapper():** Konstruktor, amely létrehozza az alapértelmezett billentyűkiosztást.
+ **getAction(int keyCode): InputAction** — Visszaadja az adott billentyűkódhoz tartozó játékbeli műveletet.
+ **setBinding(int keyCode, InputAction action): void** — Új billentyűkiosztást állít be egy adott művelethez.
+ **removeBinding(int keyCode): void** — Törli az adott billentyűkódhoz tartozó kiosztást.
- **loadDefaultBindings(): void** — Beállítja az alapértelmezett billentyűket, például WASD vagy nyilak használatát.

11.3.23 ModelObserver

- **Felelősség**

A ModelObserver interfész a modell és a grafikus nézet közötti kommunikációt biztosítja. Feladata, hogy a modell állapotváltozásairól értesítse a regisztrált grafikus komponenseket, így a felület automatikusan frissülhet.

- **Ősosztályok**

Nincs

- **Interfészek**

Nincs

- **Attribútumok**

Nincs

- **Metódusok**

+ **modelChanged(): void** — Értesíti a megfigyelőt, hogy a modell állapota megváltozott.

11.3.24 ObservableModel

- **Felelősség**

Az ObservableModel osztály a megfigyelő mintát valósítja meg. Feladata a ModelObserver objektumok regisztrálása és értesítése.

- **Össztályok**

Nincs

- **Interfészek**

Nincs

- **Attribútumok**

- **observers: List<ModelObserver>** — A regisztrált megfigyelők listája.

- **Metódusok**

+ **addObserver(ModelObserver observer): void** — Új megfigyelő regisztrálása.

+ **removeObserver(ModelObserver observer): void** — Megfigyelő eltávolítása.

+ **notifyObservers(): void** — Az összes megfigyelő értesítése.

11.3.25 GamePanel

- **Felelősség**

A GamePanel osztály a játéktér kirajzolásáért felelős.

- **Össztályok**

JPanel

- **Interfészek**

ModelObserver

- **Attribútumok**

- **game: Game** — A játékmodell referenciája.
- **vehicleRenderer: VehicleRenderer** — A járművek renderelését végzi.
- **roadRenderer: RoadRenderer** — Az utak kirajzolását végzi.
- **weatherRenderer: WeatherRenderer** — Az időjárási effekteket rajzolja ki.

- **Metódusok**

- + **paintComponent(Graphics g): void** — Kirajzolja a játéktér elemeit.
- + **modelChanged(): void** — Frissíti a panel megjelenítését.
- **renderMap(Graphics2D g2d): void** — Kirajzolja a pályát.
- **renderVehicles(Graphics2D g2d): void** — Kirajzolja a járműveket.
- **renderWeather(Graphics2D g2d): void** — Kirajzolja az időjárási effekteket.

11.3.26 VehicleRenderer

- **Felelősség**

A VehicleRenderer osztály a járművek grafikus megjelenítéséért felelős.

- **Össztályok**

Nincs.

- **Interfészek**

Nincs.

- **Attribútumok**

- **textures: Map<String, Image>** — A jármű textúrákat tárolja.

- **Metódusok**

- + **renderVehicle(Graphics2D g2d, Vehicle vehicle): void** — Kirajzol egy járművet.
- **getVehicleTexture(Vehicle vehicle): Image** — Betölti a megfelelő textúrát.

11.3.27 RoadRenderer

- **Felelősség**

A RoadRenderer osztály az utak és sávok kirajzolását végzi.

- **Össztályok**

Nincs.

- **Interfészek**

Nincs.

- **Attribútumok**

- **roadColor: Color** — Az utak színe.
- **laneColor: Color** — A sávok színe.

- **Metódusok**

- + **renderRoads(Graphics2D g2d, RoadNetwork network): void** — Kirajzolja az úthálózatot.
- + **renderLane(Graphics2D g2d, Lane lane): void** — Kirajzol egy sávot.

11.3.28 WeatherRenderer

- **Felelősség**

A WeatherRenderer osztály az időjárási effektek megjelenítéséért felelős.

- **Ősosztályok**

Nincs.

- **Interfészek**

Nincs.

- **Attribútumok**

- **snowflakes: List<Snowflake>** — A hópelyhek listája.

- **Metódusok**

- + **renderSnow(Graphics2D g2d): void** — Kirajzolja a hóesést.
- + **updateWeather(): void** — Frissíti a hópelyhek állapotát.

11.3.29 MouseController

- **Felelősség**

A MouseController osztály az egérműveletek kezeléséért felelős.

- **Ősosztályok**

Nincs.

- **Interfészek**

MouseListener, MouseMotionListener

- **Attribútumok**

- **controller: GameController** — A játék vezérlője.
- **hoveredComponent: Component** — Az aktuálisan kijelölt komponens.

- **Metódusok**

- + **mouseClicked(MouseEvent e): void** — Kattintás kezelése.
- + **mousePressed(MouseEvent e): void** — Egérgomb lenyomása.
- + **mouseReleased(MouseEvent e): void** — Egérgomb felengedése.
- + **mouseMoved(MouseEvent e): void** — Egérmozgás kezelése.
- + **mouseDragged(MouseEvent e): void** — Húzás kezelése.

11.3.30 StoreScreen

- **Felelősség**

A StoreScreen osztály a bolt grafikus felületét jeleníti meg.

- **Ősosztályok**

JFrame

- **Interfészek**

Nincs.

- **Attribútumok**

- **storePanel: StorePanel** — A bolt tartalmi panelje.
- **playerMoneyLabel: JLabel** — A játékos pénzét mutatja.

- **Metódusok**

- + **refreshStore(): void** — Frissíti a bolt tartalmát.
- + **updateMoney(int amount): void** — Frissíti a pénz kijelzését.

11.3.31 StoreController

- **Felelősség**

A StoreController a vásárlási logikát kezeli.

- **Ősosztályok**

Nincs.

- **Interfészek**

Nincs.

- **Attribútumok**

- **store: Store** — A bolt objektuma.
- **player: Player** — A játékos objektuma.

- **Metódusok**

- + **buyItem(String itemId): boolean** — Megvásárol egy elemet.
- + **canAfford(int price): boolean** — Ellenőrzi a pénzt.

11.3.32 SettingsScreen

- **Felelősség**

A SettingsScreen osztály a játék beállításait kezeli.

- **Össztályok**

JFrame

- **Interfészek**

Nincs.

- **Attribútumok**

+ **canAfford(int price): boolean**

11.3.33 AssetManager

- **Felelősség**

Az AssetManager kezeli a képeket, fontokat és egyéb erőforrásokat.

- **Össztályok**

Nincs.

- **Interfészek**

Nincs.

- **Attribútumok**

- **imageCache: Map<String, Image>** — A képek gyorsítótára.

- **fontCache: Map<String, Font>** — A fontok gyorsítótára.

- **Metódusok**

+ **getImage(String path): Image** — Betölt egy képet.

+ **getFont(String path): Font** — Betölt egy fontot.

+ **preloadAssets(): void** — Előre betölti az erőforrásokat.

11.3.34 SoundManager

- **Felelősség**

A SoundManager a hangok és zenék kezeléséért felelős.

- **Össztályok**

Nincs.

- **Interfészek**

Nincs.

- **Attribútumok**

- **backgroundMusic: Clip** — A háttérzene.

- **soundEffects: Map<String, Clip>** — A hangeffektek tárolója.

- **Metódusok**

- + **playMusic(String id): void** — Elindít egy zenét.
- + **stopMusic(): void** — Leállítja a zenét.
- + **playEffect(String id): void** — Lejátszik egy effektet.

11.3.35 Main

- **Felelősség**

A Main osztály az alkalmazás belépési pontja.

- **Össztályok**

Nincs.

- **Interfészek**

Nincs.

- **Attribútumok**

Nincs.

- **Metódusok**

- + **main(String[] args): void** — Elindítja az alkalmazást.

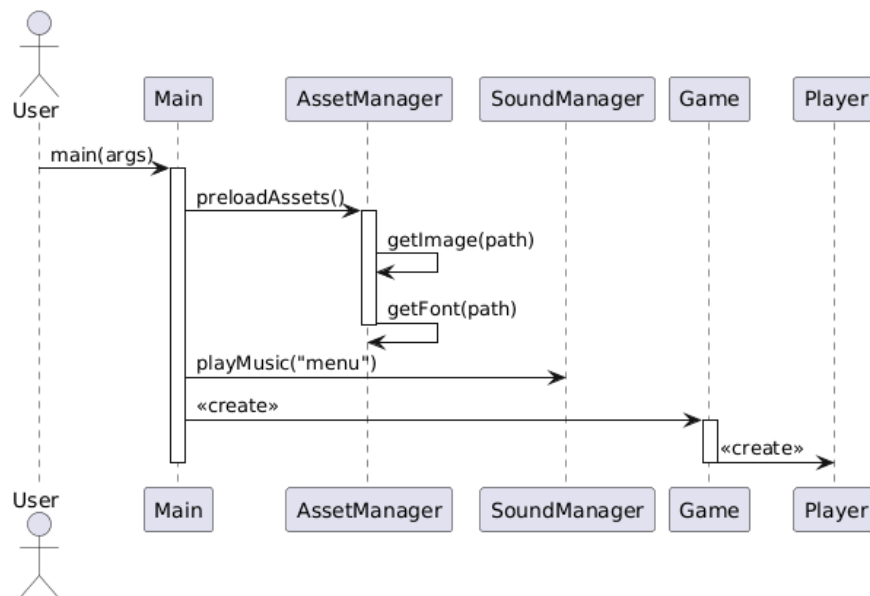
Billentyűkiosztás

Billentyű	Funkció
W / ↑	Előre / felfelé mozgás
S / ↓	Hátra / lefelé mozgás
A / ←	Balra mozgás
D / →	Jobbra mozgás
SPACE	Megállítás / akció
B	Bolt megnyitása
ESC	Menü / visszalépés

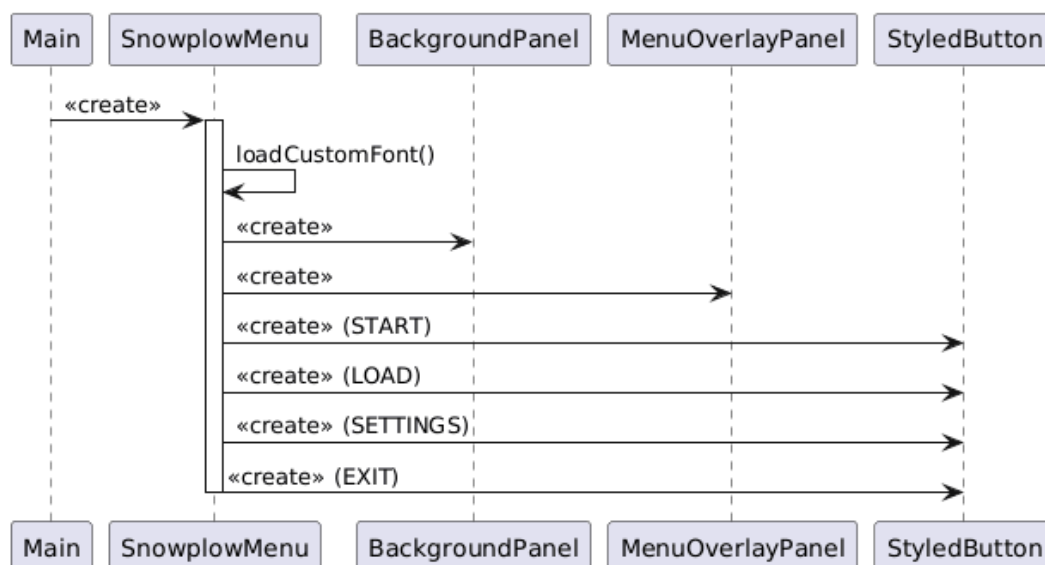
P	Szünet
ENTER	Megerősítés
C	Takarítás

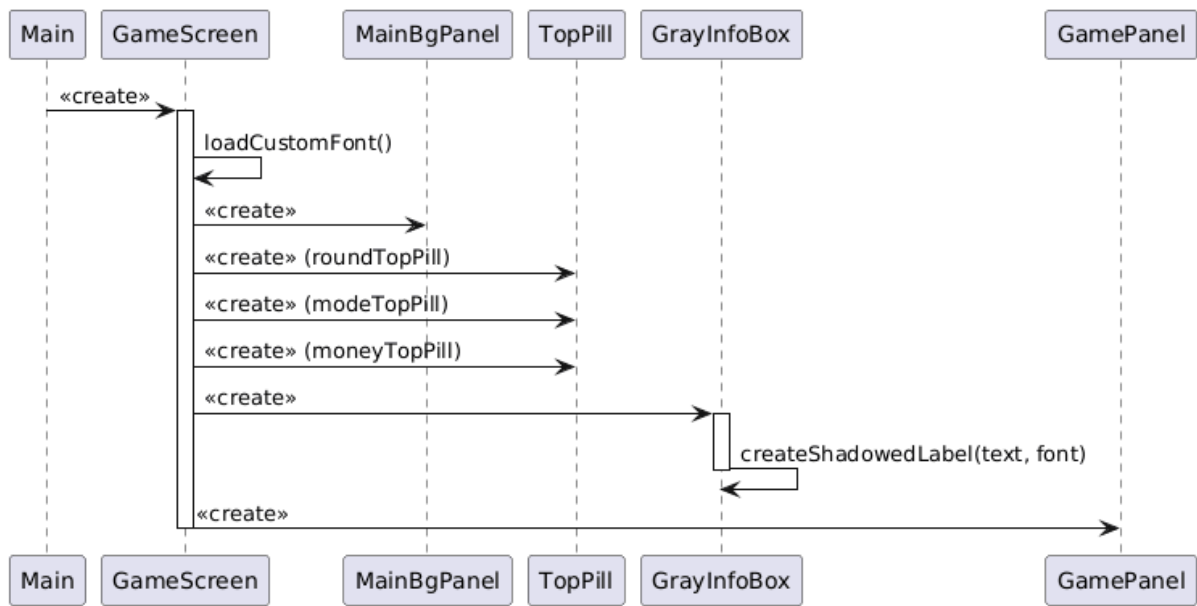
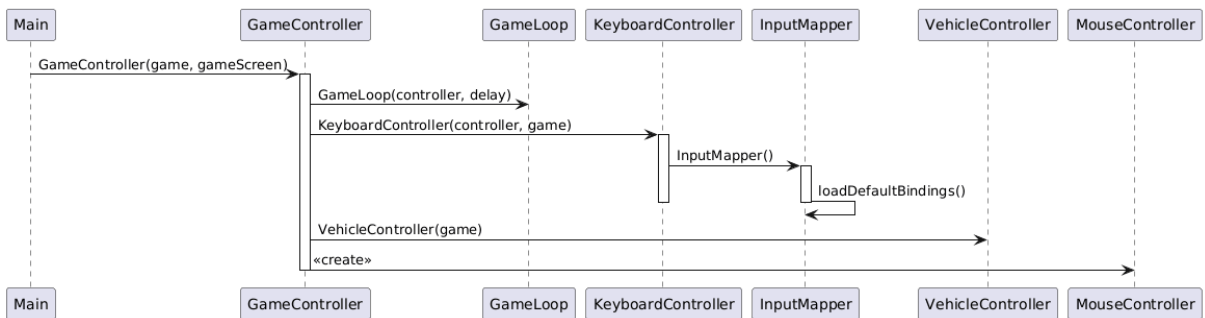
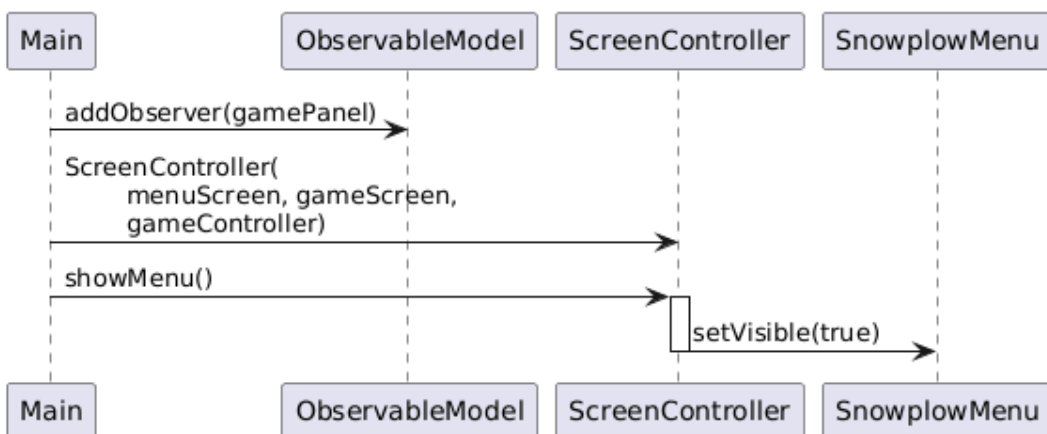
11.3 Kapcsolat az alkalmazói rendszerrel

1. Model és erőforrás inicializálás

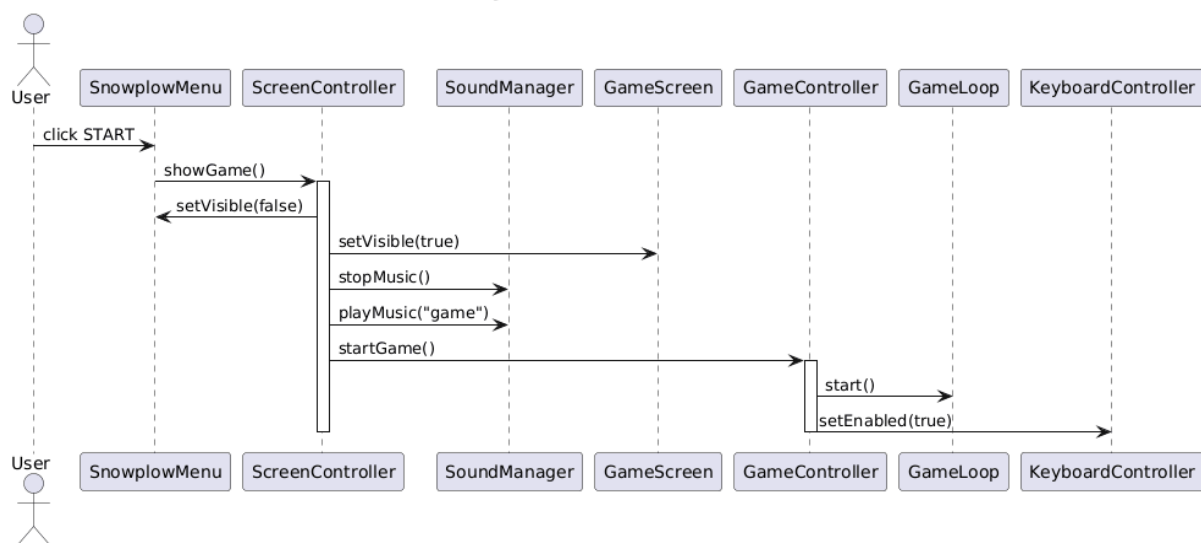


2. Menü nézet létrehozása

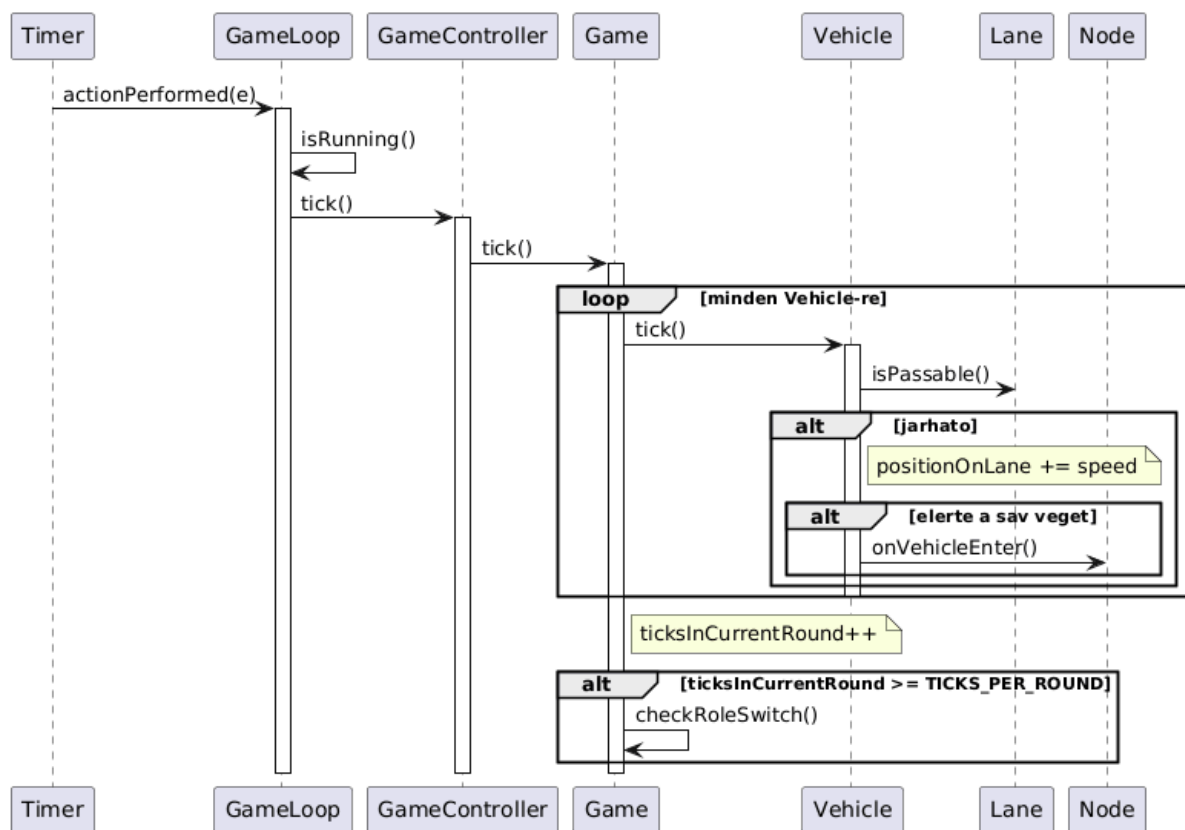


3. Játék nézet létrehozása**4. Controller réteg létrehozása****5. Observer regisztráció es menü megjelenítés**

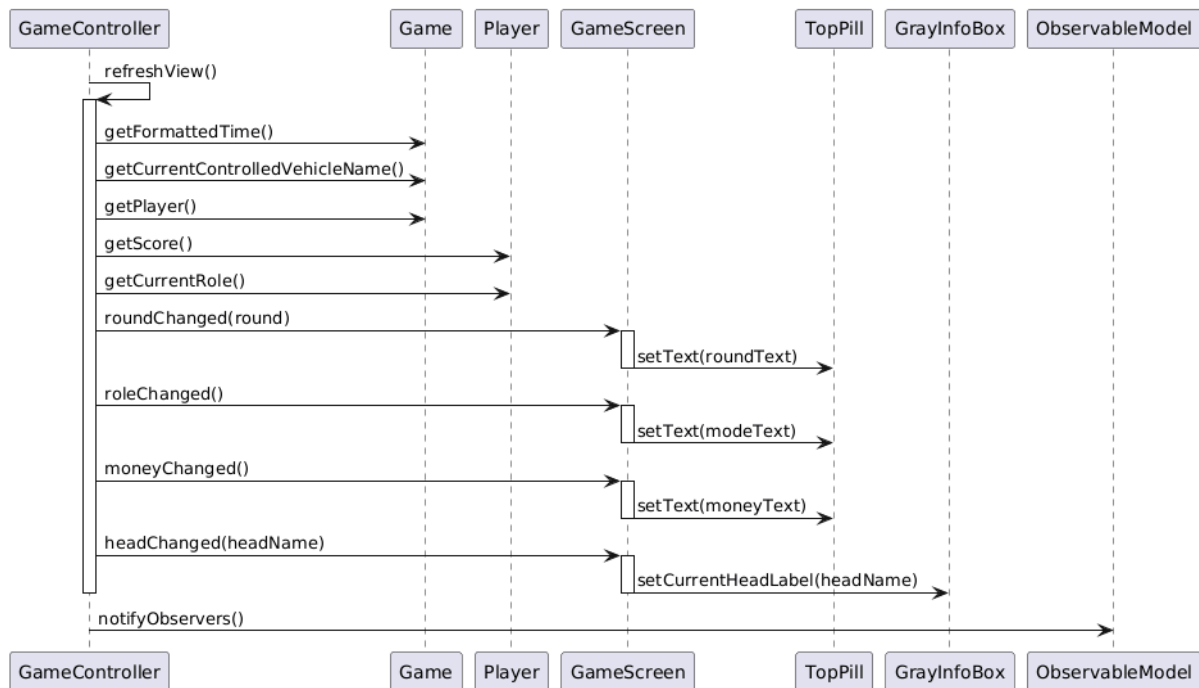
6. Játék indítás a főmenüből



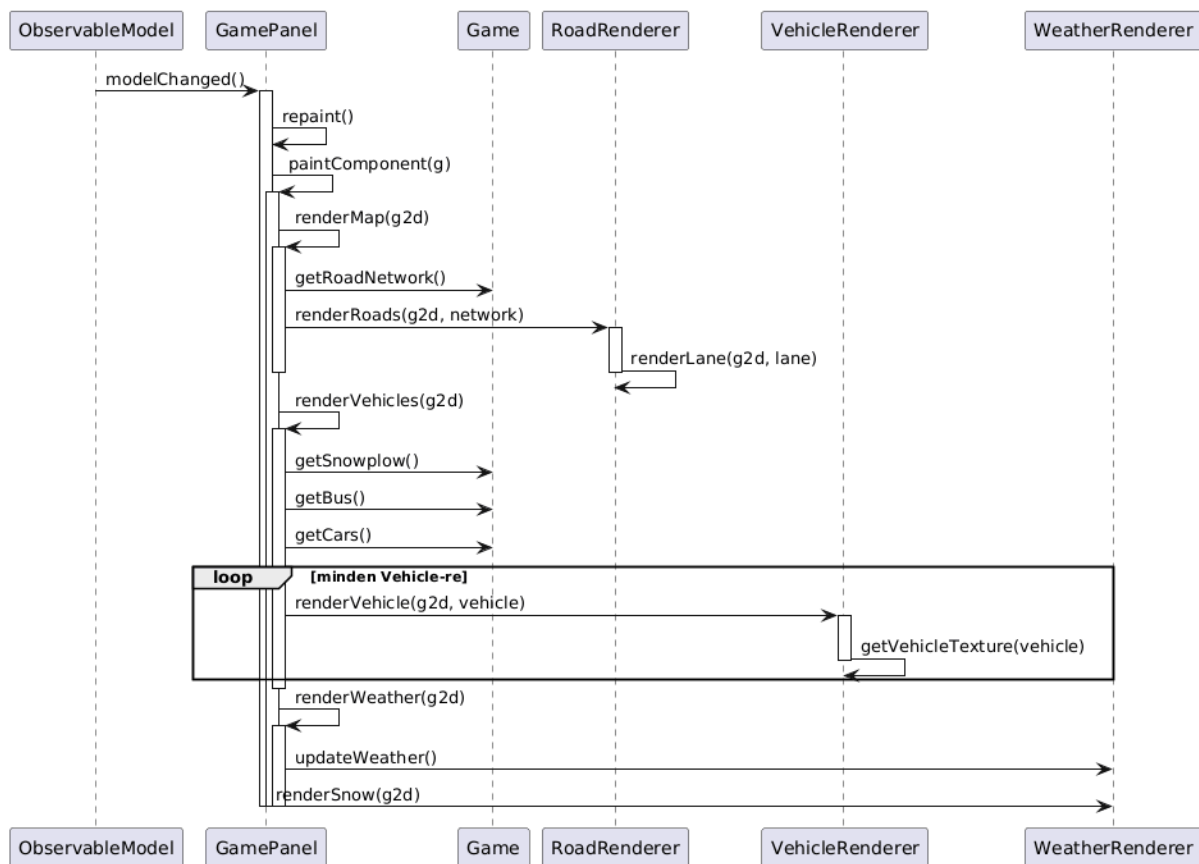
7. Game Loop tick és szimuláció



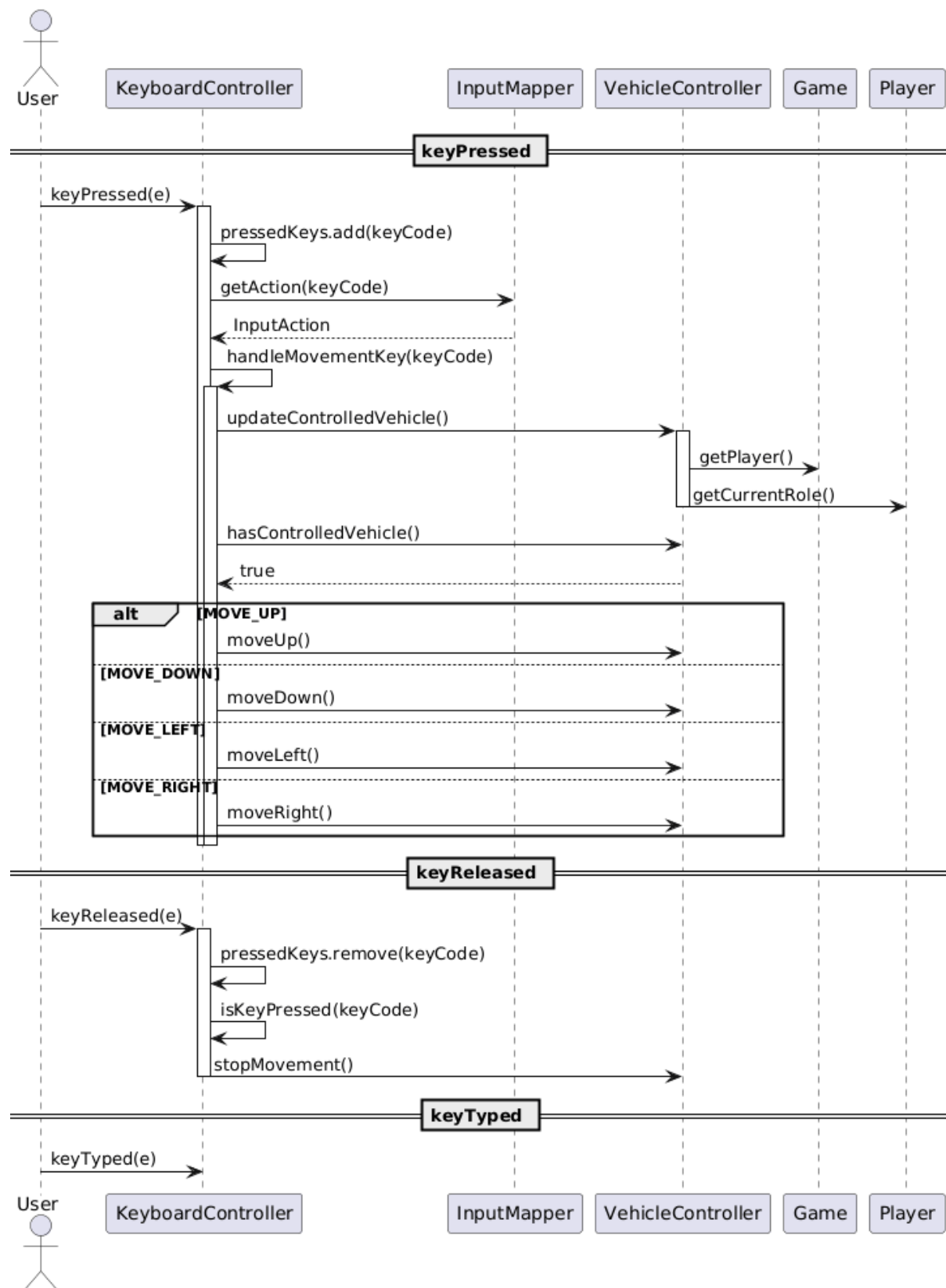
8. Nézet frissítés



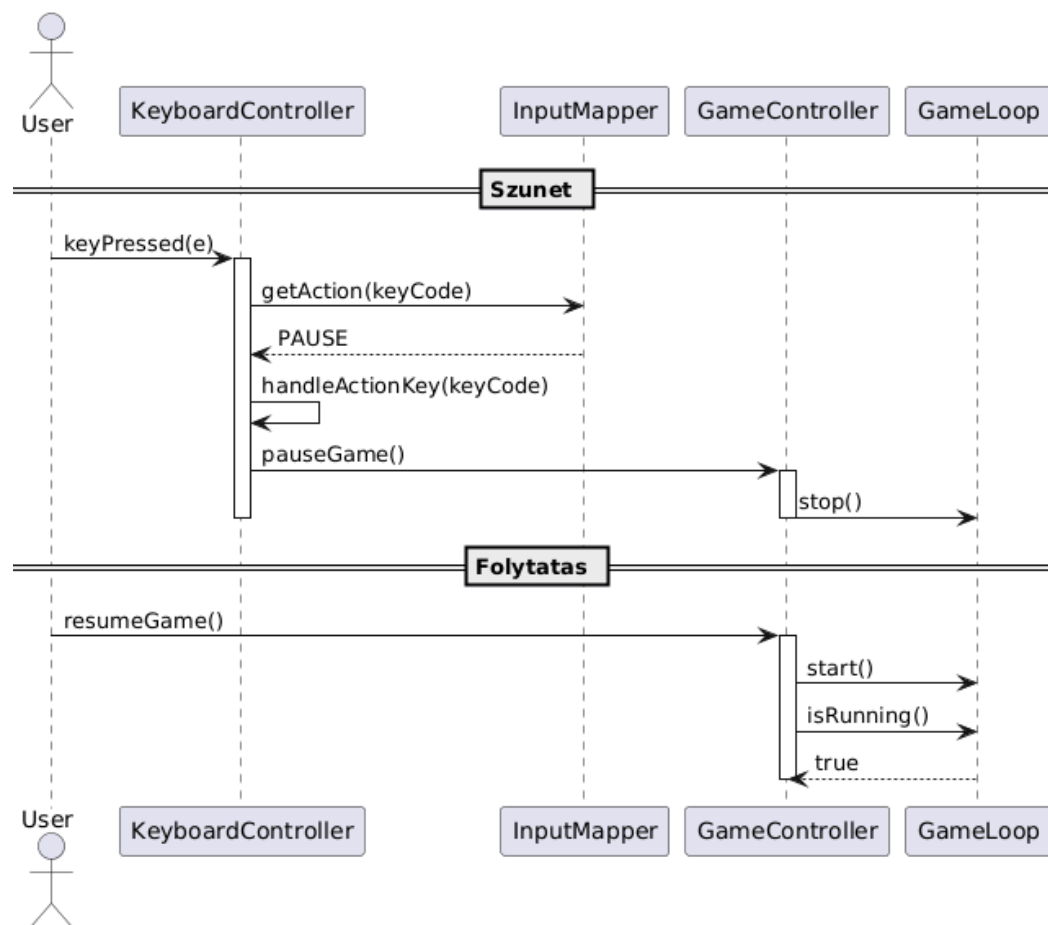
9. Observer és GamePanel renderelés



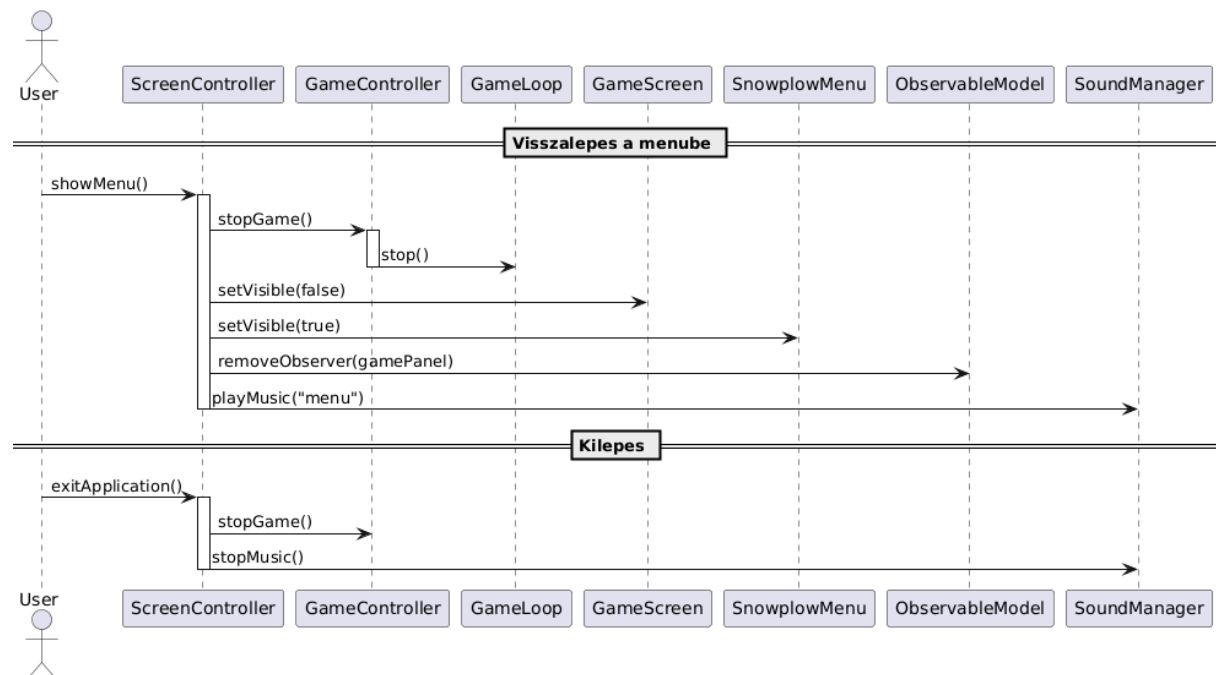
10. Billentyűzet mozgás kezelés



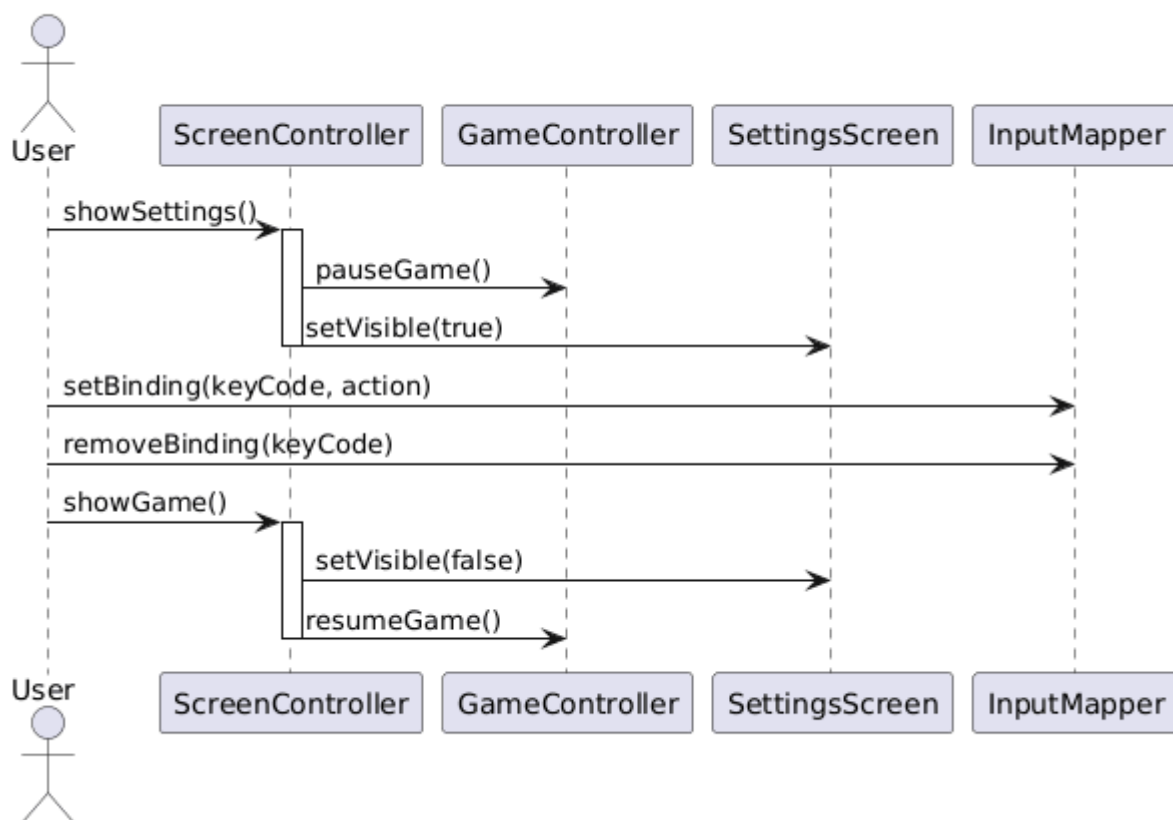
11. Szünet és folytatás



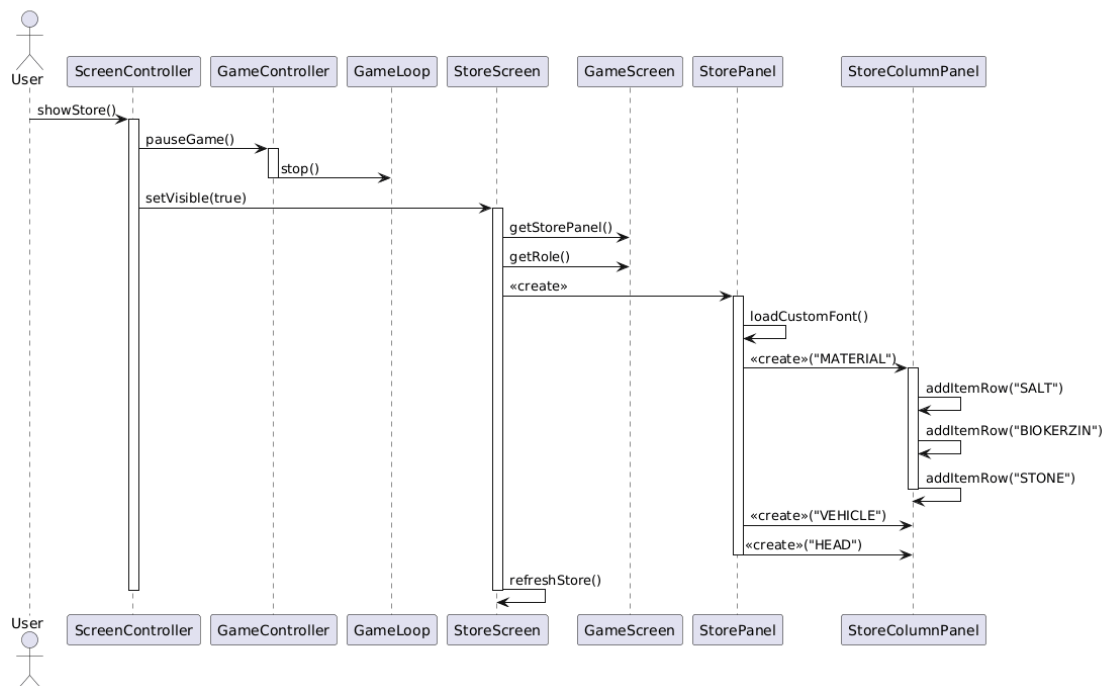
12. Menü visszalépés és kilépés



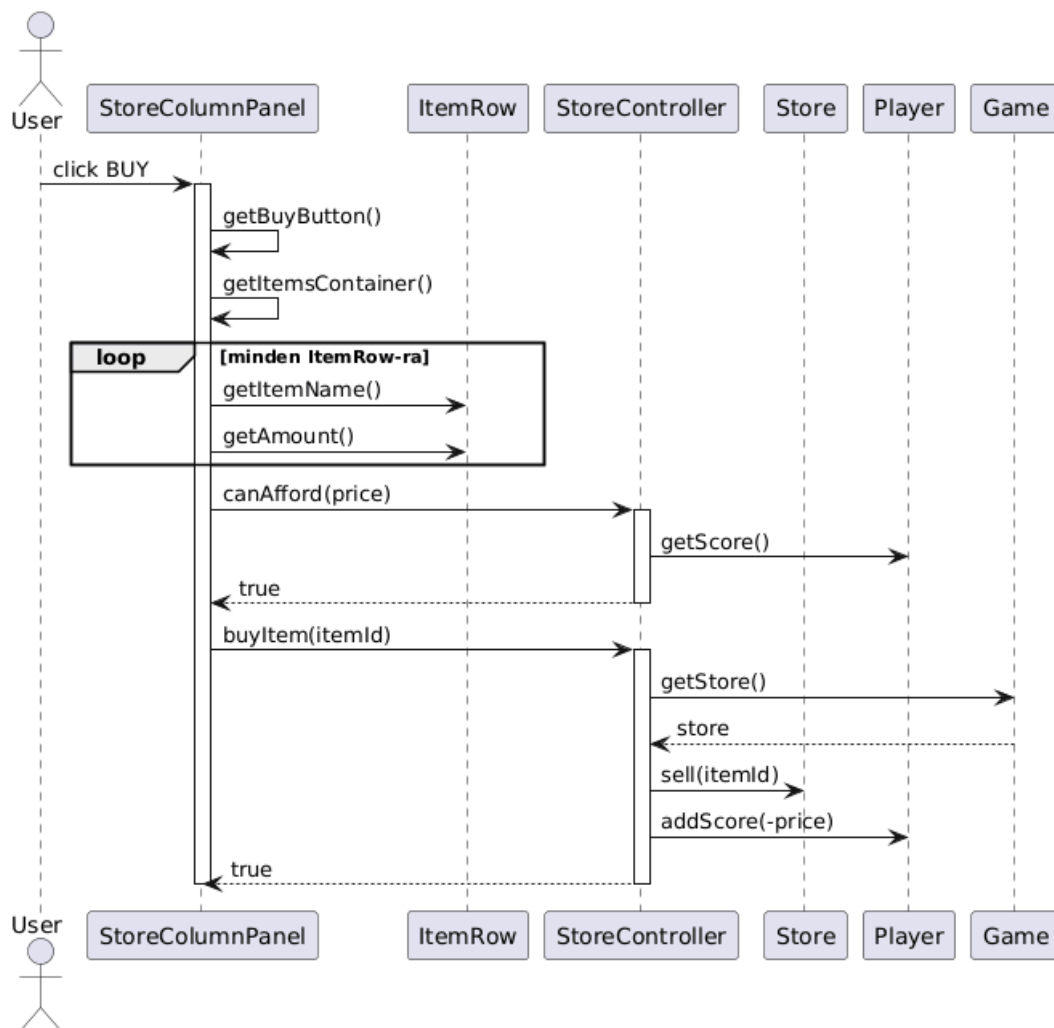
13. Beállítások és billentyű kiosztás



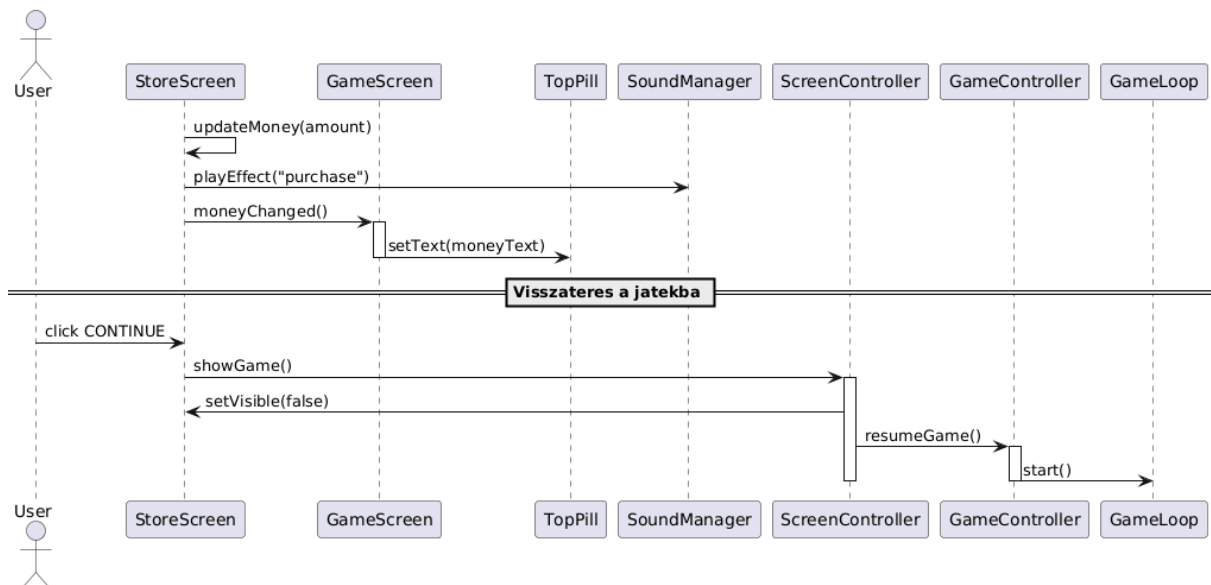
14. Bolt megnyitása és feltöltése

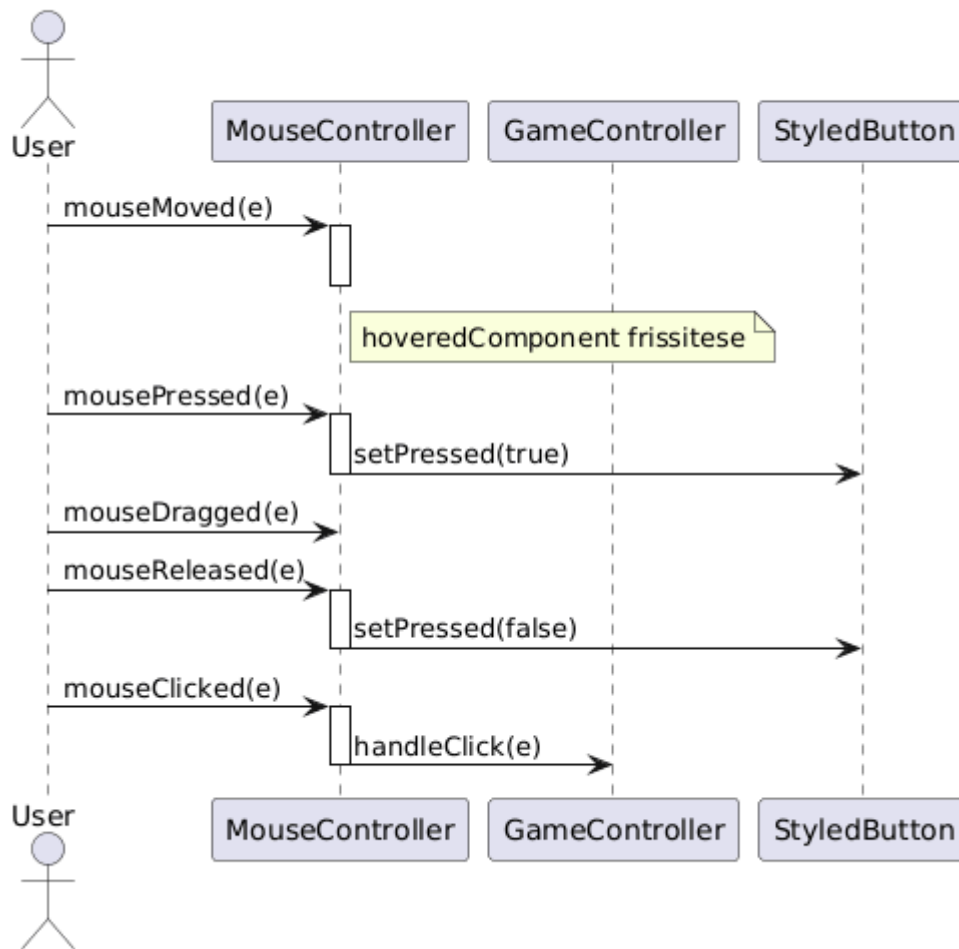


15. Vásárlás a boltban

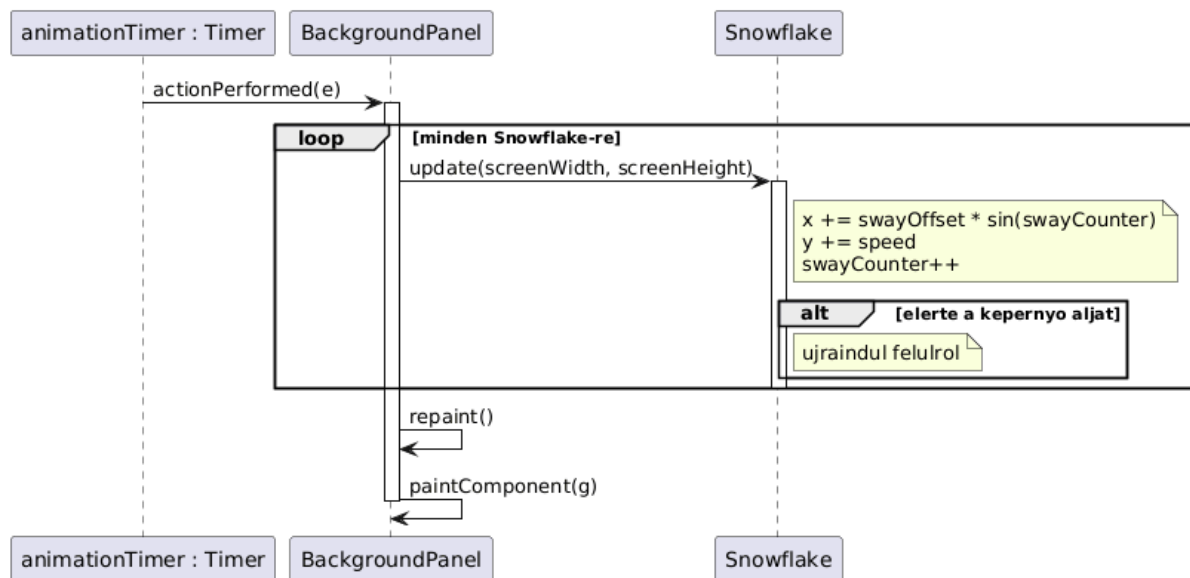


16. Bolt bezárás és pénz frissítés

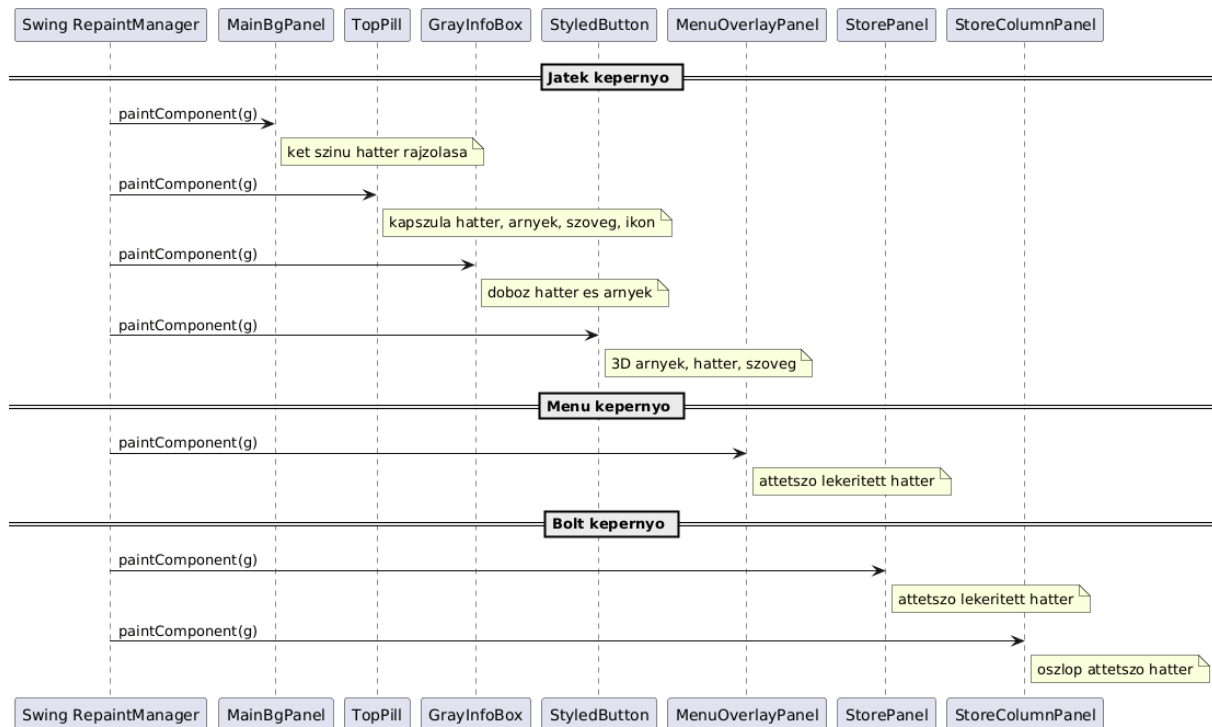


17. Egér kezelés

18. Hópehely animáció



19. Swing komponensek rajzolása



11.4 Napló

Kezdet	Időtartam	Résztevők	Leírás
2026.05.04. 18:00	2,5 óra	Bocskai Czap Jandó Tóth Péter	Értekezlet.
2026.05.05	5 óra	Dénes	grafikus panelek tervezése, megjelenés tervezése
2026.05.08.	2 óra	Jandó	A már meglévő de megváltozott osztályok leírásának megírása
2026. 05. 08.	2 óra	Tóth	Új osztályok írása
2026.05.08	5 óra	Dénes	grafikus panelek tervezése és megírása
2026.05.09.	4 óra	Bocskai	A megjelenítésért felelős osztályok leírása.
2026.05.10.	5 óra	Czap	1-19 szekvencia diagramok elkészítése.
2026. 05. 10.	5 óra	Bocskai	Osztálydiagram elkészítése
2026. 05. 10.	4 óra	Tóth	Maradék, hiányzó osztályok és “A felület működési elve” megírása

13. Grafikus változat terve

23 – githappens

Konzulens:

Haragos Gergő Viktor

Csapattagok

Czap László	NS7V2T	cz.laci04@gmail.com
Bocskai Zsófia	UGSTW9	bocskaizsofi@gmail.com
Tóth Márton	K01YXA	toth.marton21@gmail.com
Dénes Péter István	PO6MVA	denespeter03tab@gmail.com
Jandó Barnabás Tamás	AWQ77G	jandobarnabas@gmail.com

2026.05.27.

13. Grafikus változat beadása

13.1 Fordítási és futtatási útmutató

13.1.1 Fájllista

Fájl neve	Méret	Keletkezés ideje	Tartalom
controller\AssetManager.java	4173 byte	2026.05.26 20:49	Erőforrás-kezelő osztály, amely a játékban használt képeket, betűtípusokat és színeket kezeli. Singleton mintát alkalmaz, így egyetlen példányban létezik az alkalmazás futása során.
controller\GameController.java	61184 byte	2026.05.26 20:49	A játék fő vezérlője, amely összekapcsolja a játékmodellt, a nézetet és a bemeneti vezérlőket. Kezeli a pálya generálását, a játékos mozgását, a forgalmi autókat, az időjárásváltozásokat, a takarítási logikát, a pontszámítást és a körváltásokat.
controller\GameLoop.java	2053 byte	2026.05.26 20:49	A játék fő ciklusa, amely időzítő segítségével periodikusan meghívja a GameController tick() metódust. Az ActionListener interfészt implementálja a Swing Timer eseményeinek kezeléséhez.
controller\InputAction.java	822 byte	2026.05.26 20:49	A játékos által végrehajtható bemeneti műveletek felsorolása. Az egyes értékek a billentyűzetről vagy más beviteli eszközről érkező parancsokat reprezentálják.
controller\InputMapper.java	2614 byte	2026.05.26 20:49	Billentyűkódok és bemeneti műveletek közötti leképezést kezelő osztály. Alapértelmezett billentyűkiosztást tölt be a konstruktorban, amely később módosítható.
controller\KeyboardController.java	4172 byte	2026.05.26 20:49	Billentyűzet-vezérlő osztály, amely a KeyListener interfészt implementálja. A lenyomott billentyűket az InputMapper

			segítségével műveletekké alakítja, majd továbbítja a GameController-nek.
controller\MouseController.java	3622 byte	2026.05.26 20:49	Egérvezérlő osztály, amely a MouseListener és MouseMotionListener interfészeket implementálja. Az egéresemények alapján a megfelelő játékvezérlő műveleteket hívja meg.
controller\ScreenController.java	3375 byte	2026.05.26 20:49	Képernyővezérlő osztály, amely a különböző képernyők (menü, játék, beállítások, bolt) közötti navigációt kezeli.
controller\SoundManager.java	2503 byte	2026.05.26 20:49	Hangkezelő osztály, amely a háttérzene és a hangeffektusok betöltését és lejátszását végzi.
controller\StoreController.java	6001 byte	2026.05.26 20:49	A bolt vezérlője, amely a vásárlási logikát kezeli. Kezeli a különböző tárgyak (fejek, anyagok, hókotró) megvásárlását, és frissíti a bolt képernyőt a tranzakciók után.
controller\VehicleController.java	5838 byte	2026.05.26 20:49	Jármű-vezérlő osztály, amely reflexió segítségével irányítja az aktuálisan vezérelt járművet. A játékos szerepétől függően a hókotró vagy a busz vezérlését végzi.
Main.java	3577 byte	2026.05.26 20:49	A Main osztály az alkalmazás belépési pontja. Inicializálja és összehangolja az összes szükséges komponenst: a képernyőkezelőt, az erőforrás-kezelőt, a hangkezelőt és a bolt vezérlőjét.
ModelObservable.java	1214 byte	2026.05.26 20:49	Megfigyelhető modell osztály, amely az Observer tervezési mintát valósítja meg. A megfigyelők értesítést kapnak a modell változásairól.
ModelObserver.java	362 byte	2026.05.26 20:49	Megfigyelő interfész a modell változásainak figyeléséhez. Az ezt megvalósító osztályok értesítést kapnak, amikor a modell állapota megváltozik.
src\Bridge.java	512 byte	2026.05.26 20:49	A Bridge osztály egy híd típusú útszakaszt reprezentál.

src\BrokenIce.java	1287 byte	2026.05.08 16:34	A BrokenIce osztály a tört jég útállapotát reprezentálja. A tört jég kritikus, de járható útállapot, jellemzően jégtörő munkájának eredménye. Tovább kezelhető (pl. eltakarítással), hogy újra Clear állapotba kerüljön.
src\Bus.java	3542 byte	2026.05.26 20:49	A Bus osztály a városi buszjáratot reprezentálja. A busz két végállomás között közlekedik, és egy előre kijelölt útvonal mentén halad. Feladata a közlekedés lebonyolítása, a végállomások elérése, valamint a körök végrehajtása a szimulációban. A busz működése összekapcsolódik a BusDriverRole szereppel, amely az útvonal kijelöléséért és a teljesített fordulóknak nyilvántartásáért felel.
src\BusdriverRole.java	5873 byte	2026.05.26 20:49	A BusdriverRole a buszvezető szerepkört reprezentálja. A buszvezető felelős a buszok mozgatásáért, útvonalak megtervezéséért és a fordulóknak teljesítéséért két végállomás között. A szerepkör pontszáma a sikeresen teljesített fordulókból adódik.
src\Buyable.java	347 byte	2026.05.08 16:34	A Buyable interfész a megvásárolható elemek közös szerződését írja le. Az interfészt megvalósító objektumok rendelkeznek árral, amely alapján a boltban megvásárolhatók.
src\Car.java	6163 byte	2026.05.26 20:49	A Car osztály a városban lakóhely és munkahely között közlekedő személyautót reprezentálja. Az autó célja, hogy mindig a legrövidebb járható útvonalon jusson el a célállomására. Az útvonaltervezést a RoadNetwork végzi, míg a Car saját feladata a kijelölt útvonal követése és az

			aktuális közlekedési helyzethez való alkalmazkodás. Az autó reagálhat az utak járhatatlanná válására, és szükség esetén újratervezés kezdeményezhető.
src\CleanerRole.java	5773 byte	2026.05.26 20:49	A CleanerRole a takarító szerepkört reprezentálja. Felelős a hó eltávolításáért, a hókotrók irányításáért és a takarításhoz kapcsolódó gazdasági döntésekért. Emellett kezeli a játékos pénzét, vásárolhat, és működteti a hókotrók fejeit.
src\Clear.java	1422 byte	2026.05.08 16:34	A Clear osztály a tiszta (hótól mentes) út állapotát reprezentálja. A tiszta út az ideális útállapot, ahol a járművek normál sebességgel haladhatnak és nincs szükség hó eltávolítására. Ez az értékelt állapot a közlekedés szempontjából. A Clear állapot akkor áll fenn, amikor az útról sikeresen eltávolították az összes havat és jeget.
src\DeepSnow.java	1395 byte	2026.05.08 16:34	A DeepSnow osztály a vastag hó útállapotát reprezentálja. A vastag hó alatt az út nem járható járművek számára, csak a megfelelő hókotrók tudják azt kezelni. A vastag hó jelentősen nehezíti a közlekedést, és megköveteli az aktív hótakarítást az út újrahasználhatóságához.
src\DragonHead.java	1053 byte	2026.05.26 20:49	A DragonHead egy speciális tisztítófej, amely bio-kerozin felhasználásával magas hőmérséklettel olvasztja el a havat vagy jeget.
src\Game.java	10783 byte	2026.05.26 20:49	A Game osztály a szimuláció központi vezérlője. Feladata a teljes játékmenet koordinálása, a körök számának kezelése, valamint a

			játékban szereplő járművek és játékosok nyilvántartása.
src\Gravel.java	2175 byte	2026.05.26 20:49	Az osztály a sáv zúzott kővel fedett állapotát reprezentálja. A kiszórt zuzalék a jég csúszósságát megszünteti, de a söprő és hányó fej a zuzalékot ugyanúgy eltakarítja, mint a havat. A lángszóró, a jégtörő és a só a zuzalékra nem hat. A zuzalék nem tömörödik. Ha hó esik rá, akkor a hó egy idő után befedi, a hó a szokott módon letaposható és így lefagyott jéggé válik.
src\GravelSpreaderHead.java	1254 byte	2026.05.26 20:49	A GravelSpreaderHead speciális kotrófej, amely zúzott követ, azaz zuzalékot szór a jármű előtt az útfelületre. Elsődleges célja nem a hó fizikai eltávolítása, hanem a csúszós, jeges útfelület biztonságosabbá tétele.
src\Head.java	388 byte	2026.05.08 16:34	Absztrakt kotrófej. Minden fej egy adott tisztítási stratégiát valósít meg.
src\IcebreakerHead.java	1174 byte	2026.05.26 20:49	Az IcebreakerHead feladata a kemény, letapadt jégréteg mechanikus feltörése. Ez a fej elsősorban nem a teljes tisztítást végzi el, hanem a jég állapotát alakítja át kezelhetőbbé, például jégpáncélból feltört jéggé. Ezzel előkészíti a felületet más fejek, például a SweeperHead vagy ThrowerHead számára.
src\IceSheet.java	1135 byte	2026.05.08 16:34	Az IceSheet osztály a jégkéreg útállapotát reprezentálja. Jégkéreg esetén az út nem járható a járművek számára, ezért a forgalomhoz a felületet előbb kezelni kell.
src\Intersection.java	1360 byte	2026.05.26 20:49	Az Intersection osztály felel az úthálózat (RoadNetwork) csomópontjainak reprezentálásáért. Ez az

			osztály köti össze az egyes útszakaszokat (Road vagy Lane), és biztosítja a járművek (Vehicle) számára az áthaladást és a kanyarodást az egyik útszakaszból a másikra. Szerepet játszik a járművek útvonalkeresésében (routing) is, hiszen innen kérdezhető le, hogy egy adott pontból mely további utak érhetőek el.
src\Lane.java	10739 byte	2026.05.26 20:49	A Lane osztály az úthálózat gráfjának egy irányított élét reprezentálja. Ez az osztály a "gazdája" az útszakasz fizikai állapotának (hó, jég, baleset).
src\LaneState.java	789 byte	2026.05.08 16:34	A LaneState interfész egy sáv aktuális állapotát írja le. Az állapot meghatározza, hogy a sáv járható-e, mekkora dinamikus terhelés tartozik hozzá, és hogyan változik időjárási hatásra.
src\Node.java	900 byte	2026.05.08 16:34	Absztrakt alaposztály az úthálózat összes csatlakozási pontjához.
src\NormalRoad.java	1009 byte	2026.05.26 20:49	A NormalRoad egy hagyományos útszakaszt modellez. Az időjárási hatások közvetlenül érvényesülnek rajta, így a sávok állapota a hó vagy jég mennyiségének megfelelően változhat.
src\Player.java	4523 byte	2026.05.26 20:49	A Player egy valódi emberi résztvevőt reprezentál a rendszerben. A játékoshoz egy aktuális Role és egy külön kezelt pontszám tartozik.
src\Residence.java	812 byte	2026.05.26 20:49	A Residence osztály egy lakóhely típusú csomópontot reprezentál.
src\Road.java	4361 byte	2026.05.26 20:49	Absztrakt alaposztály minden úttípushoz. Kezeli a sávok listáját és az időjárási hatásokat.
src\RoadNetwork.java	5101 byte	2026.05.26 20:49	A RoadNetwork osztály felelős Zúzmaraváros teljes úthálózatának reprezentálásáért és

			kezeléséért. Ez az osztály működik a játék útvonalkereső motorjaként: nyilvántartja a csomópontokat és útszakaszokat, valamint kiszámítja a járművek számára az aktuálisan járható legrövidebb utat.
src\Role.java	487 byte	2026.05.26 20:49	A Role absztrakt osztály felel a járművek irányításáért. A szerepkörök határozzák meg, hogy a játékos milyen műveleteket végezhet és hogyan szerez pontokat a játék során. Egy játékos több szerepkört is felvehet egyszerre.
src\Route.java	2800 byte	2026.05.26 20:49	A Route osztály egy konkrét útvonaltervet reprezentál, amelyet az úthálózat útvonalkereső algoritmus állít elő a járművek számára. Feladata a célba jutáshoz szükséges sávok rendezett listájának tárolása. A szimuláció során ez az osztály felelős azért, hogy a jármű haladása közben kiszolgálja a következő sávot, amelyre a járműnek rá kell hajtania.
src\SaltSpreaderHead.java	1137 byte	2026.05.26 20:49	A SaltSpreaderHead a hókotró sókészletét használva csökkenti az utak jegesedését és javítja a tapadást. Működéséhez szükség van a hókotró sókészletére.
src\Snowplow.java	6608 byte	2026.05.26 20:49	A Snowplow osztály egy hókotrót reprezentál. Különböző fejekkel képes tisztítani az utakat.
src\Store.java	4468 byte	2026.05.26 20:49	A Store osztály a játék boltját reprezentálja. A bolt tárolja a megvásárolható elemeket (pl. fejek, járművek, anyagok), és kezeli a vásárlási műveleteket a szerepkörök számára.
src\SweeperHead.java	1584 byte	2026.05.26 20:49	A SweeperHead olyan mechanikus tisztítófej, amely a havat vagy a fellazított jeget eltakarítja. A zúzalékot

			ugyanúgy eltakarítja, mint a havat. Vastag, de még nem teljesen lefagyott hó eltávolítására alkalmas, valamint képes a korábban kiszórt zúzott kő eltávolítására is, ha arra már nincs tovább szükség az adott sávon.
src\Terminal.java	813 byte	2026.05.26 20:49	A Terminal osztály egy végállomás típusú csomópontot reprezentál.
src\ThinSnow.java	1235 byte	2026.05.26 20:49	A ThinSnow osztály a vékony hó útállapotot reprezentálja. Vékony hó esetén az út alapvetően még járható, de a felület kezelésével javítható a közlekedés biztonsága és hatékonysága.
src\ThrowerHead.java	1354 byte	2026.05.26 20:49	A ThrowerHead olyan mechanikus tisztítófej, amely a havat, zúzottkövet vagy a fellazított jeget oldalra dobja.
src\Tunnel.java	738 byte	2026.05.26 20:49	A Tunnel osztály egy alagút típusú útszakaszt reprezentál. Az alagútban az időjárás hatása nem érvényesül közvetlenül, helyette a hőszint csökken.
src\Vehicle.java	4693 byte	2026.05.26 20:49	A Vehicle absztrakt osztály a rendszerben szereplő összes jármű közös őssztálya. Felelős az alapvető mozgási logikáért és az állapotok nyilvántartásáért.
src\Weather.java	3032 byte	2026.05.26 20:49	A Weather osztály a szimuláció időjárási viszonyait reprezentálja. Felelős a havazás szimulálásáért és az utak állapotának befolyásolásáért.
src\Workplace.java	893 byte	2026.05.26 20:49	A Workplace osztály egy munkahely csomópontot reprezentál.
view\BackgroundPanel.java	2662 byte	2026.05.26 20:49	Animált hóesést és háttérképet kirajzoló panel. Ha nem találja a megadott képet, sötétszürke hátteret használ.
view\GameScreen.java	47052 byte	2026.05.26 20:49	A játék fő képernyője, amely tartalmazza a játékeret, a felső információs sávot, a jobb

			oldali vezérlőpanelt és a HUD elemeket.
view\GrayInfoBox.java	4448 byte	2026.05.26 20:49	Szürke, lekerekített sarkú információs doboz, amely a hókotró aktuális fejét mutatja és egy váltó gombot tartalmaz.
view\ItemRow.java	6005 byte	2026.05.26 20:49	Egy sor a bolt vagy beállítások listájában, amely egy rózsaszín címkét és egy léptető (Spinner) vagy vásárlás gombot tartalmaz.
view\MainBgPanel.java	936 byte	2026.05.26 20:49	Fő háttérpanel, amely kirajzolja a kétszínű felületet: sötétkék bal oldalt és menta zöld jobb oldali panelt.
view\MenuOverlayPanel.java	1129 byte	2026.05.26 20:49	Félig áttetsző, lekerekített sarkú menü panel, amely kékes-szürke háttérrel jeleníti meg a menüelemeket függőleges elrendezésben.
view\RoadRenderer.java	1222 byte	2026.05.26 20:49	Az úthálózat és sávok grafikus megjelenítéséért felelős osztály.
view\SettingsScreen.java	3286 byte	2026.05.26 20:49	A beállítások képernyő, ahol a játékos beállíthatja a játékosok számát, a kör időtartamát és az autók számát.
view\Snowflake.java	1903 byte	2026.05.26 20:49	Egyetlen hópehely adatait tároló osztály, amely kezeli a pozíciót, méretet, sebességet és az oldalirányú kilengést.
view\SnowplowMenu.java	2660 byte	2026.05.26 20:49	A játék főmenüje, amely Start, Settings és Exit gombokat tartalmaz animált havas háttérrel és áttetsző menüpanellel.
view\StoreColumnPanel.java	3577 byte	2026.05.26 20:49	A bolt egy oszlopát reprezentáló panel, amely egy címet, elemsorokat és egy vásárlás gombot tartalmaz.
view\StorePanel.java	3910 byte	2026.05.26 20:49	A bolt fő panele, amely három oszlopban jeleníti meg az anyagokat, járműveket és fejeket vásárlási lehetőséggel.
view\StoreScreen.java	7864 byte	2026.05.26 20:49	A bolt képernyő, amely megjeleníti a vásárolható elemeket, a játékos pénzt és a hókotró készleteit.

view\StyledButton.java	2808 byte	2026.05.26 20:49	Egyedi megjelenésű, 3D árnyékos gomb lekerekített sarkokkal és nyomás effektussal.
view\TopPill.java	2917 byte	2026.05.26 20:49	A felső sávban megjelenő, lekerekített rózsaszín kapszula alakú panel, amely szöveget és opcionálisan ikont jelenít meg.

13.1.2 Fordítás és telepítés

A grafikus felületet tartalmazó alkalmazás Java nyelven, Swing könyvtár használatával készült. A projekt futtatásához és fordításához Java Development Kit (JDK) szükséges.

Fordítás lépései

1. A projekt forrásfájljait egy közös projekt mappába kell elhelyezni.
2. Terminálban vagy parancssorban a projekt gyökérkönyvtárába kell navigálni.
3. A Java forrásfájlok fordítása a következő paranccsal történik:

javac -encoding UTF-8 -d bin Main.java controller*.java view*.java src*.java

A parancs lefordítja a forráskódot, és a létrejövő .class fájlokat a bin mappába helyezi.

Telepítés

A rendszer nem igényel külön telepítőt vagy külső könyvtárat. A futtatáshoz elegendő:

- a lefordított **bin** mappa,
- a szükséges erőforrásfájlok (képek, betűtípusok),
- valamint a megfelelő Java futtatókörnyezet megléte.

Az erőforrásfájlokat a projekt struktúrájának megfelelő helyen kell tárolni, hogy az alkalmazás elérje őket futás közben.

13.1.3 Futtatás

A program indításához először meg kell győződni arról, hogy a Java futtatókörnyezet elérhető a rendszeren.

A grafikus alkalmazás a következő paranccsal indítható:

java "-Dfile.encoding=UTF-8" -cp bin Main

A parancs elindítja a játék grafikus felületét tartalmazó főablakot.

A futtatás során:

- megjelenik a játéktér,
- betöltődnek a grafikus elemek,
- valamint elérhetővé válnak a játék vezérlő funkciói és kezelőfelületei.

A rendszer nem igényel internet kapcsolatot vagy külön háttér szolgáltatást a működéshez.

13.2Értékelés

Tag neve	Tag neptun	Munka százalékban
Czap László	NS7V2T	20%
Bocskai Zsófia	UGSTW9	20%
Tóth Márton	K01YXA	20%
Dénes Péter István	PO6MVA	20%
Jandó Barnabás Tamás	AWQ77G	20%

13.3 Napló

Kezdet	Időtartam	Résztevők	Leírás
2026.06.16.	2 óra	Bocskai Czap Jandó Tóth Dénes	Értekezlet. Feladatok felosztása
2026.06.18.	2 óra	Tóth	Felosztott diagram lila mezőinek elkészítése.
2026.06.18.	2 óra	Czap	Felosztott diagram szürke osztályainak megvalósítása.
2026.06.19.	2 óra	Bocskai	Felosztott diagram kék osztályainak elkészítése.
2026.06.19	3 óra	Dénes	Grafikus felületek készítése
2026.06.19	2 óra	Jandó	Felosztott diagram narancssárga osztályainak elkészítése
2026.06.21.	2 óra	Bocskai Czap Jandó Tóth Dénes	Értekezlet. Feladatok nyomon követése, egyeztetés.
2026.06.23.	3 óra	Tóth	Grafikus felület összeegyeztetése, véglegesítése.
2026.06.24.	2 óra	Tóth	Grafikus felület hibák kijavítása.
2026.06.26.	2 óra	Bocskai	Grafikus felület működésén finomítás
2026.06.26.	2 óra	Dénes	Grafikus felületek készítése
2026.06.26.	2 óra	Czap	Javadoc kommentek kiegészítése és fájllista kitöltése.
2026.06.26	1 óra	Jandó	Tesztelés

14. Összefoglalás

23 – githappens

Konzulens:

Haragos Gergő Viktor

Csapattagok

Czap László	NS7V2T	cz.laci04@gmail.com
Bocskai Zsófia	UGSTW9	bocskaizsofi@gmail.com
Tóth Márton	K01YXA	toth.marton21@gmail.com
Dénes Péter István	PO6MVA	denespeter03tab@gmail.com
Jandó Barnabás Tamás	AWQ77G	jandobarnabas@gmail.com

2026.05.29.

14. Összefoglalás

14.1 A projektre fordított összes munkaidő

Tag neve	Munkaidő (óra)
Czap László	88
Dénes Péter István	88
Jandó Barnabás Tamás	88
Tóth Márton	88
Bocskai Zsófia	88
Összesen	440

- A feltöltött programok forrásainak száma**

Fázis	Kódsorok száma
Szkeleton	4609
Prototípus	5565
Grafikus változat	9508
Összesen	19682

14.2 • Projekt összegzés

14.2.1 Mit tanultak a projektből konkrétan és általában?

A projekt során megtapasztaltuk, milyen egy csapat részeként dolgozni, összehangolni a munkát, alkalmazkodni egymás stílusához és erősségeihez, valamint hatékonyan felosztani a feladatokat. Emellett fejlődünk az összetett feladatok tervezésében, dokumentálásában és megvalósításában.

14.2.2 Mi volt a legnehezebb és a legkönnyebb?

A legnehezebb rész az volt, hogy összehangoljuk a munkát és az elvégzett feladatok egészéé álljanak össze. Időnként kihívást jelentett a feladatok felosztása, különösen, amikor a részfeladatok nagy mértékben kapcsolódtak egymáshoz.

A legkönnyebb rész a hét eleji megbeszélések és a közös gondolkodás volt

14.2.3 Összhangban állt-e az idő és a pontszám az elvégzendő feladatokkal?

A pontszám összhangban állt, az idő általában igen, de a Skeleton elkészítésére viszonylag kevés idő volt, ami okozott kellemetlenséget.

14.2.4 Ha nem, akkor hol okozott ez nehézséget?

A Skeleton elkészítésekor nagyobb nyomás alatt és gyorsabb ütemben kellett dolgoznunk ahhoz, hogy minden funkció időben elkészüljön.

14.2.5 Milyen változtatási javaslatuk van?

A tárgy több kreditet érdemelne, a befektetett energiát nem találjuk arányosnak a kreditszámmal. A határidők és a konzultációs alkalmak kiírása lehetne egyértelműbb.

14.2.6 Milyen feladatot ajánlanának a projektre?

Ehhez hasonló feladatot.

Zenekar szimuláció.

14.2.7 Egyéb kritika és javaslat